

PrimeSim™ HSPICE® User Guide: Signal Integrity Modeling and Analysis

Version W-2024.09, September 2024



Contents

New in This Release	14
Related Products, Publications, and Trademarks	14
Conventions	14
Customer Support	15
Statement on Inclusivity and Diversity	16

Part 1: Signal Integrity Analyses

1. Introduction to Signal Integrity	18
Getting Started on Signal Integrity Simulations	20
Signal Integrity Problems	20
Analog Side of Digital Logic	21
System Design Issues	22
Simulating Circuits with IBIS Models in PrimeSim HSPICE	23
Using the B-element to Instantiate Individual Buffers	24
Using the IBIS Component	25
Determining Output and Input Pins in the IBIS Component	26
Power and Ground Pins in the IBIS Component	27
Package and Pin Parasitics	27
Interconnect Model	27
Power Supply of the IBIS Buffer	28
2. Linear Network Parameter Analysis	29
LIN Analysis	29
Identifying Ports with the P-element	31
Using a Hierarchy Delimiter to Narrow the AMI Parameter Assignment	38
Using the P-element for Mixed-Mode Measurement	38
.LIN Input Syntax	39
.LIN Output Syntax	40
.PRINT and .PROBE Statements	40
Examples	41
Hybrid Parameter Calculations	42

Contents

Multi-Port Scattering (S) Parameters	42
Two-Port Transfer and Noise Calculations	43
Equivalent Input Noise Voltage and Current	44
Equivalent Noise Resistance and Conductance	45
Noise Correlation Impedance and Admittance	45
Optimum Matching for Noise	45
Noise Figure and Minimum Noise Figure	45
Associated Gain	46
Output Format for Group Delay in .sc* Files	46
Output Format for Two-Port Noise Parameters in *.sc# Files	46
Noise Parameters in 2-Port and N-Port Networks	47
Hybrid (H) Parameters	48
Group Delay	49
Summary of Printing and Probing Network Parameters	50
Specifying the Analysis Type	50
Port Reference	50
Port Numbers with More Than One Digit	51
Network Parameter Data Types	51
Mixed-Mode Ports	51
SNLIN Analysis	52
Multitone Analysis	53
SNLIN Input Syntax	53
SNLIN Output Syntax	54
.PRINT and .PROBE Statements	54
Output to Data Files	55
Examples	56
Additional Measurements From .LIN Command	57
Impedance Characterizations	57
Stability Measurements	57
Gain Measurements	57
Matching for Optimal Gain	58
VSWR	59
ZIN(i)	59
YIN(i)	59
K_STABILITY_FACTOR (Rollett Stability Factor)	59
MU_STABILITY_FACTOR (Edwards-Sinsky Stability Factor)	59
Maximum Available Power Gain - G_MAX	60
Maximum Stable Gain - G_MSG	60
Maximum Unilateral Transducer Power Gain - G_TUMAX	60
Unilateral Power Gain - GU	61
Simultaneous Conjugate Match for G_MAX	61
Equivalent Input Noise Voltage and Current - IN2, VN2, RHON	62

Contents

Equivalent Noise Resistance and Conductance - RN, GN	62
Noise Correlation Impedance and Admittance - ZCOR, YCOR	63
Optimum Matching for Noise - ZOPT, YOPT, GAMMA_OPT	63
Noise Figure and Noise Figure Minimum - NF, NFMIN	63
Associated Gain - G_As	64
Example - Extracting Bandpass Filter S-parameters	64
Extracting Mixed-Mode Scattering (S) Parameters	67
Syntax	67
Defaults	68
Output File Formats	69
Two-Port Parameter Measurement	69
Output Format and Description	69
Features Supported	70
Prerequisites and Limitations	70
Reported Statistics for the Performance Log	71
Errors and Warnings	71
Tutorial Example: LIN Analysis for a NMOS Low Noise Amplifier	71
References	78
3. Statistical Eye Analysis	80
Understanding StatEye Analysis in PrimeSim HSPICE	81
Statistical Eye Analysis Setup	82
Port Element Configuration	83
.PRINT and .PROBE Syntax	83
Output Data	85
.MEASURE Syntax	86
Understanding the Worst Pattern Measurement Result	89
Understanding n-UI Accumulated Clock Period Jitter Measurements (RMS/PP)	91
Sample Measurements	92
Output	92
Saving and Reusing Initial Transient Results	92
Examples - Statistical Analysis Setup	93
Example – StateEye Crosstalk using W Element	94
Determining Number of Bits Simulated in Statistical and Bit-by-bit Modes	96
Statistical Mode	96
Bit-by-bit Mode	97

Contents

Pulse Superposition and Edge Mode	97
Edge Control	98
Statistical Eye Full Transient Mode	99
Example - Controlling Edges	100
Example - Run StatEye with Precomputed Transient Waveform	100
Separate Rise Time and Fall Time Settings	101
Multi-Edge Responses for Accurate Modeling of Nonlinearity	101
Capturing Steady State Noise for Power Integrity	102
Example - Create SSN Eye Diagram	105
Example - ADD SSN Eye Diagram	106
Duty Cycle Distortion	107
Capturing DCD Effect on System Clock	109
Periodic and Random Jitter Effect	110
Periodic Jitter	111
Random Jitter Noise Effect	111
Specifying Random Jitter by Peak-to-Peak Value with BER	113
Specifying Custom Jitter Distribution	113
Port Element Syntax for Jitter	115
N-Cycle Jitter Modeling for Data and Clock Signals	116
Pre-Emphasis and De-Emphasis	120
Syntax for P-Element and Transient V and I Sources	122
Usage Examples	124
Example 1: Pre-Emphasis	124
Example 2: De-Emphasis	125
Built-In DFE Function for Rx Port Element in StatEye Analysis	126
Perturbation Analysis with StatEye	129
Unfolding Statistical Eye Diagrams to Waveforms	130
Multiple Data Rate Eye Diagram Analysis with StatEye	133
Accelerating StatEye's Initial Transient with Process Distribution and Multi-Threading	133
Known Limitation	134
Troubleshooting Output Warnings	134
References	134
4. Time Domain Reflectometry	136

Contents

Optimizing TDR Packaging	136
Performing a TDR Simulation in PrimeSim HSPICE	137
Basic TDR Impedance Analysis	138
Example 1: Short Circuit Termination	139
Example 2: Open Circuit Termination	139
Example 3: Mismatched Load Termination	140
Example 4: Shunt Capacitance Discontinuity	140
Example 5: Series Inductance Discontinuity	141
Testbench Netlist Overview	141
Input and Output Ports	142
Selecting the Risettime of the Incident Wave	142
Simulating the Example DUTs	143
Troubleshooting: TDR Zo Result Starts at Negative Value	145
Differential TDR Example Netlist	146
References	147

Part 2: Signal Integrity Modeling

5. S-parameter Modeling Using the S-element	149
S-parameter Model	150
Notifications and Limitations	151
Using the Scattering Parameter Element	151
S Model Syntax	152
S-element Data File Model Examples	158
Mixed-Mode S-parameters	160
Relating Voltage and Current Waves to Nodal Waves	161
Characterizing Differential Data Transfer Systems	162
Deriving a Simpler Set of Voltage and Current Pairs	163
Using the Mixed-Mode S-parameters (S-element)	164
Accelerating S-Element Time Domain Performance with Recursive Convolution	166
Multithreading Acceleration for S-element on Linux	168
Ensuring Causality in the Rational Function Model	169
Rational Function Matrix (.rfm) File Format	169
Multiport Noise Model for Passive Systems	171

Input Interface	172
Output Interface	173
S-element Noise Model	173
Two-port Noise Parameter Support in Touchstone Files	173
Input Interface	174
Output Interface	175
Notifications and Limitations	176
S-parameter Standalone Manipulation Utility	176
SPutil Program Features	177
Invoking the Utility	177
SPutil Runset Format	178
Commands	178
SPUTIL Functions	186
Status Messages	187
Causality Checking	187
Using Interpolation for Causality Checking	188
Preconditioning S-parameters	189
Group Delay Handler in Time Domain Analysis	190
S-Model Data Smoothing	190
Data Smoothing Methods	191
S-Model Syntax	191
Description	191
Example	192
Predicting an Initial Value for FMAX in S-Element Models	192
De-Embedding S-parameters	193
Small-Signal Parameter Data Frequency Table Model	195
SP Model Syntax	196
Four Valid Forms of the SP Model	197
Examples	199
References	202
6. W- and T-elements Transmission-Line Modeling	203
Overview of W-element Usage	203
W-element Capabilities	204
Using the W-element	205
Input Model 1: W-element, RLGC Model	206

Contents

Syntax	206
Sample Netlist	207
Simulation Results	208
Specifying the RLGC Model in an External File	210
Input Model 2: U-element, RLGC Model	212
Using RLGC Matrices	213
Sample Netlist	214
Input Model 3: Built-in Field-Solver Model	215
Input Model 4: Frequency-Dependent Tabular Model	215
Notation Used	216
Table Model Card Syntax	216
Examples: 4-Conductor Tx Line and RLGC Model List	217
Introducing Causality Check for W-element RLGC Table Model	219
Input Model 5: S Model	221
S-Model Conventions	222
S-Model Example	222
Input for W-element Passive Noise Model	223
Input Interface	223
Output Interface	224
Controlling W-element Accuracy	225
Setting Frequency Range of Interest for Greater Accuracy	225
Setting .OPTION RISETIME	225
Using DELAYOPT Keyword for Higher Frequency Ranges	226
Using DCACC Keyword for Lower Frequency Ranges	226
Using Time-Step Control	227
Static Time-Step Control	227
Dynamic Time-Step Control using WACC	227
Understanding Frequency-dependent Matrices and Transmission Line Theory	228
Equations and Parameters	229
Frequency-Dependent Matrices	230
Introducing the Complex Dielectric Loss Model	231
Fitting Procedure Triggered by INCLUDEGDIMAG Keyword	233
Example: Complex Dielectric Loss Model	234
Determining Matrix Properties	234
Using the PRINTZO Option	236
Defining Frequency-dependent Impedance in Mixed Mode Printing	237
File Description for *.wzo	240
Wave Propagation	241
Propagating a Voltage Step	242
Handling Line-to-Line Junctions	244
Overview of T-element Usage	246

Using T-element for Predicting Simple Tx Line Effects	246
T-element Capabilities	247
Invoking Ideal T-element Transmission Lines	247
Ideal Transmission Line Model	247
Selecting Ideal or Lossy Transmission Line Elements	249
Lossless Voltage and Current Propagation	253
References	255
7. Using the Field Solver to Extract Tx Line Parameters	257
Using the Field Solver Model	258
Filament Method	258
Modeling Geometries	260
Solver Limitation	260
Field-Solver-Related Netlist Statements	260
Field Solver Model Syntax	261
Using the Field Solver to Extract an RLGC Tabular Model	262
Syntax for Extracting RLGC Tabular Model	262
COMPUTE_TABLE/COMPUTETABLE Keyword	263
Default	263
RLGC Tabular Model Sample Output	264
Accounting for Surface Roughness Effect in W-element	265
Syntax for Scaling RS Matrix	266
Syntax for Taking RMS Surface Roughness of Conductor Materials	266
Syntax for Choosing Surface Roughness Method	266
Example: Transmission Line Surface Roughness	266
Accelerating the W-element Field Solver Using Multicore Technology	267
Visualizing Cross-Sectional Geometric Information	268
Field Solver Examples	269
Example 1: Cylindrical Conductor Above a Ground Plane	270
Example 2: Stratified Dielectric Media	272
Example 3: Two Traces Between Two Ground Planes	274
Example 4: Using Field Solver with Monte Carlo Analysis	275
Example 5: Modeling Coaxial and Shielded Twin-Lead Lines Using the Polar Field Solver	277
Coax Line	277
Shielded Twin-Lead Line	278
8. Using IBIS Files	280

Contents

Terminology	281
Verifying IBIS Files with the Golden Parser	282
Specifying Models for IBIS Buffers	282
Troubleshooting Signal Propagation Issues/Overclocking	283
Multiple Rising and Falling Waveforms Support	284
Differential Pins	284
Differential and Pseudo-Differential Buffers	286
Buffers in Subcircuits	287
Netlist Example with Output Buffer, Transmission Line, and Input Buffer	289
Using the IBIS Component Command	289
How .IBIS Creates Buffers	290
Required Keywords	291
file= <i>'file_name'</i>	291
component= <i>'component_name'</i>	292
Optional Keywords	292
package	292
pkgfile= <i>'pkg_file_name'</i>	293
[Model Selector] Support	293
Other Optional Keywords	293
Component Calls for SPICE or Verilog-A Formatted Pins	294
Component Calls for SPICE or Verilog-A Formatted [External Circuit]	295
Buffer Power	295
Buffer Power ON	295
Buffer Power OFF	299
Using IBIS Package Modeling	302
Syntax	302
Examples	303
Accessing Nets inside a Package Model	303
Using IBIS Board-Level Components	304
.EDB and .IBIS Command Syntax	305
Circuit Topology Created by the .EBD and .IBIS Commands	307
B-element Naming Rules	307
Circuit Topology Created with SPICE or Verilog-A Formatted Pins	310
SPICE or Verilog-A Formatted B-element Naming Rules	310
IBIS Board-Level Component Examples	312
Using IBIS Interconnect Modeling (ICM)	318

9. Using IBIS-AMI Equalizer Models with StatEye	319
IBIS-AMI Model Structure	320
Requirements for Using IBIS-AMI Models with PrimeSim HSPICE	321
Using the Standalone AMI Object Checking Program 'amicheck'	321
amicheck Flow	322
Windows OS 'amicheck' Setup	322
amicheck Output	323
How AMI Works in PrimeSim HSPICE	323
Input Syntax	324
Saving AMI Pulse/Step Responses and Filters	325
Visualizing Eye Diagram Output of an IBIS-AMI RX Model	325
Usage Example	326
StatEye Analysis with AMI Using Demo Example	326
Parametric Control over AMI File for Quoted Expressions	328
Backchannel Communication Interface for IBIS-AMI's Link Training Protocol	331
Input Syntax	331
Output	332
BIC_ID Naming Convention	332
Using an IBIS-AMI Mid-Channel Repeater Model	332
Mid-Channel Repeater Example	333
Results	334
References	334

Part 3: Feed-Forward Equalization with the PrimeSim HSPICE Tool

10. Introduction to Feed-Forward Equalization	336
FFE Modeling With Verilog-A	336
Built-in FFE Model Using Port Emphasis Parameters	337
E-element Delay Based Modeling	338

Appendices

A. Simulating Circuits with Signetics Drivers	340
--	-----

Simulating Circuits with Signetics Drivers—Examples	340
---	-----

B. Simulating Circuits with Xilinx FPGAs	344
Introduction to Simulating Circuits with Xilinx FPGAs	344
Syntax for IOB (xil_job) and IOB4 (xil_job4)	344
Ground-Bounce Simulation	345
Coupled Line Noise	348

Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

About the PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis

The *PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis* describes how to use the Synopsys PrimeSim HSPICE® tool to achieve and maintain signal integrity in your chip design.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the PrimeSim HSPICE Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim HSPICE simulator, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

PrimeWave™ Design Environment

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Part 1: Signal Integrity Analyses

Describes some of the factors and various analyses that can affect signal integrity in your design.

Part 1 focuses on signal integrity analyses and consists of the following chapters:

- [Chapter 1, Introduction to Signal Integrity](#)
- [Chapter 2, Linear Network Parameter Analysis](#)
- [Chapter 3, Statistical Eye Analysis](#)
- [Chapter 4, Time Domain Reflectometry](#)

1

Introduction to Signal Integrity

Describes some of the factors that can affect signal integrity in your design.

The performance of an IC design does not only depends on how many million transistors a vendor fits on a single chip. With tighter packaging space and increasing clock frequencies, packaging issues and system-level performance issues (such as crosstalk and transmission lines) become increasingly significant. At the same time, the popularity of multichip packages and increased I/O counts is forcing package design to become more like chip design.

This user guide consists of three parts and a set of appendices:

Part	Content
Part 1, Signal Integrity Analyses	Chapter 1, Introduction to Signal Integrity Chapter 2, Linear Network Parameter Analysis Chapter 3, Statistical Eye Analysis Chapter 4, Time Domain Reflectometry
Part 2, Signal Integrity Modeling	Chapter 5, S-parameter Modeling Using the S-element Chapter 6, W- and T-elements Transmission-Line Modeling Chapter 7, Using the Field Solver to Extract Tx Line Parameters Chapter 8, Using IBIS Files Chapter 9, Using IBIS-AMI Equalizer Models with StatEye
Part 3, Feed-Forward Equalization with the PrimeSim HSPICE Tool	Chapter 10, Introduction to Feed-Forward Equalization
Appendixes	Appendix A, Simulating Circuits with Signetics Drivers

Part	Content
	Appendix B, Simulating Circuits with Xilinx FPGAs

PrimeSim HSPICE ships hundreds of examples for your use. Find signal integrity-related demo files under Signal Integrity Examples, S-parameter Examples, IBIS Examples, Transmission (W-element) Line Examples, and Transmission Lines Examples, in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

To copy and paste proven syntax, use the demonstration files shipped with your installation of PrimeSim HSPICE (see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists*). Attempting to copy and paste from the manual or help documentation may present unexpected results as the text used in formatting may include hidden characters and white space for visual clarity.

Note:

The measurement system in this manual always refers to MKS units (meter, kilogram, second measurement), unless otherwise stated. PrimeSim HSPICE expects length and width units of meters. But PrimeSim HSPICE does directly support units of “mil” (0.001 inch, 25.4e-06 meters) as input. For example, you can specify a transmission line with a length of 0.4 inches:

```
T1 IN 0 OUT 0 Z0=50 f=1meg L=400mil
```

To get the results you expect, use caution when mixing units of measure. For reference, here are other “m” units. Do not confuse Mega, which you express as “meg” or “x”, with “m” (mili):

1m = 1e-3 (mili)

1meg = 1x = 1e6 (mega)

1u = 1e-6 (micro)

The following sections discuss these topics:

- [Getting Started on Signal Integrity Simulations](#)
- [Simulating Circuits with IBIS Models in PrimeSim HSPICE](#)

Getting Started on Signal Integrity Simulations

In a signal integrity simulation, you must model the following components:

- Driver cell including parasitic pin capacitances and package lead inductances
- Transmission lines
- A receiver cell with parasitic pin capacitances and package lead inductances
- Terminations or other electrical elements on the line

Model the transmission line as closely as possible—that is, to maintain the integrity of the simulation, include all electrical elements exactly as they appear on the backplane or printed circuit board.

You can use readily-available I/O drivers from ASIC vendors and the PrimeSim HSPICE device models advanced lossy transmission lines to simulate the electrical behavior of the board interconnect, bus, or backplane. You also can analyze the transmission line behavior under various conditions.

In addition, PrimeSim HSPICE preserves the necessary electrical characteristics with full transistor-level library circuits.

PrimeSim HSPICE can simulate systems by using:

- System-level behavior, such as local component temperature and independent models, to accurately predict electrical behavior.
- Automatic inclusion of library components by using the `SEARCH` option.
- Lossy transmission line models that:
 - Support common-mode simulation.
 - Include ground-plane reactance.
 - Include resistive loss of conductor and ground plane.
 - Allow multiple signal conductors.
 - Require minimum CPU computation time.

Signal Integrity Problems

[Table 1](#) lists some of the signal integrity problems that might cause failures in high-speed designs.

Table 1 *High-Speed Design Problems and Solutions*

Signal Integrity Problem	Causes	Solution
Noise: delta I (current)	Multiple simultaneously-switching drivers; high-speed devices create larger delta I	Adjust or evaluate location, size, and value of decoupling capacitors.
Noise: coupled (crosstalk)	Closely-spaced parallel traces	Establish design rules for lengths of parallel lines.
Noise: reflective	Impedance mismatch	Reduce the number of connectors and select proper impedance connectors.
Delay: path length	Poor placement and routing; too many or too few layers; chip pitch	Choose MCM or other high-density packaging technology.
Propagation speed	Dielectric medium	Choose the dielectric with the lowest dielectric constant.
Delay: rise time degradation	Resistive loss and impedance mismatch	Adjust width, thickness, and length of line.

The following sections discuss these topics:

- [Analog Side of Digital Logic](#)
- [System Design Issues](#)

Analog Side of Digital Logic

Circuit simulation of a digital system becomes necessary when the analog characteristics of the digital signals become electrically important. The integrity of the digital quality of the signals require careful circuit analysis.

The roadblocks to successful high-speed digital designs are noise and signal delays. Digital noise can originate from several sources. The fundamental digital noise sources are:

- Line termination noise - Additional voltage reflected from the load back to the driver caused by an impedance mismatch. Digital output buffers do not accurately control the output impedance. Most buffers have different rising and falling edge impedances.
- Ground bounce noise - Generated where leadframes or other circuit wires cannot form into transmission lines. The resulting inductance creates an induced voltage in the ground circuit, supply circuit, and output driver circuit. Ground bounce noise lowers the noise margins for the rest of the system.
- Coupled line noise - Induced from lines that are physically adjacent. This noise is generally more severe for data lines that are next to clock lines.

As system cycle times approach the speed of electromagnetic signal propagation for the printed circuit board, consideration of the line length becomes critical. The system noises and line delays interact with the electrical characteristics of the gates and might require circuit-level simulation.

System Design Issues

Exceeding the noise quota might not cause a system to fail. Maximum noise becomes a problem only when PrimeSim HSPICE accepts a digital input. If a digital system engineer can decouple the system, PrimeSim HSPICE can accept a much higher level of noise.

Some common methods that a digital system engineer can use to decouple a system include:

- Multiple ground and power planes on the printed circuit board (PCB), multichip module (MCM), and pin grid array (PGA)
- Separating signal traces with ground traces
- Decoupling capacitors
- Series resistors on output buffer drivers
- Twisted-pair line driving

In present systems designs, you must select the best packaging methods at three levels:

- PCB
- MCM
- PGA

Extra ground and power planes are often necessary to lower the supply inductance and to provide decoupling:

- Decoupling capacitors must have very low internal inductance to be effective for high-speed designs
- Newer designs frequently use series resistance in the output drivers to lower circuit ringing
- Critical high-speed driver applications use twisted differential-pair transmission lines

A system engineer must determine how to partition the logic. The propagation speed of signals on a printed circuit board is about 6 in/ns. As digital designs become faster, wiring interconnects become a factor in how you partition a logic.

The critical wiring systems are:

- IC-level wiring
- Package wiring for SIPs, DIPs, PGAs, and MCMs
- Printed circuit-board wiring
- Backplane and connector wiring
- Long lines—power, coax, or twisted pair

If you use ASIC or custom integrated circuits as part of your system logic partitioning strategy, you must make decisions about integrated circuit level wiring. The more-familiar decisions involve selecting packages and arranging packages on a printed circuit board. Large systems generally have a central backplane, which becomes the primary challenge at the system partition level.

Use the following equation to estimate wire length when transmission line effects become noticeable:

$$\text{critical length} = (\text{rise time}) * \text{velocity} / 8$$

For example, if rise time is 1 ns and board velocity is 6 in./ns, then distortion becomes noticeable when wire length is 3/4 in. The PrimeSim HSPICE circuit simulator contains a field solver to extract full loss transmission line models and the linear analysis (.LIN) feature can extract S-parameter models for packages or complex interconnects.

Simulating Circuits with IBIS Models in PrimeSim HSPICE

There are two ways to instantiate IBIS buffers in your PrimeSim HSPICE testbench. You can instantiate buffers one at a time with the B-element or you can instantiate all the buffers for a given component using the .IBIS component card. With the IBIS component, package parasitics can be annotated automatically. With the B-element, these must be added as PrimeSim HSPICE circuit elements if desired. For a complete discussion of the IBIS model components, see [Chapter 8, Using IBIS Files](#).

The following sections discuss these topics:

- [Using the B-element to Instantiate Individual Buffers](#)
- [Using the IBIS Component](#)
- [Power Supply of the IBIS Buffer](#)

Using the B-element to Instantiate Individual Buffers

If you decide to select and instantiate individual buffers using the B-element syntax, you create the node list and choose what to name the nodes. The list of required nodes for a buffer depends on the buffer type. IBIS models are supported according to the current PrimeSim HSPICE-supported IBIS standard. For a complete list of supported buffers, see Buffer Types in *PrimeSim Continuum User Guide: Elements*. For illustration, here are six common types:

- Input buffer:

```
B_INPUT nd_pc nd_gc nd_in nd_out_of_in
```

- Output buffer:

```
B_OUTPUT nd_pu nd_pd nd_out nd_in [nd_pc nd_gc]
```

- Input ECL Buffer:

```
B_INPUT_ECL nd_pc nd_gc nd_in nd_out_of_in
```

- Output ECL Buffer:

```
B_OUTPUT_ECL nd_pu nd_out nd_in [nd_pc nd_gc]
```

- Tri-state buffer:

```
B_3STATE nd_pu nd_pd nd_out nd_in nd_en [nd_pc nd_gc]
```

- Input/Output buffer:

```
B_IO nd_pu nd_pd nd_out nd_in nd_en nd_out_of_in [nd_pc nd_gc]
```

In the preceding examples, the required nodes are listed first and the optional nodes are in brackets ([]). The position in the list, not the name, selects the node function. You can name the nodes using any valid PrimeSim HSPICE node name. In these examples, *pu* and *pd* are pullup and pulldown; *pc* and *gc* are power clamp and ground clamp; *nd* simply stands for node.

The following is a sample B-element instantiation of an output (driver) buffer:

```
b_out1 nd_pu nd_pd out1 in1  
+ file = 'at16245.ibs'  
+ model = 'AT16245_OUT'
```

This is the minimum syntax to add a B-element. Note that both the file and model names are case-sensitive. There are numerous additional options, many of which are shared with the IBIS component.

For more details, see Specifying Required and Optional Common Keywords in *PrimeSim Continuum User Guide: Elements*.

Using the IBIS Component

To use the .IBIS component card, your IBIS file must have a [Component] section. When you use the IBIS component, PrimeSim HSPICE automatically creates node names for all the buffer pins in the component. These node names are created by concatenating the component name, the pin name, and the pin's function. For node naming, the component name is the user-supplied name given to the .IBIS card—not the component name.

For example, if the root of derived node name is

```
.IBIS ddr256
```

The output node of an output buffer (driver) might look like:

```
ddr256_q1_o
```

where `ddr256` is the component name, `q1` is the pin name, and `_o` denotes the driver output.

PrimeSim HSPICE adds underscore symbols (`_`) in between for readability. You do not have to create these nodes yourself in the netlist. They exist logically, and you can make connections to them simply by using the derived name.

Just as in the B-element node list, the nodes that are created for each buffer depend on the buffer type. For example, a tri-state buffer generates an enable node, where a simple input or output buffer does not.

An easy way to get started with the IBIS component is to create a skeleton netlist with only the IBIS component and a simple analysis. When you probe all signals, you see a list of all the known nodes for each buffer in your waveform tool.

Here is an example of the minimum syntax to instantiate an IBIS component and generate a plotfile:

```
* Instantiate an IBIS component to see the logical nodes created
.ibis ddr256
+ file = 'hyb256_400.ibs'
+ component = 'hyb25d256'
.tran 1n 5n
.option post probe dcstep=1
.probe v(ddr*)
.end
```

Since all logical nodes of the IBIS component start with `ddr256_`, using `.probe v(ddr256_*)` outputs all the nodes created by the IBIS component instantiation. The `dcstep` option prevents PrimeSim HSPICE from printing numerous warnings about nodes with no path to ground.

The following sections describe these topics:

- [Determining Output and Input Pins in the IBIS Component](#)
- [Power and Ground Pins in the IBIS Component](#)
- [Package and Pin Parasitics](#)
- [Interconnect Model](#)

Determining Output and Input Pins in the IBIS Component

How you connect the nodes of a buffer instantiated by an IBIS component depends on if it is an input, output or I/O function. There is a special pin for I/O buffers named “outofin”. When an IO buffer is in receive mode, the input is external to the component and sits on your SI path. The output of that input buffer (or “outofin”) is inside the component. If you probe the outofin node, you see it is an ideal, behavioral digital waveform. The regular input buffer does not have a separate “outofin” node. Instead, the “_i” node acts as outofin.

Buffer Type	Derived Logical Node Name	Example
Input buffers - receivers		
The input (receiving) node of an input buffer:	'cname'_'pin_name'	ddr256_q1
The output node of an input buffer:	cname'_'pin_name'_'i	ddr256_q1_i
Output buffers - drivers		
Input node of an output buffer:	'cname'_'pin_name'_'i	ddr256_q1_i
Output node of an output buffer:	'cname'_'pin_name'	ddr256_q1
Die side output node, if package model used:	'cname'_'pin_name'_'o	ddr256_q1_o
IO buffers - input/output		
Input of an IO buffer in input mode:	'cname'_'pin_name'	ddr256_q1
Output of an IO buffer in input mode:	'cname'_'pin_name'_'outofin	ddr256_q1_outofin
Input of an IO buffer in output mode:	'cname'_'pin_name'_'i	ddr256_q1_i
Output node of an IO buffer in output mode:	'cname'_'pin_name'	ddr256_q1
Die side output node of an IO buffer in output mode, if package model used:	'cname'_'pin_name'_'o	ddr256_q1_o

Buffer Node Names	General Guidelines
Die side output node, if package model used:	'cname'_'pin_name'_'o

Buffer Node Names	General Guidelines
Output node or input node for input buffers:	'cname'_pin_name'
Enable node for buffers with enable function:	'cname'_pin_name'_en
Input node or outofin node for input buffers:	'cname'_pin_name'_i
Outofin node for buffers other than input buffers:	'cname'_pin_name'_outofin

Power and Ground Pins in the IBIS Component

You may wish to externally power your IBIS buffer in certain situations, such as to simulate Simultaneous Switching Noise, or “ground bounce”. Nodes of power and ground pins are created in the “logical” netlist but no buffer is created for them. The pins can also have specific pin or general package parasitics associated with them such as the other IO pins in the component.

Package and Pin Parasitics

The IBIS component gives you the option to automatically annotate the general R_pkg, L_pkg and C_pkg parasitics in the [Package] section, or the pin specific R_pin, L_pin and C_pin parasitics in the [Pin] section, or the complicated package model declared in the [Package Model] section. Please refer the keyword `package` of the `.IBIS` command.

Interconnect Model

IBIS 7.0 supports broadband interconnect models describing connections between the pins of a component, die pads, and its I/O buffers. The interconnect models define the interconnections to either IBIS-ISS subcircuits or n-port networks described by Touchstone files. PrimeSim HSPICE automatically creates X and S elements with proper connection based on the data in the IBIS file.

Note:

If IBIS-ISS subcircuits are used in the component, then the files containing the subcircuits should be explicitly included using the `.INC` command in the PrimeSim HSPICE netlist.

For an IBIS file with version 7.0, if interconnect models are used in an IBIS component, then any package data in ([Pin], [Package], or [Package Model]) are ignored. For backward compatibility, you can use the `package` keyword of the `.IBIS` command. For more information, see `.IBIS` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Power Supply of the IBIS Buffer

For the B-element: The physical pull-up and pull-down nodes are, by default, connected to supply and ground by the *power=on* directive. If you physically connect these nodes to an external voltage source and ground, be sure to set *power=off* in your netlist.

For the IBIS component logical netlist: If there is [Pin Mapping] for this component in the IBIS file, then physical pull-up and pull-down nodes are connected automatically to power and ground pins of the component. You should externally power your buffer from such pins manually. So *power=off* is the default in this situation.

If there is no [Pin Mapping] for this component in the IBIS file, then, like the B-element card, the physical pull-up and pull-down nodes are, by default, connected to the supply and ground by the *power=on* directive. You need not add an external voltage source for them.

2

Linear Network Parameter Analysis

Describes how to perform an AC sweep to extract small-signal linear network parameters in PrimeSim HSPICE and advanced analog simulations.

The chapter covers LIN analysis, RF measurements from `.LIN` command, extracting mixed-mode S (scattering) and noise parameters, and additional measurements.

PrimeSim HSPICE ships hundreds of examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

For descriptions of individual commands referenced in this chapter, see the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

These topics are covered in the following sections:

- [LIN Analysis](#)
- [Additional Measurements From .LIN Command](#)
- [Example - Extracting Bandpass Filter S-parameters](#)
- [Extracting Mixed-Mode Scattering \(S\) Parameters](#)
- [Tutorial Example: LIN Analysis for a NMOS Low Noise Amplifier](#)
- [References](#)

LIN Analysis

The `.LIN` command extracts noise and linear transfer parameters for a general multi-port network.

When used with the `.AC` command, `.LIN` makes available a broad set of linear port-wise measurements:

- Multi-port scattering [S] parameters
- Noise parameters

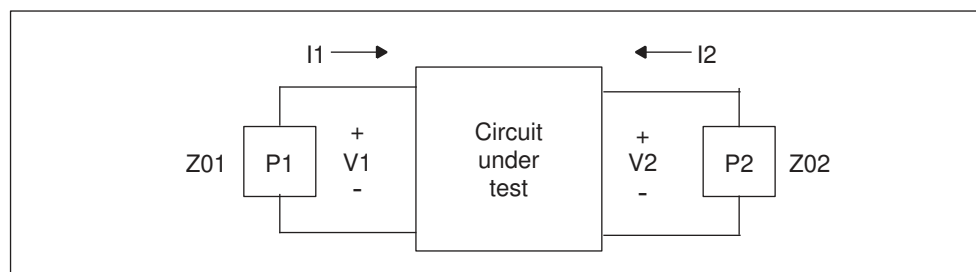
- Stability factors
- Gain factors
- Matching coefficients

The LIN analysis is similar to basic small-signal, swept-frequency `.AC` analysis, but it also automatically calculates a series of noise and small-signal transfer parameters between the terminals identified using port (P) elements.

PrimeSim HSPICE can output the result of group delay extraction and two-port noise analysis to either a `*.sc` file, a Touchstone 1.0/2.0 file, or a CITI file.

The `.PRINT/.PROBE/.MEAS` output syntax for `.LIN` supports S/Y/Z/H (hybrid) parameters, group delay, noise parameters, stability factors, gain factors, and matching coefficients.

Figure 1 Basic circuit in LIN analysis



The following sections discuss these topics:

- [Identifying Ports with the P-element](#)
- [Using the P-element for Mixed-Mode Measurement](#)
- [.LIN Input Syntax](#)
- [.LIN Output Syntax](#)
- [Multi-Port Scattering \(S\) Parameters](#)
- [Two-Port Transfer and Noise Calculations](#)
- [Noise Parameters in 2-Port and N-Port Networks](#)
- [Hybrid \(H\) Parameters](#)
- [Group Delay](#)
- [Summary of Printing and Probing Network Parameters](#)

Identifying Ports with the P-element

The `.LIN` command computes the S- (scattering), Y- (admittance), Z- (impedance), and H- (hybrid) parameters directly based on the location of the port (P) elements in your circuit, and the specified values for their reference impedances.

The port element identifies the ports used in LIN analysis.

Syntax

```
Pxxx node1 node2 [node3 node4 node5] port=portindex
+ [Z0=val]                                $Port Impedance
+ [DC mag] [AC mag phase] [HBAC mag phase] $Voltage/Power Info
+ [HB mag phase harm tone modharm modtone]
+ [transient waveform]
+ [ENCODE=DW8B10B [RD_INIT=0|1]]
+ [ENCODE=DW64b66b [TXC=txcdata]]
+ [ENCODE=DW128b130b|DW128b132b [HEAD=DATA|]] CONTROL
+ [TRANFORHB=[0|1]] [DCOPEN=[0|1]]
+ [power=[0|1|2|W|dbm]]                    $Power Switch
+ [RDC=val] [RAC=val]                      $ Source Impedance Overrides
+ [RHBAC=val] [RHB=val] [RTRAN=val]
+ [Emphasis_Level=val] [Emphasis_Time=val]  $ Emphasis
+ $ **** DFE ****
+ [DFE_tap=n1 n2 ... nN DFE_coeff=val1, val2, ... valN]

+ [DFE_TAP_OPT=N]
+ [DFE_VTH=val]
+ [DFE_CLK=0|1]
+ [DCD=val] [DCD_TYPE=0|1|2|3]             $ Duty Cycle Distortion
+ [PJ=val] [PJ_TYPE=0|1|2|3|4|5|6]         $ Period Jitter
+ [RJ=val1, [val2, ... valN]] [VN=val] $ Rand Jitter Stateye
+ [AMI_OBJ=filename] [AMI_PARAM=filename] [AMI_CDR=0|1] $AMI
+ [AMI_CORNER=idx] $AMI
+ [MODULATION=PAM4]
+ [PAM4_MAPPING=string]
+ [IBIS_AMI_FILE=filename IBIS_AMI_MODEL=modelname]
+ [AMI_PARAM_xx = string|quoted string]
+ [IBIS_AMI_USE_CCOMP = 0|1]
+ [IBIS_AMI_CCOMP_ORDER = 1|2]
+ [IBIS_AMI_VRANGE_MODE = 1|2]
+ [IBIS_AMI_USE_RAMP = 0|1]
+ [IBIS_AMI_USE_VRANGE = 0|1]
+ [IBIS_AMI_USE_ZSRC = 0|1]
```

Parameter	Description
<code>node2 [node3 node4 node5]</code>	The port element can have either two, or three, or five terminals. The three-terminal model can be used for differential ports (see Using the Port Element for Mixed-Mode Measurement). You can also add a five-terminal model (see Using the Five Terminal Port Element) mainly to take a four-port S-parameter block for the IBIS-AMI's "Ts4file Analog Buffer Models" (BIRD 158).
<code>port=portindex</code>	The port index. Numbered sequentially beginning with 1 with no shared port indexes.
<code>z0=val (or Zo=val)</code>	Port impedance (Ohms). (Default: 50). Sets port characteristic impedance used for LIN analysis, sets port termination impedance for other analyses, and also sets source impedance when you use the port element as a signal source.
<code>DC mag</code>	DC voltage or power source value.
<code>AC mag phase</code>	AC voltage or power source value.
<code>HBAC mag phase</code>	(PrimeSim HSPICE advanced analog analyses) HBAC voltage or power source value.
<code>HB mag phase harm tone modharm modtone</code>	(PrimeSim HSPICE advanced analog analyses) HB voltage, current, or power source value. Allows multiple HB specifications with different <code>harm</code>, <code>tone</code>, <code>modharm</code>, and <code>modtone</code> values. <code>phase</code> is in degrees. <code>harm</code> and <code>tone</code> are indices corresponding to the tones specified in the <code>.HB</code> statement. Indexing starts at 1 (corresponding to the first harmonic of a <code>tone</code>). <code>modtone</code> and <code>modharm</code> specify sources for multi-tone simulation. A source specifies a tone and a harmonic, and up to 1 offset tone and harmonic (<code>modtone</code> for tones and <code>modharm</code> for harmonics). Then it describes the signal as: $V \text{ (or } I) = \text{mag} * \cos(2 * \pi * (\text{harm} * \text{tone} + \text{modharm} * \text{modtone}) * t + \text{phase})$
<code>transient_waveform</code>	(Transient analysis) Voltage or power source waveform. Any one of waveforms: AM, EXP, PULSE, PWL, SFFM, SIN, PAT, or PRBS. Does <i>not</i> allow multiple transient descriptions.
<code>ENCODE=DW8b10b</code>	Keyword to specify 8b/10b encoding.
<code>RD_INIT=0 1</code>	Initial value of Running Disparity. <i>Running Disparity.1</i> is the name of the one-bit memory that recalls the bias of the last unbalanced code word. 0: Specifies that a Running Disparity value of zero is synonymous with negative Running Disparity (-). 1: Specifies that a Running Disparity value of one is synonymous with positive Running Disparity (+).
<code>ENCODE=DW64b66b</code>	Keyword to specify 64b/66b encoding. Note: <i>DW64b66b</i> encoding is valid only for the PAT <code>transient_waveform</code> type.

Parameter	Description
<code>TXC=txcdata</code>	The input 64-bit pattern signal will be divided into 8 blocks with 8 bits each; <code>txcdata</code> is an 8-bit string with only 0 and 1s. It indicates the block type (data block or control block) for each 8-bit block. 0: Specifies that the corresponding input block is a data block. 1: Specifies that the corresponding input block is a control block. For example: <code>TXC=10000000</code> indicates that the first block in the input pattern signal (bits 1 to 8) is a control block, and blocks 2, 3, 4, 5, 6, 7, and 8 (that is, 9-16, 17-24, 25-32, 33-40, 41-48, 49-56, and 57-64 bits respectively) are data blocks.
<code>ENCODE=DW128b130b DW128b132b</code>	Keyword to specify 128b/130b or 128b/132b encoding. Note: <code>DW128b130b</code> and <code>DW128b132b</code> encoding are valid only for the <code>PAT transient_waveform</code> type.
<code>HEAD = DATA CONTROL</code>	Indicates whether the pattern input signal is a data signal or control signal. Default is <code>DATA</code>. <code>DATA</code>: Specifies that the 128-bit pattern input signal is a DATA signal. <code>CONTROL</code>: Specifies that the 128-bit pattern input signal is a CONTROL signal.
<code>TRANFORHB=[0 1]</code>	PrimeSim HSPICE advanced analog analyses 0 (default): Ignores the transient description if you specify an HB value or a DC value. If you do not provide a DC or HB value and <code>TRANFORHB=0</code>, then HB analysis treats the source as a DC source, and the DC source value is the <code>time=0</code> value. 1: HB analysis uses the transient description if its value is VMRF, SIN, PULSE, PWL, or LFSR. If the type is a non-repeating PWL source, the simulation treats the <code>time=infinity</code> value as a DC analysis source value. For example, the simulation treats the following statement as a DC source with <code>value=1</code> for HB analysis: <code>v1 1 0 PWL (0 0 1n 1 1u 1) + TRANFORHB=1</code> In contrast, the following statement is a 0V DC source: <code>v1 1 0 PWL (0 0 1n 1 1u 1) + TRANFORHB=0</code> The simulation treats the following statement as a periodic source with a <code>1u</code> period that uses PWL values: <code>v1 1 0 PWL (0 0 1n 1 0.999u 1 1u 0) R + TRANFORHB=1</code> To override the global <code>TRANFORHB</code> option, explicitly set <code>TRANFORHB</code> for a voltage or current source.
<code>DCOPEN</code>	Switch for open DC connection if you have not set <code>DC mag</code>. 0 (default): P-element behaves as an impedance termination 1: P-element behaves as an open circuit in DC operating point analysis. LIN analysis mainly uses <code>DCOPEN=1</code> so the P-element does not affect the self-biasing device under test by opening the termination at the operating point.
<code>RDC=val</code>	(DC analysis) Series resistance (overrides <code>z0</code>).
<code>RAC=val</code>	(AC analysis) Series resistance (overrides <code>z0</code>).

Parameter	Description
RHBAC= <i>val</i>	(PrimeSim HSPICE advanced analog HBAC analysis) Series resistance (overrides <i>z0</i>).
RHB= <i>val</i>	(PrimeSim HSPICE advanced analog HB analysis) Series resistance (overrides <i>z0</i>).
RTRAN= <i>val</i>	(Transient analysis) Series resistance (overrides <i>z0</i>).
power=[0 1 2 W dbm]	Power switch. Causes treatment of the signal amplitude quantities specified with the port element to as power levels instead of voltage levels. The port element then becomes a power source, realized as a voltage source in series with the port impedance (<i>z0</i>). The simulation calculates the voltage source value internally as that is necessary to realize the appropriate RMS available power. For transient analysis, this is only appropriate for DC and SIN sources. • When 0 (default), power entry disabled. • When 1 or W, power given in units of Watts. • When 2 or dBm, power given in units of dBm (dB relative to 1 mW).
Emphasis_Level	Aids in .STATEYE analysis pre-emphasis and de-emphasis.
Emphasis_Time	Aids in .STATEYE analysis pre-emphasis and de-emphasis.
DFE_tap	Integer number(s) to specify relative feedback tap location.
DFE_coeff	Real number(s) to specify corresponding feedback coefficient. Note: If DFE_tap is specified, DFE_coeff is required.
DFE_TAP_OPT	Integer number to specify the number of DFE taps or coefficients. If DFE_TAP_OPT is specified and the following analysis conditions are met, the tool automatically extracts the optimized DFE coefficients and applies them to the DFE operation: • The STATEYE MODE keyword is set to EDGE or STAT. • The P-element MODULATION=PAM4 keyword is not set. Note: • The tool ignores this keyword and prints a warning message in the following situations: ◦ If you specify the number of DFE taps as less than or equal to 0 ◦ If the analysis conditions are not met (see the preceding list) • If DFE_TAP_OPT is specified, the tool ignores the DFE_tap and DFE_coeff keywords.
DFE_vth	A real number to specify logical threshold of the system. Default is 0.
DFE_CLK	Works with CLOCK_PORT and CLOCK_MODE=2 keywords in .STATEYE command. When DFE_CLK=1, StatEye passes clock ticks at CLOCK_PORT to the built-in DFE model which will use the given clock ticks in slicer operation. The default is 0. See also Applying External Clock to the Built-In DFE Model .

Parameter	Description
Note: For more information on DFE function, see Built-In DFE Function for Rx Port Element in StatEye Analysis .	
DCD	Aids in <code>.STATEYE</code> analysis; specifies peak percentage of the duty cycle distortion (DCD). Default is 0.
DCD_TYPE	Aids in <code>.STATEYE</code> analysis; specifies variation type. Default type for non-zero DCD is 1 (constant). • 0: No DCD • 1: Constant DCD • 2: Uncorrelated triangular DCD variation • 3: Uncorrelated sinusoidal DCD variation
PJ	Aids in <code>.STATEYE</code> analysis; specifies periodic jitter (voltage) magnitude. Default is 0.
PJ_TYPE	Aids in <code>.STATEYE</code> analysis. Specifies an array of integer numbers for periodic jitter variation type. Default type for nonzero PJ is 3 (uncorrelated sinusoidal voltage variation): • 0: No periodic jitter • 1: Constant voltage shift • 2: Uncorrelated triangular voltage variation • 3: Uncorrelated sinusoidal voltage variation • 4: Constant time shift • 5: Uncorrelated triangular time variation • 6: Uncorrelated sinusoidal time variation Note: • Triangular assumes random change between 0 and specified value with equal probability. • Sinusoidal assumes the probability of the jitter amount has sinusoidal distribution. • Types 4, 5, and 6 are deterministic jitters.
RJ	An array of the real numbers to specify the standard deviation of the Gaussian random jitter. The array must be in the order of the port element index. By default, the simulation does not add random jitter.
VN	(Voltage noise) Aids in <code>.STATEYE</code> analysis. Specify the standard deviation of the Gaussian random voltage noise. By default, the simulation does not add random voltage noise.
AMI_OBJ	Aids in <code>.STATEYE</code> analysis; specifies an Algorithmic Modeling Interface (AMI) shared object (typically, <code>lib*.so</code> for UNIX, and <code>*.dll</code> for Windows)
AMI_PARAM	Aids in <code>.STATEYE</code> analysis; specifies an AMI parameter file (*.ami)
AMI_CDR	Aids in <code>.STATEYE</code> analysis. Switch to use <code>AMI_GetWave</code> 's clock data recovery (CDR) output in eye diagram generation. Default is 0 (disabled state).

Parameter	Description
AMI_CORNER	Specifies an index number. The IBIS keyword "... (Corner val1 val2 ...)" defines the corner values in the IBIS-AMI file specified by AMI_PARAM. When you use AMI_CORNER, <i>idx</i> is used to choose the <i>idx</i>th value defined by the "... (Corner ...)" keyword. For example, if AMI_CORNER = 2 is specified in a port element and the associated AMI parameter file has the following: <pre>(My_AMI_param1 ... (Corner 1 2 3)) (My_AMI_param2 ... (Corner 4 5 6)) (My_AMI_param3 ... (Corner 7 8 9))</pre> Then, 2, 5 and 8 are chosen for My_AMI_param{1,2 and 3} respectively.
MODULATION=PAM4	Activates pulse amplitude modulation (PAM) 4 signaling when the source has bit pattern (LFSR/PAT) input. By default, the source element uses the conventional two logic level signaling.
PAM4_MAPPING= <i>string</i>	Specifies mapping from a pair of bits to four-level symbols. PAM4_Mapping takes a four-character string argument, a combination of '0' '1' '2' and '3' which represent pairs of binary symbols "00", "01", "10", and "11" respectively. The first, second, third, and fourth character positions in the argument string correspond to v0 (lowest), v1, v2, and v3 (highest) voltage levels, respectively. For example, PAM4_MAPPING = "0132" maps "00", "01", "11", and "10" to v0, v1, v2 and v3, respectively. Default value for PAM4_MAPPING (when MODULATION=PAM4) is "0132".
IBIS_AMI_FILE= <i>filename</i> IBIS_AMI_MODEL= <i>modelname</i>	Arguments for direct IBIS (*.ibs) file call. PrimeSim HSPICE retrieves the following information based on the specified pair of file and model names: • AMI object and parameter file names for the appropriate computer platform (Linux or Windows) • Parasitic information such as C_comp • Input source information such as source impedance, rise/fall time, and voltage range • AMI_CORNER=<i>n</i> argument which currently is used to specify the corner index of the AMI parameter (*.ami) file. This corner index will also be applied in retrieving these quantities from the IBIS (*.ibs) file. A port element applies these parasitic and input source properties in all the PrimeSim HSPICE simulations. AMI information is used in the StatEye simulations.

Parameter	Description
<code>AMI_PARAM_xx = string quoted string</code>	Argument for direct AMI parameter control from the PrimeSim HSPICE netlist. - PrimeSim HSPICE extracts "xx" as the target name, then assigns the argument string when the name is found in the AMI parameter (*.ami) file. For example, when <code>AMI_PARAM_Ignore_Bits=1000</code> is specified, PrimeSim HSPICE assigns 1000 to <code>Ignore_Bits</code> in the AMI parameter file. - When the argument value is quoted using single quotation character, PrimeSim HSPICE first evaluates the string as an expression and assigns the result to the AMI parameter file. - You can have multiple declarations of this argument. - Since the keywords in both the PrimeSim HSPICE netlist and the AMI parameter file are case insensitive, parameter name matching will be carried out in a case-insensitive manner. However, PrimeSim HSPICE preserves the case of the given argument in the assignment.
<code>IBIS_AMI_USE_CCOMP = 0 1</code>	Specifies whether or not to use <code>Ccomp</code> value in the *.ibs file for port element's load capacitance in IBIS-AMI simulation. Default is 1.
<code>IBIS_AMI_CCOMP_ORDER = 1 2</code>	Selects the <code>AMI_CORNER</code> indexing scheme for <code>C_comp</code> when <code>C_comp</code> value is selected up from *.ibs file for IBIS-AMI simulation.
<code>IBIS_AMI_VRANGE_MODE = 1 2</code>	Specifies how to use the <code>VRANGE</code> value in the *.ibs file in IBIS-AMI simulation. 1 (default): Uses <code>VRANGE</code> as differential voltage. The incident source will have (-<code>VRANGE</code> to <code>VRANGE</code>) voltage swing. 2: Uses <code>VRANGE</code> as peak-to-peak voltage. The incident source will have (-0.5*<code>VRANGE</code> to 0.5*<code>VRANGE</code>) voltage swing.
<code>IBIS_AMI_USE_RAMP = 0 1</code>	Specifies whether or not to use the <code>RAMP</code> information in the *.ibs file for incident's rise/fall time in IBIS-AMI simulation. Default is 1.
<code>IBIS_AMI_USE_VRANGE = 0 1</code>	Specifies whether or not to use the <code>VRANGE</code> information in the *.ibs file for incident's voltage swing magnitude in IBIS-AMI simulation. Default is 1.
<code>IBIS_AMI_USE_ZSRC = 0 1</code>	Specifies whether or not to use linearized pull-up/pull-down information in the *.ibs file for port element's impedance in IBIS-AMI simulation. Default is 1.

A full description of the port element and its syntax is found in Port Element in the *PrimeSim Continuum User Guide: Elements*.

Using a Hierarchy Delimiter to Narrow the AMI Parameter Assignment

When using IBIS-AMI models with AMI parameter file (*.ami), which contains multiple parameters with the same leaf names under the different hierarchy such as:

```
(EQ1
    (Mode (Usage In) (Type Integer) (List 2 0 1) (Default 2)
        (Description "Mode for equalizer 1 : 0=Off, 1=Fixed, 2=Adapt")
    )
)
(EQ2
    (Mode (Usage In) (Type Integer) (List 2 0 1) (Default 2)
        (Description "Mode for equalizer 2 : 0=Off, 1=Fixed, 2=Adapt")
    )
)
```

The `AMI_PARAM_xx` keyword with parameter name “xx” assigns the value to all the parameters with that name. For example,

```
P1 ..
+ AMI_PARAM_Mode = 1
```

results in the AMI parameter string as follows:

```
(EQ1 (Mode 1)) (EQ2 (Mode 1))
```

To selectively assign the parameter value, use the hierarchy delimiter ‘/’ in the `AMI_PARAM_xx` keyword so as to separately identify the same leaf name, *Mode* from different hierarchy, *EQ1* and *EQ2*. For example:

```
P1 ..
+ AMI_PARAM_EQ1/MODE = 1
results in
(EQ1 (Mode 1)) (EQ2 (Mode 2))
```

```
P1 ..
+ AMI_PARAM_EQ2/MODE = 1
results in
(EQ1 (Mode 2)) (EQ2 (Mode 1))
```

Note:

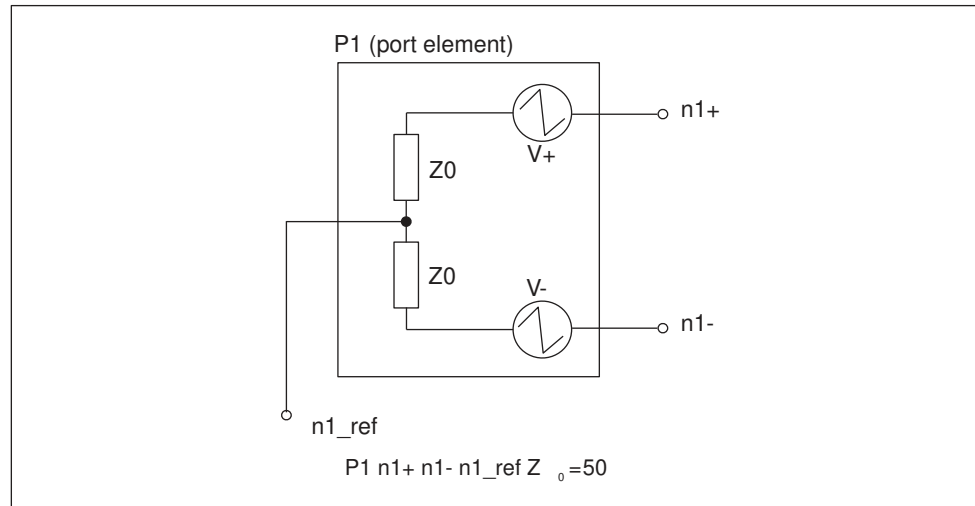
In this example, if no assignment is assigned from port element, then default value 2 is used.

Using the P-element for Mixed-Mode Measurement

You can use a port element with three terminals as the port element for measuring the mixed mode S-parameters. Except for the number of external terminals, the syntax of the

port element remains the same. The LIN analysis function internally sets the necessary drive mode (common/differential) of these mixed mode port elements. For analyses other than the LIN analysis (such as DC, AC, TRAN, and so on), the mixed-mode P-element acts as a differential driver that drives positive nodes with half of their specified voltage and the negative nodes with a negated half of the specified voltage. [Figure 2](#) shows the block diagram of the mixed mode port element.

Figure 2 Mixed mode port element



The port element can also be used as a signal source with a built-in reference impedance. For further information on its use as a signal source, see *Sources and Stimuli* in the *PrimeSim Continuum User Guide: Elements*.

.LIN Input Syntax

```
.LIN [sparcalc=1|0 [type=s|y] [modelname = modelname]]
+ [filename = filename]
+ [format=selem|citi|touchstone | touchstone2]
+ [noisecalc=1|0] [gdcalc=1|0]
+ [mixedmode2port=dd|dc|ds|cd|cc|cs|sd|sc|ss]
+ [dataformat=ri|ma|db] [FREQDIGIT=x] [SPARDIGIT=x]
+ [listfreq=(frequencies|none|all|freq1 freq2...)]
+ [listcount=num] [listfloor=val] [listsources=1|0|yes|no]
+ COMMENT_FILE={_}filename{_]
+ [PORTS = P1[, P2 ... Pn]]
```

The `FREQDIGIT` and `SPARDIGIT` keyword values allow you to extend the precision number of digits for Touchstone 1.0/2.0, CITI, and `.sc#` files. The default is 6 for both keywords.

For argument descriptions, see the `.LIN` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

.LIN Output Syntax

This section describes the syntax for the `.PRINT` and `.PROBE` statements used for LIN analysis.

The following sections discuss these topics:

- [.PRINT and .PROBE Statements](#)
- [Examples](#)
- [Hybrid Parameter Calculations](#)

.PRINT and .PROBE Statements

```
.PRINT AC Xmn | Xmn (TYPE) | X(m,n) | X(m,n) (TYPE) | Hmn |  
+ Hmn (TYPE) | H(m,n) | H(m,n) (TYPE) | LINPARAM |  
+ LINPARAM (TYPE)  
.PROBE AC Xmn | Xmn (TYPE) | X(m,n) | X(m,n) (TYPE) | Hmn |  
+ Hmn (TYPE) | H(m,n) | H(m,n) (TYPE) | LINPARAM |  
+ LINPARAM (TYPE)
```

Argument	Description
Xmn, X(m,n)	One of these parameter types: • S (scattering parameters) • Y (admittance parameters) • Z (impedance parameters) mn refers to a pair of port numbers.
Hmn, H(m,n)	Complex hybrid (H-) parameters. mn refers to a pair of port numbers, where m and n are positive integer numbers. If m, n=0 or m, n>2, PrimeSim HSPICE issues a warning and ignores the output request. • To calculate a one-port H parameter, you must specify at least one port (P) element. • To calculate a two-port H parameter, you must specify two or more port (P) elements. For additional information, see Hybrid Parameter Calculations .

Argument	Description
LINPARAM	Two-port noise parameters: • NFMIN (noise figure minimum) • NF (Noise figure) • VN2 (Equivalent input noise voltage squared) • IN2 (Equivalent input noise current squared) • RHON (Correlation coefficient between input noise voltage and input noise current) • RN (Noise equivalent resistance) • GN (Noise equivalent conductance) • ZCOR (Noise correlation impedance) • YCOR (Noise correlation admittance) • ZOPT (Optimum source impedance for noise) • YOPT (Optimum source admittance for noise) • GAMMA_OPT (source reflection coefficient that achieves the minimum noise figure) • ZOPT (source impedance that achieves minimum noise) • RN (noise equivalent resistance) • K_STABILITY_FACTOR (Rollett stability factor) • MU_STABILITY_FACTOR (Edwards & Sinsky stability factor) • G_MAX (maximum available/operating power gain) • G_MSG (Maximum stable gain) • G_TUMAX (Maximum unilateral transducer power gain) • G_U (Unilateral power gain) • G_MAX_GAMMA1 (source reflection coefficient that achieves maximum available power gain) • G_MAX_GAMMA2 (load reflection coefficient that achieves maximum operating power gain) • G_MAX_Z1=Source impedance needed to realize G_MAX (complex, Ohms) • G_MAX_Z2=Load impedance needed to realize G_MAX (complex, Ohms) • G_MAX_Y1=Source admittance needed to realize G_MAX (complex, Siemens) • G_MAX_Y2=Load admittance needed to realize G_MAX (complex, Siemens) • G_AS (associate gain—maximum gain at the minimum noise figure) • VSWR(n) (voltage standing-wave ratio at the n port) • G_MSG (maximum stable gain) • G_TUMAX (maximum unilateral transducer power gain) • G_U (unilateral power gain)
TYPE	Data type definitions: • R=Real • I=Imaginary • M=Magnitude • P=PD=Phase in degrees • PR=Phase in radians • DB=decibels

Examples

```
.print AC S11 S21(DB) S(2,3)(D) S(2,1)(I)
.print AC NFMIN GAMMA_OPT G_AS
.probe AC RN G_MAX ZOPT Y(3,1)(M) Y31(P)
```

Hybrid Parameter Calculations

The hybrid parameters are transformed from S-parameters:

- For a one-port circuit, the calculation is:

Equation 1

$$H_{11} = Z_{01} \frac{(1 + S_{11})}{(1 - S_{11})}$$

- For a two-port circuit, the calculation is:

Equation 2

$$H_{11} = Z_{01} \frac{(1 + S_{11})(1 + S_{22}) - S_{12}S_{21}}{(1 - S_{11})(1 + S_{22}) + S_{12}S_{21}}$$

Equation 3

$$H_{12} = \sqrt{\frac{Z_{02}}{Z_{01}}} \frac{2S_{12}}{(1 - S_{11})(1 + S_{22}) + S_{12}S_{21}}$$

Equation 4

$$H_{21} = \sqrt{\frac{Z_{02}}{Z_{01}}} \frac{-S_{21}}{(1 - S_{11})(1 + S_{22}) + S_{12}S_{21}}$$

Equation 5

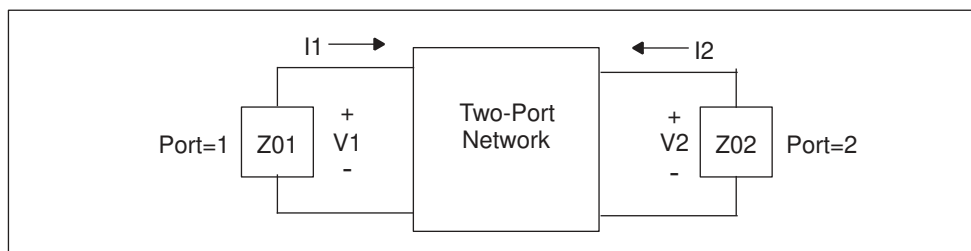
$$H_{22} = \frac{1}{Z_{02}} \frac{(1 - S_{11})(1 - S_{22}) - S_{12}S_{21}}{(1 - S_{11})(1 + S_{22}) + S_{12}S_{21}}$$

For networks with more than two ports when computing the 1,2 H index parameters, PrimeSim HSPICE assumes that ports numbered 3 and above terminate in their port reference impedance (z_0). The above two-port calculations therefore remain appropriate because S_{11} , S_{12} , S_{21} , and S_{22} remain valid, and simulation can ignore higher order S-parameters.

Multi-Port Scattering (S) Parameters

S-parameters represent the ratio of incident and scattered (or forward and reflected) normalized voltage waves. [Figure 3](#) shows a two-port network.

Figure 3 Two-port network



The following equations define the incident (forward) waves for this two-port network:

$$a_1 = \frac{v_1 + Z_{01}I_1}{2 \cdot \sqrt{Z_{01}}} \quad a_2 = \frac{v_2 + Z_{02}I_2}{2 \cdot \sqrt{Z_{02}}}$$

The following equations define the scattered (reflected) waves for this two-port network:

$$b_1 = \frac{v_1 - Z_{01}I_1}{2 \cdot \sqrt{Z_{01}}} \quad b_2 = \frac{v_2 - Z_{02}I_2}{2 \cdot \sqrt{Z_{02}}}$$

The following equations define the S-parameters:

$$S_{11} = \left. \frac{b_1}{a_1} \right|_{a_2=0} \quad S_{12} = \left. \frac{b_1}{a_2} \right|_{a_1=0}$$

$$S_{21} = \left. \frac{b_2}{a_1} \right|_{a_2=0} \quad S_{22} = \left. \frac{b_2}{a_2} \right|_{a_1=0}$$

Each S-parameter is a complex number, which can represent gain, isolation, or a reflection coefficient.

Example

The following examples show how you can represent a gain, isolation, or reflection coefficient:

```
.PRINT AC S11(DB) $ Input return loss
.PRINT AC S21(DB) $ Gain
.PRINT AC S12(DB) $ Isolation
.PRINT AC S22(DB) $ Output return loss
```

Two-Port Transfer and Noise Calculations

Two-port noise analysis is a linear AC noise analysis method that determines the noise figure of a linear two-port for an arbitrary source impedance.

Several output parameter measurements are specific to two-port networks. The LIN analysis supports two-port calculations for 3 or more ports if port=1 is the input and port=2 the output. All other ports terminate in their characteristic impedance. This is equivalent to operating on the two-port [S] submatrix extracted from the multi-port [S] matrix. This occurs for both signal and noise calculations. A warning appears if $N < 2$ and you specified two-port quantities.

Noise and signal port-wise calculations do not require that port elements use a ground reference. You can therefore measure fully-differential circuits.

.LIN command generates a set of noise parameters. The analysis assumes a noise model consisting of:

- A shunt current noise source, called I_n , at the input of a noiseless two-port linear network
- A series voltage noise source, called V_n , at the input of a noiseless two-port linear network
- A source with impedance, called Z_s , that drives this two-port network
- The two-port network drives a noiseless load, called Z_l

The following sections discuss these topics:

- [Equivalent Input Noise Voltage and Current](#)
- [Equivalent Noise Resistance and Conductance](#)
- [Noise Correlation Impedance and Admittance](#)
- [Optimum Matching for Noise](#)
- [Noise Figure and Minimum Noise Figure](#)
- [Associated Gain](#)
- [Output Format for Group Delay in *.sc* Files](#)
- [Output Format for Two-Port Noise Parameters in *.sc# Files](#)

Equivalent Input Noise Voltage and Current

For each analysis frequency, PrimeSim HSPICE computes a noise equivalent circuit for a linear two-port. The noise equivalent circuit calculation results in an equivalent noise voltage and current, and their correlation coefficient.

- **VN2**: Equivalent input noise voltage squared (Real, V^2)
- **IN2**: Equivalent input noise current squared (Real, A^2)
- **RHON**: Correlation coefficient between the input noise voltage and the input noise current (complex, unitless)

Equivalent Noise Resistance and Conductance

These measurements are the equivalent resistance and conductance, which generate the equivalent noise voltage and current values at a temperature of $T=290K$ in a 1Hz bandwidth.

- **RN**: Noise equivalent resistance (Real, Ohms)
- **GN**: Noise equivalent conductance (Real, Siemens)

Noise Correlation Impedance and Admittance

These measurements represent the equivalent impedance and admittance that you can insert at the input of the noise equivalent circuit to account for the correlation between the equivalent noise voltage and the current values.

- **ZCOR**: Noise correlation impedance (Complex, Ohms)
- **YCOR**: Noise correlation admittance (Complex, Siemens)

Optimum Matching for Noise

These measurements represent the optimum impedance, admittance, and reflection coefficient value that result in the best noise performance (minimum noise figure).

- **ZOPT**: Optimum source impedance for noise (Complex, Ohms)
- **YOPT**: Optimum source admittance for noise (Complex, Siemens)
- **GAMMA_OPT**: Optimum source reflection coefficient (Complex, unitless)

Because **ZOPT** and **YOPT** can commonly take on infinite values when computing optimum noise conditions, calculations for optimum noise loading reflect the **GAMMA_OPT** coefficient.

Noise Figure and Minimum Noise Figure

Noise figure represents the ratio of the SNR (signal to noise ratio) at the input to the SNR at the output. You can set the input source impedance to **ZOPT** to obtain the minimum noise figure.

- **NFMIN**: Minimum noise figure (source at **ZOPT**) (real, unitless, power ratio)
- **NF**: Noise figure (value obtained with source impedance at **Zc[1]**) (real, unitless, power ratio)

Associated Gain

This measurement assumes that the input impedance matches the minimum noise figure, and the output matches the maximum gain.

G_{AS} is the associated gain—maximum power gain at N_{FMIN} (real, power ratio)

Output Format for Group Delay in .sc* Files

All of the $S/Y/Z/H$ parameters support a group delay calculation. The output syntax of `.PRINT` and `.PROBE` statements for group delay is:

$X_{mn}(T) \mid X_{mn}(TD) \mid X(m,n)(T) \mid X(m,n)(TD)$

- $X=S, Y, Z,$ or H
- m, n =port number (1 or 2 for H parameter)

The output of group delay matrices in `.sc*` files lets PrimeSim HSPICE directly read back the group delay information. The tabulated data uses the regular PrimeSim HSPICE model syntax with the `SP` keyword:

```
*| group delay parameters
.MODEL SMODEL_GD SP N=2 SPACING=POI INTERPOLATION=LINEAR
+ MATRIX=NONSYMMETRIC VALTYPE=REAL
+ DATA=3
+           1e+08
+           0           5e-09
+           5e-09           0
+ { ...data... }
```

`model name` is the model name of the S -parameters, plus `_GD`.

`GROUPDELAY=[0|1]` in the top line indicates group delay data:

```
*| N=2 DATA=3 NOISE=0 GROUPDELAY=1
*| NumOfBlock=1 NumOfParam=0
```

Output Format for Two-Port Noise Parameters in *.sc# Files

Output of two-port noise parameter data in `*.sc#` files shows the tabulated data with the following quantities in the following order:

```
*| 2-port noise parameters
*| frequency Fmin[dB] GammaOpt (M) GammaOpt (P) RN/Z0
*| { ...data... }
```

In this syntax:

- $F_{min}[dB]$ =minimum noise figure (dB)
- $\Gamma_{Opt}(M)$ =magnitude of the reflection coefficient needed to realize F_{min}

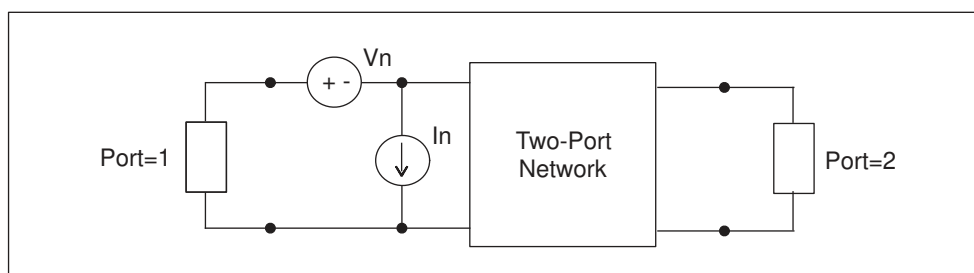
- $\text{GammaOpt}(P)$ = phase (degrees) of the reflection coefficient needed to realize $F_{\min}$
- R_N/Z_0 = normalized noise resistance

Both GammaOpt and R_N/Z_0 values are normalized with respect to the characteristic impedance of the port=1 element (that is, Z_{01}).

Noise Parameters in 2-Port and N-Port Networks

You can use the LIN analysis to compute the equivalent two-port noise parameters or multi-port noise parameters for a network. The `noisecalc=1` option automatically calculates the following equivalent circuit values.

Figure 4 Noise equivalent circuit



- V_n is the equivalent input-referred noise voltage source.
- I_n is the equivalent input-referred noise current source.
- $I_n V_n$ is their correlation.

The `noisecalc=1` option activates N-port noise analysis. Invoking an N-port noise analysis produces an `*.nc#` file to output the N-port noise correlation matrix in the following format, which is similar to a `*.sc#` file.

```
*| N=2 DATA=1 NOISE=0 GROUPDELAY=0 COMPLEX_DATAFORMAT=RI
*| NumOfBlock=2 NumOfParam=1 ParamNames=res
*| ParamSweep res:50 100
*|-----
*| res=50
.MODEL NFQMODEL SP N=2 SPACING=POI INTERPOLATION=LINEAR
MATRIX=NONSYMMETRIC
+ DATA=1
+          100
+    0.457141      0    -0.457141      0
+   -0.457141      0     0.457141      0
*|-----
*| res=100
.MODEL NFQMODEL1 SP N=2 SPACING=POI INTERPOLATION=LINEAR
MATRIX=NONSYMMETRIC
```

```
+ DATA=1
+           100
+      0.514284      0      -0.514284      0
+     -0.514284      0       0.514284      0
```

The noise analysis data is output in the `.lis` and `.ac*` files in the following format:

```
Frequency = 100.000 Hz
NF (mag) = 1.0+1.02857
NF (dB) = 3.07189
Element Contribution to NF (mag) (dB)
r1      TOT      1.02857 122.327m
```

All the 2-port noise parameters are deduced and output in the `.lis` file with `.PRINT/.PROBE` commands.

PrimeSim HSPICE can output the result of `.LIN` noise analysis to a `*.sc`, Touchstone 1.0/2.0, or CITI file. Note this limitation for Touchstone files: because Touchstone files currently provide only two-port noise parameters, this type of noise model only supports two-port noise analysis for both passive and active systems.

Hybrid (H) Parameters

LIN analysis can calculate the complex two-port H (hybrid) parameter of a multi-terminal network.

The H parameters of a two-port network relate the voltages and currents at input and output ports:

$$V1 = h11 \cdot I1 + h12 \cdot V2$$

$$I2 = h21 \cdot I1 + h22 \cdot V2$$

In the preceding equations:

- $H = \begin{bmatrix} h11 & h12 \\ h21 & h22 \end{bmatrix}$ Hybrid matrix
- V1=Voltage at input port
- I2=Current at output port
- V2=Voltage at output port
- I1=Current at input port

You can add the hybrid H parameter matrixes of two networks to describe networks that are in series at their input and in parallel at their output.

.LIN command calculates H parameters based on the scattering parameters of the networks. LIN analysis can extract one-port and two-port network H parameters. For networks with more than two ports, .LIN command assumes that the ports numbered 3 and above terminate in their port characteristic impedance ($Z_c[i], i > 2$).

Group Delay

Group delay measures the transit time of a signal through a network versus frequency. It reduces the linear portion of the phase response to a constant value, and transforms the deviations from linear phase into deviations from constant group delay (which causes phase distortion in communications systems). The average delay represents the average signal transit time through a network system.

PrimeSim HSPICE can output the result of .LIN group delay measurement to a .sc*, Touchstone 1.0/2.0, or CITIfile.

Group delay is a function of frequency:

$$gd(w) = \frac{-d(\text{phase})}{d(w)}$$

where:

- gd = Group delay at the f frequency, $2\pi f = w$
- phase = phase response at the f frequency
- w = radians frequency

All complex S, Y, Z, and H parameters support a group delay calculation.

Syntax

```
Xmn(T) | Xmn(TD) | X(m,n)(T) | X(m,n)(TD)
X=S, Y, Z, or H (parameters)
m,n=port index (1 or 2 for H-parameters)
```

The results of the group delay calculation are scalar real numbers in units of seconds. For .LIN, group delay values are a function of frequency. The calculation is:

$$r_{ij} = |r_{ij}|e^{jf_{ij}(w)}$$

Differentiating the complex logarithm with respect to omega results in:

$$\frac{1}{r_{ij}} \frac{dr_{ij}}{dw} = \frac{1}{|r_{ij}|} \frac{d|r_{ij}|}{dw} + j \frac{df}{dw}$$

The group delay is the negative derivative of the phase. Simulation can compute it from the imaginary component of the derivative w.r.t. frequency of the measurement:

$$\tau(w) = -\frac{df}{dw} = -im\left[\frac{1}{r_{ij}} \frac{dr_{ij}}{dw}\right]$$

Summary of Printing and Probing Network Parameters

When used in combination with P (port) elements and the `.AC` command, `.LIN` makes available network parameters such as `noise`, `S`, `H` and `Y/Z`. You can save these network parameters values in a plot file with `.PROBE` and view them with a waveform tool, or use `.PRINT` to record the values in your listing file in a tabular format.

The following sections discuss these topics:

- [Specifying the Analysis Type](#)
- [Port Reference](#)
- [Port Numbers with More Than One Digit](#)
- [Network Parameter Data Types](#)
- [Mixed-Mode Ports](#)

Specifying the Analysis Type

Since you may have more than one analysis in your netlist (AC/DC/TRAN), when using `.PRINT` or `.PROBE` you have the option to specify which analysis to wish to reference. If the only analysis you are performing is the `.AC` combined with `.LIN` to extract network parameters, you can omit this and therefore the following are equivalent:

```
.PROBE AC S21(DB) S12(DB)
.PROBE S21(DB) S12(DB)
```

Port Reference

The numbers in the port reference (e.g., `S21`, `S12` above) refer to the `PORT=` number in your netlist. In this single-ended example, `P_IN` is port 1 and `P_OUT` is port 2.

```
P_IN IN 0 Z0=50 PORT=1 AC=1
P_OUT OUT 0 Z0=50 PORT=2
```

So, in the print command:

```
.PRINT AC S21(DB)
```

You are printing the insertion loss (or gain) observed at port 2 from port 1.

Port Numbers with More Than One Digit

PrimeSim HSPICE can understand S12, S34, S91, etc., but port numbers with more than one digit (e.g., > 9) require putting parenthesis around the port reference and separating them with a comma. For example:

```
.PROBE S (12,2) (DB)
```

Network Parameter Data Types

The previous examples reference the DB data type. If you omit the data type argument, the default is actually magnitude. For example:

```
.PRINT S11
```

is equivalent to:

```
.PRINT S11 (M)
```

Both will return the magnitude of the return loss. These are the available data type definitions:

- R = Real
- I = Imaginary
- M = Magnitude (default)
- P=PD = Phase in degrees
- PR = Phase in radians
- DB = decibels

Mixed-Mode Ports

You can also specify whether to reference the common, mixed or cross-mode network parameters in your .PRINT or .PROBE statements. By default, PrimeSim HSPICE determines the mode of the network parameters from the port configuration. For example, in this mixed mode (differential) port configuration:

```
P_IN IN_P IN_N 0 Z0=50 PORT=1 AC=1  
P_OUT OUT_P OUT_N 0 Z0=50 PORT=2
```

The following statements will return the same values:

```
.PROBE S21 (DB)  
.PROBE SDD21 (DB)
```

But you can explicitly specify the mode of the network parameters to reference as follows:

- SDD = differential mode S parameters
- SCC = common mode S parameters
- SDC or SCD = cross mode S parameters

SNLIN Analysis

The `.SNLIN` command invokes the SNLIN analysis. This analysis is used for:

- Frequency translation scattering parameter extraction for a periodically-driven circuits that exhibits frequency translation effects, such as mixers.
- Noise parameter extraction for a periodically-driven circuits that exhibits frequency translation effects.

Frequency translate S parameter is used to describe N-port circuits that exhibits frequency translation, such as mixers. It is a small-signal analysis like `.LIN`, except that the circuit is first linearized about a periodically varying operating point, instead of a simple DC operating point. In this manner, it is possible to compute S parameters between circuit ports that convert signals from one frequency band to another. Frequency translate S parameter describes the capability of a periodically linear time varying systems to shift signals in frequency. Similar to the S parameters of a linear-time-varying system, it is defined as below:

$$b = S \times a$$

$$s[i, j] = \frac{b[i]}{a[j]}$$

The incident and reflected waves are defined as:

$$a[i, n](w) = \frac{(V[i](w + nw_0) + Z_0[i]^* I[i](w + nw_0))}{2\sqrt{(Z_0[i])}}$$

$$b[i, n](w) = \frac{(V[i](w + nw_0) - Z_0[i]^* I[i](w + nw_0))}{2\sqrt{(Z_0[i])}}$$

Parameter	Description
w_0	Tone, refers to to the fundamental frequency
n	Harmonic index

Parameter	Description
i	Port index
$d[i, n](w)$	The input wave at the frequency $w + nw_0$ for the i'th port.
$b[i, n](w)$	The reflected wave at the frequency $w + nw_0$ for the i'th port.
$V[i](w + nw_0)$	The fourier coefficient at the frequency $w + nw_0$, of the voltage at port i.
$I[i](w + nw_0)$	The fourier coefficient at the frequency $w + nw_0$, of the voltage at port i.
$Z_0[i]$	The reference impedance at port i.
$S[i, j]$	S parameter from the j th port to the i th port.

Multitone Analysis

For a multitone analysis, the calculation for multitone a, b vector definition is:

$$a[i, n_1, n_2 \dots n_N](\omega) = \frac{V[i](\omega + n_1\omega_1 + n_2\omega_2 + \dots + n_N\omega_N) + Z_o[i] * I[i](\omega + n_1\omega_1 + n_2\omega_2 + \dots + n_N\omega_N)}{2\sqrt{Z_o[i]}}$$

$$b[i, n_1, n_2 \dots n_N](\omega) = \frac{V[i](\omega + n_1\omega_1 + n_2\omega_2 + \dots + n_N\omega_N) - Z_o[i] * I[i](\omega + n_1\omega_1 + n_2\omega_2 + \dots + n_N\omega_N)}{2\sqrt{Z_o[i]}}$$

SNLIN Input Syntax

```
.SNLIN
+ [freq_sweep]
+ [NOISECALC=1|0|on|off]
+ [FILENAME=file_name]
+ [DATAFORMAT=ri|ma|db| ]
+ [MIXEDMODE2PORT=dd|cc|cd|dc|sd|sc|cs|ds| ]
```

For argument descriptions, see the .LIN command in the PrimeSim Continuum *Reference Manual: Commands and Control Options*.

SNLIN Output Syntax

This section describes the syntax for the `.PRINT` and `.PROBE` statements used for SNLIN analysis.

`.SNLIN` outputs to the `*.sl0` and `*.prints0` files.

The following sections discuss these topics:

- [.PRINT and .PROBE Statements](#)
- [Output to Data Files](#)
- [Examples](#)

.PRINT and .PROBE Statements

```
.print SNLIN Smn|Smn(TYPE) | S(m,n) | S(m,n)(TYPE) |  
.probe SNLIN Smn | Smn(TYPE) | S(m,n) | S(m,n)(TYPE) |  
.print SNLIN SXYmn|SXYmn(TYPE) | SXY(m,n) | SXY(m,n)(TYPE) |  
.probe SNLIN SXYmn|SXYmn(TYPE) | SXY(m,n) | SXY(m,n)(TYPE) |  
.print SNLIN NF | SSNF | DSNF  
.probe SNLIN NF | SSNF | DSNF
```

Argument	Description
m and n	m and n refers to port number index. Both of them must be with in the range $0 < m, n < \text{port_number}$.
x and y	x and y are used for mixed mode S parameter output. It can be any one of the following: • D - differential • C - common • S - single end
TYPE	• R = Real • I = Imaginary • M = Magnitude • P = PD = Phase in degrees • D = DB = Decibels • DBM = Decibels per milli
NF and SSNF	Refers to the single side band noise figure as a function of the input frequency band points.
DSNF	Refers to the double side band noise figure as a function of the input frequency band points.

Output to Data Files

The extracted S parameters can also be the output to data files. The file format can only be the Touchstone file. For the Touchstone file output, the frequency translation flag, and the port harmonic indices must be mentioned in the header to declare the translated frequency band. The following is an example.

```
! NPort=2 DATA=4 NOISE=0 GROUPDELAY=0
! NumOfBlock=1 NumOfParam=0 FrequencyTranslate=1
!
! PORT1 = [p1_h1 p1_h2 ... p1_hNn ... +/-1]
!
! PORT2 = [p2_h1 p2_h2 ... p2_hNn ... +/-1]
!
! ...
! PORTm = [pm_h1 pm_h2 ... pm_hNn ... +/-1]
! SS_TONE = i

# TONES = [F1 F2 ... Fn]
# Hz S MA R      50.0000
.....
```

Frequency translates S parameter data with this port harmonic combination.

```
# TONES = [F1 F2 ... Fn]          ! exist only when large signal tone sweep
# Hz S MA R      50.0000
.....
```

Frequency translate S parameter data with this harmonic combination.

If mixed mode S parameters are extracted, the mixed-mode information is be specified in the file header at the very beginning, such as:

```
! NPort=2 DATA=1000 NOISE=0 GROUPDELAY=0
! NumOfBlock=1 NumOfParam=0 FrequencyTranslate=1
! MIXEDMODE=1 DATATYPE=D1D2C1C2
!
! S(1,1) = SDD(1,1)
! S(1,2) = SDD(1,2)
! S(1,3) = SDC(1,1)
! S(1,4) = SDC(1,2)
! S(2,1) = SDD(2,1)
! S(2,2) = SDD(2,2)
! S(2,3) = SDC(2,1)
! S(2,4) = SDC(2,2)
! S(3,1) = SCD(1,1)
! S(3,2) = SCD(1,2)
! S(3,3) = SCC(1,1)
! S(3,4) = SCC(1,2)
! S(4,1) = SCD(2,1)
! S(4,2) = SCD(2,2)
! S(4,3) = SCC(2,1)
! S(4,4) = SCC(2,2)
!
```

```
! PORT1 = [p1_h1 p1_h2 ... p1_hNn +/-1]
! PORT2 = [p2_h1 p2_h2 ... p2_hNn +/-1]
! ...
! PORTm = [pm_h1 pm_h2 ... pm_hNn +/-1]
! SS_TONE = i

# TONES = [F1 F2 ... Fn]
# Hz S MA R      50.0000
```

Frequency translate S parameter data with this harmonic combination.

```
# TONES = [F1 F2 ... Fn]           ! exist only when large signal
tone sweep
# Hz S MA R      50.0000
```

Frequency translate S parameter data with this harmonic combination.

Examples

This example does single-tone analysis, with frequency translation:

- Extract two-port S parameters transferred from RF (1G-del_f) to IF (del_f). The LO signal is specified by normal voltage source Vlo.
- The frequency on port 1 is in RF band:1G-del_f, the frequency on port 2 is in IF band: del_f; del_f is swept from 0 to 100meg.
- The results are output to the ex1.s2p file.

```
p1 RFin gnd port = 1 SNLIN=(1, -1)
p2 IFout gnd port = 2 SNLIN=(0,1)
Vlo Loin gnd 1.0 cos(1.0 0.5 1g)
.SN tones=1G nharm=5
```

```
.SNLIN
```

```
+ lin 5 0 100meg
+ noisecalc = off
+ filename = ex1
+ dataformat = ma
```

The following example is for singletone analysis, with frequency translation:

- Extract two-port S parameters transferred from RF (1G-del_f) to IF (del_f). The LO signal is specified by normal voltage source Vlo.
- The frequency component is picked at port 1 is del_f; the frequency component picked at port 2 is 1GHz*2-del_f; the frequency component picked at port 3 is 1GHz +del_f; del_f is swept from 0 to 100meg.
- SNNOISE calculation is required.
- The results are output to the ex2.s3p file.


```
p1 1 0 port=1 SNLIN=(0, 1)
p2 2 0 port=2 SNLIN=(2, -1)
p3 3 0 port=3 SNLIN=(1, 1)

.PARAM smfreq=100meg
.SN tones=1G nharms=2

.SNLIN
+ lin 5 0 100meg
+ noisecalc=on
+ filename=ex2
```

Additional Measurements From .LIN Command

In addition to S , Y , Z , and H parameters, a LIN analysis can include other output measurements discussed in the following sections:

- [Impedance Characterizations](#)
- [Stability Measurements](#)
- [Gain Measurements](#)
- [Matching for Optimal Gain](#)

Impedance Characterizations

- $VSWR(i)$ = Voltage standing wave ratio at port i (real, unit-less)
- $ZIN(i)$ = Input impedance at port i (complex, Ohms)
- $YIN(i)$ = Input admittance at port i (complex, Siemens)

Stability Measurements

- $K_STABILITY_FACTOR$ = Rollett stability factor (real, unit-less)
- $MU_STABILITY_FACTOR$ = Edwards & Sinsky stability factor (real, unit-less)

Gain Measurements

- G_MAX = Maximum available/operating power gain (real, power ratio)
- G_MSG = Maximum stable gain (real, power ratio)
- G_TUMAX = Maximum unilateral transducer power gain (real, power ratio)
- G_U = Unilateral power gain (real, power ratio)

Matching for Optimal Gain

- G_MAX_GAMMA1 = Source reflection coefficient needed to realize G_MAX (complex, unitless)
- G_MAX_GAMMA2 = Load reflection coefficient needed to realize G_MAX (complex, unitless)
- G_MAX_Z1 = Source impedance needed to realize G_MAX (complex, Ohms)
- G_MAX_Z2 = Load impedance needed to realize G_MAX (complex, Ohms)
- G_MAX_Y1 = Source admittance needed to realize G_MAX (complex, Siemens)
- G_MAX_Y2 = Load admittance needed to realize G_MAX (complex, Siemens)

The following sections discuss these topics:

- [VSWR](#)
- [ZIN\(i\)](#)
- [YIN\(i\)](#)
- [K_STABILITY_FACTOR](#) (Rollett Stability Factor)
- [MU_STABILITY_FACTOR](#) (Edwards-Sinsky Stability Factor)
- [Maximum Available Power Gain - \$G_MAX\$](#)
- [Maximum Stable Gain - \$G_MSG\$](#)
- [Maximum Unilateral Transducer Power Gain - \$G_TUMAX\$](#)
- [Unilateral Power Gain - \$GU\$](#)
- [Simultaneous Conjugate Match for \$G_MAX\$](#)
- [Equivalent Input Noise Voltage and Current - \$IN2\$, \$VN2\$, \$RHON\$](#)
- [Equivalent Noise Resistance and Conductance - \$RN\$, \$GN\$](#)
- [Noise Correlation Impedance and Admittance - \$ZCOR\$, \$YCOR\$](#)
- [Optimum Matching for Noise - \$ZOPT\$, \$YOPT\$, \$GAMMA_OPT\$](#)
- [Noise Figure and Noise Figure Minimum - \$NF\$, \$NFMIN\$](#)
- [Associated Gain - \$G_As\$](#)

VSWR

The *Voltage Standing Wave Ratio* represents the ratio of maximum to minimum voltages along a standing wave pattern due to a port's impedance mismatch. All ports other than the port of interest terminate in their characteristic impedances. VSWR is a real number related to that port's scattering parameter:

$$VSWR[i] = \frac{1 + |S_{ii}|}{1 - |S_{ii}|}$$

ZIN(i)

The Input Impedance at the i port is the complex impedance into a port with all other ports terminated in their appropriate characteristic impedance. It is related to that port's scattering parameter:

$$ZIN[i] = Z_{0i} \frac{1 + S_{ii}}{1 - S_{ii}}$$

YIN(i)

The Input Admittance at the i port is the complex admittance into a port with all other ports terminated in their appropriate characteristic impedance. It is related to that port's scattering parameter:

$$YIN[i] = \frac{1}{Z_{0i}} \frac{1 - S_{ii}}{1 + S_{ii}}$$

K_STABILITY_FACTOR (Rollett Stability Factor)

The *Rollett* stability factor is:

$$K = \frac{1 - |S_{11}|^2 - |S_{22}|^2 + |\Delta|^2}{2|S_{12}||S_{21}|}$$

Δ determines the two-port S matrix calculated from this equation:

$$\Delta = S_{11}S_{22} - S_{12}S_{21}$$

An amplifier where $K > 1$ is unconditionally stable at the selected frequency.

MU_STABILITY_FACTOR (Edwards-Sinsky Stability Factor)

The following equation defines the Edwards-Sinsky stability factor.

$$\mu = \frac{1 - |S_{11}|^2}{|S_{22} - \Delta S_{11}^*| + |S_{21}S_{12}|}$$

$$\Delta = S_{11}S_{22} - S_{12}S_{21}$$

An amplifier with $\mu > 1$ is considered unconditionally stable at the specified frequency.

Maximum Available Power Gain - G_MAX

This is the gain value that can be realized if the two-port is simultaneously conjugate-matched at both input and output (with no additional feedback):

$$G_{max} = \frac{|S_{21}|}{|S_{12}|} (K - \sqrt{K^2 - 1})$$

K is the Rollett stability factor. Special cases of G_{MAX} are handled in the following manner:

- If $|S_{12}|=0$ and ($|S_{11}|=1$ or $|S_{22}|=1$), $G_{MAX}=|S_{21}|^2$
- If $|S_{12}|=0$ and $|S_{11}| \neq 1$ and $|S_{22}| \neq 1$, $G_{MAX}=G_{TUMAX}$
- If $|S_{12}| \neq 0$ and $K \leq 1$, $G_{MAX}=G_{MSG}$

When values for $K \leq 1$, the Maximum Available Power Gain is undefined, and PrimeSim HSPICE returns the Maximum Stable Gain.

Maximum Stable Gain - G_MSG

For a two-port that is conditionally stable ($K < 1$), the following equation calculates the maximum stable gain:

$$G_{MSG} = \frac{|S_{21}|}{|S_{12}|}$$

To achieve this gain, resistively load the unstable two-port so that $K=1$, and then simultaneously conjugately match the input and output ports. G_{MSG} is therefore equivalent to G_{MAX} with $K=1$. In terms of admittance parameters:

$$G_{MSG} = \frac{|y_{21}|}{|y_{12}|}$$

MSG is equivalent to the Maximum Available Power Gain if $K=1$.

Maximum Unilateral Transducer Power Gain - G_TUMAX

This is the highest possible gain that a two-port with no feedback (that is, $S_{12}=0$) can achieve.

$$G_{tymax} = \frac{|S_{21}|^2}{(1 - |S_{11}|^2)(1 - |S_{22}|^2)}$$

Unilateral Power Gain - GU

This is the highest gain that the active two-port can ever achieve by embedding in a matching network that includes feedback. The frequency where the unilateral gain becomes unity defines the boundary between an active and a passive circuit. The frequency is usually referred to as $f_{\max}$, the maximum frequency of oscillation.

To realize this gain, PrimeSim HSPICE neutralizes the feedback of the two-port, and simultaneously conjugate-matches both input and output:

$$G_U = \frac{\left| \frac{s_{21}}{s_{12}} - 1 \right|^2}{2K \left| \frac{s_{21}}{s_{12}} \right| - 2\operatorname{Re} \left\{ \frac{s_{21}}{s_{12}} \right\}}$$

Simultaneous Conjugate Match for G_MAX

A simultaneous conjugate match is required at the source and load to realize the $G_{\max}$ gain value. The source reflection coefficient at the input must be:

$$\Gamma_1 = \frac{C_1^*}{|C_1|} \left[\frac{B_1}{2|C_1|} \sqrt{\frac{B_1^2}{|2C_1|^2} - 1} \right]$$

$$B_1 = 1 - |s_{22}|^2 + |s_{11}|^2 - |\Delta|^2$$

$$C_1 = s_{11} - \Delta s_{22}^* \quad \Delta = s_{11}s_{22} - s_{12}s_{21}$$

The load reflection coefficient ($G_{\max_GAMMA_2}$) is:

$$\Gamma_2 = \frac{C_2^*}{|C_2|} \left[\frac{B_2}{2|C_2|} \sqrt{\frac{B_2^2}{|2C_2|^2} - 1} \right]$$

In the preceding equation:

$$B_2 = 1 - |s_{11}|^2 + |s_{22}|^2 - |\Delta|^2$$

$$C_2 = s_{22} - \Delta s_{11}^* \quad \Delta = s_{11}s_{22} - s_{12}s_{21}$$

You can obtain useful solutions only when:

$$\frac{B_1}{|2C_1|} > 1 \quad \frac{B_2}{|2C_2|} > 1$$

These equations also imply that $K > 1$.

PrimeSim HSPICE derives calculations for the related impedances and admittances from the preceding values.

For G_MAX_Z1:

$$Z_1 = Z_{01} \frac{1 + \Gamma_1}{1 - \Gamma_1}$$

For G_MAX_Z2:

$$Z_2 = Z_{02} \frac{1 + \Gamma_2}{1 - \Gamma_2}$$

For G_MAX_Y1:

$$Y_1 = \frac{1}{Z_{01}} \frac{1 - \Gamma_1}{1 + \Gamma_1}$$

For G_MAX_Y2:

$$Y_2 = \frac{1}{Z_{02}} \frac{1 - \Gamma_2}{1 + \Gamma_2}$$

Equivalent Input Noise Voltage and Current - IN2, VN2, RHON

For each analysis frequency, PrimeSim HSPICE computes a noise-equivalent circuit for a linear two-port. The noise analysis assumes that all ports terminate in noiseless resistances. For circuits with more than two ports, ports identified as 3 and above terminate, and the analysis considers only ports 1 and 2. The noise-equivalent circuit calculation results in an equivalent noise voltage and current, and their correlation coefficient. These values are:

$$VN2 = \overline{|v_n|^2} \quad IN2 = \overline{|i_n|^2}$$

$$RHON = \rho_n = \frac{\overline{i_n v_n^*}}{\sqrt{\overline{|i_n|^2} \overline{|v_n|^2}}}$$

Equivalent Noise Resistance and Conductance - RN, GN

These measurements are the equivalent resistance and conductance that would generate the equivalent noise voltage and current values at a temperature of $T_0 = 290k$ in a 1 Hz bandwidth (that is, $\Delta f = 1Hz$).

$$RN = R_n = \frac{\overline{|v_n|^2}}{4kT_0\Delta f} \quad GN = G_n = \frac{\overline{|i_n|^2}}{4kT_0\Delta f}$$

Noise Correlation Impedance and Admittance - ZCOR, YCOR

These measurements represent the equivalent impedance and admittance that you can insert at the input of the noise-equivalent circuit to account for the correlation between the equivalent noise voltage and the current values.

$$ZCOR = \rho_n \sqrt{\frac{|v_n|^2}{|i_n|^2}} = \frac{i_n^* v_n}{|i_n|^2} = R_{cor} + jX_{cor}$$

$$YCOR = \rho_n \sqrt{\frac{|i_n|^2}{|v_n|^2}} = \frac{i_n v_n^*}{|v_n|^2} = G_{cor} + jB_{cor}$$

Optimum Matching for Noise - ZOPT, YOPT, GAMMA_OPT

The equivalent input noise sources and their correlation make it possible to compute the impedance, admittance, and reflection coefficient values that, if presented at the input of the noisy two-port, result in the best noise performance. These values are:

$$Z_{opt} = \sqrt{\frac{R_n}{G_n}} - X_{cor} - jX_{cor} \quad Y_{opt} = \frac{1}{Z_{opt}} = \sqrt{\frac{G_n}{R_n}} - B_{cor} - jB_{cor}$$

$$GAMMA_OPT = \Gamma_{opt} = \frac{Z_{opt} - Z_{01}}{Z_{opt} + Z_{01}}$$

Noise Figure and Noise Figure Minimum - NF, NFMIN

If you set the input source impedance to Z_{OPT} , the two-port operates with the minimum Noise Figure. The definition of the Noise Figure (F) is unusual because it involves the available gain of the two-port, not its transducer gain. You can express it in the following form:

$$F = 1 + \frac{N_a}{G_a k T_0 \Delta f}$$

- G_a is the available power gain.
- N_a is the available noise power at the output of the two-port (due solely to the two-port's noise and not to the input impedance).
- k is Boltzmann's constant.
- T_0 is the 290 Kelvin reference temperature.

The NMIN minimum noise figure value is computed as:

$$NFMIN = F_{min} = 1 + 2G_n \left(R_{cor} + \sqrt{\frac{R_n}{G_n}} - X_{cor} \right)$$

where $NF_{min} \geq 1$. For input source impedance values other than Z_{OPT} , the Noise Figure varies as a function of the input source reflection coefficient, according to:

$$F = F_{min} + \frac{R_n |\Gamma_s - \Gamma_{opt}|^2}{2Z_{01} |1 + \Gamma_{opt}|^2}$$

The PrimeSim HSPICE Noise Figure measurement (NF) returns the noise figure value if the input terminates in the port characteristic impedance (that is, $\Gamma_s = 0$). This value is:

$$NF = F_{min} + \frac{R_n |\Gamma_{opt}|^2}{2Z_{01} |1 + \Gamma_{opt}|^2} = F_{min} + \frac{G_n}{Z_{01}} |Z_{01} - Z_{opt}|^2$$

Associated Gain - G_{As}

PrimeSim HSPICE also includes a measurement named Associated Gain, which assumes that the γ_s in-out impedance is matched for the minimum noise figure (that is, $\Gamma_s = \Gamma_{opt}$), while the output is matched for the maximum gain.

$$G_{AS} = \frac{|s_{21}|^2 (1 - |\Gamma_{opt}|^2)}{|1 - s_{11} \Gamma_{opt}|^2 (1 - |s'_{22}|^2)}$$

In the preceding equation: $s'_{22} = s_{22} + \frac{s_{12}s_{21}\Gamma_{opt}}{1 - s_{11}\Gamma_{opt}}$

Example - Extracting Bandpass Filter S-parameters

This example uses linear analysis to extract the S-parameters in the Touchstone 1.0/2.0 format for a bandpass filter. The input impedance of the filter is also measured. The circuit for this example is `fbplin.sp`, which is located in the directory `$installdir/demo/hspice/sparams`.

```
*band pass filter
c1  in  2    3.166pf
l1  2   3    203nh
c2  3   0    3.76pf
c3  3   4    1.75pf
c4  4   0    9.1pf
l2  4   0   36.81nh
c5  4   5    1.07pf
c6  5   0    3.13pf
l3  5   6   233.17nh
c7  6   7    5.92pf
c8  7   0    4.51pf
c9  7   8    1.568pf
c10 8   0    8.866pf
l4  8   0   35.71nh
```


Chapter 2: Linear Network Parameter Analysis

Example - Extracting Bandpass Filter S-parameters

```
c11 8 9 2.06pf
c12 9 0 4.3pf
l5 9 10 200.97nh
c13 10 out 2.97pf
```

To extract the S-parameters, port elements are inserted at the input and output of the filter.

```
p1 in 0 port=1
p2 out 0 port=2
```

Set up the linear analysis to extract the S-parameters of the filter. The center frequency of the filter is 250 MHz.

```
.ac lin 250 200meg 300meg
.lin format=touchstone
```

Probe the magnitude and phase of the input return loss (s_{11}), the insertion loss (s_{21}) and the input impedance (z_{in}).

```
.probe ac s11(db) s11(p) s21(db) s21(p)
.probe ac zin(1)(m) zin(1)(p)
```

The frequency in the passband where the input impedance is 50 ohm and the phase in the passband where the input impedance is 50 ohm can be measured.

```
.meas ac cross50 when zin(1)=50 td=230meg
.meas ac phase50 find zin(1)(p) when zin(1)(m)=50 td=230meg
```

Results can be seen in [Figure 5](#) and [Figure 6](#).

Figure 5 Input return loss magnitude and phase



Figure 6 Input impedance magnitude and phase



Extracting Mixed-Mode Scattering (S) Parameters

The LIN analysis includes a keyword for extracting mixed-mode scattering (S) parameters.

Syntax

```
.LIN ... [ mixedmode2port= dd|dc|ds|cd|cc|cs|sd|sc|ss ]
```

The following keywords in the `.PRINT` and `.PROBE` statements specify the elements in the mixed mode S-parameter matrices:

`Sxy|Yxy|Zxy nm (t)`

Argument	Description
<code>x,y</code>	One of the following: • D (differential) • C (common) • S (single-ended) For example: • SCC = Common mode S-parameters • SDC or SCD = Cross mode S-parameters If you omit <code>x,y</code> , then PrimeSim HSPICE uses the value set for the <code>mixedmode2port</code> .
<code>S_{cc}</code>	Common-mode S-parameters
<code>S_{cd}</code> and <code>S_{dc}</code>	Mode-conversion or cross-mode S-parameters
<code>m, n</code>	port number
<code>type</code>	One of the following: • DB: Magnitude in decibels • I: Imaginary part • M: Magnitude (default) • P: Phase in degree • R: Real part

The following sections discuss these topics:

- [Defaults](#)
- [Output File Formats](#)
- [Two-Port Parameter Measurement](#)
- [Output Format and Description](#)
- [Features Supported](#)

- [Prerequisites and Limitations](#)
 - [Reported Statistics for the Performance Log](#)
 - [Errors and Warnings](#)
-

Defaults

Availability and default value for the `mixedmode2port` keyword depends on the port configuration.

Example 1

`p1=p2=single`

where:

- Available: `ss`
- Default: `ss`

Example 2

`p1=p2=balanced`

where:

- Available: `dd, cd, dc, cc`
- Default: `dd`

Example 3

`p1=balanced p2=single`

where:

- Available: `ds, cs`
- Default: `ds`

Example 4

`p1=single p2=balanced`

where:

- Available: `sd, sc`
- Default: `sd`

Output File Formats

An `.sc#` file format for the mixed mode:

- The S-element model has additional keywords, such as `mixedmode` and `datatype`, if the netlist includes one or more balanced ports.
- The `mixedmode2port` keyword prints in the header line.
- The other S-element keywords also appear in the header lines.

Touchstone 1.0/2.0 format for the mixed mode:

The following lines for data mapping are added to the head of the Touchstone output file if the netlist includes one or more balanced ports.

```
! S11=SDD11
! S12=SDD12
! S13=SDC11
! S14=SDC12
```

Two-Port Parameter Measurement

Two-port parameter measurement function takes the first 2 ports, then reads the corresponding parameter with the drive condition specified by the `mixedmode2port` keyword.

Output Format and Description

File Type	Description
*.ac#	Output from both the <code>.PROBE</code> and <code>.PRINT</code> statements.
*.printac#	Output from the <code>.PRINT</code> statement. Available in PrimeSim HSPICE when <code>.option LIS_NEW</code> is set.
.sc#	The extracted S-parameters/2-port noise parameters are written to a <code>.sc#</code> file by using the S-element format. If you want to simulate the S element, you can reference the <code>*.sc#</code> file in your netlist.
.s#p	Touchstone 1.0/2.0 file
*.citi#	CITI file
*.nc#	File to output noise matrix. Generated by <code>.lin noisecalc=1</code>

```
* N=numOfPorts DATA=numOfFreq NOISE=[0|1] GROUPDELAY=[0|1]
* NumOfBlock=numOfSweepBlocks NumOfParam=numOfSweptParameters
* MIXEDMODE=[0|1] DATATYPE=mixedModeDataTypeString
```

```
.MODEL mname S
+ N=numOfPorts FQMODEL=SFQMODEL TYPE=S Z0=*** *** ...
.MODEL SFQMODEL SP N=numberOfPorts SPACING=POI
  INTERPOLATION=LINEAR MATRIX=NONSYMMETRIC
+ DATA= numberOfData
+ freq1
+ s1lreal s1limag s12real s12imag ... s1Nreal s1Nimag
...
+ sNlreal sNlimag ... sNNreal sNNimag
...
+ freqNumberOfData
+ s1lreal s1limag s12real s12imag ...s1Nreal s1Nimag
...
+ sNlreal sNlimag ... sNNreal sNNimag

* 2-port noise parameter
* frequency Fmin [dB] GammaOpt (M) GammaOpt (P) RN/Z0
* 0.10000E+09 0. 1.0000 0. 1.0281
* ...
```

The 2-port noise section starts with “*” so that you can include this file in your PrimeSim HSPICE input netlists.

Features Supported

LIN analysis in PrimeSim HSPICE supports the following features:

- Automatic calculation of bias-dependent S , Y , and H parameters. No additional sources required.
- Calculation of noise parameters (triggered by options).
- Calculation of group delay matrices (triggered by options).

In addition, PrimeSim HSPICE supports all existing PrimeSim HSPICE models for noise analysis; PrimeSim HSPICE view port 1 as the input and port 2 as the output.

Prerequisites and Limitations

The following prerequisites and limitations apply to LIN analysis in PrimeSim HSPICE:

- Requires one `.LIN` statement to specify calculation options.
- Requires one `.AC` statement to specify frequency sweep and parameter sweep.
- Requires at least one `P` element, numbered from port 1 to `N`.
- For noise analysis, PrimeSim HSPICE views port 1 as the input and port 2 as the output.

Reported Statistics for the Performance Log

- Simulation time
 - DC op time
 - Total simulation time
- Memory used
 - Total memory

Errors and Warnings

- If the circuit contains fewer than two P-elements and `noisecal=1`, then the 2-port noise calculation is skipped.
- If the circuit contains fewer than two P-elements, you cannot use the `.PRINT`, `.PROBE`, or `.MEAS` commands with any two-port noise or gain parameters.
- If the circuit contains more than two P-elements, all two-port parameters are computed. By default, `port=1` is the input and `port=2` is the output. All other ports terminate in their reference impedances.

Tutorial Example: LIN Analysis for a NMOS Low Noise Amplifier

The `.LIN` command simplifies the calculation of linear multi-port transfer parameters and noise parameters. In the LIN analysis, Port (P) elements are used to specify port numbers and their characteristic impedances. The analysis automatically computes the frequency-dependent complex transfer coefficients between ports. The result is a convenient means to get scattering parameters, noise parameters, stability parameters, and gain coefficients. The `.LIN` command renders obsolete the `.NET` command. By default, the output from the `.LIN` command is saved in the `*.sc0` file format that can, in turn, be referenced as a model file for the S-parameter element.

To set up a linear transfer parameter analysis, the PrimeSim HSPICE input netlist must contain the following:

- An `.AC` command to activate small-signal AC analysis, and to specify a frequency sweep. Also, use the `.AC` command to specify any other parameter sweeps of interest.
- A `.LIN` command with the `.AC` command to activate small-signal linear transfer analysis. The `.AC` command specifies the base frequency sweep for the LIN analysis. The LIN analysis automatically performs multiple AC and NOISE analyses as needed to compute all complex signal transfer parameters.

- The necessary number of port (P) elements, numbered sequentially beginning with one to define the terminals of the multi-port network. For example, a two-port circuit must contain two port elements with one listed as port=1 and the other as port=2. The port elements define the ordering for the output quantities from the `.LIN` command (for example, the terminals for port=1 are used for S11, Y11, and Z11 measurements).

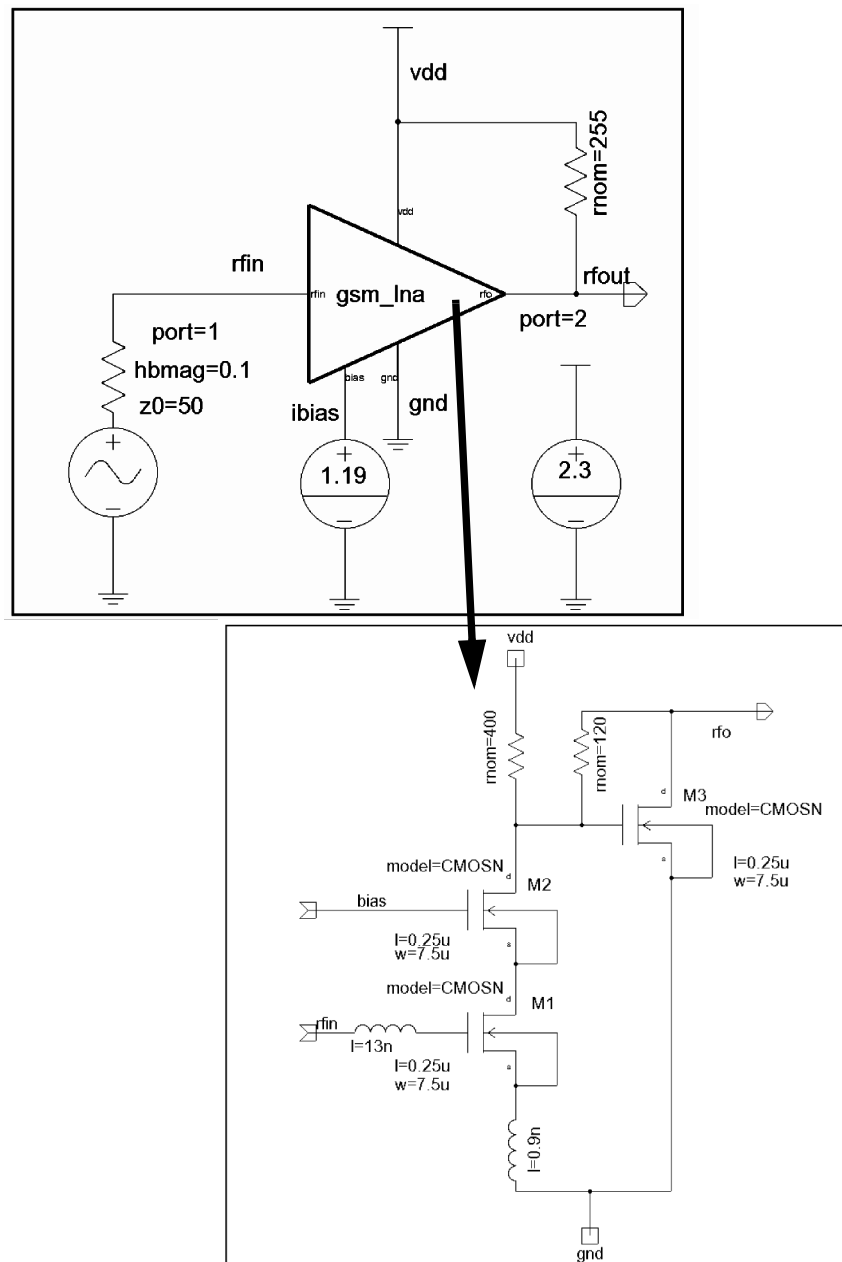
Much of the LIN analysis is automated so the PrimeSim HSPICE input netlist often does not require the following:

- AC signal sources. The `.LIN` command computes transfer parameters between the ports with no additional AC sources needed.
- DC sources. You can analyze a purely passive circuit without adding sources of any kind.

The following tutorial example shows how to set up a LIN analysis for an NMOS low noise amplifier circuit. This netlist is shipped with PrimeSim HSPICE distribution as `gsmlna.sp` and is available in the following directory: `$installdir/demo/hspice/rf_examples/`.

Compare this analysis to the use of the `.HB` command on the same circuit. See Example 2 — Using HB Analysis for a Low Noise Amp in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

Figure 7 Schematic showing instantiation for low noise amplifier



```

** NMOS 0.25um Cascode LNA for GSM applications
** setup for s-parameter and noise parameter measurements
**
.temp 27
.options post
.param Vdd=2.3
.global gnd
**

```

Chapter 2: Linear Network Parameter Analysis

Tutorial Example: LIN Analysis for a NMOS Low Noise Amplifier

```

** Cascode LNA tuned for operation near 1 GHz
**
M1 _n4 _n3 _n5 _n5 CMOSN l=0.25u w=7.5u
+   as=15p ad=15p ps=19u pd=19u m=80
M2 _n6 _n1 _n4 _n4 CMOSN l=0.25u w=7.5u
+   as=15p ad=15p ps=19u pd=19u m=80
M3 rfo _n6 gnd gnd CMOSN l=0.25u w=7.5u
+   as=15p ad=15p ps=19u pd=19u m=40
r1 _vdd _n6 400
l1 _n5 gnd l=0.9nH
l2 rfin _n3 l=13nH
vvb _n1 gnd dc=1.19 $ bias for common base device
vdd _vdd gnd dc=Vdd
rfb rfo _n6 120 $ feedback
**
** 50 Ohm input port (incl. bias), 255 Ohm output port.
**
P1 rfin gnd port=1 z0=50 dc = 0.595 $ input port includes DC bias
P2 rfo _vdd port=2 z0=255 $ port doubles as pull-up resistor
**
** Measure s-parameters and noise parameters
**
.AC DEC 50 100MEG 5G
.LIN noisecal=1 sparcalc=1 format=touchstone
.PRINT S11(DB) S21(DB) S12(DB) S22(DB) NFMIN
**
.PROBE NFMIN G_AS K_STABILITY_FACTOR
** Approximate parameters for TSMC 0.25 Process (MOSIS run T17B)
**
.MODEL CMOSN NMOS (
3.1          TNOM      = 27          TOX      = 5.8E-9          LEVEL      = 49
+XJ          = 1E-7          NCH          = 2.3549E17          VTH0          = 0.3819327
+K1          = 0.477867          K2          = 2.422759E-3          K3          = 1E-3
+K3B         = 2.1606637          W0          = 1E-7          NLX          = 1.57986E-7
+DVT0W       = 0          DVT1W         = 0          DVT2W         = 0
+DVT0        = 0.5334651          DVT1        = 0.7186877          DVT2        = -0.5
+U0          = 289.1720829          UA          = -1.300598E-9          UB          = 2.3082E-18
+UC          = 2.841618E-11          VSAT         = 1.482651E5          A0          = 1.6856991
+AGS         = 0.2874763          B0          = -1.833193E-8          B1          = -1E-7
+KETA        = -2.395348E-3          A1          = 0          A2          = 0.4177975
+RDSW        = 178.7751373          PRWG         = 0.3774172          PRWB         = -0.2
+WR          = 1          WINT          = 0          LINT          = 1.88839E-8
+XL          = 3E-8          XW          = -4E-8          DWG          = -1.2139E-8
+DWB         = 4.613042E-9          VOFF         = -0.0981658          NFACTOR      = 1.2032376
+CIT         = 0          CDSC         = 2.4E-4          CDSCD         = 0
+CDSCB       = 0          ETA0         = 5.128492E-3          ETAB         = 6.18609E-4
+DSUB        = 0.0463218          PCLM         = 1.91946          PDIBLC1      = 1
+PDIBLC2     = 4.422611E-3          PDIBLCB      = -0.1          DROUT        = 0.9817908
+PSCBE1      = 7.982649E10          PSCBE2       = 5.200359E-10          PVAG         = 9.31443E-3
+DELTA       = 0.01          RSH          = 3.7          MOBMOD       = 1
+PRT         = 0          UTE          = -1.5          KT1          = -0.11
+KT1L        = 0          KT2          = 0.022          UA1          = 4.31E-9
+UB1         = -7.61E-18          UC1          = -5.6E-11          AT           = 3.3E4
+WL          = 0          WLN          = 1          WW           = 0
+WWN         = 1          WWL          = 0          LL           = 0
+LLN         = 1          LW           = 0          LWN          = 1
+LWL         = 0          CAPMOD       = 2          XPART        = 0.5
+CGDO        = 5.62E-10          CGSO         = 5.62E-10          CGBO         = 1E-12
+CJ          = 1.641005E-3          PB           = 0.99          MJ           = 0.4453094
+CJSW        = 4.179682E-10          PBSW         = 0.99          MJSW         = 0.3413857
+CJSWG       = 3.29E-10          PBSWG        = 0.99          MJSWG        = 0.3413857

```

Chapter 2: Linear Network Parameter Analysis

Tutorial Example: LIN Analysis for a NMOS Low Noise Amplifier

```
+CF      = 0          PVTH0    = -8.385037E-3    PRDSW    = -10
+PK2     = 2.650965E-3  WKETA    = 7.293869E-3    LKETA     = -6.070E-3)
*
.END
```

A LIN analysis also includes the following:

- **.LIN** command:

```
.LIN noisecal=1 sparcalc=1 format=touchstone
```

This invokes a LIN analysis and activates noise calculations and S- parameter output files in Touchstone format.

- Two port elements:

```
P1 rfin gnd port=1 z0=50 dc=0.595
```

Specifies that an input port is assumed between terminals `rfin` and ground, that it is has a 50 ohm termination, and it has a built-in DC bias of 0.595 V. The output (second) port is:

```
P2 rfo _vdd port=2 z0=255
```

This syntax specifies that the output port is between terminals `rfo` and `_vdd`, and is being used as a pull up resistor with impedance of 255 ohm.

- A **.PRINT** command for plotting the output S-parameters in dB and the noise figure minimum.

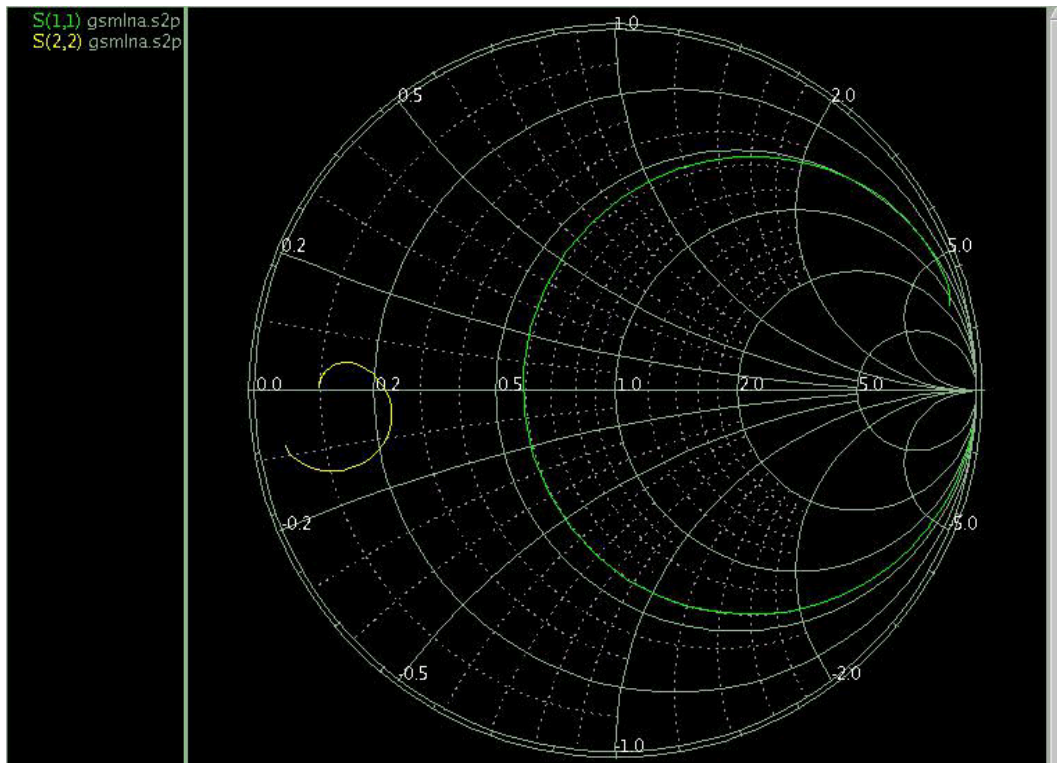
To run this netlist, type the following command:

```
hspicerf gsmlna.sp
```

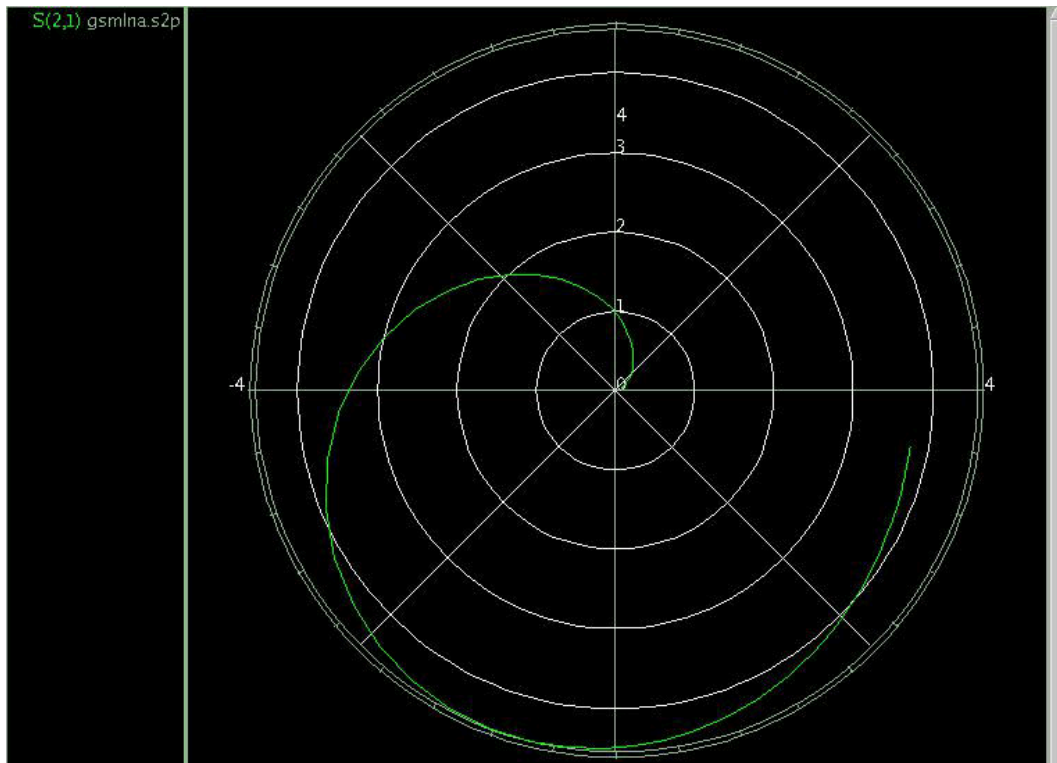
This produces three output files, named `gsmlna.ac0`, `gsmlna.s2p`, and `gsmlna.printac0`, containing the AC analysis results, S-parameter and noise parameter results, and the requested PRINT data.

To view the output:

1. Type **wv** at the prompt to invoke WaveView.
2. Click the **Open Waveform** icon and select the `gsmlna.s2p` file from the Open: Waveform Files dialog box.
3. Select **Smith Chart** from the Panel section of the **Waveform** tab to open a blank Smith chart.

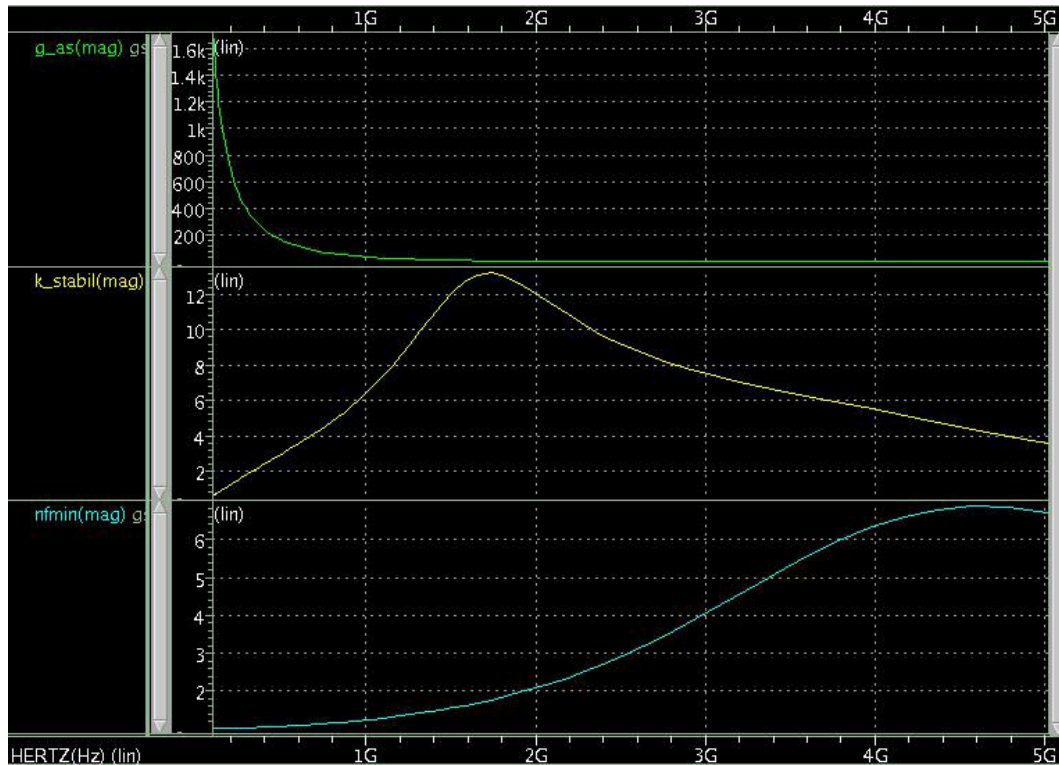


4. Select the $S(1,1)$ and $S(2,2)$ signals in the signal browser. Drag and drop the selected signals in the Smith Chart.
5. Select **Polar Plot** from the Panel section of the **Waveform** tab to open a blank Polar chart.
6. Select the $S(2,1)$ signal in the signal browser. Drag and drop the signal in the Polar chart to plot the complex gain of the LNA.



7. Click the **Open Waveform** icon and select the `gms1na.ac0` file from the Open: Waveform Files dialog box.
8. Click the **WaveView** icon in the Add section of the **Waveform** tab to open a new waveview.

9. Select the `g_as` (associated gain), `k_stability_factor` (Rollet stability factor) and `nfmin` (the noise figure minimum) signals in the signal browser. Drag and drop the selected signals in the new waveview.



References

- [1] Goyal, Ravender. "S-Parameter Output From SPICE Program", *MSN & CT*, February 1988, pp. 63 and 66.
- [2] Robert J. Weber "Introduction to Microwave Circuits", IEEE Press.
- [3] Behzad Razavi, "Design of Analog CMOS Integrated Circuits", McGraw Hill.
- [4] Reinhold Ludwig, Pavel Bretchko, "RF Circuit Design Theory and Applications".
- [5] G.D. Vendelin, Design of Amplifiers and Oscillators by the S-Parameter Method, John Wiley & Sons, 1982.
- [6] R.S. Carson, High-Frequency Amplifiers, 2nd Edition, John Wiley & Sons, 1982.
- [7] G. Gonzalez, Microwave Transistor Amplifiers: Analysis and Design, 2nd Edition, Prentice-Hall, 1997.

- [8] M.L. Edwards and J.H. Sinsky, "A single stability parameter for linear 2-port networks," IEEE 1992 MTT-S Symposium Digest, pages 885-888.
- [9] H. Rothe and W. Dahlke, "Theory of noisy fourpoles", Proc. IRE, volume 44, pages 811-818, June 1956.
- [10] David E. Bockeman, "Combined Differential and Common-Mode Scattering Parameters: Theory and Simulation," IEEE trans. on MTT Volume 43, Number 7, Jul. 1995.

3

Statistical Eye Analysis

Describes statistical eye diagram analysis by using the .STATEYE command.

Analysis of high-speed serial interfaces requires processing millions of bits of data making analysis by traditional analysis tools computationally expensive. Eye diagrams aid extensively in the evaluation of high-speed serial interfaces and as a fundamental performance metric for high-speed serial interfaces in the bit error rate (BER). Statistical eye diagram techniques enable you to quickly and accurately evaluate eye diagrams and BER.

PrimeSim HSPICE includes several full demonstration examples for your use:

- For paths to the StatEye demo files, see Signal Integrity Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the StatEye demo files.
- For a demonstration of Stateye analysis, go to <https://www.synopsys.com/verification/ams-verification/hspice.html> and click **PrimeSim HSPICE TIPS & TRICKS** to load the **Eye Diagrams** (PrimeSim HSPICETips & Tricks #2) mini webinar.
- For a discussion of the StatEye feature working with IBIS-AMI, see [Chapter 9, Using IBIS-AMI Equalizer Models with StatEye](#).
- For a mini webinar demo of StatEye with IBIS-AMI, see <https://www.synopsys.com/verification/ams-verification/hspice.html> > **PrimeSim HSPICE TIPS & TRICKS > IBIS-AMI** (PrimeSim HSPICE Tips & Tricks #3).

The following sections discuss these topics:

- [Understanding StatEye Analysis in PrimeSim HSPICE](#)
- [Statistical Eye Analysis Setup](#)
- [Determining Number of Bits Simulated in Statistical and Bit-by-bit Modes](#)
- [Edge Control](#)
- [Capturing Steady State Noise for Power Integrity](#)
- [Duty Cycle Distortion](#)
- [Capturing DCD Effect on System Clock](#)

- [Periodic and Random Jitter Effect](#)
- [N-Cycle Jitter Modeling for Data and Clock Signals](#)
- [Pre-Emphasis and De-Emphasis](#)
- [Built-In DFE Function for Rx Port Element in StatEye Analysis](#)
- [Perturbation Analysis with StatEye](#)
- [Unfolding Statistical Eye Diagrams to Waveforms](#)
- [Multiple Data Rate Eye Diagram Analysis with StatEye](#)
- [Accelerating StatEye's Initial Transient with Process Distribution and Multi-Threading](#)
- [Known Limitation](#)
- [Troubleshooting Output Warnings](#)
- [References](#)

Understanding StatEye Analysis in PrimeSim HSPICE

StatEye analysis in PrimeSim HSPICE represents eye diagrams by using-probability-density function (PDF). When a probability of a target waveform's (V_{ISI}) crossing between voltage v_1 and v_2 is given using nonnegative, continuous function as $f(x)$:

Equation 6

$$P[v_1 \leq V_{ISI} \leq v_2] = \int_{v_1}^{v_2} f(x) dx$$

where, $f(x)$ is defined as a probability density function of V_{ISI} with the unit of one over interval. In PrimeSim HSPICE `.StatEye`, [1/mV] is used for the unit of eye diagram probably density function. Here, $f(x)$ satisfies:

Equation 7

$$P[-\infty \leq V_{ISI} \leq \infty] = \int_{-\infty}^{\infty} f(x) dx = 1$$

Then, the BER can be evaluated as an accumulation of probability density from the center of the eye up to the target voltage point v_1 as:

Equation 8

$$BER[v_1] = \int_{v_c}^{v_1} f(x) dx$$

where, v_c is the center of the eye diagram. Assuming a given bit stream's even probability for each symbol, the boundary conditions of the BER are given as:

Equation 9

$$\text{BER}[-\infty] = \int_{-\infty}^{v_c} f(x) dx = 0.5$$

$$\text{BER}[\infty] = \int_{v_c}^{\infty} f(x) dx = 0.5$$

Statistical Eye Analysis Setup

The `.STATEYE` command performs pulse/step response examinations before the statistical computation and performs this initial time domain analysis as a common transient analysis. Therefore, you can apply all the `.print/.PROBE/.MEASURE` commands with the `tran` keyword.

Use the port element to designate the incident (input) and probe (output) ports for the system under analysis. You can specify ports can as single-ended or mixed-mode. You can apply random jitter to each incident and probe point in the system.

Each incident port acts as random bit pattern source with specified voltage magnitude. .

Probe ports function as observation points where you can define `.PRINT`, `.PROBE`, and `.MEASURE` statements.

To perform the statistical eye diagram analysis, use the analysis command `.STATEYE`.

The following sections discuss these topics:

- [Port Element Configuration](#)
- [.PRINT and .PROBE Syntax](#)
- [Output Data](#)
- [.MEASURE Syntax](#)
- [Understanding the Worst Pattern Measurement Result](#)
- [Understanding n-UI Accumulated Clock Period Jitter Measurements \(RMS/PP\)](#)
- [Sample Measurements](#)
- [Output](#)
- [Saving and Reusing Initial Transient Results](#)
- [Examples - Statistical Analysis Setup](#)
- [Example – StateEye Crosstalk using W Element](#)

Port Element Configuration

Incident ports act as pure random bit stream generators in `.STATEYE` analysis unless you specify either `LFSR` or `PAT` source keywords. These keywords force the incident port to act as a specific bit pattern generator. If an incident port element does not have a time-domain voltage magnitude specification, PrimeSim HSPICE uses the default values, `v_high=1.0`, `v_low=-1.0`.

You can print or probe the following statistical eye diagram analysis:

- Eye diagram at a port
- Bit error rate map at a port
- Bathtub curve of the bit error rate at a port for specified voltage
- Bathtub curve of the bit error rate at a port for a specified time
- Emphasis (see [Pre-Emphasis and De-Emphasis](#))
- Duty cycle distortion (see [Duty Cycle Distortion](#))
- Perjitter analysis (see [Periodic and Random Jitter Effect](#))

For P-element syntax, refer to [Identifying Ports with the P-element](#).

If you define port elements, but do not specify them as incident or probe ports, PrimeSim HSPICE assumes them to be DC voltage sources with no time dependence.

.PRINT and .PROBE Syntax

```
.print stateye eye(port_idx)
.print stateye eyeBW(port_idx)
.print stateye eyeC(port_idx)
.print stateye eyeV(port_idx,v)
.print stateye eyeT(port_idx,t)
.print stateye ber(port_idx)
.print stateye berC(port_idx)
.print stateye bathtubV(port_idx,v)
.print stateye bathtubT(port_idx,t)
.print stateye eyeUF(port_idx, bit_idx, bit_idx2)
```

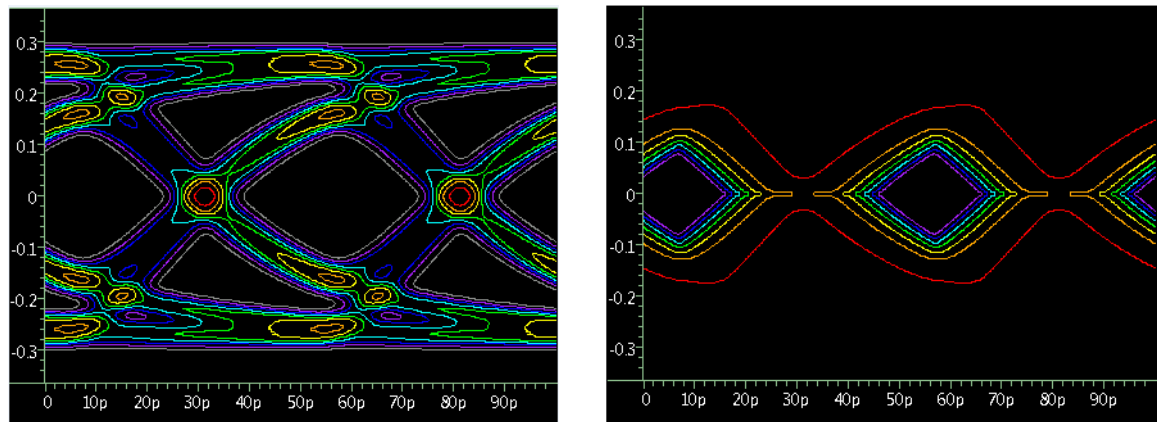
```
.probe stateye eye(port_idx)
.probe stateye eyeBW(port_idx)
.probe stateye eyeC(port_idx)
.probe stateye eyeV(port_idx,v)
.probe stateye eyeT(port_idx,t)
.probe stateye ber(port_idx)
.probe stateye berC(port_idx <,BER_value>)
```

```
.probe stateye bathtubV(port_idx,v)
.probe stateye bathtubT(port_idx,t)
.probe stateye eyeUF(port_idx, bit_idx, bit_idx2)
```

where:

- `eye(port_idx)` specifies the port to output the eye diagram.
- `eyeBW(port_idx)` specifies the port to output the eye diagram as black and white data.
- `eyeC(port_idx)` specifies the port to output the eye diagram as contour data (see [Figure 8](#)).
- `eyeV(port_idx,v)` specifies the port to output as a cross-section of the eye diagram at specified port at a specified voltage.
- `eyeT(port_idx, t)` specifies cross-section of the eye diagram at specified port at a specified time.
- `ber(port_idx)` specifies the port to output the bit error rate.
- `berC(port_idx, ber_value)` specifies the port to output the bit error rate as contour data (see [Figure 8](#)). When `BER_value` is specified, the output contains single contour at a given BER level. When no `BER_value` is specified, multiple contour levels are included. See also `BERC_Levels` keyword description for more information about contour levels.
- `bathtubV(port_idx, v)` specifies the port and voltage to output a bathtub curve.
- `bathtubT(port_idx, t)` specifies the port and time to output a bathtub curve.
- `eyeUF(port_idx, bit_idx, bit_idx2)` specifies the port and bit index to output an unfolded time domain waveform (see [Unfolding Statistical Eye Diagrams to Waveforms](#)). When one bit index is specified, StatEye outputs a time domain waveform around the specified bit position. When two bit indices are specified, StatEye outputs a bit stream waveform from the first index to the second index. In this case, you can use “-1” for either or both indices where -1 in the first argument means the beginning of the simulation and -1 in the second argument means the end of the simulation.

Figure 8 (Left) eyeC(port_idx) and (right) berC(port_idx) contour waveforms



Output Data

This section discusses output for `.PROBE`, `.PRINT`, and `.MEASURE` commands and initial transient results.

Probing Output

The `.probe stateye` command generates `netlist.stet#` and `netlist.stev#` for following purposes:

- `netlist.stet#`: `eye(t,v)`, `eyeBW(t,v)`, `eyeV(t)`, `ber(t,v)` and `bathtubV(t)`
- `netlist.stev#`: `eyeT(v)` and `bathtubT(v)`

Printing Output

The `.print stateye` command generates the data directory `netlist.printSte#/.` Then the command stores a convenient `gnuplot` script for each `.print stateye` target with necessary data files. For example:

- The `.print stateye eye(2)` statement creates the `netlist.printSte0/eye_2.plt` script file.
- The `.print stateye bathtubV(2,0.1)` creates the `netlist.printSte0/bathtub_2_0.1000e+00.plt` script file.

On the `gnuplot` command shell, you can load these scripts directly to display the target data. For example:

```
% gnuplot
gnuplot> load "printSte0/eye_2.plt"
gnuplot> load "netlist.printSte0/bathtub_2_0.1000e+00.plt"
```

Measurement Output

The `.measure stateye` command generates the `netlist.mste#` file and stores the measurement results.

StatEye Initial Transient Results

Storage of probe and measurement results takes place in the `netlist.pNeM.tr#` files, where:

- `N`: Incident port index
- `M`: Edge index
- `#`: Alter index

Typically, `netlist.p1e1.tr0` is the probe result file for the pulse response of the first incident in single edge mode.

.MEASURE Syntax

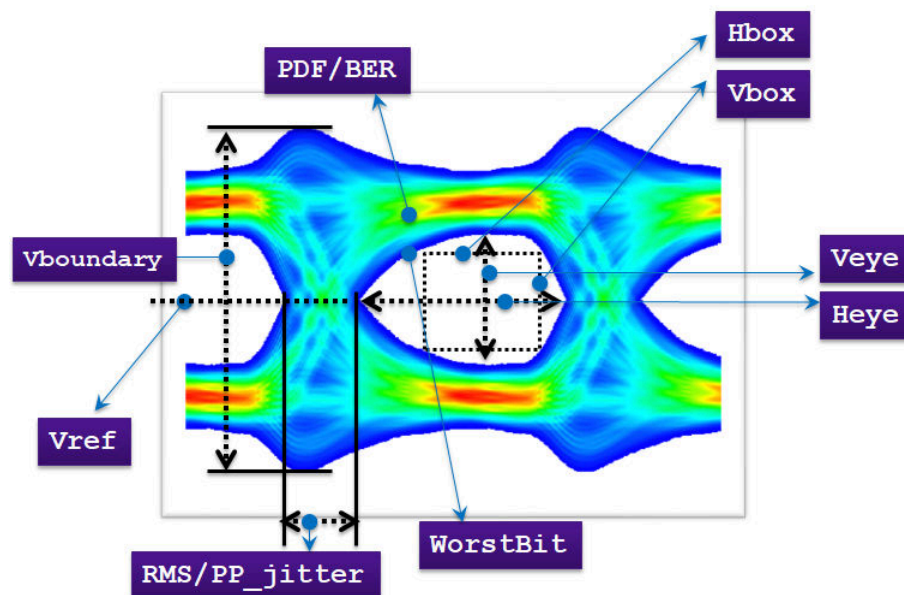
You can measure the following statistical eye diagram analysis results:

- Vertical eye opening at a specified time
- Horizontal eye opening, RMS, and peak-to-peak jitter at a specified voltage
- A bit pattern that produces the worst eye opening at a specified time in a high or low state (See [Figure 9](#)):
- Crosstalk measurement
- Accumulated period jitter, RMS, and peak-to-peak with the lowest target probability specified as a tolerance (default is `tol=1e-20`)

```
.measure stateye result_name Veye port_idx [time=val]
+ [tol=val] [eyetype=1|2]
.measure stateye result_name Veye_AVG port_idx
+ [tol=val] [eyetype=1|2]
.measure stateye result_name Heye port_idx [volt=val]
+ [tol=val] [eyetype=1|2]
.measure stateye result_name WorstBits port_idx [time=val]
+ [state=low|high]
.measure StatEye result_name Eye port# time=val volt=val
.measure StatEye result_name BER port# time=val volt=val
.measure stateye result_name rmsjitter port_idx [volt=val]
+ [eyetype=1|2]
.measure stateye result_name ppjitter port_idx [volt=val]
+ [tol=val] [eyetype=1|2]
.measure stateye result_name VBOX port_idx time=val
+ [tol=val] [eyetype=1|2]
```

```
.measure stateye result_name HBOX port_idx volt=val
+ [tol=val] [eyetype=1|2]
.measure stateye result_name Vboundary port_idx [time=val]
+ [tol=val] [eyetype=1|2]
.measure stateye result_name VREF port_idx
+ [tol=val] [eyetype=1|2]
.measure stateye result_name CROSSTALK probe_port_idx
.measure stateye result_name TREF port_idx
+ [tol=val] [eyetype=1|2]
.measure stateye result_name RMSaccPerJitter clk_idx <nUI = n>
.measure stateye result_name PPaccPerJitter clk_idx <nUI = n> <tol=value>
```

Figure 9 Measurements show vertical and horizontal eyes, worst bit pattern, and voltage with time eye



where:

- `Veye port_idx [time=val] [tol=val]` specifies the measurement port, time, and tolerance for the vertical eye opening. (For more on `tol`, see [Vertical/Horizontal Eye-Opening Tolerance](#).) If you do not specify time, PrimeSim HSPICE measures the maximum eye height and reports the time of the maximum eye opening in the listing file. `eyetype=1` (default) specifies a PDF-based eye opening measurement. `eyetype=2` specifies a BER-based eye opening measurement.
- `Veye_AVG port_idx[tol=val]` specifies the measurement port and tolerance for the average eye opening. (For more on `tol`, see [Vertical/Horizontal Eye-Opening Tolerance](#).) This measurement computes the average vertical eye opening over the time interval for which eye height is nonzero.

- `Heye port_idx [volt=val] [tol=val]` specifies the measurement port, voltage, and tolerance for the horizontal eye opening. (For more on `tol`, see [Vertical/Horizontal Eye-Opening Tolerance](#).) If you do not specify voltage, StatEye measures the maximum eye width, and PrimeSim HSPICE reports the voltage position in the listing file.
- `WorstBits port_idx time=val [state=low|high]` specifies the port, time, and state of the bit pattern that produces the worst eye opening. The default values of tolerance (`tol`) and logic state (`state`) are `1e-20` and `high`, respectively. If you do not specify time, StatEye measures the worst bit pattern at the maximum vertical eye opening point, and PrimeSim HSPICE reports the measured time point in the listing file.
- `Eye` and `BER` measurements pick up pinpoint probability at time and voltage points that you specify.
- `rmsjitter/ppjitter port_idx [volt=val]` specifies the measurement port and voltage for the RMS or peak-to-peak jitter. If you do not specify voltage, PrimeSim HSPICE measures jitter at `voltage=0`. `tol=val` specifies the tolerance for the peak-to-peak jitter measurement. `eyetype=1` (default) specifies a PDF-based eye opening measurement. `eyetype=2` specifies a BER-based eye opening measurement.
- `VBOX` measures the vertical eye opening at a given time from the reference time. The `.STATEYE` specifies how to retrieve the reference time for `VBOX` and `WorstBits` measurements.
- `HBOX` measurement first captures the voltage at which the horizontal maximum eye opening is given as reference. Then `HBOX` measures horizontal eye opening at given voltage from the reference voltage.
- `Vboundary port_idx[time=val][tol=val]` specifies the measurement port, time, and tolerance for the vertical eye boundary where the maximum possible voltage swing is above probability tolerance. If you do not specify time, PrimeSim HSPICE measures the maximum eye boundary and reports the time at which the maximum value is given. `eyetype=1` (default) specifies a PDF-based eye boundary measurement. `eyetype=2` specifies a bit error rate-BER-based eye boundary measurement.
- `VREF` measurement returns the reference voltage level at which the eye diagram gives the maximum horizontal eye opening.
- `CROSSTALK probe_port_idx` lists the magnitude of the crosstalk noise from each incident port observed at the specified `probe_port_idx`.
- `TREF` measurement returns the reference time at which the eye diagram gives the maximum vertical eye opening.

Vertical/Horizontal Eye-Opening Tolerance

Tolerance for eye width/height measurements means that the eye is considered open where PDF/BER values are below the tolerance value, where the default is `tol=1e-20`.

Without random jitter, changes to the tolerance value have little effect on the eye-opening result. Confirm this by viewing vertical/horizontal PDF/BER cross sections (eyeT, eyeV, bathtubT, and bathtubV). Then, if you add random jitter to the eye, you can see the usefulness of the `tol=val` control since random Gaussian distribution is unbounded. Thus PDF/BER never reach zero.

Worst Bits Output Format

In the worst bits output, the number within parenthesis is the incident port index which becomes important when you have multiple data lanes that include the effect of crosstalk.

In case of a multiple data lane system, the worst bits result that looks something like:

```
(1) 001111111100 (2) 111101111101
```

This means that you will get the worst possible eye edge with this bit combination at the two incident inputs.

In cases that have different bit lengths, the worst bits result that looks something like:

```
(1) 011111 (3) 1111111111111101
```

The shorter bit sequence is left padded with don't cares (-):

```
Incident port 1 bit sequence: -----01111  
Incident port 3 bit sequence: 1111111111111101
```

The values of the preceding bits do not matter and will still produce the worst bits.

Understanding the Worst Pattern Measurement Result

The worst pattern catcher results are stored in both the log (*.lis) file and measurement (*.mste0) file. Following is the example of the result pattern set.

```
(1:241) 10101001011110011111 (2:36359) 00000100101010010110
```

The worst patterns set above consists of two N patterns where N is the number of incidents including main signal and bits from crosstalk aggressors. Each pattern contains

header part and body (pattern) part. The header part contains 1, 2 or 3 numbers depending on the StatEye analysis set up as follows:

1. When using the multilevel (PAM-N) logic, the header format contains 3 numbers:

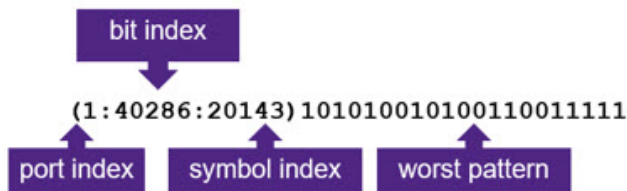
```
(port_index : bit_index : symbol_index)
```

where,

- port_index is the input port index.
- bit_index is the bit position of the right most bit of the worst pattern.
- symbol_index is the symbol position corresponding to the bit position.

For example, [Figure 10](#) indicates that, the signal from port-1 produces the worst waveform situation at 40286-th bits, which means the 20143-th symbol (PAM-4 in this case). And the worst pattern displays the pattern towards the worst point. Here, the right most bit represents the 40286-th bit.

Figure 10 Using the multilevel (Pam-N) logic



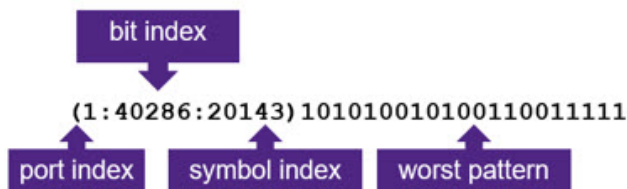
2. When using system clock-based DCD (DCD_mode=2), the header contains 2 numbers:

```
(port_index : bit_index)
```

where,

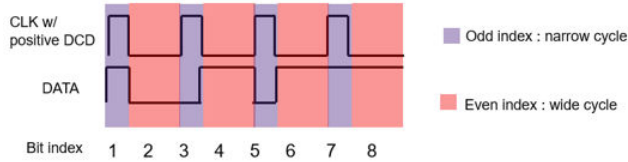
- port_index is the input port index
- bit_index is the bit position of the right most bit of the worst pattern.

Figure 11 Using system clock-based DCD



In `DCD_mode=2`, narrow and wide clock cycles toggle. When positive DCD is given to the incident port, the bits in odd indices are in narrow cycle and bits in even indices are in wide cycle.

Figure 12 Narrow and Wide Cycles



- Other than these conditions, the header contains 1 number where `port_index` is the input port index.

Understanding n-UI Accumulated Clock Period Jitter Measurements (RMS/PP)

The root mean square (RMS) and peak-to-peak (PP) values are measured from the following period jitter distribution:

$$T = \sum_{p=m}^{m+nUI-1} (UI_p - UI_{ideal}) \text{ for } (p = 1, 2, \dots, nBit - nUI)$$

where,

T is the nUI accumulated period jitter.

UI_{ideal} and UI_p are the ideal period and the period at the p -th bit/symbol position.

nUI is the number of jitter accumulation.

n^{Bit} is the total number of bits/symbols.

For peak-to-peak jitter, the lowest target probability can be specified as a tolerance where the default is `tol=1e-20`.

Note:

This measurement is available on clock output port in numerical sequence mode.

See the `Clock_Port` and `Clock_Mode` keyword in the `.STATEYE` command reference.

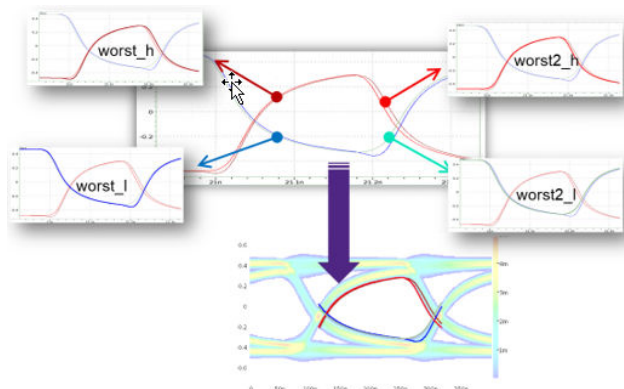
Sample Measurements

```
.StatEye ...
+ WORST_WAVEFORM=1
* 1 : worst waveform into binary (tr0) file
* 2: worst waveform into text (.txt) file
.measure stateye wst_h WorstBits 3 time=0e-12 state=high
.measure stateye wst_l WorstBits 3 time=0e-12 state=low
.measure stateye wst2_h WorstBits 3 time=70e-12 state=high
.measure stateye wst2_l WorstBits 3 time=70e-12 state=low
```

Output

```
***** Measured values for the netlist *****
worst_h= (1)000000000000000000010(2)000011111111111110101
worst_l= (1)100000111111011111101(2)110000110000010000010
worst2_h= (1)000000000000000000010(2)000011011111111110110
worst2_l= (1)100000111111011111101(2)110000010000010000001
% ls my_netlist.steuf0/
worst3_0.000000e+00_h.tr0 worst3_7.000000e-11_h.tr0
worst3_0.000000e+00_l.tr0 worst3_7.000000e-11_l.tr0
```

Figure 13 Output



Saving and Reusing Initial Transient Results

StatEye analysis shows superior performance in generating eye diagrams over conventional transient analysis. However, when a system has a large number of circuit elements and/or has many incident ports, the computational time for initial pulse/step response examinations becomes significant. Therefore, it is useful to be able to save and load the initial transient analysis data. This capability is especially useful when running multiple StatEye analyses without changing network configuration, but changing noise properties and/or post process (.PRINT/.PROBE/.MEASURE) targets. Using the data

loading function you can import externally generated pulse/step responses. The `.StatEye` command takes the `Save_tr` and `Load_tr` keywords for such purposes.

For the save/load functionality, the name of data storage directory is `netlist.save#/(#:` alter number). In the storage directory, StatEye saves/seeks the following files for each purpose:

- `incident#.txt` stores pulse responses (for `Edge=1`)
- `incident#_r.txt` stores rising step responses (for `Edges=2`)
- `incident#_f.txt` stores falling step responses (for `Edges=2`)
- `incident#_eN.txt`, N is 1-4 for `Edges=4`
- `incident#_eN.txt`, N is 1-8 for `Edges=8`

where # is the index number (`PORT=n`) of the incident port element. Save/Load data files are in simple table data format. The first column is always the “time” entry then `probe#` (# index number (`PORT=n`) of the probe port element) entries follow in the order of `probe_port` keyword specification in the `.StatEye` statement.

Example (`stateye.save0/incident1.txt`)

```
time probe1 probe2
0.0000000000e+00 -4.9945800254e-01 0.0000000000e+00
1.0000000000e-12 -4.9945800254e-01 2.7317300241e-17
2.0000000000e-12 -4.9945800254e-01 7.4348539204e-17
3.0000000000e-12 -4.9945800254e-01 1.3213235524e-16
...
```

Examples - Statistical Analysis Setup

Circuit Example:

```
* StatEye example circuit
* Define parameters
.param _BW=0.05n
.param trf = '_BW*0.01'
.param opfreq = '1.0/_BW'
* define ports
* Port P1 in the input or incident port
* Port P2 is the output or probe port
P1 in 0 port=1 LFSR (1 -1 0 'trf' 'trf''opfreq' 1 [5,2] )
P2 out 0 port=2
* Channel or devices to be simulated
X1 in out CONN
* Stateye command
.stateye T='_BW' trf='0.1*_BW' incident_port=1 probe_port=2
* Simulation options
.options post accurate
```

```
* Subcircuit definition
.subckt CONN _in _out
Rtest _in _out 50 RS=1e-3 $ skin effect resistor
Cin _in 0 0.3p
Cout _out 0 0.3p
.ends
* print and probe statements to output stateye results
* output eye diagram in color and black and white at the probe port
* output bit error rate (ber) at the probe port
.probe stateye eye(2) ber(2) eyeBW(2)
.print stateye eye(2) ber(2) eyeBW(2)
* print and probe statements to output bathtub curves
* Output bathtub curve at a time point and bathtub curve at a voltage
point
.probe stateye bathtubT(2,1e-11) bathtubV(2,0.0)
.print stateye bathtubT(2,1e-11) bathtubV(2,0.0)
* Print and probe statements to output the eye size
* Output eye cross section at a time point and at a voltage point
.probe stateye eyeV(2,0.0) eyeT(2,1.0e-11)
.print stateye eyeV(2,0.0) eyeT(2,1.0e-11)
* Measurement statements
* measure the vertical eye size at a time point
* measure the horizontal eye size at a voltage point
* measure the worst bits at a time point
.measure stateye veyel Veye 2 time=1e-11 tol=1e-10
.measure stateye heyel Heye 2 volt=0.0 tol=1e-10
.measure stateye worstl WorstBits 2 time=10p
.end
```

The PrimeSim HSPICE demonstration example, stateye_ex1.sp, is in
\$installdir/demo/hspice/si.

Example – StateEye Crosstalk using W Element

Circuit Example:

```
* superposing LTI responses

P1 in1 0 port=1
+ LFSR (1 -1 0 '0.01*_BR' '0.01*_BR''1.0/_BR' 1 [13,5,2] )
P2 in2 0 port=2
+ LFSR (1 -1 0 '0.01*_BR' '0.01*_BR''1.0/_BR' 1 [15,14,6,1] )
P3 in3 0 port=3
+ LFSR (1 -1 0 '0.01*_BR' '0.01*_BR''1.0/_BR' 1 [14,13,2,1] )

P4 out1 0 port=4
P5 out2 0 port=5
P6 out3 0 port=6

.param _BR=0.1n
```

```
.stateye T='_BR' trf='0.01*_BR'
+ incident_port= 1 2 3
+ probe_port= 4
+ save_tr=ascii

.opt post accurate
.probe stateye eye(4)
.measure stateye xtalk4 crosstalk 4

W1 in1 in2 in3 gnd out1 out2 out3 gnd FSmodel=line3 N=3 l=0.1
+ delayopt=3
+ includegdimag=yes

.material diel dielectric er=4.3, losstangent=0.02
.material copper metal conductivity=57.6meg

.shape rect rectangle width=400e-6
+height=40e-6
+nw=20 nh=20

.layerstack stack1
+layer=(copper,10e-6)
+layer=(diel,200e-6)

.fsoptions opt1 printdata=yes computegd=yes computers=yes

.model line3 W Modeltype=fieldsolver,
+layerstack=stack1,
+fsoptions=opt1,
+Rlgcfile=line2.rlgc,
+conductor=(shape=rect,origin=(0,100e-6),material=copper)
+conductor=(shape=rect,origin=(500e-6,100e-6),material=copper)
+conductor=(shape=rect,origin=(1000e-6,100e-6),material=copper)

.end
```

Ports 1, 2, and 3 are incident ports. Ports 4, 5, and 6 are termination ports; and port 4 is designated as probe port in the StatEye analysis.

The crosstalk measurement on port 4 is written in the `.mste0` file and lists the magnitude of the crosstalk noise from each.

The PrimeSim HSPICE demonstration example, `xt_meas.sp`, is in `$installdir/demo/hspice/si/stat_xtalk`.

Determining Number of Bits Simulated in Statistical and Bit-by-bit Modes

This section covers the following topics

- [Statistical Mode](#)
- [Bit-by-bit Mode](#)
- [Pulse Superposition and Edge Mode](#)

Statistical Mode

The number of bits simulated is directly related to the length of the pulse response. The pulse response length is reported in the log file (.lis). For example if the length reported is 8.5T where T is the length of the UI, the number of bits simulated is $2^{8.5} = 362$ bits. To increase the number of bits, use keyword `ISI_DEPTH` (Iter-Symbol Interference Depth) in the `.STATEYE` command. Specifying `ISI_DEPTH=20`, the number of bits simulated is $2^{20}=1048576$.

Note:

By default, PrimeSim HSPICE determines the length of the time window (8.5T) based on settling of the pulse response within a certain tolerance.

For example:

When `ISI_DEPTH=5`, all possible bit patterns simulated are:

```
00000
00001
00010
00011
.
.
11100
11101
11110
11111
```

The `.STATEYE` analysis produces the output of all possible patterns above and generates corresponding eye diagram. This can be thought of as a sequence of these bits combination.

The length of the sequence above becomes $2^{ISI_DEPTH}=32$ bits

The `.STATEYE TRAN_INIT` option limits the initial transient to 60 UIs. This would limit the `ISI_DEPTH` in case `ISI_DEPTH` is specified higher than `TRAN_INIT`.

Note:

The first all zero does not practically occur in PRBS (LFSR cannot have seed=0). But statistically, all zero and ones are possible patterns.

Bit-by-bit Mode

When port elements are specified as LFSR or PAT sources, by default `.STATEYE` analysis is run in bit-by-bit mode. The length of the simulated pattern determines the number of bits simulated.

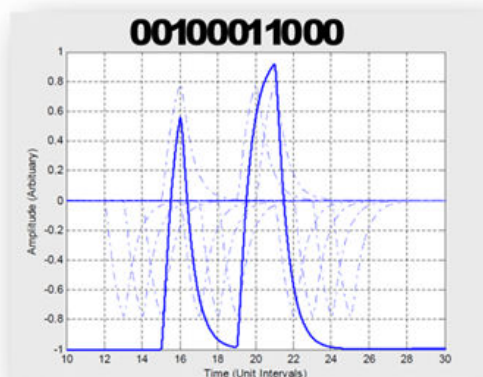
In the STATEYE transient mode (`mode=tran`), the number of bits is set by default to 1000 (`MAX_BITS=1000`). When edge mode (`mode=edge`) is used the number of bits is $2^{21} = 2097152$ (approximately 2million bits). `MAX_BITS` and `PATTERN_REPEAT` can be increased if more bits are desired. For more information, see keywords `PATTERN_REPEAT` and `MAX_BITS` in the `.STATEYE` section of the *PrimeSim Continuum Reference Manual: Commands and Control Options*.

Switching from bit-by-bit mode to statistical mode can be achieved by adding `MODE=STATISTICAL` in the STATEYE analysis.

Pulse Superposition and Edge Mode

In `.STATEYE` edge mode, the edge superposition method is used. The pulse response is generated based on the superposition of multiple edges. If specified, jitter with its corresponding variance is then added to the pulse. The STATEYE analysis convolves the distribution of the jitter on top of the ISI based PDF. Same treatment of jitter is implemented in the STATEYE transient mode.

Figure 14 Edge superposition method

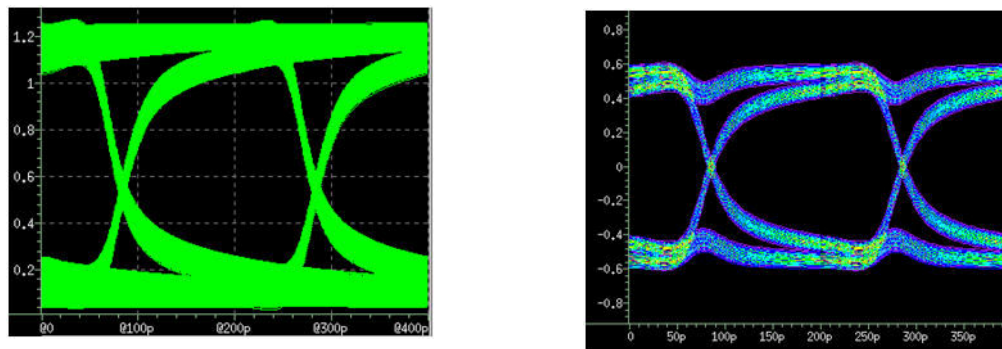


Edge Control

Statistical eye analysis method generates eye diagrams by superposing single bit responses based on symbols in a given bit stream. While the statistical approach significantly reduces the computational cost for eye diagram generation compared to the conventional transient-based approach, it may not be accurate when the target system is strongly nonlinear.

Typical accuracy degradation occurs when a nonlinear buffer is imbalanced and the negative pulse response becomes different from a positive one. In these cases, an eye diagram becomes virtually nonsymmetric (Figure 15). As a result, the conventional statistical method is unable to capture an eye diagram with great accuracy due to imbalanced buffer.

Figure 15 (Left) Vertically nonsymmetric eye diagram due to an imbalanced buffer; generated by transient analysis with a pseudo random bit stream (PRBS) (right) Conventional .STATEYE result with imbalanced buffer

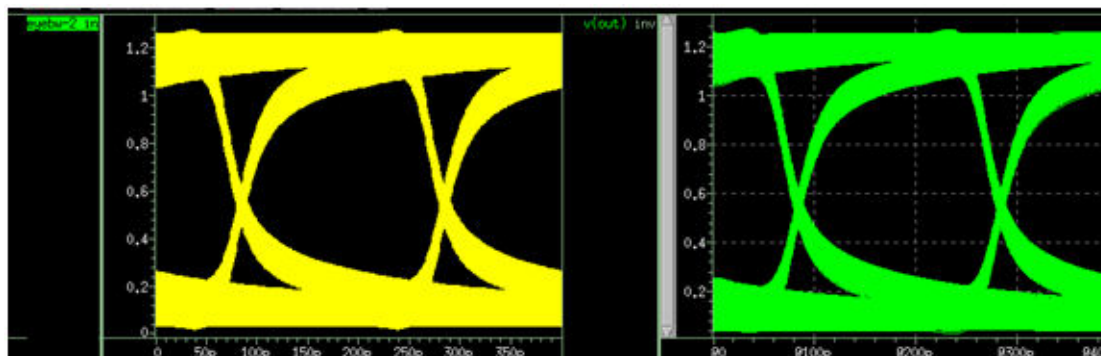


To overcome the difficulty, you can use the `Edge` keyword in the `.STATEYE` command. Here you can represent a pulse response, $P(t)$, by using the rising edge step, $R(t)$, and falling edge step, $F(t)$, responses with T as the unit interval (width of the input pulse):

$$P(t) = R(t) + F(t-T)$$

Here, since nonlinear buffers usually have circuit-bias dependencies, you must specify these edge response acquisitions at desired bias points. Then the statistical process must correctly account for these bias points. Because of the double edge-based statistical computations, the `.STATEYE` command realizes higher levels of consistency with conventional transient-based results (Figure 16).

Figure 16 Comparison between .STATEYE and transient-based eye diagrams. On left is the eye diagram of .STATEYE with double edge mode; the right shows the conventional transient-based eye diagram



The following sections present these edge-related topics:

- [Statistical Eye Full Transient Mode](#)
- [Example - Controlling Edges](#)
- [Example - Run StatEye with Precomputed Transient Waveform](#)
- [Separate Rise Time and Fall Time Settings](#)
- [Multi-Edge Responses for Accurate Modeling of Nonlinearity](#)

Statistical Eye Full Transient Mode

Statistical eye diagram analysis algorithm consider a systems' inter-symbol interference (ISI) and other linear memory effects. However, it is difficult to consider buffers nonlinear memory effects since superposition theory gradually breaks down as the amount of nonlinearity increases. By using multiple edge patterns, the StatEye accuracy can be increased dramatically. However, ideally, there should be a method to fully verify a nonlinear system that is faster than traditional transient analysis. The StatEye full transient mode will run a transient analysis and generate the PDF eye diagrams, and BER eye diagrams. The full transient mode uses all the available StatEye features such as random (RJ) and deterministic (DJ) and equalization using pre- and de-emphasis. The full transient mode is compatible with transistor level, IBIS, and IBIS-AMI models.

By Specifying `MODE=TRAN`, StatEye uses full transient analysis mode instead of edge mode. The `EDGE` keyword will be ignored when full transient analysis mode is specified.

Transient Probe Data Information about .STATEYE in Transient Mode (mode=tran)

When .STATEYE analysis is running in transient mode (with `mode=tran`), `.probe` statements with `tran` notation will be stored in `netlist.e0.tr#` (#: alter number) file. And

when IBIS-AMI filtering objects are attached to incident and/or probe port, transient probe data in `netlist.e0.tr#` mean:

1. Probe data at incident terminals are waveforms filtered by transmitter side AMI model (output from AMI).
2. Probe data at probe terminals are waveforms to be filtered by receiver side AMI model (input to AMI).

To see the time domain waveforms filtered by receiver side AMI object, use:

```
.probe stateye eyeUF(M, N)
```

Where `M` is the port index, and `N` is the bit position.

Example - Controlling Edges

```
.param BW=0.2n
.param trf = 'BW*0.01'
.param opfreq = '1.0/BW'
.param vdd = '3.0'

.STATEYE T='BW' TRF='trf' incident_port=1 probe_port=2
+ V_HIGH='vdd' V_LOW=0.0
+ edge=2
```

Example - Run StatEye with Precomputed Transient Waveform

Circuit Example:

```
* transient

.opt post probe accurate wdf=1

.param UI=0.1n
.param trf = 'UI*0.1'
.param opfreq = '1.0/UI'
.param tstop='500*UI'

*** dummy channel ***
P1 in 0 port=1
P2 out 0 port=2

.stateye
+ T='UI' trf='trf'
+ incident_port=1 probe_port=1 2
+ mode=tran
+ STREAM_FILE='tran_wdf.tr0'
+ STREAM_SIGNAL='v(inp,inn)' 'v(outp,outn)'
+ ignore_bits=100
```

```
.probe stateye eye(1) eye(2)

.end
```

The PrimeSim HSPICE demonstration example, `ste_wdf.sp`, is in `$installdir/demo/hspice/si/tran_stateye`.

Separate Rise Time and Fall Time Settings

In double edge mode (`Edge=2`), StatEye can handle vertically asymmetric eye diagrams by using `TR/TF` keywords within the `.STATEYE` command and the existing `TRF` keyword. You must specify either `TR/TF` or `TRF` in the `..STATEYE` command. If you specify both `TR/TF` and `TRF`, the last keyword takes priority.

Multi-Edge Responses for Accurate Modeling of Nonlinearity

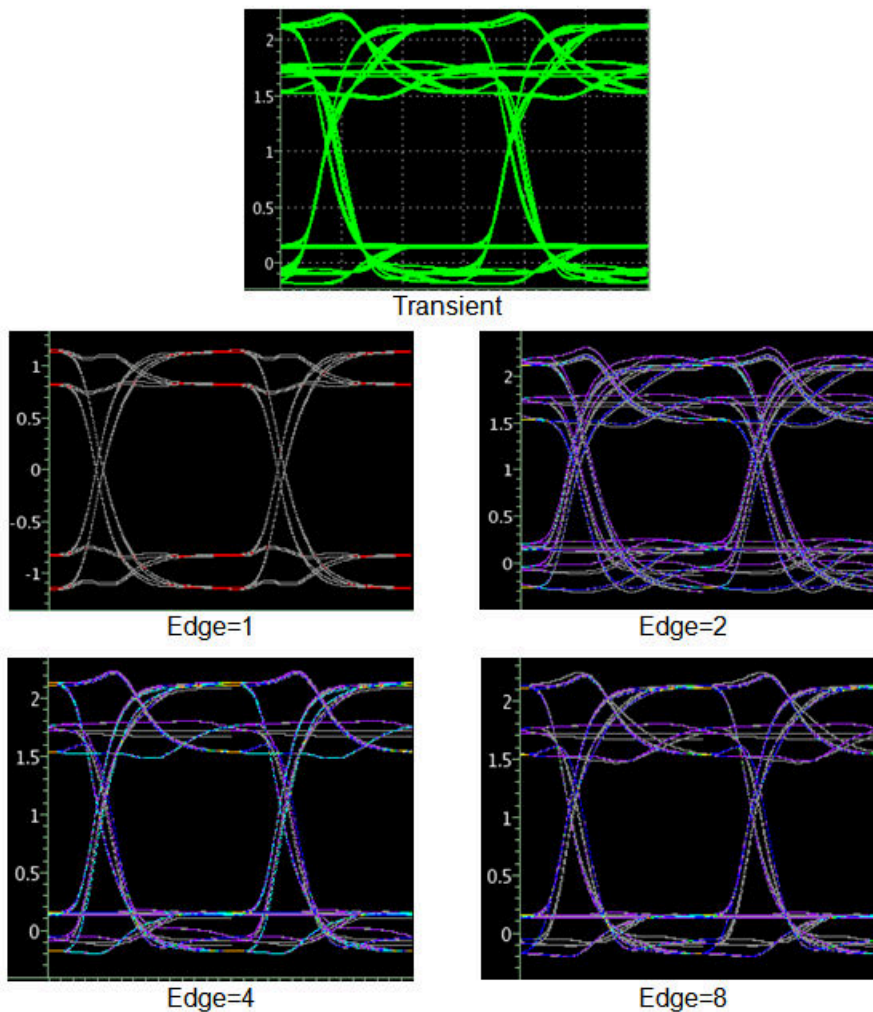
PrimeSim HSPICE StatEye performs initial time domain analyses with predetermined edge patterns for each `EDGE` mode for each incident port (shown in [Table 2](#)). Then, StatEye applies the best-matched pattern response in the statistical process.

Table 2 *Edge value and resulting pattern*

Edge Values	Edge Patterns
1 (single pulse)	01000...
2 (rise/fall steps)	01111..., 1000...
4	0111..., 1000..., 01000..., 011000...
8	0111..., 1000..., 01000..., 011000..., 0111000..., 01111000..., 010111..., 010100...

For a system with highly nonlinear memory effect in drivers and receiver circuits, [Figure 17](#) shows a comparison of resulting eye diagrams from transient analysis versus StatEye with `Edge=1, 2, 4, and 8`. As the number of edge patterns increases, you can model higher nonlinearity accurately.

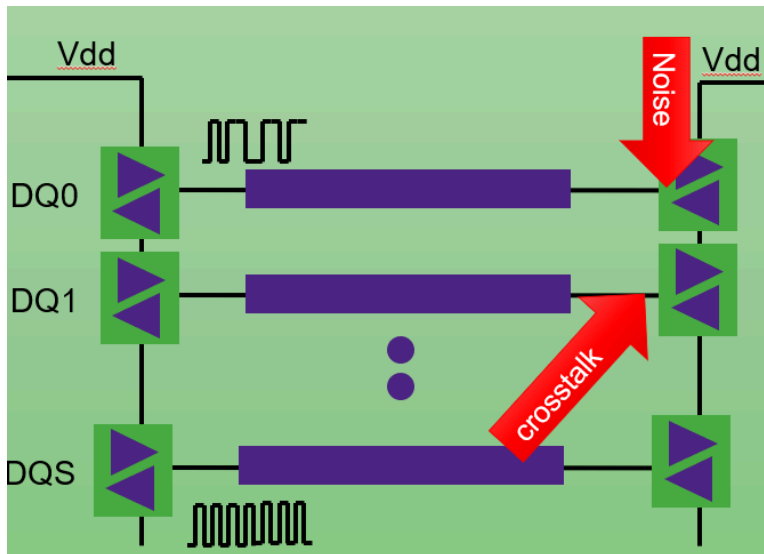
Figure 17 Edge mode comparisons with transient analysis-based eye diagram (top)



Capturing Steady State Noise for Power Integrity

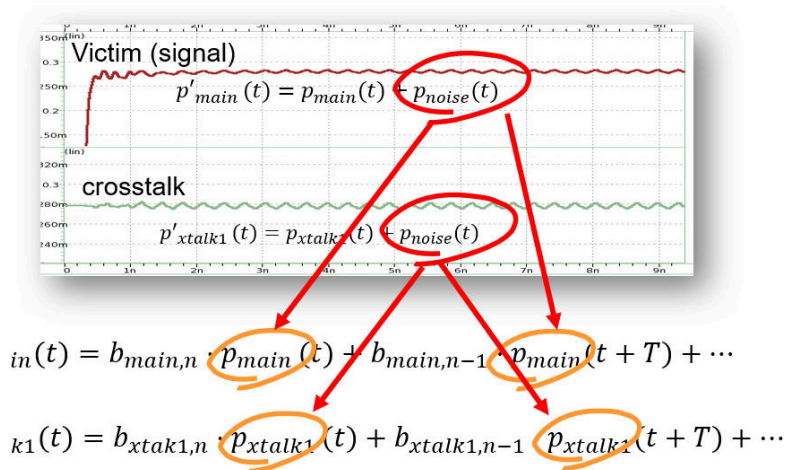
When the power supply for a system has voltage noise, the noise causes jitter on data transfer on the channel. Such jitter is called power supply induced jitter (PSIJ). There are multiple causes of the PSIJ, such as clock effect on the voltage regulator, simultaneous switching noise (SSN) and so on. Many of these are aligned with system clock period. Since these PSIJ effects appear independently of input data pattern, PrimeSim HSPICE treats this as steady state noise.

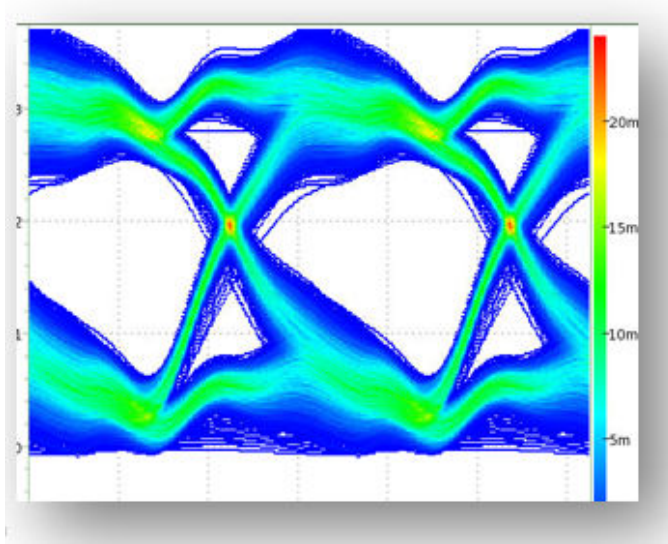
Figure 18 System with steady state noise



When input data independent steady state noise appears, StatEye's edge superposition algorithm requires special handling of such noise components; otherwise effect may be over counted because the same noise appears on the top of all the edge response waveforms as shown in Figure 19. As a result, eye diagram may become overly pessimistic as shown in Figure .

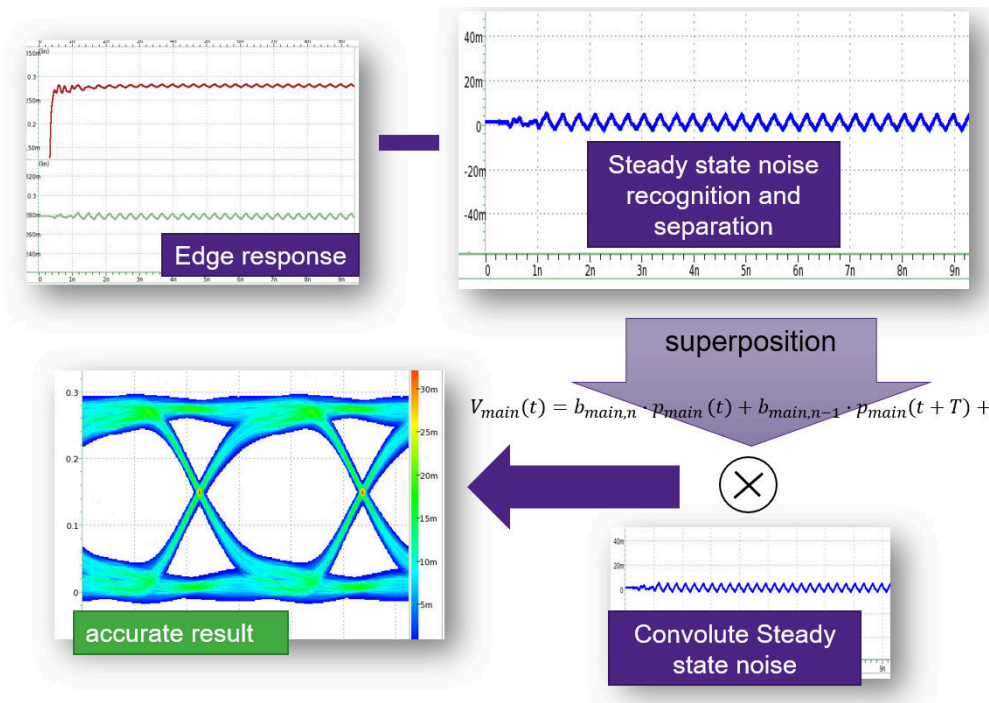
Figure 19 Steady state noise over counted in edge response superposition





To accurately account the steady state noise, StatEye introduces the `steady_state` keyword. With this keyword, StatEye precharacterizes the steady state noise, removes the noise effect from edge responses, and then performs edge superposition.

Figure 20 Analysis flow with the `steady_state` keyword



Finally, precharacterized steady state noise convolves to obtain the overall system eye diagram accurately as shown in [Figure 20](#).

- ## Example - Create SSN Eye Diagram

[illegible]

```
.pat start= b00
.pat swl=start sw

.end
```

Reusing edge responses from initial transient analysis on the original PDN circuit, StatEye generates edge superposition-based eye diagram (PDF) of the simultaneous switching noise and writes it in the `make_ssn.stev0` file. The probability distribution matures as you increase `max_pattern` in `.StatEye` command. Also, `make_ssn.steuf0/` directory contains reproduced SSN (time domain) waveform.

The PrimeSim HSPICE demonstration example, `make_ssn.sp`, is in `$installdir/demo/hspice/si/ddr_sso/MakeSSN`.

Example - ADD SSN Eye Diagram

Circuit Example:

```
.param pvdd=0.8    $Typical
.param f=1333.33   $signal frequency in MHz
.param vol =0      $ siganl output low voltage
.param voh=pvdd    $ signal output hight voltage $$we want to add noise to
                    this.
.param tper='(1/f)*1e-6'
.param tr=40p tf=40p
.param dbit='tper/2' tdq='3*tper/4'

.opt post probe
.stateye
+ T='tper/2' trf='tr'      $ UI, rise/fall time
+ incident_port=1 3 4      $ incident ports
+ probe_port=7             $ probe port
+ load_tr=ascii            $ reuse edge response
*+ max_pattern = 320        $ pattern length
*+ max_pattern = 1000       $ pattern length
+ max_pattern = 5000        $ pattern length
*+ max_pattern = 1000000    $ pattern length
+ unfold_length = 500      $ length of bits for eyeUF
+ align_eye=sync           $ eye alignment
+ XTALK_TYPE=DDP           $ crosstalk type
+ VD_RANGE = 1.6           $ vertical display range
+ edges = 8                $ use 8 edge patterns
+ initial_low = 'tper'     $ start edge response w/ low
+ TD_IN = '10*tper'        $ initial time for edge response
+ t_resolution = 400       $ horizontal resolution
+ v_resolution = 400       $ vertical resolution

.probe stateye eye(7)
.meas stateye vref7 Vref 7 tol=1e-16
.meas stateye hbox7 hbox 7 volt = 60e-3 tol=1e-16
```

Reusing edge responses from initial transient analysis on the original nonlinear netlist, StatEye performs edge superposition based eye diagram generation. Then second alter adds SSN jitter on victim output.

The PrimeSim HSPICE demonstration example, `add_ssn.sp`, is in `$installdir/demo/hspice/si/ddr sso/AddSSN`.

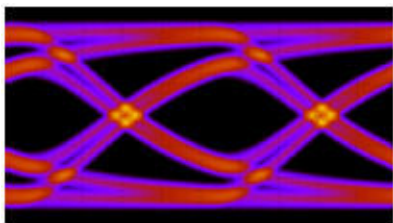
StatEye analysis with the double-edge mode is capable of evaluating duty cycle distortion (DCD) effect. DCD occurs when transmitter drivers show unwanted fluctuation in their logical threshold. When the logical threshold value shifts higher than its ideal value, a transmitter's output has a longer low state than expected ([Figure 21](#)).

Figure 21 Illustration of the DCD effect with 10% distortion

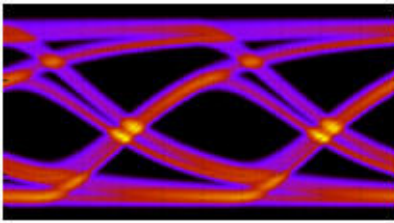


DCD typically displays the eye diagrams. StatEye analysis with the double-edge mode (Edge=2) is capable of evaluating this effect.

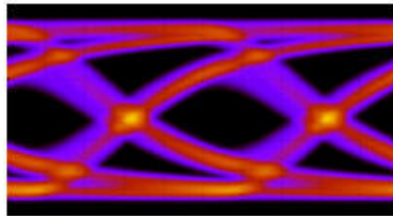
Figure 22 DCD_TYPES



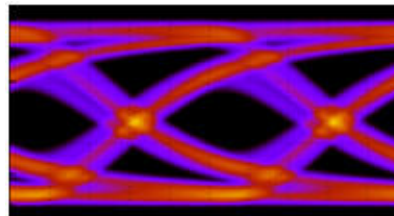
(0) No DCD



(1) 10% constant DCD



(2) 10% triangular DCD variation



(3) 10% sinusoidal DCD variation

Port Element Syntax for Duty Cycle Distortion

An incident port element takes `DCD` and `DCD_TYPE` keywords to specify the duty cycle distortion. See [DCD Example](#).

```
Pi n1 n2 port=1 + [DCD=value] [DCD_TYPE=0|1|2|3]
```

DCD	Aids in .STATEYE analysis; specifies peak percentage of the duty cycle distortion (DCD). Default is 0.
DCD_TYPE	Aids in .STATEYE analysis; specifies variation type. Default type for non-zero DCD is 1 (constant). • 0: No DCD • 1: Constant DCD • 2: Uncorrelated triangular DCD variation • 3: Uncorrelated sinusoidal DCD variation

DCD Example

```
* StatEye w/ 10% DCD
.param _BW=0.05n
.param trf = '_BW*0.01'
.param opfreq = '1.0/_BW'

P1 in 0 port=1
+ DCD=10 DCD_TYPE=1          $ adds 10% constant DCD
P2 out 0 port=2
X1 in out CONN

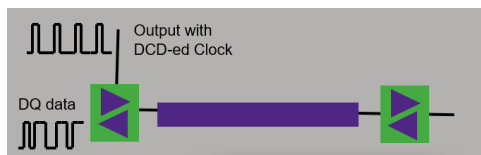
.stateye T='_BW' trf='0.1*_BW' incident_port=1 probe_port=2
+ edges=2
.opt post accurate
.probe stateye eye(2)
.print stateye eye(2)

.subckt CONN _in _out
Rtest _in _out 50 RS=1e-3 $ skin effect resistor
Cin _in 0 0.3p
Cout _out 0 0.3p
.ends
.end
```

Capturing DCD Effect on System Clock

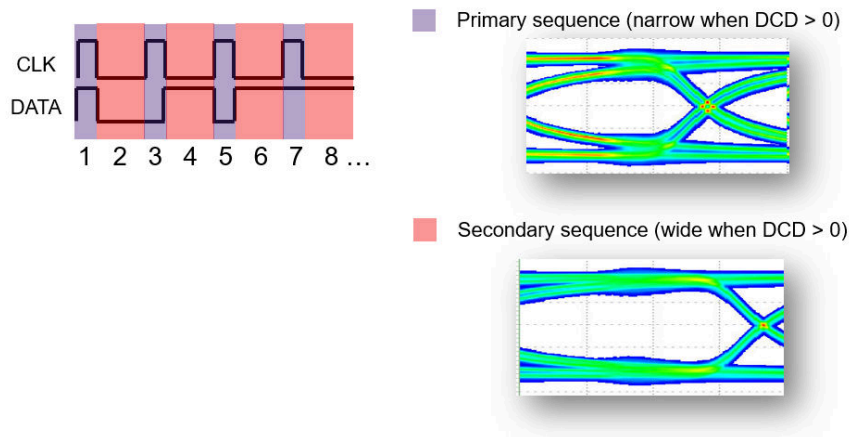
When a clock that contains duty cycle distortion (DCD) is fed to the output buffer which drives the channel, that also causes distortion in the channel output as shown in [Figure 23](#).

Figure 23 System with DCD-ed clock



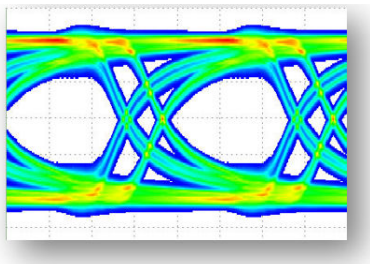
Under such condition, primary bit (1st 2nd 3rd ...) sequence is in narrower cycle and secondary bit (2nd, 4th, 6th ...) sequence is in wider cycle or vice versa depending on the sign of DCD on the system clock.

Figure 24 Primary and secondary bit sequences under DCD-ed clock



To capture this effect, StatEye analysis with the designated keyword `DCD_mode=2` generates probability density function (PDF) eye diagrams for primary and secondary sequence separately as shown in Figure 24. And these two PDFs will be merged. This method prevents inter-sequence crosstalk from being miscounted; therefore resulting eye diagrams as well as worst bit pattern measurements accurately capture the system clock DCD effect. The `DCD_mode=2` supports constant DCD only.

Figure 25 Eye diagram under DCD-ed system clock



Periodic and Random Jitter Effect

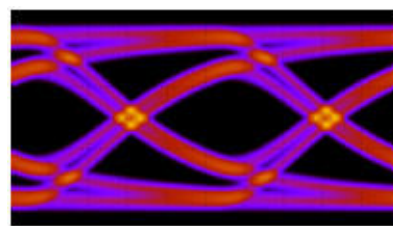
The following sections present these topics:

- [Periodic Jitter](#)
- [Random Jitter Noise Effect](#)
- [Specifying Random Jitter by Peak-to-Peak Value with BER](#)
- [Specifying Custom Jitter Distribution](#)
- [Port Element Syntax for Jitter](#)

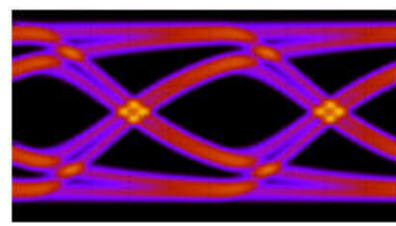
Periodic Jitter

Periodic jitter is an effect of unwanted periodic voltage/time interferences added to the target signal. For example, an electromagnetic field from a power supply line adds a low frequency sinusoidal voltage variation to the target signal. When a periodic interference appears at an incident port, the shaping of the jitter is due to channel characteristics before appearing at probing points (see [Figure 26](#)). StatEye analysis can evaluate this type of effect. Incident and/or port elements take `PJ` and `PJ_TYPE` keywords to specify periodic jitter.

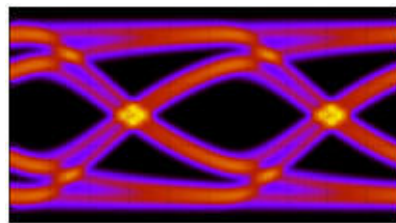
Figure 26 StatEye with periodic jitter at transmitter (Tx) side



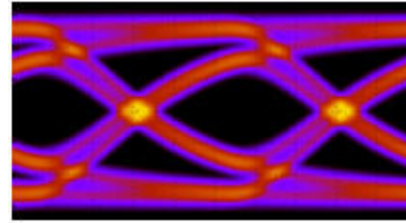
(0) No perjitter



(1) 100-mV constant shift at Tx



(2) 100-mV pp uncorrelated triangular Tx-PJ



(3) 100-mV pp uncorrelated sinusoidal Tx-PJ

Random Jitter Noise Effect

StatEye can also display random noise effects (see [Figure 27](#)) by adding `RJ` and `VN` keywords to port elements. For descriptions of keywords, see [Port Element Syntax for Jitter](#).

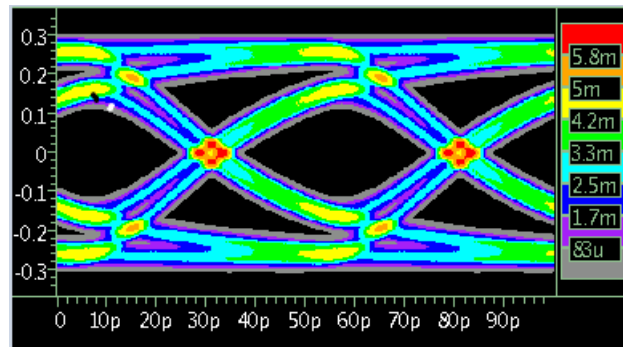
StatEye analysis incident and probe ports include random Gaussian jitter when you use the `[RJ=value]` keyword. In [Equation 10](#), the RJ value specifies the root mean square (RMS) value of the distribution (known as the standard deviation, σ), of the following Gaussian distribution:

Equation 10

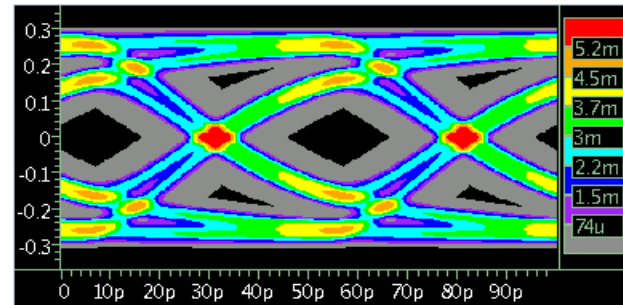
$$f(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{z^2}{2\sigma^2}}$$

The `VN` keyword adds vertical uncertainty by specifying the standard deviation of the Gaussian random voltage noise.

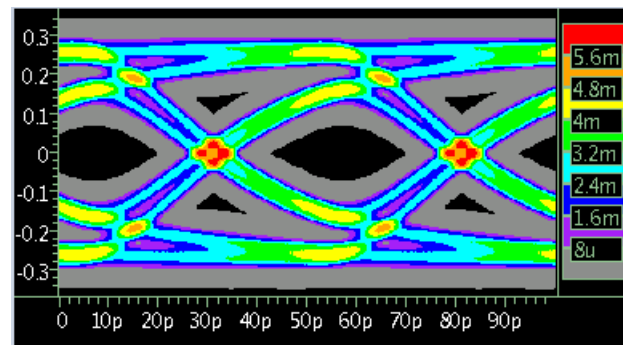
Figure 27 *StatEye with random noise effects*



Without random noise



Adding 1-p sec random jitter
to the probe port



Adding 5-mV voltage noise
to the probe port

Specifying Random Jitter by Peak-to-Peak Value with BER

Another convenient way to specify the random noise distribution is to use the peak-to-peak value associated with the specified BER. Since BER is a cumulative error probability of $f(z)$ outside of specific “peak value” as:

Equation 11

$$\text{BER} = \frac{1}{2} \int_{\text{peak_value}}^{\infty} f(z) \, dz$$

Then, the RMS jitter value can relate to the RJPP with a specific BER by using the inverse of complementary error function as:

Equation 12

$$\text{RMS Jitter} (= \sigma) = \text{RJPP} / (2\sqrt{2} \operatorname{erfc}^{-1}(2 \times \text{BER}))$$

For example, PrimeSim HSPICE allows [RJPP=value] and [RJPP_BER=value] to specify the following:

Equation 13

$$\begin{aligned} \text{RMS Jitter} &= (\text{RJPP} / 8.52978) [\text{RJPP_BER} = 1e-5] \\ &= (\text{RJPP} / 10.3987) [\text{RJPP_BER} = 1e-7] \\ &= (\text{RJPP} / 11.9956) [\text{RJPP_BER} = 1e-9] \end{aligned}$$

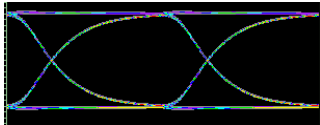
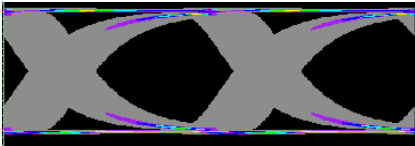
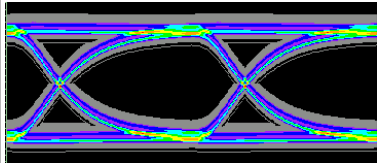
Note:

An RMS Jitter specification overrides a peak-to-peak jitter specification.

Specifying Custom Jitter Distribution

The StatEye analysis convolutes custom jitter distribution described in an external data file. A port element statement specifies the file with `JITTER_FILE=filename` keyword. The jitter file can specify either time- (horizontal) wise or voltage- (vertical) wise distribution. In the first row of the data file, “time” or “volt” can be used as x-axis variable. The second variable name must be “PDF” or “probability.”

	Time wise jitter	Voltage wise jitter
Data file name	jit_t.txt	Jit_v.txt

Data file contents	time PDF -8e-12 0.00443185 -7.6e-12 0.00791545 -7.2e-12 0.013583 -6.8e-12 0.0223945 -6.4e-12 0.0354746 -6e-12 0.053991 -5.6e-12 0.0789502 -5.2e-12 0.110921 -4.8e-12 0.149727 -4.4e-12 0.194186 -4e-12 0.241971 -3.6e-12 0.289692 -3.2e-12 0.333225 -2.8e-12 0.36827 -2.4e-12 0.391043 -2e-12 0.398942 -1.6e-12 0.391043 -1.2e-12 0.36827 -8e-13 0.333225 -4e-13 0.289692 -1.31266e-27 0.241971 4e-13 0.194186 8e-13 0.149727 1.2e-12 0.110921 1.6e-12 0.0789502 2e-12 0.053991 2.8e-12 0.0223945 3.2e-12 0.013583 3.6e-12 0.00791545 4e-12 0.00443185 4.4e-12 0.00238409 4.8e-12 0.00123222 7.6e-12 3.9613e-06	volt PDF -0.08 3.64553e-09 -0.064 1.49655e-05 -0.06 8.02981e-05 -0.056 0.000367141 -0.052 0.00143045 -0.048 0.00474927 -0.044 0.0134367 -0.04 0.0323946 -0.032 0.116512 -0.028 0.173817 -0.024 0.220972 -0.02 0.239419 -0.016 0.221207 -0.012 0.174769 -0.008 0.119678 -0.004 0.0755103 4.16334e-17 0.053991 0.004 0.0578051 0.008 0.0824237 0.012 0.117307 0.016 0.147675 0.02 0.159657 0.024 0.147323 0.028 0.115879 0.032 0.0776747 0.036 0.0443684 0.04 0.0215964 0.044 0.00895781 0.048 0.00316618 0.052 0.000953635 0.056 0.000244761 0.06 5.35321e-05 0.076 2.47305e-08
P-element statement	Px n1 n2 ... + JITTER_FILE=jit_t.txt	Px n1 n2 ... + JITTER_FILE=jit_v.txt
Result (w/o jitter)	w/ time wise jitter	w/ voltage wise jitter
		

Port Element Syntax for Jitter

```
Pi n1 n2 port=1
+ [PJ=[val1 .. valN.] ] [PJ_TYPE=[0|1|2|3|4|5|6]]
+ [RJ=[val1 ... valN]] [RJ_MEAN=[val1...valN] ] [VN=val]
+ [RJPP=val RJPP_BER=val]
+ [VNPP=val VNPP_BER=val]
+ [JITTER_FILE=filename]
```

Syntax	Description
RJ	Random jitter aids in .STATEYE analysis. Specifies an array of the real numbers for the standard deviation of the Gaussian random jitter. By default, StatEye adds no random timing jitter. RJ=val/ overrides RJPP and RJPP_BER keywords.
RJ_MEAN	Specifies an array of the real numbers for the mean values of Gaussian random jitter specified by the RJ keyword. Default mean value for nonzero RJ is zero.
PJ	Aids in .STATEYE analysis. Specifies an array of real numbers for peak-to-peak values of periodic jitter in voltage or time. Default is 0.
PJ_TYPE	Aids in .STATEYE analysis. Specifies an array of integer number for periodic jitter variation type. Default type for nonzero PJ is 3 (uncorrelated sinusoidal voltage variation). • 0: No periodic jitter • 1: Constant voltage shift • 2: Uncorrelated triangular voltage variation • 3: Uncorrelated sinusoidal voltage variation • 4: Constant time shift • 5: Uncorrelated triangular time variation • 6: Uncorrelated sinusoidal time variation
VN	Random voltage noise aids in .STATEYE analysis. Specifies an array of real numbers for the standard deviation of the Gaussian random noise. By default, StatEye adds no random noise.
RJPP	Peak-to-peak jitter
RJPP_BER	BER for the peak-to-peak jitter specification
VNPP	Peak-to-peak jitter with voltage noise (VN)
VNPP_BER	BER for the peak-to-peak jitter specification with voltage noise (VN)
JITTER_FILE	Specifies custom jitter file name

Periodic Jitter Example

```
* StatEye w/ 100mV PJ at Tx
.param _BW=0.05n
.param trf = ' _BW*0.01 '
.param opfreq = '1.0/_BW'

P1 in 0 port=1
```

```
+ PJ=1e-1 PJ_TYPE=2
P2 out 0 port=2
X1 in out CONN

.stateye T='_BW' trf='0.1*_BW' incident_port=1 probe_port=2
.opt post accurate
.probe stateye eye(2)
.print stateye eye(2)

.subckt CONN _in _out
Rtest _in _out 50 RS=1e-3 $ skin effect resistor
Cin _in 0 0.3p
Cout _out 0 0.3p
.ends
.end
```

N-Cycle Jitter Modeling for Data and Clock Signals

In the `/hspice/StatEye_clk_skew/nUI_sj.sp` directory, use the following setting in the `sj.sp` and `n_sj.sp` files:

- `UI=1n`
- Sinusoidal jitter (SJ) with `freq=1e8` (1/10UI) and `phase=90` degrees
- Long 4.5UI skew to achieve maximum DQ and DQS jitter difference (jitter `freq=1e8`)

1. The netlists settings:

`sj.sp`: Adds SJ to DQ and DQS. Alter-0 has DQ/DQS skew within a UI (`CLK_skew=0.5*UI`) and alter-1 has 4.5*UI skew

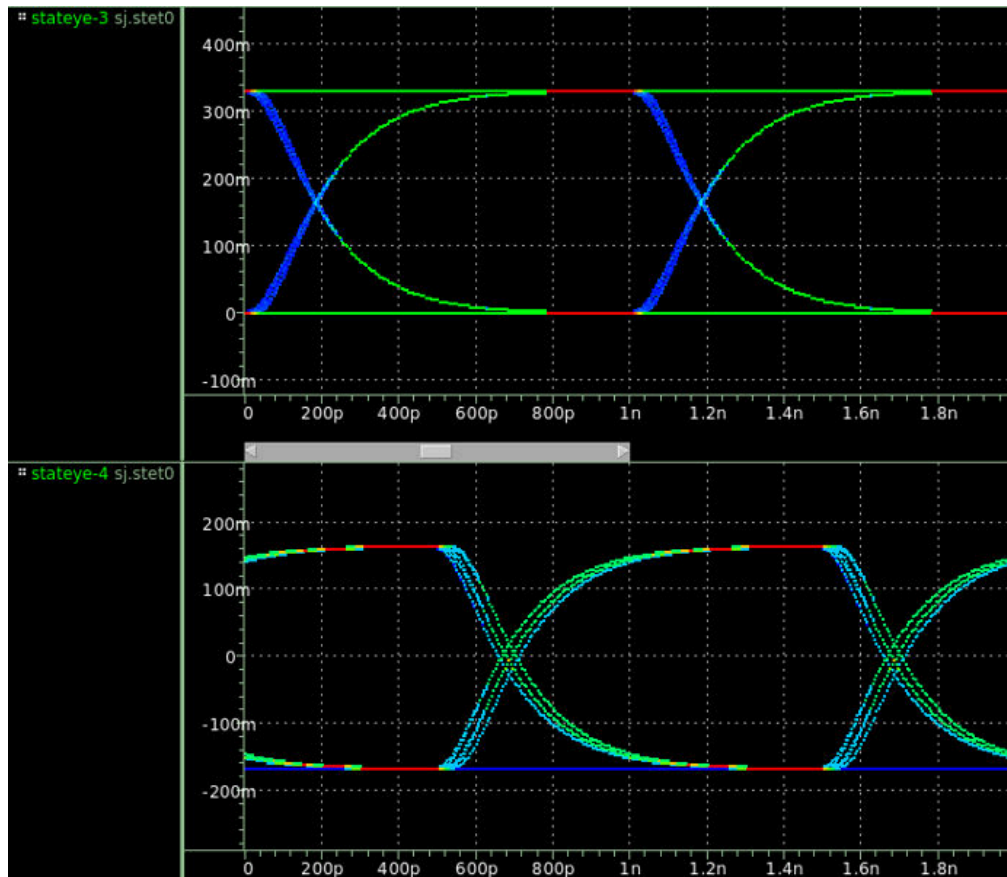
`n_sj.sp`: Same as netlist above but without jitter to compare with injected jitter

2. Use WaveView to load the presimulated results session:

```
%wv -x view_eye.sx
```

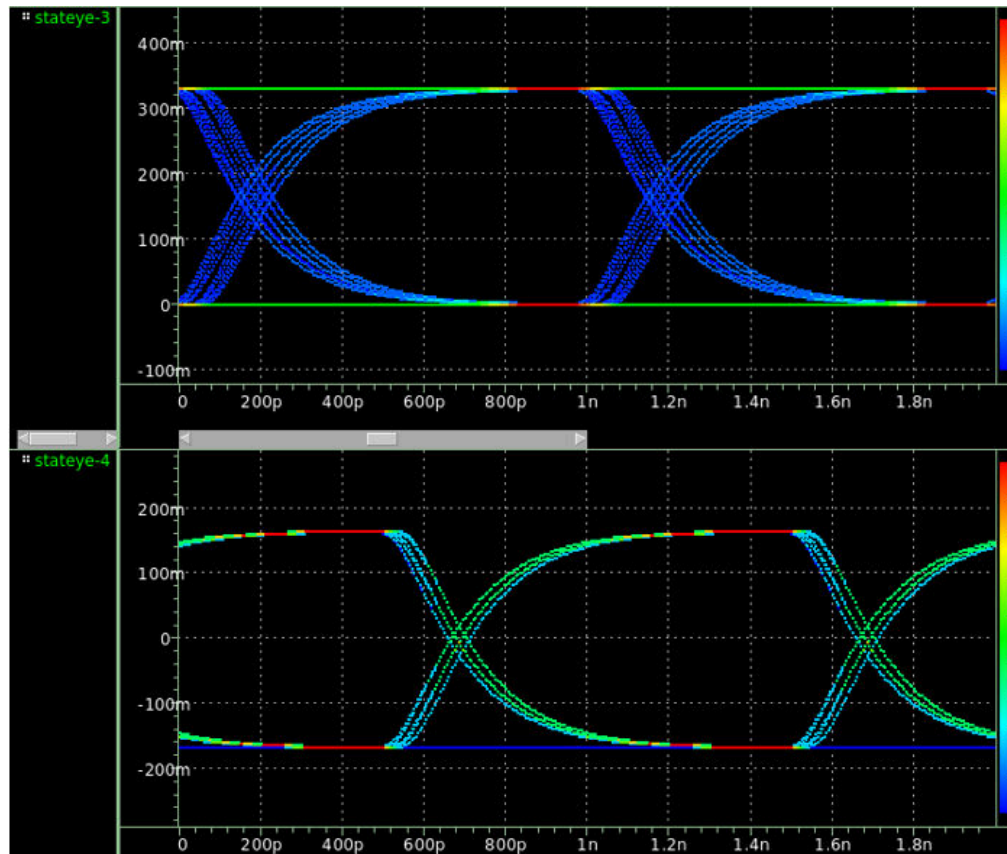
First WaveView (`alter_0`): port-3 shows no jitter, port-4 (bottom) has jitter. This is because DQ and DQS have same in-phase jitter canceling out.

Figure 28 First WaveView (alter_0)



In WaveView (alter_1), port-3 shows 2x the jitter of port-4. This is because the DQ and DQS jitter is out of phase ($4.5 \cdot \text{CLK_skew}$).

Figure 29 WaveView (alter_1)



3. Viewing the unfolded eyes waveforms of port 4: eyeUF(4,0):

```
% wv -x view_clk.sx
```

- Top: skew=0.5UI w/ and w/o jitter
- Bottom: skew=4.5UI w/ and w/o jitter

Since jitter sequence is applied from the beginning of the sequence, jitter amount of the two yellow circles match, and orange circles match. However, yellow versus orange do not match.

Therefore, jitter on clock at each timing is different between skew=0.5*UI and skew=4.5*UI.

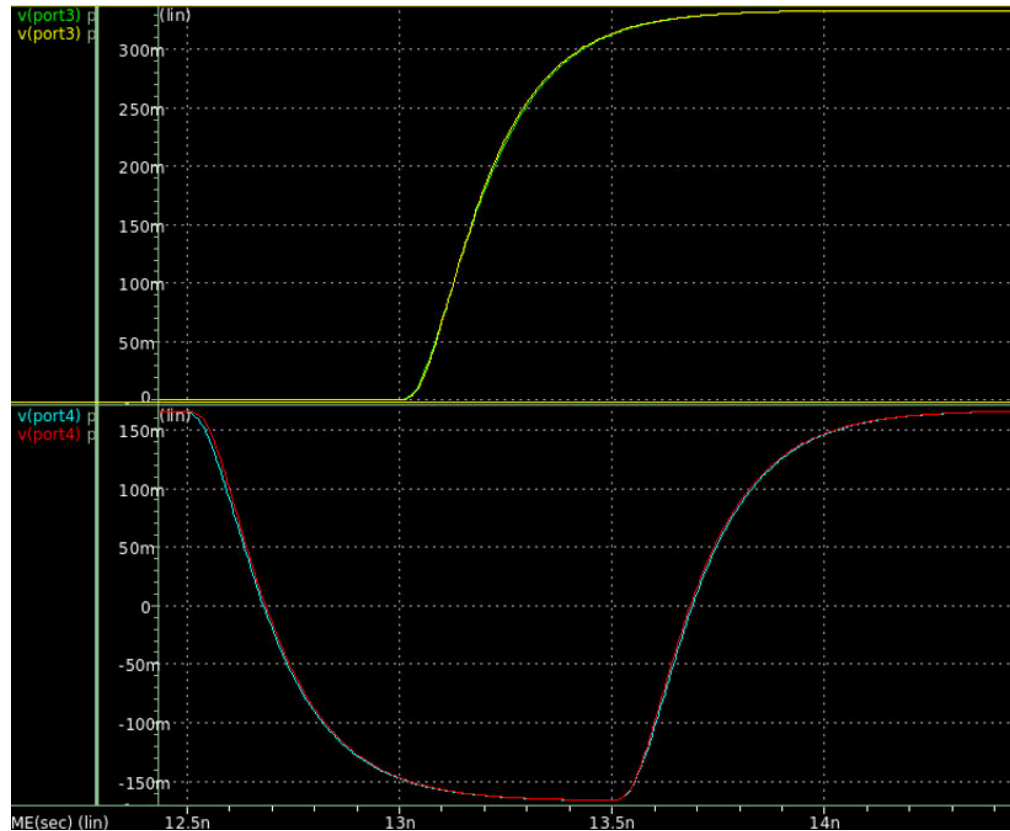
4. Jitter relationship between DQ and DQS, using unfolded waveforms from port-3 and port-4: alter-0 (0.5UI skew):

```
% wv -x view_skew0.5.sx
```

Top: DQ (w/ and w/o jitter. Bottom: DQS (w/ w/o jitter)

Since skew is within a UI, you can see the same jitter at each edge transition between DQ and DQS. Therefore, if you use the DQS as trigger, the jitter is canceled.

Figure 30 Jitter relationship between DQ and DQS

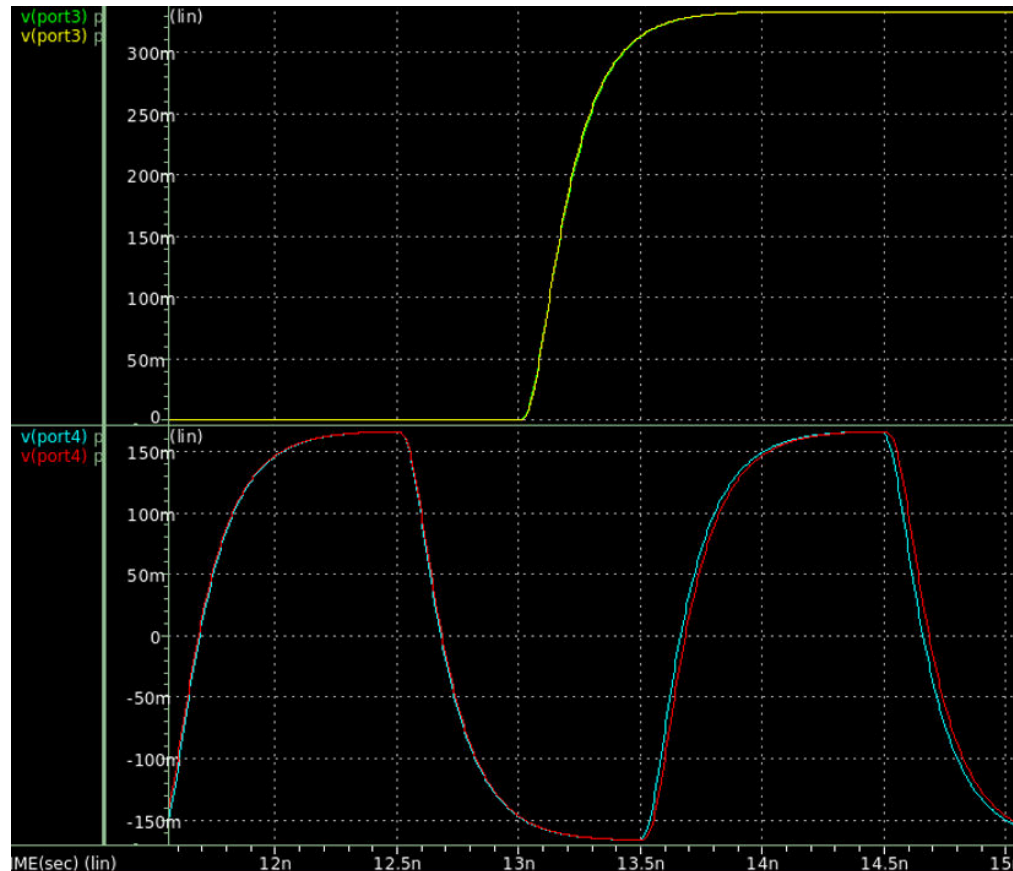


```
alter-1 (4.5UI skew):  
%wv -x view_skew4.5.sx
```

Top: DQ (w/ and w/o jitter) bottom DQS (w/ and w/o jitter)

Since skew is within more than UI, the different jitter is observed at each edge transition between DQ and DQS.

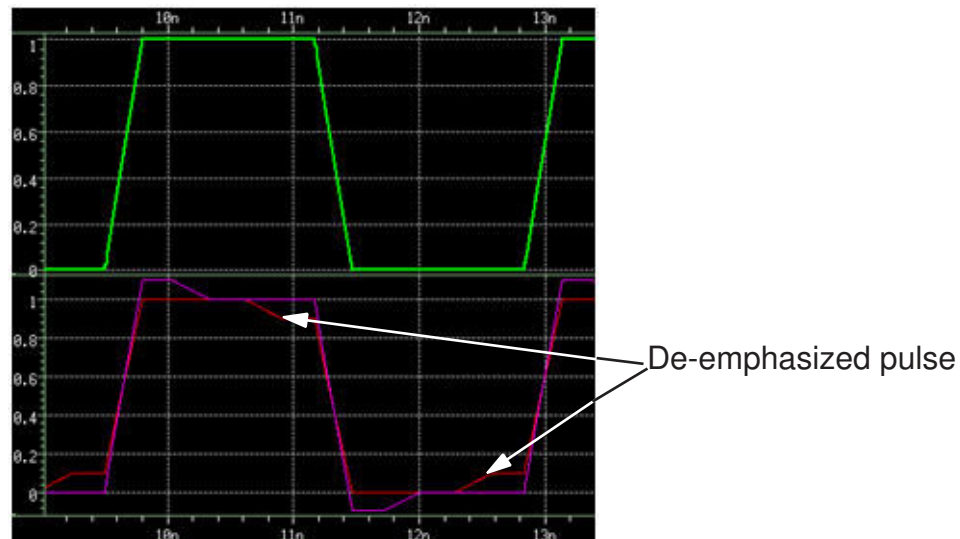
Figure 31 Edge transition between DQ and DQS



Pre-Emphasis and De-Emphasis

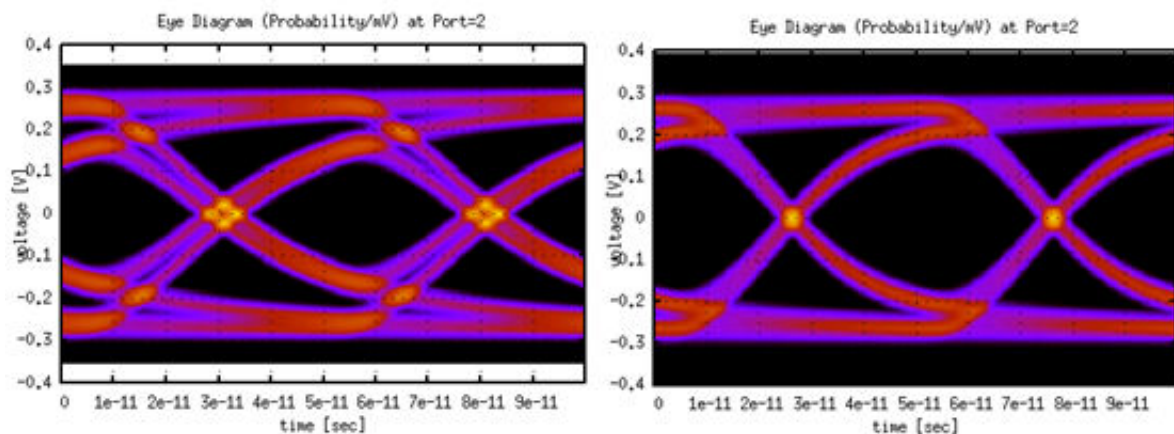
Eye diagrams enable you to estimate and evaluate high-speed serial data links. As the data rate increases, however, the design goal of sufficient eye opening becomes difficult to achieve. In over-10 gigabit-per-second (bps) designs, the loss of high frequency components in the rising and falling edges of the digital bit streams becomes significant. To compensate for the loss, preshaping the incident waveform to increase the amount of high frequency components becomes essential. You can use techniques called *pre-emphasis* and *de-emphasis* for this purpose. Pre-emphasis adds high frequency components to the rising and falling edges of the incident waveforms as shown in purple in Figure 32. The de-emphasis voltage swing from the waveform portions specifies time after the rising and falling edges. The red waveform in Figure 32 illustrates that the rising and falling edges are also de-emphasized.

Figure 32 Emphasized pulse sources: (green) original waveform; (purple) 10% pre-emphasized pulse; (red) 10% de-emphasized pulse



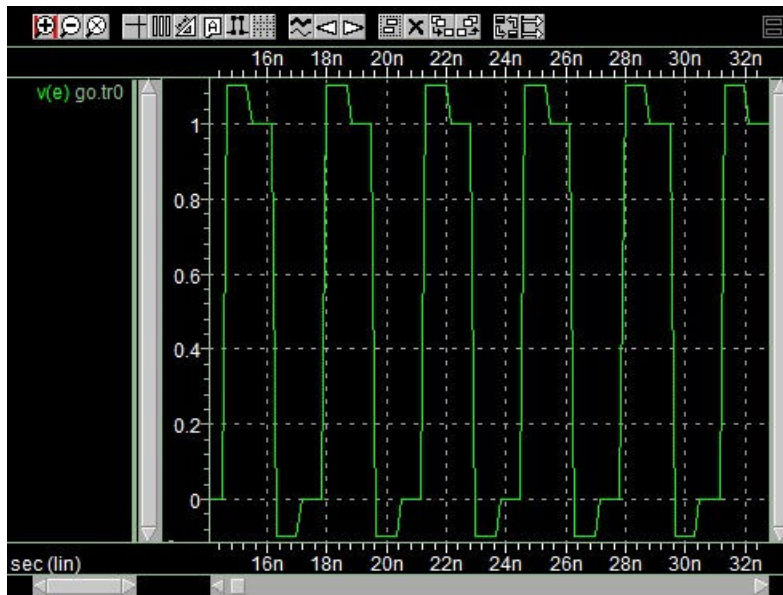
You can achieve clearer eye openings by using these emphasis techniques for the same system with the same digital symbol voltage level as shown in [Figure 33](#).

Figure 33 The effect of incident pre-emphasis: (left) original eye diagram; (right) eye diagram with additional 30% of pre-emphasis to the incident port



[Figure 34](#) shows how the amount of pre- or de-emphasis represents a fraction of 1. For example, an `emphasis_level = -.5` (or 50%) would correspond to 6-dB transmitter de-emphasis.

Figure 34 Effect of emphasis on a pulse source



To add pre-emphasis and de-emphasis, apply the keywords `Emphasis_Level` and `Emphasis_Time` to the port-element, the transient voltage and the current sources.

The following section discuss these topics:

- [Syntax for P-Element and Transient V and I Sources](#)
- [Usage Examples](#)

Syntax for P-Element and Transient V and I Sources

Port Element Syntax

```
Pxxx node1 node2 [node3 node4 node5] port=portindex
+ [Z0=val] $Port Impedance
+ [DC mag] [AC mag phase] [HBAC mag phase] $Voltage/Power Info
+ [HB mag phase harm tone modharm modtone]
+ [transient_waveform]
+ [ENCODE=DW8B10B [RD_INIT=0|1]]
+ [ENCODE=DW64b66b [TXC=txcdata]]
+ [ENCODE=DW128b130b|DW128b132b [HEAD=DATA|CONTROL]]
+ [TRANFORHB=[0|1]] [DCOPEN=[0|1]]
+ [power=[0|1|2|W|dbm]] $Power Switch
+ [POWERSIN=[0|1|2|W|dbm]]
+ [POWERHB=[0|1|2|W|dbm]]
+ [POWERHBAC=[0|1|2|W|dbm]]
+ [POWERSNAC=[0|1|2|W|dbm]]
+ [RDC=val] [RAC=val] $ Source Impedance Overrides
+ [RHBAC=val] [RHB=val] [RTRAN=val]
```

```
+ [Emphasis_Level=val] [Emphasis_Time=val]          $ Emphasis
+ $ **** DFE ****
+ [DFE_tap=n1 n2 ... nN DFE_coeff=val1, val2, ... valN]
+ [DFE_VTH=val]
+ [DFE_CLK=0|1]
+ [DCD=val] [DCD_TYPE=0|1|2|3]          $ Duty Cycle Distortion
+ [PJ=val] [PJ_TYPE=0|1|2|3|4|5|6]      $ Period Jitter
+ [RJ=val1, [val2, ... valN]] [VN=val] $ Rand Jitter Stateeye
+ [IBIS_AMI_FILE=filename IBIS_AMI_MODEL=modelname]
+ [AMI_OBJ=filename] [AMI_PARAM=filename] [AMI_CDR=0|1]$AMI
+ [AMI_CORNER=idx]$AMI
+ [AMI_Remove_DC_Offset = 0|1]
+ [AMI_Recover_DC_Offset = 0|1]
+ [MODULATION=PAM4]
+ [PAM4_MAPPING=string]
+ [AMI_PARAM_xx = string|quoted string]
+ [IBIS_AMI_VRANGE_MODE = 1|2]
+ [IBIS_AMI_USE_RAMP = 0|1]
+ [IBIS_AMI_USE_VRANGE = 0|1]
+ [IBIS_AMI_USE_ZSRC = 0|1]
```

Pulse Voltage and Current Source Syntax

```
Vxxx n+ n- PU[LSE] [(v1 v2 [td [tr [tf [pw [per]]]]]) []]
+ [Emphasis_Level=val] [Emphasis_Time=val]
Ixxx n+ n- PU[LSE] [(v1 v2 [td [tr [tf [pw [per]]]]]) []]
+ [Emphasis_Level=val] [Emphasis_Time=val]
```

SIN Voltage and Current Source Syntax

```
Vxxx n+ n- SIN [(v0 va [freq [td [q [j]]]]) []]
+ [PERJITTER=val] [SEED=val] [Emphasis_Level=val]
+ [Emphasis_Time=val]
Ixxx n+ n- SIN [(v0 va [freq [td [q [j]]]]) []]
+ [PERJITTER=val] [SEED=val] [Emphasis_Level=val]
+ [Emphasis_Time=val]
```

PWL Voltage and Current Source Syntax

```
Vxxx n+ n- PWL [(t1 v1 [t2 v2 t3 v3...] [R [=repeat]]
+ [TD=delay] []) [Emphasis_Level=val]
+ [Emphasis_Time=val]
Ixxx n+ n- PWL [(t1 v1 [t2 v2 t3 v3...] [R [=repeat]]+ [TD=delay]
[]) [Emphasis_Level=val]
+ [Emphasis_Time=val]
```

Exponential Voltage and Current Source Syntax

```
Vxxx n+ n- EXP [(v1 v2 [td1 [t1 [td2 [t2]]]]) []]
+ [Emphasis_Level=val] [Emphasis_Time=val]
Ixxx n+ n- EXP [(v1 v2 [td1 [t1 [td2 [t2]]]]) []]
+ [Emphasis_Level=val] [Emphasis_Time=val]
```

PAT Voltage and Current Source Syntax

```
Vxxx n+ n- PAT [(] vhi vlo td tr tf tsample data [RB=val]
+ [R=repeat] [)] [Emphasis_Level=value]
+ [Emphasis_Time=val]
Ixxx n+ n- PAT [(] vhi vlo td tr tf tsample data [RB=val]
+ [R=repeat] [)] [Emphasis_Level=val]
+ [Emphasis_Time=val]
```

Pseudo-Random Bit Source Syntax

```
Vxxx n+ n- LFSR [(] vlow vhigh tdelay trise tfall rate seed
+ [[] taps []] [rout=val] [)] [Emphasis_Level=val]
+ [Emphasis_Time=val]
Ixxx n+ n- LFSR [(] vlow vhigh tdelay trise tfall rate seed
+ [[] taps []] [rout=val] [)] [Emphasis_Level=val]
+ [Emphasis_Time=val]
```

Single Frequency FM Voltage and Current Source Syntax

```
Vxxx n+ n- SFFM [(] vo va [fc [mdi [fs]]] [)]
+ [Emphasis_Level=val] [Emphasis_Time=val]
Ixxx n+ n- SFFM [(] vo va [fc [mdi [fs]]] [)]
+ [Emphasis_Level=val] [Emphasis_Time=val]
```

Single Frequency AM Voltage and Current Source Syntax

```
Vxxx n+ n- AM [(] sa oc fm fc [td] [)]
+ [Emphasis_Level=val] [Emphasis_Time=val]
Ixxx n+ n- AM [(] sa oc fm fc [td] [)]
+ [Emphasis_Level=val] [Emphasis_Time=val]
```

Usage Examples

- [Example 1: Pre-Emphasis](#)
- [Example 2: De-Emphasis](#)

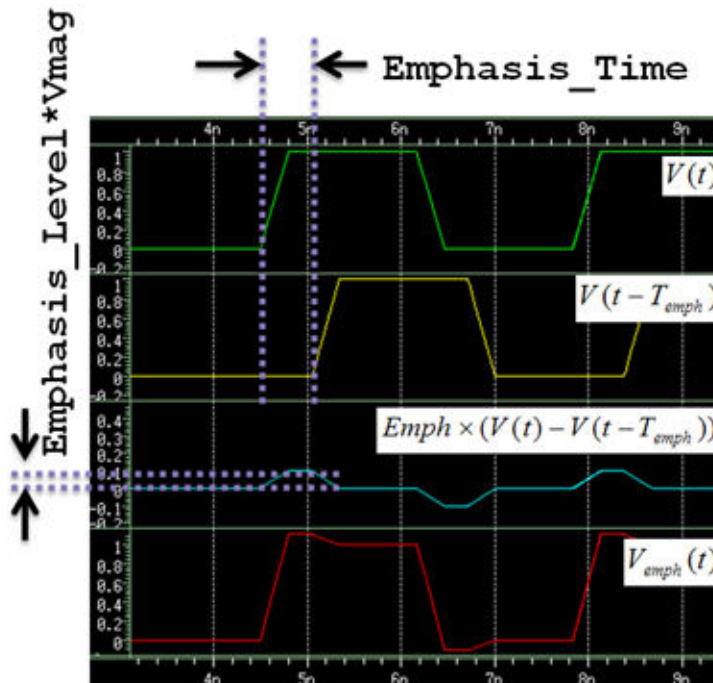
Example 1: Pre-Emphasis

In this example, the voltage source v_e adds $V_{add}(t) = 0.1 \cdot \left(V(t) - V\left(t - \frac{1}{4}T\right) \right)$ to the original pulse waveform to emphasize the rising and falling edges of the original pulse ([Figure 35](#)).

```
.param Vmag = 1.0
.param T     = 3.33n
.param trf   = 'T/20'
.param td    = 4.5n
```

```
Vde de 0 Emphasis_Level =0.1 Emphasis_Time = 'T/4'
+ pulse (0 mag td trf trf 'T/2-trf' T)
```

Figure 35 Pre-emphasis added by specifying positive Emphasis_Level



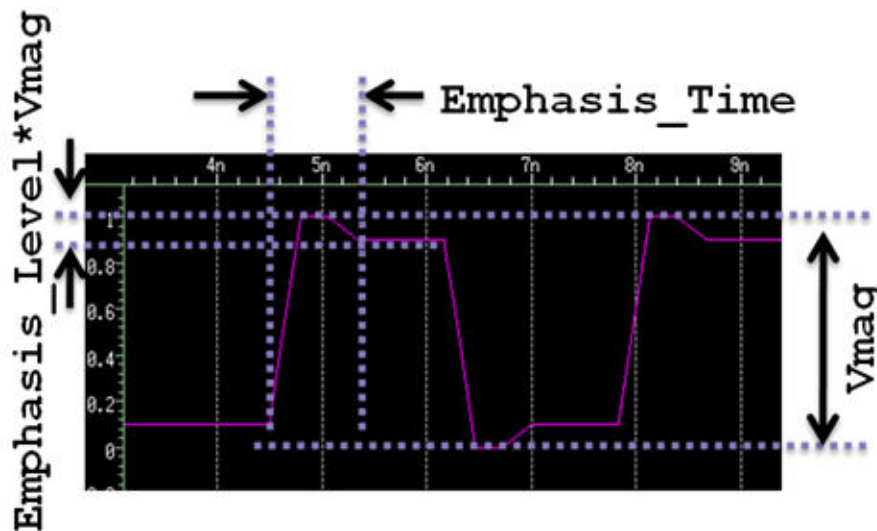
Example 2: De-Emphasis

In this example, the voltage source V_{de} applies de-emphasis by adding $V_{sub}(t) = 0.1 \cdot \left(V(t) - V\left(t - \frac{1}{4}T\right) \right)$ to the original waveform. A negative $Emphasis_Level$ specifies the de-emphasis. Figure 36 shows the pulse waveform after you reduce $Emphasis_Time$ in magnitude.

```
.param Vmag = 1.0
.param T = 3.33n
.param trf = 'T/20'
.param td = 4.5n
```

```
Vde de 0 Emphasis_Level =-0.1 Emphasis_Time = 'T/4'
+ pulse (0 mag td trf trf 'T/2-trf' T)
```

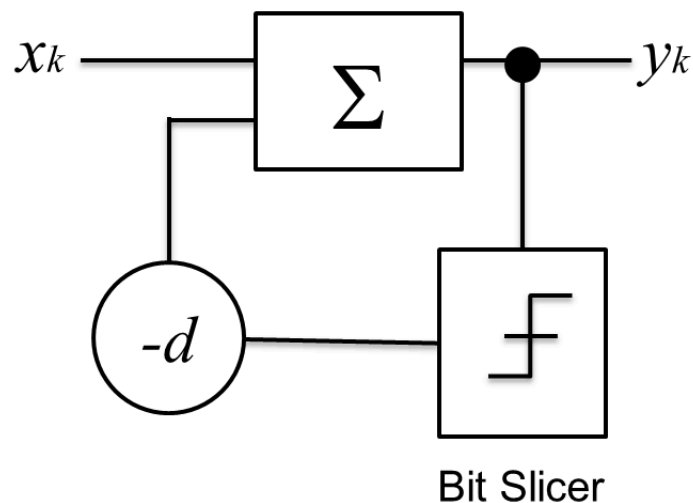
Figure 36 De-emphasis applied by specifying negative Emphasis_Level



Built-In DFE Function for Rx Port Element in StatEye Analysis

Figure 37 shows a block diagram of the decision feedback equalizer (DFE). The system applies the waveform filter represented in Equation 14 to equalize the output bit stream waveform.

Figure 37 DFE block diagram



Equation 14

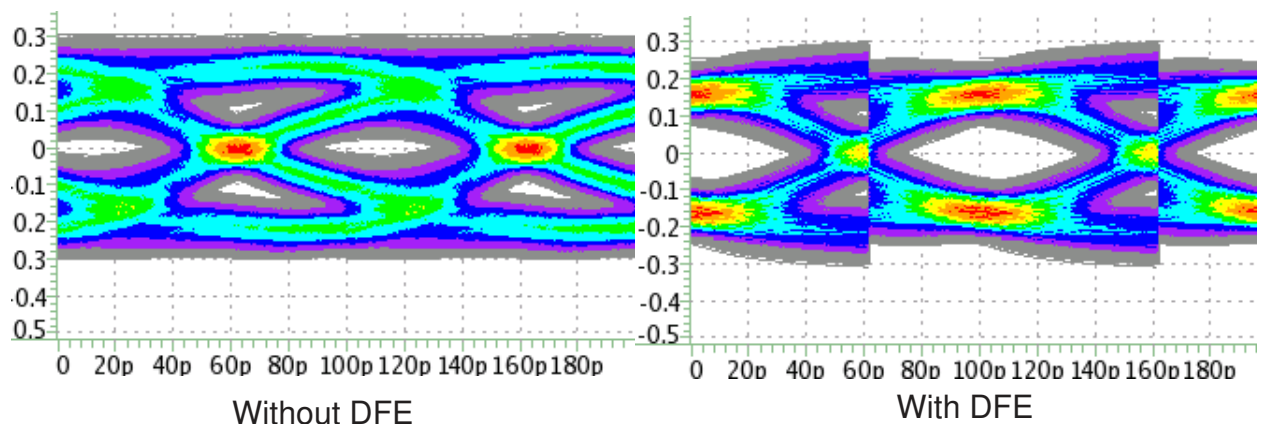
$$y_k = x_k - d \times \text{sgn}(y_{k-m} - v_{th})$$

Where:

- x_k is the DFE input voltage
- y_k is the DFE output voltage
- d is the feedback coefficient
- k is the sample index in UI (bit position)
- m is the relative bit position to be fed back and
- v_{th} is the logical threshold of the system.

An example netlist is provided in the PrimeSim HSPICE demo package in `../demo/hspice/si/SNPS_AMI/biDFE.sp`. This example uses the AMI model which represents a transmitter-side feed-forward equalizer and a built-in DFE as the receiver-side equalizer. The following figure compares eye diagrams with and without the DFE effect.

Figure 38 Eye diagrams without and with DFE



Applying External Clock to the Built-In DFE Model

The DDR5 next generation memory interface can use equalizer(s) in transmitter and/or receiver circuitries. For receiver, DFE will be deployed. Since the DDR system uses designated clock lane (DQS), the DFE implementation for DDR simulation will take the external clock signal into account in its slicer process.

For DFE's external clock input, set `DFE_CLK=1` for the probe side port element.

The demo netlist in `$installdir/demo/hspice/si/BiDFE/clk_fwd.sp` provides an example of this feature.

Chapter 3: Statistical Eye Analysis Built-In DFE Function for Rx Port Element in StatEye Analysis

```
.StatEye ....
+ incident_port=1, 2
+ probe_port=3, 4
+ Clock_Port=(4, 0.0)
+ Clock_Mode =2
```

In the preceding statements in the example, the `StatEye` command sets up one data (DQ) lane and one clock (DQS) lane. The `Clock_Port` command specifies port-4 as the probing point of the DQS signal with trigger at 0V. And `Clock_Mode=2` sets the clock jitter injection as a numerical perturbation sequence. Then, the following statements add timing variation (PJ) with different frequency to port-1 (DQ input) and port-2 (DQS input).

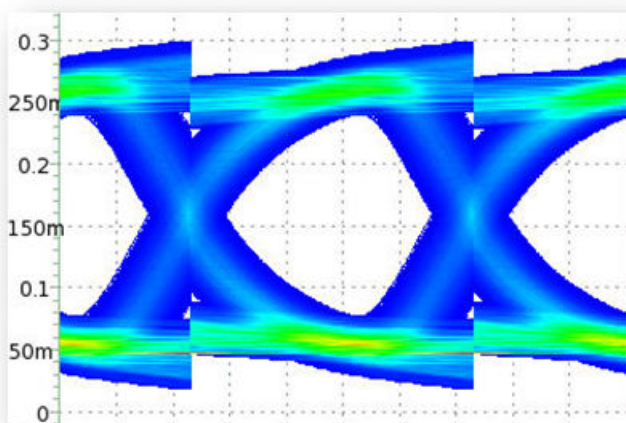
```
P1 DQ0_n 0 port=1
+ pj = 10p
+ pj_type=6
+ pj_freq=1e8
+ LFSR (0 1 0 _TRF _TRF _BPS 1 [21,1] )

* clock : 90 degree phase shifted
P2 DQS_n 0 port=2
+ pj = 10p
+ pj_type=6
+ pj_freq=1e6
+ PAT (0.5 -0.5 '0.5*_UI' _TRF _TRF _UI b01010101 R=10000000)
```

Finally, to add built-in DFE at receiver side of the DQ lane, the following statements are used:

```
Pdq0_f dq0_f 0 port=3
+ DFE_TAP=1 2 DFE_COEFF=0.02 0.01 + DFE_VTH=0.16
+ DFE_CLK=1
```

Figure 39 Results of the StatEye simulation in `clk_fwd.sp`



The results show that a non-ideal clock signal is considered in the DFE slicer.

Perturbation Analysis with StatEye

With the `PTB_DATA` keyword in StatEye analysis, PrimeSim HSPICE loads a perturbed set of edge responses. When the `PTB_DATA` keyword is specified, PrimeSim HSPICE computes a perturbation transfer function at each voltage by numerical difference between nonperturbed and perturbed data. Then StatEye applies vertical fluctuation on the resulting PDF eye diagram. The contents and file format of the perturb data directory are the same as regular edge response data generated by StatEye analysis with the `SAVE_TR` keyword. Therefore, you can simply run one StatEye analysis with an arbitrary parameter set to obtain the perturbed edge response data.

The following example shows the supply voltage fluctuation effect on the eye diagram.

In the netlist, the initial run computes an eye diagram with a nominal supply voltage. Then the first `.alter` block changes the supply voltage to 90 percent of the nominal level and saves the initial edge responses under the `perturb.save1/` directory. Finally, the second `.alter` block sets a nominal supply voltage again with `PTB_DATA=perturb.save1` to perform perturbation analysis using the perturbed edge response generated in the previous alter.

Example Netlist: perturb.sp

```
* VDD Perturbation Analysis
.param BW=0.2n
.param trf = 'BW*0.01'
.param opfreq = '1.0/BW'
.param vdd_nom = '3.0'
.param vdd = vdd_nom

P1 in 0 port=1
P2 out 0 port=2

.include 'buff.inc'
x1 in m vdd buffer
x2 m mm vdd buffer
x3 mm out connector

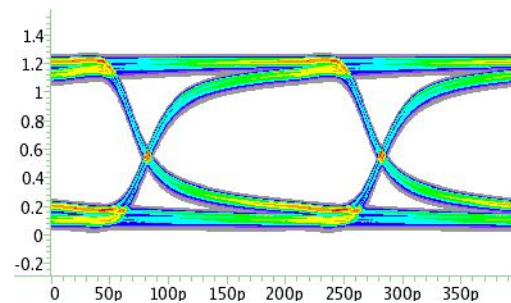
.opt post accurate
.StatEye T='BW' TRF='trf'
+ incident_port=1 probe_port=2
+ V_HIGH='vdd' V_LOW=0.0
+ edges=2 save_tr=ascii
.probe stateye eye(2)

.alter $ 90% vdd
.param vdd = 'vdd_nom*0.9'

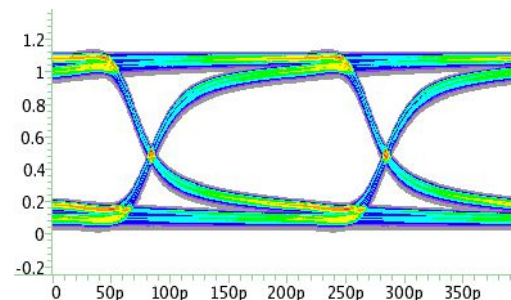
.alter $ perturb analysis
.param vdd = 'vdd_nom'

.StatEye T='BW' TRF='trf'
+ incident_port=1 probe_port=2
+ V_HIGH='vdd' V_LOW=0.0
+ edges=2
+ PTB_DATA=perturb.save1
* load alter-1 result
* as perturbation data .end
```

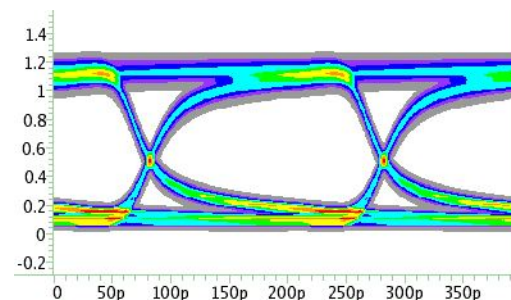
Initial Run With Nominal VDD



First alter with 90-percent VDD



Second alter with perturbation analysis



Unfolding Statistical Eye Diagrams to Waveforms

The following describes the method for .STATEYE analysis to unfold resulting eye diagrams to output partial time domain waveforms when you provide a specific pattern to the incident port.

Under this condition, `.STATEYE` allows the following `.PROBE` and `.PRINT` commands to extract partial time domain waveforms:

```
.PROBE eyeUF(port_index, bit_index)
.PRINT eyeUF(port_index, bit_index)
```

PrimeSim HSPICE extracts partial time domain waveforms about the bit stream location, `bit_index`, from the resulting eye diagram at a probing port when you specify `port_index`. The length of the partial time domain waveform is 100 bits. When `-1` is specified as the `bit_index`, PrimeSim HSPICE outputs the time domain waveform of the entire bit sequence.

Note:

- Unfolding of eye diagrams applies only to pattern-specific eye diagram generation.
- When using `-1` as the `bit_index`, it is strongly recommended to use the `MAX_PATTERN` keyword to limit the length of bits, to avoid a very large output file.

Output

The `.PROBE` command outputs a binary time domain data file readable to WaveView. The unfolded time domain files are stored under `netlist.steuf#`. The naming convention of the binary data file is:

```
netlist.steuf#/portN_M
```

The `.PRINT` command outputs the text time domain data file under the `netlist.printSte0` directory. The naming convention of the text data file is:

```
Netlist.printSte#/data_v_t_M_
```

Also, the `.PRINT` command produces a script file which you can upload to the `gnuplot` program:

```
Netlist.printSte#/eyeUF_N_M
```

where `#`, `M`, and `N` are the alter index number, probe port index number, and bit index number, respectively.

Example: Unfolding StateEye Diagram

An example netlist is provided in the PrimeSim HSPICE demo package in `demo/hspice/si/stateye_unfold.sp`. The example PrimeSim HSPICE netlist performs `.STATEYE` analysis with an unfolding time domain waveform around the 500th bit ([Figure 40](#)).

```
*
* StatEye Unfolding Example
*
```

Chapter 3: Statistical Eye Analysis

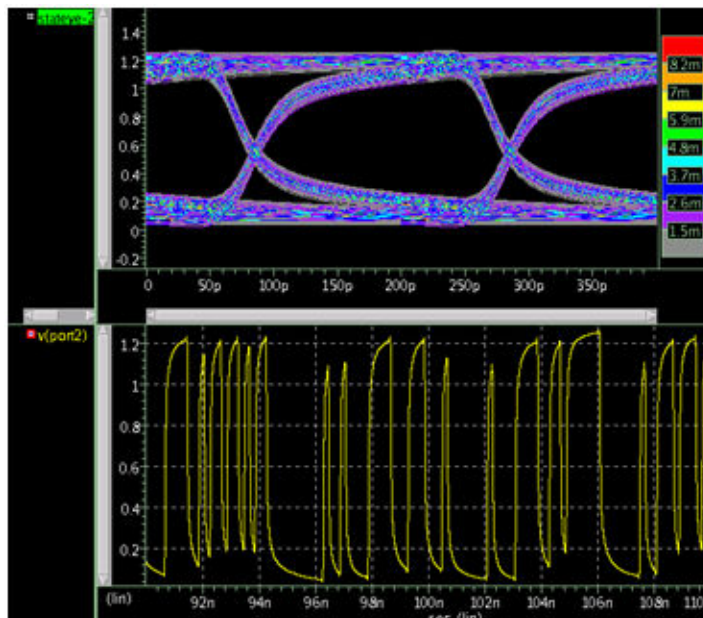
Unfolding Statistical Eye Diagrams to Waveforms

```

.param BW=0.2n
.param trf = 'BW*0.01'
.param opfreq = '1.0/BW'
.param vdd = '3.0'
P1 in 0 port=1 LFSR 0 'vdd' 0 'trf' 'trf' 'opfreq' 1 [14,9,5,2]
P2 out 0 port=2
x1 in m vdd INV
x2 m mm vdd INV
x3 mm out CONN
.opt post accurate
.StatEye T='BW' TRF='trf' incident_port=1 probe_port=2
+ V_HIGH='vdd' V_LOW=0.0
+ edges=2
.print stateye eye(2) eyeUF(2,500)
.probe stateye eye(2) eyeUF(2,500)
.model nch nmos level=54
.model pch pmos level=54
Vd vdd 0 'vdd'
.subckt inv in out vdd
mn1 out in 0 0 nch l=0.25u w=30u
mp1 out in vdd vdd pch l=0.25u w=60u
.ends
.subckt CONN _in _out
Rtest _in _out 50 RS=1e-3 $ skin effect resistor
Cin _in 0 0.3p
Cout _out 0 0.3p
.ends
.end

```

Figure 40 StatEye result with unfolded waveform around 500th bit



Multiple Data Rate Eye Diagram Analysis with StatEye

StatEye allows you to inject incident signals with different data rates using the optional `T_Factor` keyword.

`T_factor = n1 [, n2, ... nN]` multiplies the unit interval of the incident bit stream at each port.

Example:

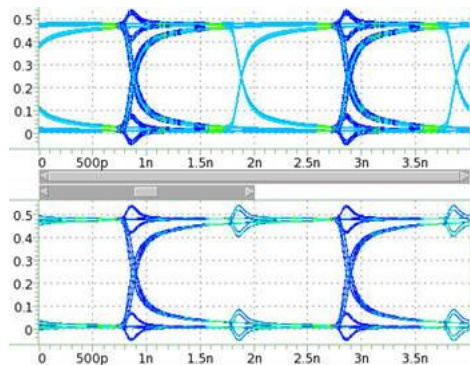
```
.param _UI=1n
.param _TRF='0.05*_UI'

.stateye
+ T = '_UI'
+ TRF = '_TRF'
+ incident_port= 1 2
+ probe_port= 3 4
+ T_Factor= 1 2      $ UI x2 for 2nd incident
+ save_tr=ascii
+ TD_nUI=4
+ align_eye=sync
```

This example injects the bit stream with 1 ns unit interval at port 1 and the bit stream with 2 ns unit interval at port 2.

Note:

`TD_nUI` uses the unscaled unit interval.



Accelerating StatEye's Initial Transient with Process Distribution and Multi-Threading

StatEye deploys distributed processing (DP) for the initial edge response transient. When the target is a highly complex nonlinear system with crosstalk couplings that require multiple incidents with multiple edge responses, applying process distribution (with the

`-dp` command-line argument) in conjunction with multi-threading (or MT, with the `-mt` command-line argument) can significantly reduce the overall runtime.

For more information on invoking PrimeSim HSPICE with DP, see Running Multiple Simulations in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

For more information on invoking PrimeSim HSPICE with MT, see Running Multithreaded Simulations in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*.

Known Limitation

`.StatEye` assumes an equivalent to a bit stream response that you can obtain by superposition of single bit responses. This assumption may not apply when you have strongly nonlinear components in the system under test.

Troubleshooting Output Warnings

When running a StatEye test case, you may see the following messages:

```
Warning: (line 60) port 1 is not specified as .stateye probe_port
Warning: [STAT_EYE error] invalid probe port index for stateye
measurement
```

For StatEye analysis, `.PRINT`, `.PROBE`, and `.MEASURE` statements are taken at the probe port. The statements must match the "probe_port" defined in the `.stateye` statement. For example if the `.STATEYE` command is:

```
.stateye T='BW' trf='0.1*BW' incident_port=1 probe_port=2
```

You should identify the probe port in your output statement. The following example shows valid `.PRINT`, `.PROBE`, and `.MEASURE` statements for this analysis:

```
.print stateye eye(2)           * eye diagram of specified port
.probe stateye eyeV(2,V)        * cross-section of eye at V
.print stateye bathtubT(2,T)    * port and time for bathtub curve
.probe stateye ber(2)           * output BER plot for port
.measure stateye res_name Veye 2 * maximum vertical eye measurement
```

PrimeSim HSPICE issues no warnings when the port listed in the output statement matches the `probe_port` specified in the `.STATEYE` command.

References

- [1] IBIS Version 5.0 Specification: http://eda.org/pub/ibis/ver5.0/ver5_0.txt.

- [2] IBIS (I/O Buffer Information Specification) home page: <http://www.eigroup.org/ibis/default.htm>

4

Time Domain Reflectometry

This chapter discusses using digitized time domain reflectometry (TDR) in conjunction with PrimeSim HSPICE to select design components.

The following sections discuss these topics:

- [Optimizing TDR Packaging](#)
- [Performing a TDR Simulation in PrimeSim HSPICE](#)
- [Basic TDR Impedance Analysis](#)
- [Testbench Netlist Overview](#)
- [Input and Output Ports](#)
- [Selecting the Risetime of the Incident Wave](#)
- [Simulating the Example DUTs](#)
- [Troubleshooting: TDR Zo Result Starts at Negative Value](#)
- [Differential TDR Example Netlist](#)
- [References](#)

Optimizing TDR Packaging

The packaging plays an important role in determining the overall speed, cost, and reliability of a system. With small feature sizes and high levels of integration, a significant portion of the total delay is the time required for a signal to travel between chips.

Time domain reflectometry (TDR) is the closest measurement to actual digital component functions. It provides a transient display of the impedance versus time for pulse behavior.

Performing a TDR Simulation in PrimeSim HSPICE

When performing a signal integrity analysis, a bit stream is often the first stimulus that you use to validate the system. If you obtain poor results, it can be useful to first perform a TDR simulation on either the system or the individual models to find unexpected impedance discontinuities or mismatches. A TDR simulation measures the reflections that result from an ideal step edge traveling through a transmission medium.

The following is an example PrimeSim HSPICE testbench to demonstrate TDR simulations with a variety of classic discontinuities. The included testbench is completely functional and uses single-ended lossless transmission lines and lumped elements to demonstrate the analysis but you can be easily adapt use of the more complex W-element or S-parameter models. The last section of this description supplies a differential equivalent example netlist.

PrimeSim HSPICE provides a number of methods to model transmission lines and media. These include:

- Lumped models with R-, L-, and C-elements connected by lossless (T-element) transmission lines
- S-parameter data (S-element), either from instrument measurements or extracted during previous PrimeSim HSPICE simulations
- The frequency-dependent lossy W-element. The W-element has several different modeling options:
 - The PrimeSim HSPICE field-solver based on materials and geometries
 - RLGC per unit length matrices
 - S-parameter data called by the W-element SMODEL parameter
 - A network of behavioral sources

A TDR simulation measures the reflections that result from an ideal step edge traveling through a transmission medium. This allows you to analyze impedance discontinuities and mismatches. The medium that you study is often an interconnect topology, and could be a combination of circuit board traces, cables, connectors and even the wire and bonding between an IC and its package. The following uses an example PrimeSim HSPICE testbench to demonstrate TDR simulations with a variety of classic discontinuities.

An interconnect simulation often contains a combination of these methods. Singularly or collectively you can refer to them as the Device Under Test (DUT). It is common for the manufacturer to provide the DUT model, or portions of it (as in the case of a connector) or it is the output by a signal-integrity analysis tool which creates a model by analyzing the packaging or layout data.

Basic TDR Impedance Analysis

The resulting waveform of the TDR simulation is the combination of the incident wave and reflections that occur when the step edge encounters impedance variations.

This example netlist uses the following names and equations:

- **Vincident**: An ideal copy of the step edge source you use as a reference
- **Vmeasured**: Voltage at the input to the DUT which is the combination of incident and reflected waves
- **Vreflect**: The reflected portion only which you obtain by subtracting the incident portion from the measured voltage
- **Impedance (Zo)**: Calculated from the above parameters:

$$\frac{Z_{ref} \cdot (V_{incident} + V_{reflect})}{(V_{incident} - V_{reflect})}$$

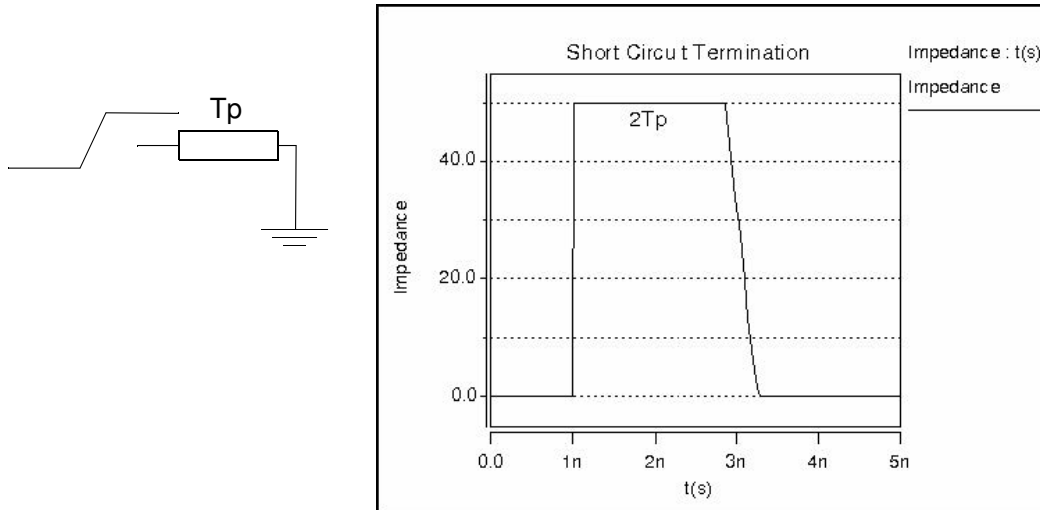
Use this calculation to graph the response of the transmission line in unit of characteristic impedance which is often more useful than plotting the voltage of the TDR measured wave. In at least one case, however, the open circuit termination is more useful to plot the measured voltage. Note that due to the voltage divider effect, the measured voltage is 1/2 of the incident wave.

The following examples demonstrate classic impedance mismatches and discontinuities:

- [Example 1: Short Circuit Termination](#)
- [Example 2: Open Circuit Termination](#)
- [Example 3: Mismatched Load Termination](#)
- [Example 4: Shunt Capacitance Discontinuity](#)
- [Example 5: Series Inductance Discontinuity](#)

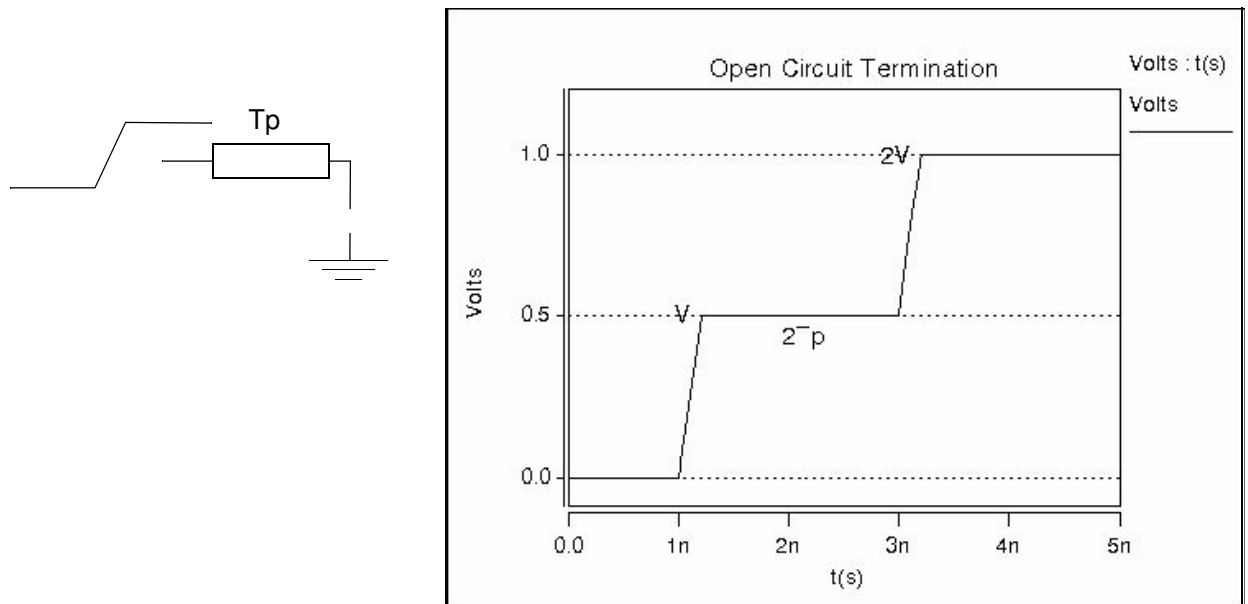
Example 1: Short Circuit Termination

Figure 41 Short circuit termination



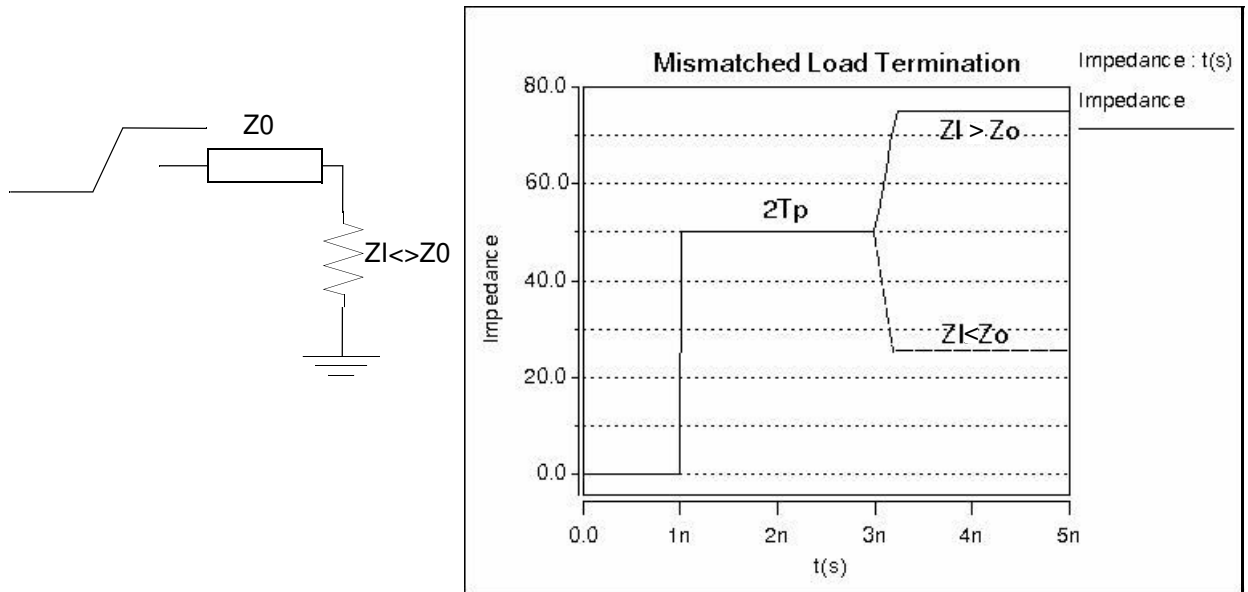
Example 2: Open Circuit Termination

Figure 42 Open circuit termination



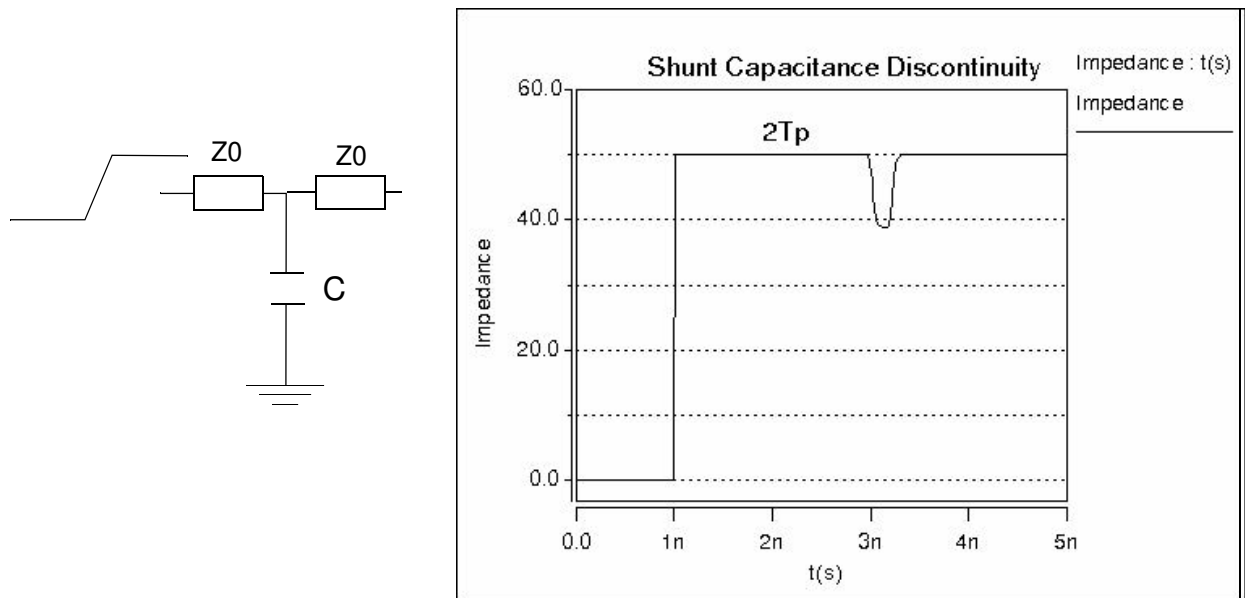
Example 3: Mismatched Load Termination

Figure 43 Mismatched load termination



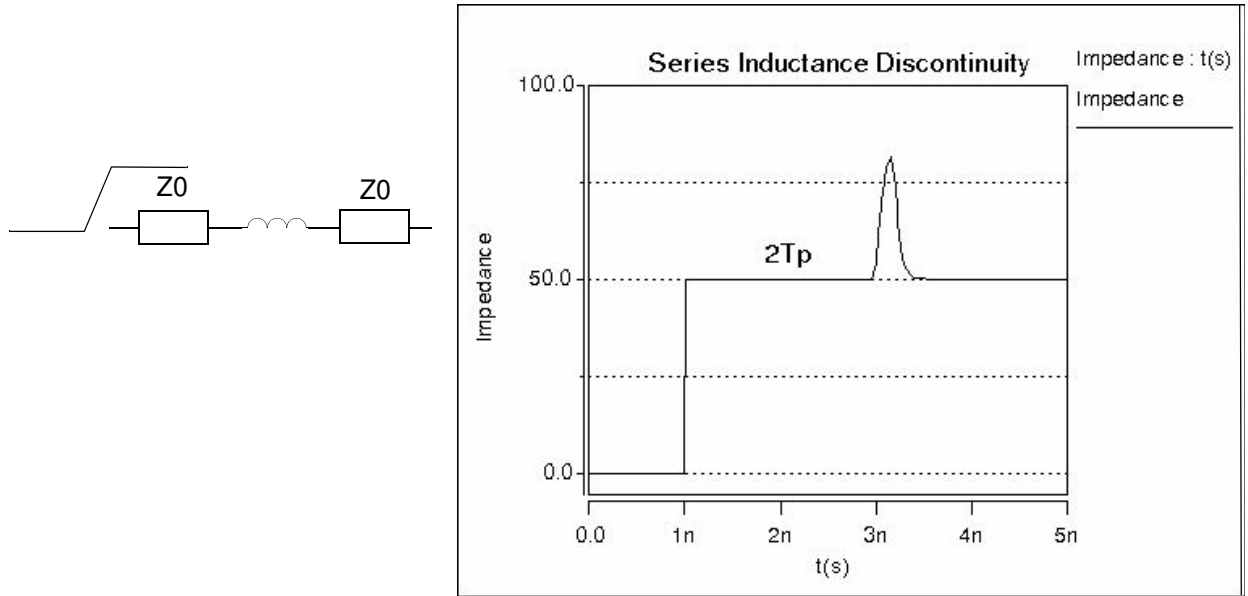
Example 4: Shunt Capacitance Discontinuity

Figure 44 Shunt capacitance discontinuity



Example 5: Series Inductance Discontinuity

Figure 45 Series inductance discontinuity



Testbench Netlist Overview

In creating a PrimeSim HSPICE TDR test bench, this example uses the "P" (port) element for both the input sources and output terminations because it can:

- Represent either single-ended or differential signals, and
- Provide both a source voltage and an impedance value in a single construct

The first example simulates a single-ended TDR example. Additional syntax for a differential testbench and DUT appears in [Differential TDR Example Netlist](#).

Four main sections comprise the testbench:

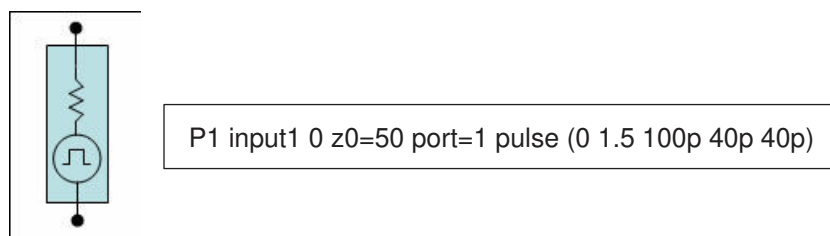
1. The port that represents the source voltage and impedance for the incident wave
2. The Device Under Test (DUT)
3. An output termination port
4. A reference port and matching termination port

The reference port functions to create an ideal version of the incident pulse which is separate from the DUT. The tool uses this port to calculate the reflected portion of the TDR measurement.

Input and Output Ports

The P-element is the basis for the input and output ports. As you see above, you can visualize it as an impedance and a voltage source in series:

Figure 46 PrimeSim HSPICE port element



In the example shown in [Figure 46](#), a port named “P1” connects between “input1” and ground and has a characteristic impedance of 50 ohm. It supplies a single 40 ps rising edge to input1 after a 100 ps delay. If you omit the voltage source argument, the port serves as a simple termination. Each port must have a unique sequential numerical designation in the “port=” argument. You also use the P-element in the LIN analysis to extract S-parameters.

For more information on port elements, see Port Element in the PrimeSim Continuum User Guide: Elements.

Selecting the Risettime of the Incident Wave

One way to determine the risetime of the incident wave is using the knee frequency. To guarantee reliable operation of a digital system, you should develop and verify the circuit design for frequencies below the knee frequency. Most energy in digital pulses concentrates below the knee; the behavior of a circuit at the knee frequency determines its ability to process a step edge. The knee frequency for any digital signal relates to the rise and fall time of its digital edges rather than its clock rate.

You can calculate the risetime based the desired knee frequency and this calculation depends on the basis for the measurement: a 10-90% or 20-80% measurement.

- For a 10-90% measurement, $T_{rise} = .5/F_{knee}$
- For a 20-80% measurement, $T_{rise} = .35/F_{knee}$

For example, the risetime measured at 20-80% needed for a 10 GHz knee frequency:

$$T_{rise} = \frac{.35}{10e9\text{Hz}} = 35\text{ps}$$

In a PrimeSim HSPICE trapezoidal pulse source function, specify the risetime from 0-100%.

So, for a 20-80% risetime, multiply the desired risetime by 1.67 to obtain the “tr” parameter and for a 10-90% measured risetime, multiply the desired risetime by 1.25. For example:

$(35\text{ps measured at 20-80\%}) \cdot 1.67 = \text{tr parameter} = 58.5\text{ps}$

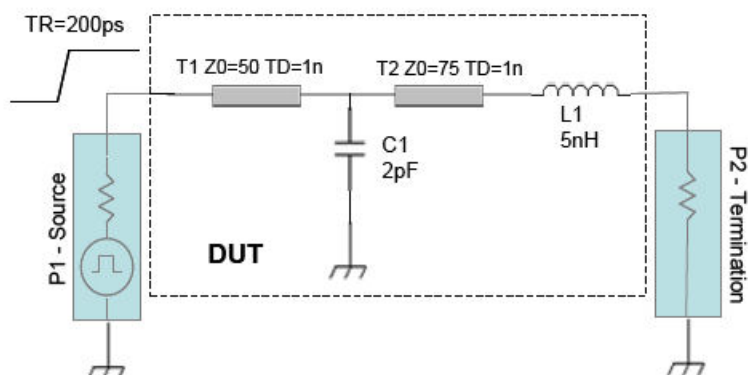
Simulating the Example DUTs

The sample netlist includes three separate example DUTs:

1. An inductive discontinuity
2. A capacitive shunt discontinuity
3. A transmission line followed by a shunt capacitance discontinuity, an impedance mismatch in the second transmission line and a series inductance

The design in [Figure 47](#) shows a combination of behaviors. An example netlist for simulation is provided following [Figure 47](#) and the results are shown in [Figure 48](#).

Figure 47 Testbench and DUT



The example testbench below, “An example PrimeSim HSPICE TDR Netlist with DUT3 Uncommented”, is completely functional and uses simple lossless transmission lines and lumped elements to demonstrate the analysis, but it could just as easily use more complex W-element or S-parameter models. A differential example follows at the end of this section, and contains appropriate modifications to the ports, DUTs and equations.

An Example PrimeSim HSPICE TDR Netlist with DUT3 Uncommented

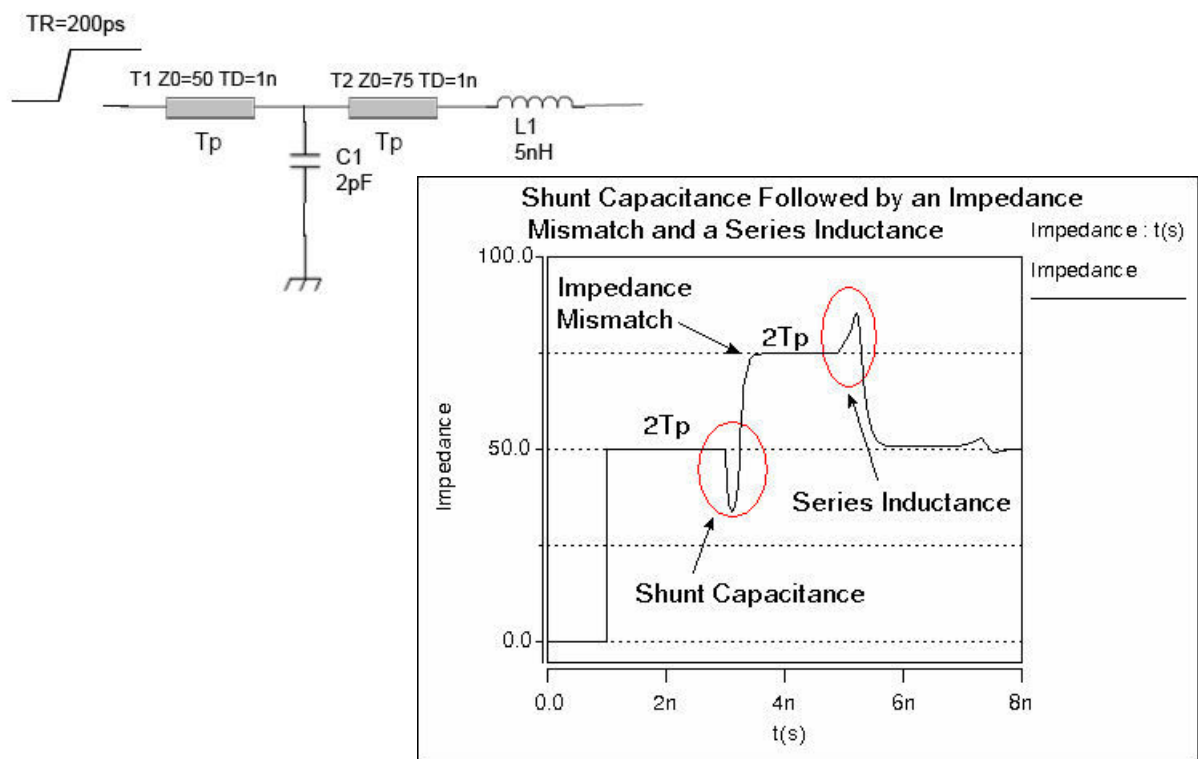
```
** Single-ended TDR example **
*
.opt post probe runlvl=5
```

```
.tran 0.1n 20n
*
.param zref=50.0 vlo=0 vhi=1 td=1n tr=.2n tf=.2n pw=50n per=200n
*
* P1 is the input port for the incident wave. It provides
* both an impedance and a pulse source
*
Psource dut_in 0 z0=zref port=1 pulse(vlo vhi td tr tf)
*
* DUT1 - An inductive discontinuity
*T1 dut_in 0 node1 0 Z0=50 td=1n
*L1 node1 node2 5e-9
*T2 node2 0 dut_out 0 Z0=50 td=1n
*
* DUT2 - A capacitive shunt discontinuity
*T1 dut_in 0 node1 0 Z0=50 td=1n
*C1 node1 0 1e-12
*T2 node1 0 dut_out 0 Z0=50 td=1n
*
* DUT3 - Series inductance - shunt capacitance
* with impedance mismatch in the second transmission
* line
T1 dut_in 0 node1 0 Z0=50 td=1n
C1 node1 0 2e-12
T2 node1 0 node2 0 Z0=75 td=1n
L1 node2 dut_out 5e-9
*
* Pterm is the output port of the DUT and provides termination
*
Pterm dut_out 0 z0=50 port=2
*
* Pref and Prefterm are a reference source and termination to use
* as the ideal incident wave in the reflection calculation
*
Pref vref 0 z0=zref port=3 pulse(vlo vhi td tr tf)
Prefterm vref 0 z0=zref port=4
*
* Calculations of the reflected wave and impedance
*
.probe tran vincident = par('v(vref)')
.probe tran vmeasured = par('v(dut_in)')
.probe tran vreflect = par('vmeasured-vincident')
.probe tran Z0 = par('zref*(vincident+vreflect)/(vincident-vreflect)')
*
.end
```


Procedural Notes:

1. Increase the default `RUNLVL` from 3 to 5 to observe the subtleties in the discontinuities of the waveforms.
2. Alternately, you can specify: `.option runlvl=6 delmax=10p`. Since the pulse is a single event (non-repeating), you can omit the `pw` (pulse width) and `per` (period) arguments, although a warning message is reported. Since `delmax` forces the maximum time step length, it may sacrifice the efficiency of the dynamic time step control. Therefore, unless you have a specific need for dense time points, do *not* specify `delmax`. Uncomment only one DUT at a time to observe the example discontinuities and impedance mismatches.

Figure 48 Result of the DUT3 Analysis



Troubleshooting: TDR Zo Result Starts at Negative Value

You may see TDR Zo results start at negative 50 ohm (single-ended) or negative 100 ohm (differential). This is due to a change in the precision of expression evaluation in PrimeSim HSPICE. You can safely ignore this, since the TDR result following the incident wave is the same. If you would rather see it to start at zero, add a very small value (such as $1e-20$) to

the denominator in the Z_0 equation. Using the formula in [Basic TDR Impedance Analysis](#), modify the equation as follows:

```
.probe tran Zo = par('zref*(vincident+vreflect)/
(vincident-vreflect+1e-20)')
```

The Z_0 result starts at zero ohm and can be shared without the need to explain the initial starting impedance.

Differential TDR Example Netlist

```
** Differential TDR example **
*
.opt post probe runlvl=5
.tran 0.1n 20n
.param zref=50.0 vlo=0 vhi=1 td=1n tr=.2n tf=.2n
* P1 is the differential input port for the incident wave.
* It provides both an impedance and a pulse source
*
Psource dut_inp dut_inn 0 z0=zref port=1 pulse(vlo vhi td tr tf)
*
* DUT1 - An inductive discontinuity
* T1P dut_inp 0 node1p 0 Z0=50 td=1n
* T1N dut_inn 0 node1n 0 Z0=50 td=1n
* L1 node1p node2p 5e-9
* L2 node1n node2n 5e-9
* T2P node2p 0 dut_outp 0 Z0=50 td=1n
* T2N node2n 0 dut_outn 0 Z0=50 td=1n
*
* DUT2 - A capacitive shunt discontinuity
* T1P dut_inp 0 node1p 0 Z0=50 td=1n
* T1N dut_inn 0 node1n 0 Z0=50 td=1n
* C1 node1p 0 1e-12
* C2 node1n 0 1e-12
* T2P node1p 0 dut_outp 0 Z0=50 td=1n
* T2N node1n 0 dut_outn 0 Z0=50 td=1n
*
* DUT3 - A shunt capacitance followed by an impedance mismatch
* and a series inductance
T1P dut_inp 0 node1p 0 Z0=50 td=1n
T1N dut_inn 0 node1n 0 Z0=50 td=1n
C1 node1p 0 2e-12
C2 node1n 0 2e-12
T2P node1p 0 node2p 0 Z0=75 td=1n
T2N node1n 0 node2n 0 Z0=75 td=1n
L1 node2p dut_outp 5e-9
L2 node2n dut_outn 5e-9
*
* Pterm is the dut_output port of the DUT and provides termination
*
Pterm dut_outp dut_outn 0 z0=50 port=2
```

```
*
* Pref and Prefterm are a reference source and termination to use
* as the ideal incident wave in the reflection calculation
*
Pref refp refn 0 z0=zref port=3 pulse(vlo vhi td tr tf)
Prefterm refp refn 0 z0=zref port=4
*
* Calculations of the reflected wave and impedance
*
.probe tran vincident = par('v(refp)-v(refn)')
.probe tran vmeasured = par('v(dut_inp)-v(dut_inn)')
.probe tran vreflect = par('vmeasured-vincident')
.probe tran Zdiff
    = par('2*zref*(vincident+vreflect)/(vincident-vreflect)')
*
.end
```

References

TDR Impedance Measurements: A Foundation for Signal Integrity, Copyright © 2001, Tektronix, Inc.

Part 2: Signal Integrity Modeling

Describes the signal integrity modeling.

Part 2 focuses on signal integrity modeling and consists of the following chapters:

- [Chapter 5, S-parameter Modeling Using the S-element](#)
- [Chapter 6, W- and T-elements Transmission-Line Modeling](#)
- [Chapter 7, Using the Field Solver to Extract Tx Line Parameters](#)
- [Chapter 8, Using IBIS Files](#)
- [Chapter 9, Using IBIS-AMI Equalizer Models with StatEye](#)

5

S-parameter Modeling Using the S-element

Describes the S-parameter, S-parameter modeling, and other topics related to the S-element.

PrimeSim HSPICE provides numerous examples. For paths to S-parameter-related demo files, see S-parameter Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

You can use the S-element to describe a multi-terminal network within most PrimeSim HSPICE and RF analyses. (The exception is Shooting-Newton SN analysis.) For more information about using the S-element (S-parameter) for signal integrity analysis, see S-element (Scattering Parameter Data) in the *PrimeSim Continuum User Guide: Elements*.

The following sections discuss these topics:

- [S-parameter Model](#)
- [Using the Scattering Parameter Element](#)
- [S Model Syntax](#)
- [S-element Data File Model Examples](#)
- [Mixed-Mode S-parameters](#)
- [Accelerating S-Element Time Domain Performance with Recursive Convolution](#)
- [Multiport Noise Model for Passive Systems](#)
- [S-element Noise Model](#)
- [S-parameter Standalone Manipulation Utility](#)
- [Preconditioning S-parameters](#)
- [Group Delay Handler in Time Domain Analysis](#)
- [S-Model Data Smoothing](#)
- [Predicting an Initial Value for FMAX in S-Element Models](#)
- [De-Embedding S-parameters](#)

- [Small-Signal Parameter Data Frequency Table Model](#)
- [References](#)

S-parameter Model

You can use small-signal parameters at the network terminals to characterize linear or nonlinear networks that have sufficiently small signals. (For details, see [Chapter 2, Linear Network Parameter Analysis](#).) After you set the parameters, you can simulate the block in any external circuit. Designers of high frequency circuits widely use S-parameters to characterize a linear network.

In multiport networks, S-parameters (**S**) take the following definition:

Equation 15

$$\mathbf{b} = \mathbf{S} \cdot \mathbf{a}$$

In [Equation 15](#), **a** is an incident wave vector, and **b** is a reflected wave vector, defined as follows:

Equation 16

$$\mathbf{a} = \mathbf{Y}_r^{1/2} \cdot \mathbf{v}_f = \mathbf{Z}_r^{1/2} \cdot \mathbf{i}_f$$

Equation 17

$$\mathbf{b} = \mathbf{Y}_r^{1/2} \cdot \mathbf{v}_b = \mathbf{Z}_r^{1/2} \cdot \mathbf{i}_b$$

[Equation 16](#) and [Equation 17](#) use the following definitions:

- $\mathbf{v}_f$ is the forward voltage vector.
- $\mathbf{v}_b$ is the backward voltage vector.
- $\mathbf{i}_f$ is the forward current vector.
- $\mathbf{i}_b$ is the backward current vector.
- $\mathbf{Z}_r$ is the characteristic impedance matrix of the reference system.
- $\mathbf{Y}_r$ is the characteristic admittance matrix.
- $\mathbf{Z}_r$ and $\mathbf{Y}_r$ satisfy the relationship $\mathbf{Y}_r = \mathbf{Z}_r^{-1}$

The S-parameters are frequency-dependent. When all ports terminate with impedance-matching without a voltage/current source, the forward wave becomes zero. This is because there is no reflection if the ports have no voltage/current source.

Notifications and Limitations

Because the S-element supports two types of noise models, the priority is:

- For multiport ($N \neq 2$) S-elements, PrimeSim HSPICE only considers passive noise models in noise analysis. If `NOISE=0`, the tool considers the system as noiseless.
- For two-port S-elements, if a Touchstone file provides two-port noise parameters, those two-port noise parameters generate the noise model. If the file does not provide two-port noise and `NOISE=1`, a passive noise model is used. Otherwise, the tool considers the system as noiseless.
- PrimeSim HSPICE does not support bias-dependent S-parameters.

Using the Scattering Parameter Element

The S-element gives you a convenient way to describe a multi-terminal network. You can use the S-element with the generic frequency-domain model (`.MODEL SP`). You also can use the S-element with data files that describe frequency-varying behavior of a network and provide discrete frequency-dependent data such as a Touchstone 1.0/2.0 file and a CITI file.

In particular, the S-parameter in the S-element represents the generalized scattering parameter (S) for a multi-terminal network.

The S-parameter and the Y-parameter satisfy the following relationship:

Equation 18

$$\mathbf{Y} = \mathbf{Y}_{rs} (\mathbf{I} - \mathbf{S}) (\mathbf{I} + \mathbf{S})^{-1} \mathbf{Y}_{rs}$$

where $\mathbf{Y}_r$ is the characteristic admittance matrix of the reference system. The following formula relates $\mathbf{Y}_r$ to the $\mathbf{Z}_r$ characteristic impedance matrix:

Equation 19

$$\mathbf{Y}_r = \mathbf{Z}_r^{-1} \mathbf{Y}_{rs} \mathbf{Y}_{rs} = \mathbf{Y}_r' \mathbf{Z}_{rs} \mathbf{Z}_{rs} = \mathbf{Z}_r$$

Similarly, you can convert the Y-parameter to the S-parameter as follows:

Equation 20

$$\mathbf{S} = (\mathbf{I} + \mathbf{Z}_{rs} \mathbf{Y} \mathbf{Z}_{rs})^{(-1)} (\mathbf{I} - \mathbf{Z}_{rs} \mathbf{Y} \mathbf{Z}_{rs})$$

For detailed information on the S-element syntax and examples, see S-element (Scattering Parameter Data) in *PrimeSim Continuum User Guide: Elements*.

S Model Syntax

Use the following syntax to describe specific S models:

```
.MODEL Smodel_name S [N=dimension]
+ [TSTONEFILE=filename | CITIFILE=filename |
+ RFMFILE=file_name.rfm | BNPFIL=filename]
+ [TYPE=s|y] [Z0=value| vector_value]
+ [FBASE=base_frequency] [FMAX=maximum_frequency]
+ [INTERPOLATION=STEP|LINEAR|SPLINE|HYBRID]+ [INTDAT TYP=[RI|MA|DBA]]
+ [HIGHPASS=0|1|2|3|4] [LOWPASS=0|1|2|3]
+ [DELAYHANDLE=1|2|0|ON|OFF] [DELAYFREQ=val]
+ [MIXEDMODE=0|1]
+ [DATATYPE=data_string] [XLINELENGTH=val]
+ [NoiPassiveChk [1|0]]
+ [SMOOTH=val] [SMOOTHPTS=val]
+ [RATIONAL_FUNC=[0|1] [RATIONAL_FUNC_REUSE=0|1|2]
+ [RATIONAL_FUNC_FOR_AC=0|1]
+ [RATIONAL_FUNC_WINDOWING=[0|1]]
+ [PASSIVE=[0|1] [PASSIVE_TOL=val] [COLSUM_LIMIT=val]
+ [ENFORCE_PASSIVE=0|1] [STAMP=AUTO|Y|S|YSTS|SSTS|DEEMBED]
+ [PRECFAC=val] FQMODEL=sp_model_name
+ [NOISE=[1|0]] [DTEMP=value]
+ [CACHE_DIR=dir_str]
+ [CAUSALITY=[0|1]]
+ [CAUSALITY_FMAX_SCALE = val]
+ [WRITE_CAUSALITY_ENFORCED_FILE = [0|1]]
+ [WRITE_FITTED_FILE = 1|0]
+ [CAUSALITY_ENFORCED_FILE_NAME = file_name]
```

Keyword	Description
Smodel_name	Name of the S model.
S	Specifies that the model type is an S model.
N	S model dimension, which is equal to the terminal number of an S-element and excludes the reference node.
TSTONEFILE	Specifies the name of a Touchstone file v 1.0/2.0. Data contains frequency-dependent array of matrixes. Touchstone v1.x files must follow the .s#p file extension rule, where # represents the dimension of the network. You can specify the Touchstone file by specifying the filename within single quotes or by using string parameters. Example of using <code>str()</code> to specify a Touchstone file: <pre>.subckt sparam n1 n2 tsfile=str('ss_ts.s2p') S1 n1 n2 0 mname=s_model .model s_model S TSTONEFILE=str(tsfile) .ends x1 A B sparam tsfile=str('ss_ts.s2p') ...</pre> For details, see <i>Touchstone® File Format Specification</i> by the IBIS Open Forum (https://ibis.org/).

Keyword	Description
CITIFILE	Specifies the name of the CITI file, which is a data file that contains frequency-dependent data. Note that string parameters are supported for calling a CITIFILE. For details, see <i>Using Instruments with ADS</i> by Agilent Technologies (http://www.agilent.com).
RFMFILE	Specifies S-element rational function (RFM) file. See Accelerating S-Element Time Domain Performance with Recursive Convolution .
BNPFILE	Specifies Broadband Network Parameter (BNP) file (proprietary). Note that when using the BNP file, there is no need for INTERPOLATION or LOW_PASS keywords. For HIGH_PASS, other than all options are HIGH_PASS=4 supported. Note: If you get a warning such as BNP file read failure at f=xx, this means that the BNP API is returning a read failure. It is likely that this BNP file does not cover the f=xx frequency point.
TYPE	Parameter type: - S: (scattering) (default) - Y: (admittance)
Z0 (or ZO)	Characteristic impedance value of the reference line (frequency-independent). For multi-terminal lines (N>1), PrimeSim HSPICE assumes that the characteristic impedance matrix of the reference lines are diagonal, and their diagonal values are set to Z0. You also can set a vector value for nonuniform diagonal values. Use ZO to specify more general types of a reference-line system. The default is 50.
FBASE	Base frequency used for transient analysis. PrimeSim HSPICE uses this value as the base frequency point for IFFT: - If FBASE is not set, PrimeSim HSPICE uses a reciprocal of the transient period as the base frequency. - If FBASE is set to a value smaller than the reciprocal value of transient period, transient analysis performs circular convolution by using the reciprocal value of FBASE as a base period.
FMAX	Maximum frequency for transient analysis. Used as the maximum frequency point for IFFT. See Predicting an Initial Value for FMAX in S-Element Models .
INTERPOLATION	The interpolation method: - STEP: piecewise step. - SPLINE: b-spline curve fit. - LINEAR: piecewise linear (default). - HYBRID: PrimeSim HSPICE combines different interpolation/extrapolation methods, and switches automatically between them to get the best accuracy. If needed, it also does causality correction down to DC. It is most useful for the S-parameters showing local resonances, and provides the proper interpolation and low-frequency extrapolation method for each entry of the S matrix, which shows different behaviors. For best accuracy, low frequency examples should be provided.
INTDATTYPE	Data type for the interpolation of the complex data. It's used only with linear and spline interpolation. - RI: real-imaginary based interpolation. - DBA: dB-angle based interpolation. - MA: magnitude-angle based interpolation (default).

Keyword	Description
HIGHPASS	Specifies high-frequency extrapolation: • 0: Use zero in Y dimension (open circuit). • 1: Use the given spectrum for IFFT process. No extrapolation is done. This is only available for S-type-based model. • 2: Use linear extrapolation with the highest two points. • 3: Apply window function (default). • 4: Estimates average derivatives of the phase and magnitude from highest 10% of sampling points. Extrapolation is performed using the highest sampling point and these derivatives. This option overrides <code>EXTRAPOLATION</code> in <code>.MODEL SP</code>.
LOWPASS	Method to extrapolate lower frequency points: • 0: Cut off. • 1: Make use of the S-matrix at the magnitude of the lowest given frequency point; Set the magnitude value of each entry as the element of DC matrix. The sign of each value is determined by the real part of the extrapolated value at DC point. (default) • 2: Perform linear extrapolation using the magnitude of the lowest two points. • 3: Perform rational function approximation based on low end frequency extrapolation. The <code>LOWPASS</code> option overrides <code>EXTRAPOLATION</code> in <code>.MODEL SP</code>.
DELAYHANDLE	Extracts the system delay before constructing the system impulse response. This might improve transient accuracy when the system does have delay, such as a transmission line system, and <code>RATIONAL_FUNC=0</code> is used. When <code>RATIONAL_FUNC=1</code> (default) is used, it is recommended to not specify the <code>DELAYHANDLE</code> keyword. The S-element automatically detects and handles the propagation delay. If <code>DELAYHANDLE</code> is 2, the S-element extracts the propagation delay according to the model property over the whole frequency band, instead of calculating group delay upon single frequency point as that when <code>DELAYHANDLE=1</code>. This option is now only available when the <code>rational_func=1</code>. If <code>DELAYHANDLE</code> is 2, the S-element extracts the propagation delay according to the model property over the whole frequency band, instead of calculating group delay upon single frequency point as that when <code>DELAYHANDLE=1</code>. This option is now only available when <code>rational_func=1</code>. If you set <code>DELAYHANDLE=OFF</code> (or 0), but <code>DELAYFREQ</code> is not zero, PrimeSim HSPICE simulates the S-element in the delay mode. For Y-parameter models, <code>DELAYHANDLE</code> is not supported and will be ignored. See also Group Delay Handler in Time Domain Analysis.
DELAYFREQ	Delay frequency for transmission-line type parameters. The default is <code>FMAX</code>. If the <code>DELAYHANDLE</code> is set to <code>OFF</code>, but <code>DELAYFREQ</code> is nonzero, PrimeSim HSPICE still simulates the S-element in the delay mode.
MIXEDMODE	Sets to 1 if the parameters are represented in the mixed mode.
DATATYPE	A string used to determine the order of the indices of the mixed-signal incident or reflected vector. The string must be an array of a letter and a number (<code>Xn</code>) where: • X = D to indicate a differential term • X = C to indicate a common term • X = S to indicate a single (grounded) term • n = the port number

Keyword	Description
XLINLENGTH	The line length of the transmission line system where the S-parameters are extracted. This keyword is required only when the S Model is used in a W-element.
NoiPassiveChk	Checks S-parameter for passivity in noise analysis (only): • 1 (default): Checks for passivity; if it fails at any frequency, thermal noise is turned off for the specific frequency point. • 0: Disables the passivity checker; thermal noise is always turned on.
SMOOTH	An integer value to choose one of following methods: • 0: No smoothing (default) • 1: Mean • 2: Median • 3: 2nd order polynomial fit • 4: 4th order polynomial fit See S-Model Data Smoothing .
SMOOTHPTS	An integer value to specify width of the smoothing window on each side of the target point. In total, $2*x + 1$ point is taken at each point calculation.
RATIONAL_FUNC	• 0: Performs the same as conventional S-element. FBASE/FMAX-based linear convolution is performed. • 1 (default): Performs rational function approximation then recursive convolution; also handles noncausal S-parameters. • 2: Performs rational function approximation to both time and frequency domain simulation. When <code>RATIONAL_FUNC=1</code> is used, it is recommended to not specify the <code>DELAYHANDLE</code> keyword. S-element automatically detects and handles the propagation delay. See also <code>.OPTION S_RATIONAL_FUNC</code> in the <i>PrimeSim HSPICE Reference Manual: Commands and Control Options</i> .
RATIONAL_FUNC_REUSE	• 0: Discard previously extracted rational function data and re-run the rational function approximation. • 1: Reuse rational function data if available. • 2: (default) Reuse rational function data if available and make no change in parameter source file (time stamp), FBASE, FMAX, HIGHPASS, LOWPASS, and passivity enforcement configurations; otherwise, rerun the rational function approximation.
RATIONAL_FUNC_FOR_AC	When used in conjunction with <code>RATIONAL_FUNC=1</code> , <code>RATIONAL_FUNC_FOR_AC = 1</code> , causes the S-element to use the extracted rational function in frequency domain evaluations. Default is 0.
RATIONAL_FUNC_WINDOWING	When you specify this argument with a value 1, to an S-parameter model, a window function is applied that will smoothly roll off the propagation toward the maximum frequency when generating the rational function. The default maximum frequency is the highest sampling point in the given S-parameter data. When <code>FMAX</code> is specified, the value of <code>FMAX</code> is used for the maximum frequency. When you specify this argument with a value 0 (default), no windowing function is applied. To apply windowing globally to all S-parameter models in the design, see <code>.OPTION S_RATIONAL_FUNC_WINDOWING</code> in the <i>PrimeSim HSPICE Reference Manual: Commands and Control Options</i>.

Keyword	Description
PASSIVE	Activates the passive checker to help debug the passive models. The default is 0 for the S-element where 0=deactivate and 1=activate. Using the tolerance value specified by <code>PASSIVE_TOL</code> keyword, the eigenvalues of matrix $(I-S^*S')$, $ev[i]$, will be checked. - If any frequency point violates $RE(ev[i]) > -(TOL*0.1)$, PrimeSim HSPICE issues a warning containing a list of violating frequencies with an "E" flag. Also, the checker verifies potential passivity violations by checking the summation of each S-parameter matrix column. - If $Sum > COLSUM_LIMIT$, PrimeSim HSPICE issues a warning containing a list of those violating frequencies with a "C" flag.
PASSIVE_TOL	Tolerance for eigenvalue checking activated by the <code>PASSIVE</code> keyword. Default value is 0.01.
COLSUM_LIMIT	Maximum value allowed for S-parameter matrix column summation. The limiting value will be used for passivity checker and passivity enforcement. Default value is 1.0
ENFORCE_ PASSIVE	With the <code>ENFORCE_PASSIVE=1</code> keyword, the S-element checks passivity of all the given frequency sampling points. Once passivity violations are found, the S-element seeks a minimum amount of loss property which restores passivity of all the violated points. Then the S-element adds the loss to all the given frequency points. Note: Passivity enforcement function generates cache data and stores it in a <code><model_name>.penf</code> file. When the cache file exists, the PrimeSim HSPICE S-element reuses passivity enforcement result without re-enforcement.
STAMP	- AUTO (default): Automatically selects an optimized stamping method - Y: Conventional admittance based stamp. - S: Scattering parameter based stamp (Note 1). - YSTS: Admittance parameter based state space stamp (Note 2). - SSTS: Scattering parameter based state space stamp (Note 2). - DEEMBDED: Produces a negated stamp to de-embed given an S-parameter block from the adjacent DUT connected in series (see De-Embedding S-parameters). Note: Use <code>STAMP=SSTS</code> when running Shooting Newton (SN) based analyses using the PrimeSim HSPICE tool.
PRECFACTOR	In almost all cases, you do not need to specify a value for this parameter. This parameter specifies the precondition factor keyword used for the precondition process of the S-parameter. A precondition is used to avoid an infinite admittance matrix. The default is 0.75, which is good for most cases. See also, Preconditioning S-parameters .
FQMODEL	Specifies the name of the frequency model file behavior of the S-, Y-, or Z-parameters. The <code>.MODEL</code> statement of sp type defines the frequency-dependent matrices array.
NOISE	Activates thermal noise. 1 (default): element generates thermal noise. 0: element is considered noiseless.

Keyword	Description
DTEMP	Temperature difference between the element and the circuit, expressed in °C. The default is 0.0. Element temperature is calculated as: $T = \text{Element temperature (}^{\circ}\text{K)} \\ = 273.15 \text{ (}^{\circ}\text{K)} + \text{circuit temperature (}^{\circ}\text{C)} \\ + \text{DTEMP (}^{\circ}\text{C)}$ Where circuit temperature is specified using either the .TEMP statement, or by sweeping the global TEMP variable in .DC, .AC, or .TRAN statements. When a .TEMP statement or TEMP variable is not used, the circuit temperature is set by .OPTION TNOM, which defaults to 25°C unless you use .OPTION SPICE, which raises the default to 27°C.
CACHE_DIR	Specifies the directory to write (for reuse) the generated cache files, such as rational function model generation.
CAUSALITY	Turns on or off the causality enforcement algorithm for the IFFT model: • 0 (default): OFF ** • 1: ON ** When causality enforcement is turned on, the HIGHPASS and FMAX keyword of S element or model is ignored. • For PrimeSim HSPICE: default is 0. • For PrimeSim: default is 1.
CAUSALITY_FMAX_SCALE	A real value val, which is used to manually set FMAX=val * Fmax_given in causality enforcement algorithm, where Fmax_given is the highest frequency value in the S model. Default is 3. Only valid when CAUSALITY=1.
WRITE_CAUSALITY_ENFORCED_FILE	Turns on or off the causality enforced result output to a Touchstone format file. • 0 (default): do not output • 1: output Only valid when CAUSALITY=1
WRITE_FITTED_FILE	Writes the rational function fitted s-parameters file when the RATIONAL_FUNC or S_RATIONAL_FUNC commands are used. Default is 1.
CAUSALITY_ENFORCED_FILE_NAME	Touchstone file name for the causality enforced result output. • Only valid when CAUSALITY=1. When file name is not specified, while WRITE_CAUSALITY_ENFORCED_FILE=1, the default file name is constructed by S model name, such as model_name_enforced.s#p.

The FQMODEL, TSTONEFILE, CITIFILE, and RFMFILE parameters describe the frequency-varying behavior of a network. Only specify one of the parameters in an S model card. If more than one method is declared, only the first one is used, and PrimeSim HSPICE issues a warning message.

For full example demo files of the S model usage, see S-parameter Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

S-element Data File Model Examples

The S model statement samples shown in Example 1 and Example 2 generate the same results.

Example 1

S model statement code example:

```
s1 n1 n2 n3 n_ref mname=smodel
.model smodel s n=3 z0=50 fbase=25e6 fmax=1e9
s1 n1 n2 n3 n_ref z0=50 fbase=25e6 fmax=1e9
```

Example 2

In this example, the S model statement has the characteristic impedance equal to 100 instead of the 50 as defined in `smodel`. The impedance changes because the parameters defined in the S element statement have higher priority than the parameters defined in the S model statement:

```
s1 n1 n2 n3 n_ref mname=smodel z0=100
.model smodel s n=3 z0=50 fbase=25e6 fmax=1e9
```

Example 3

In this example, `tstonefile` and `citifile` are both declared in `smodel`. PrimeSim HSPICE accepts `tstonefile`, ignores `citifile`, and issues a warning message. It is illegal to define a `tstonefile` and `CITIfile` `smodel` in the same statement. This prevents conflicts in the frequency-varying behavior description of the network. From the `tstonefile` file extension `.s3p`, you can tell that the network has three ports:

```
s1 n1 n2 n3 n_ref mname=smodel
.model smodel s tstonefile=expl.s3p
               citifile=expl.citi0
```

Example 4

In this example, `n` is declared both in the S-element statement and the S model statement.

```
s1 n1 n2 n3 n_ref mname=smodel
.model smodel s n=3
```

Example 5

This example shows a generic S-parameter statement using port elements. For information on port elements, see [Identifying Ports with the P-element](#).

```
**S-parameter example
.PTION post
.probe v(n2)
P1 n1 0 port=1 Z0=50 ac=1v PULSE 0v 5v 5n 0.5n 0.5n 25n
```

```
P2 n2 0 port=2 Z0=50
.ac lin 500 1Hz 30MegHz
.tran 0.1ns 10ns
* reference node is set
S1 n1 n2 0 mname=s_model
* S parameter
.model s_model S TSTONEFILE = ss_ts.s2p
Rt1 n2 0 50
.end
```

Example 6

This example shows the option line and noise parameters of a Touchstone file:

```
!
! touchstone file example
!
! # Hz S MA R 50.0000
0.00000 0.637187 180.000 0.355136 0.00000
0.355136 0.00000 0.637187 180.000
.....
! # HZ S DB R 50.0000
! 0.00000 -3.91466 180.000 -8.99211 0.00000
! -8.99211 0.00000 -3.91466 180.000
! .....
!
! # Hz S RI R 50.0000
! 0.00000 -0.637187 0.00000 0.355136 0.00000
! 0.355136 0.00000 -0.637187 0.00000
! .....
!
! 2-port noise parameter
! frequency[Hz] Nfmin[dB] GammaOpt(M) GammaOpt(P) RN/Z0
0.0000 0.29166 0.98916 180.00 0.11055E-03
0.52632E+08 6.2395 0.59071 -163.50 0.32868
0.10526E+09 7.7898 0.44537 175.26 0.56586
! .....
! end of file
```

Example 7

This example shows an S-parameter statement using port elements and the referenced CITI file. For information on port elements, see [Identifying Ports with the P-element](#).

```
**S-parameter
.OPTION post
.probe v(n2)
P1 n1 0 port=1 Z0=50 ac=1v PULSE 0v 5v 5n 0.5n 0.5n 25n
P2 n2 0 port=2 Z0=50
.ac lin 500 1Hz 30MegHz
.tran 0.1ns 10ns
*reference node is set
*S1 n1 n2 0 mname=s_model
```

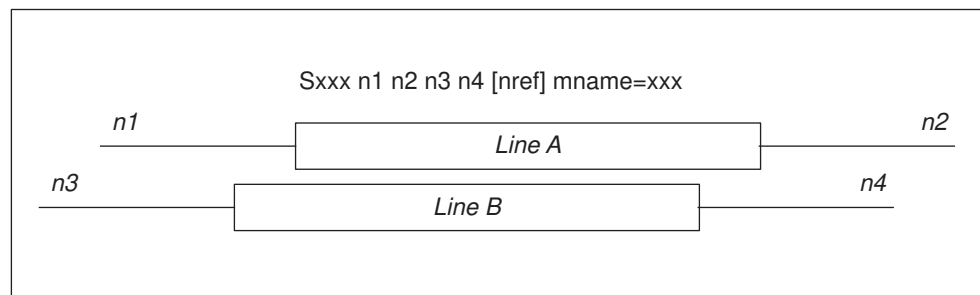
```
* use default reference node
S1 n1 n2 mname=s_model
* S parameter
.model s_model S CITIFILE = ss_citi.citi Z0=50
Rt1 n2 0 50
.end
```

Mixed-Mode S-parameters

Mixed-mode refers to a combination of differential and common mode characteristics in PrimeSim HSPICE linear network (LIN) analysis by using the S-element. It is useful to understand mixed-mode S-parameters since they present an overview of the wiring channels. The S-parameters also provide a view of inherent behaviors of differential and common-mode signal propagation characteristics.

PrimeSim HSPICE accepts both conventional single-ended S-parameters and mixed-mode S-parameters. In PrimeSim HSPICE, since the mixed-mode S-parameters undergo conversion to the single-ended S-parameters to fit with ground-referenced nodal analyses, there is no difference in simulation results between single-ended S-parameters and the equivalent mixed-mode representations of them.

Figure 49 Node indexing convention of the ground referenced (single ended) S-parameter



- Nodes 1 and 3 are the ports for one end of the transmission-line pair.
- Nodes 2 and 4 are the ports for the opposite end of the transmission-line pair.

The following sections discuss these topics:

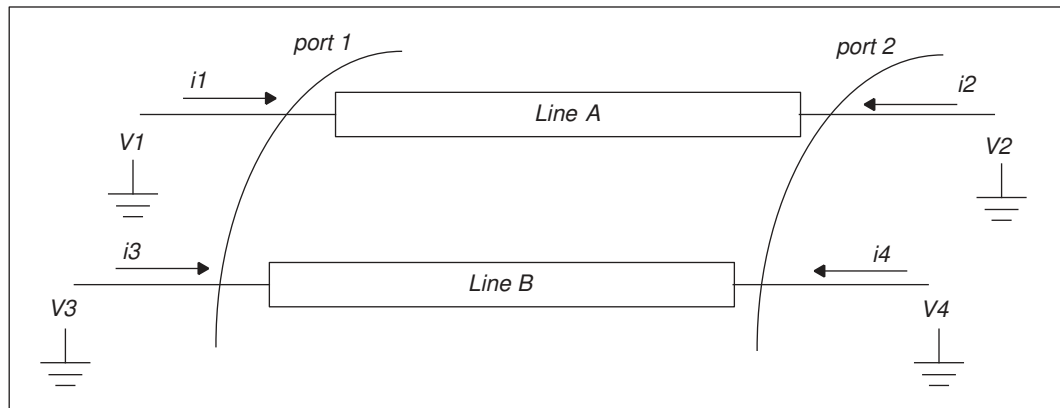
- [Relating Voltage and Current Waves to Nodal Waves](#)
- [Characterizing Differential Data Transfer Systems](#)
- [Deriving a Simpler Set of Voltage and Current Pairs](#)
- [Using the Mixed-Mode S-parameters \(S-element\)](#)

Relating Voltage and Current Waves to Nodal Waves

Figure 50 and Equation 21 through Equation 24 include common and differential mode voltage and current waves, relating them to nodal waves. Although you can apply mixed-mode data propagation to an arbitrary number of pairs of transmission lines, Figure 50 shows using a single pair model.

This figure also shows a schematic of symmetric coupled pair transmission lines commonly used for the differential data transfer system.

Figure 50 Schematic of symmetric coupled-pair transmission line



Solving the telegrapher's equation, you can represent nodal voltage and current waves of the data transfer system as:

Equation 21

$$v_1 = A_1 e^{-\gamma_e x} + A_2 e^{\gamma_e x} + A_3 e^{-\gamma_o x} + A_4 e^{\gamma_o x}$$

Equation 22

$$v_3 = A_1 e^{-\gamma_e x} + A_2 e^{\gamma_e x} - A_3 e^{-\gamma_o x} - A_4 e^{\gamma_o x}$$

Equation 23

$$i_1 = \frac{A_1}{Z_e} e^{-\gamma_e x} - \frac{A_2}{Z_e} e^{\gamma_e x} + \frac{A_3}{Z_o} e^{-\gamma_o x} - \frac{A_4}{Z_o} e^{\gamma_o x}$$

Equation 24

$$i_3 = \frac{A_1}{Z_e} e^{-\gamma_e x} - \frac{A_2}{Z_e} e^{\gamma_e x} - \frac{A_3}{Z_o} e^{-\gamma_o x} + \frac{A_4}{Z_o} e^{\gamma_o x}$$

where:

- Y_e is the propagation constant for even mode waves.
- Y_o is the propagation constant for odd mode waves.
- Z_e is the characteristic impedance for even mode waves.
- Z_o is the characteristic impedance for odd mode waves.
- A_1 and A_3 represent phasor coefficients for the forward propagating modes.
- A_2 and A_4 represent phasor coefficients for the backward propagating modes.

Each voltage and current pair at each node represents a single propagating signal wave that references to the ground potential. This type of expression is nodal wave representation.

Characterizing Differential Data Transfer Systems

[Equation 25](#) through [Equation 28](#) use differential and common mode waves to characterize differential data transfer systems. The difference of the nodal wave defines the voltage and current of the differential wave:

Equation 25

$$v_{dm} \equiv v_1 - v_3$$

Equation 26

$$i_{dm} \equiv \frac{1}{2} (i_1 - i_3)$$

Common mode voltage and current definitions are:

Equation 27

$$v_{cm} \equiv \frac{1}{2} (v_1 + v_3)$$

Equation 28

$$i_{cm} \equiv i_1 + i_3$$

Deriving a Simpler Set of Voltage and Current Pairs

In the following equations, substituting [Equation 15](#), [Equation 16](#) and [Equation 17](#) into [Equation 15](#) derives a simpler set of voltage and current pairs:

Equation 29

$$v_{dm} = 2(A_3 e^{-\gamma_o x} + A_4 e^{\gamma_o x})$$

Equation 30

$$v_{cm} = A_1 e^{-\gamma_e x} + A_2 e^{\gamma_e x}$$

Equation 31

$$i_{dm} = \frac{A_3}{Z_o} e^{-\gamma_o x} - \frac{A_4}{Z_o} e^{\gamma_o x}$$

Equation 32

$$i_{cm} = 2\left(\frac{A_1}{Z_e} e^{-\gamma_e x} - \frac{A_2}{Z_e} e^{\gamma_e x}\right)$$

You also can relate characteristic impedances of each mode to the even and odd mode characteristic impedances:

$$Z_{dm} \equiv 2Z_o \text{ and } Z_{cm} \equiv \frac{Z_e}{2}$$

Having defined a generalized parameter power, you can then define differential normalized waves:

$$a_{dm1} \equiv \frac{v_{dm} + Z_{dm} i_{dm}}{2\sqrt{Z_{dm}}} \Big|_{x=0} \text{ and } a_{dm2} \equiv \frac{v_{dm} + Z_{dm} i_{dm}}{2\sqrt{Z_{dm}}} \Big|_{x=L}$$

$$b_{dm1} \equiv \frac{v_{dm} - Z_{dm} i_{dm}}{2\sqrt{Z_{dm}}} \Big|_{x=0} \text{ and } b_{dm2} \equiv \frac{v_{dm} - Z_{dm} i_{dm}}{2\sqrt{Z_{dm}}} \Big|_{x=L}$$

Similarly, you can define common mode normalized waves as:

$$a_{cm1} \equiv \frac{v_{cm} + Z_{cm} i_{cm}}{2\sqrt{Z_{cm}}} \Big|_{x=0} \text{ and } a_{cm2} \equiv \frac{v_{cm} + Z_{cm} i_{cm}}{2\sqrt{Z_{cm}}} \Big|_{x=L}$$

$$b_{cm1} \equiv \frac{v_{cm} - Z_{cm} i_{cm}}{2\sqrt{Z_{cm}}} \Big|_{x=0} \text{ and } b_{cm2} \equiv \frac{v_{cm} - Z_{cm} i_{cm}}{2\sqrt{Z_{cm}}} \Big|_{x=L}$$

You can then specify S-parameters for mixed-mode waves as ratios of these waves as shown in [Equation 33](#):

Equation 33

$$\begin{bmatrix} b_{dm1} \\ b_{dm2} \\ b_{cm1} \\ b_{cm2} \end{bmatrix} = S_{mixed} \begin{bmatrix} a_{dm1} \\ a_{dm2} \\ a_{cm1} \\ a_{cm2} \end{bmatrix}, S_{mixed} = \begin{bmatrix} S_{dd} & S_{dc} \\ S_{cd} & S_{cc} \end{bmatrix}$$

where:

- S_{dd} is the differential-mode S-parameter.
- S_{cc} is the common-mode S-parameter.
- S_{cd} and S_{dc} represent the mode-conversion or cross-mode S-parameters.

Based on these definitions, you can linearly transform nodal wave (standard) S-parameters and mixed mode S-parameters: $\mathbf{M} \cdot \mathbf{S}_{standard} \cdot \mathbf{M}^{-1} = \mathbf{S}_{mixed}$

The M transformation matrix is: $\mathbf{M} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$

Using the Mixed-Mode S-parameters (S-element)

The S-element can recognize and parse the mixed-mode S-parameters when the keyword `mixedmode=1` is used. Any keywords besides `mixedmode` and `datatype` remain the same. Use the following syntax for a mixed-mode S-parameter:

```
Sxxx p1+ [p1-] p2+ [p2-] p3+ [p3-]...[n_ref] mname=Smodel
.MODEL Smodel S ...
[+ mixedmode=[0 1]]
[+ datatype=XiYjZk...]
```

Keyword	Description
pn+, pn-	Positive and negative terminals of the port n, respectively. The port numbers must be in an increasing order corresponding to the S matrices notation. • If the port is in mixed-mode (balanced), it requires that both positive and negative terminal names are in series. • If the port is single-ended, mixed-mode requires only one terminal name.
mixedmode	When <code>mixedmode=1</code> , PrimeSim HSPICE passes the information to the S-element that the S-parameters are in mixed-mode. The default is 0 (standard mode).

Keyword	Description
<code>datatype</code>	A string that determines the order of indices of the incident or reflected vectors (a and b) in Equation 25. The string must be an array of pairs that consists of a letter and a number (for example, Xn), where X = • D or d to indicate differential term. • C or c to indicate common term. • S, s, G, or g to indicate single (grounded) term and n = port number.

The definition `datatype = D1D2C1C2` is the default for a 2-balanced port network and specifies the nodal relationship of the following equation:

Equation 34

$$\mathbf{a}_{\text{standard}} = [a_{1+} a_{1-} a_{2+} a_{2-}] \Leftrightarrow \mathbf{a}_{\text{mixed}} = [a_{d1} a_{d2} a_{c1} a_{c2}]^T$$

where:

- a_{1+} is the incident wave goes into positive terminal of the port 1.
- a_{1-} is the incident wave goes into negative terminal of the port 1.
- a_{2+} is the incident wave goes into positive terminal of the port 2.
- a_{2-} is the incident wave goes into negative terminal of the port 2.

You also can derive the nodal relationship of the reflection wave in the same way. PrimeSim HSPICE assigns nodes from the given S-matrices to the S-element in the order of $\mathbf{a}_{\text{standard}}$. For example, incident and reflected waves at the positive terminal of the $1a_{1+}$, b_{1+} appear at the first node of the S-element.

The definition `datatype=D1C1S2` specifies the nodal relationship of the following equation:

Equation 35

$$\mathbf{a}_{\text{standard}} = [a_{1+} a_{1-} a_2]^T \Leftrightarrow \mathbf{a}_{\text{mixed}} = [a_{d1} a_{c1} a_{s2}]^T$$

The default is `datatype=D1D2...DnC1C2...Cn`, which is available for systems with mixed-mode (balanced) ports only.

Mixed-Mode S-parameter Netlist Examples

Example 1: Differential Transmission Line Pair

For an example netlist of a differential transmission line pair, go to the following directory:

`$installdir/demo/hspice/sparam/mixedmode_s.sp`

Example 2: Differential Amplifier

For an example netlist of a differential amplifier, go to the following directory:

```
$installdir/demo/hspice/sparam/diffamp_s.sp
```

Accelerating S-Element Time Domain Performance with Recursive Convolution

By default, the recursive convolution method is enabled by using the S-element keyword `RATIONAL_FUNC`. The original S-parameters are fit to their equivalent rational functions by using the original sampling points. This avoids interpolation or extrapolation. PrimeSim HSPICE saves the rational function to a `*.yrf` file to use recursive convolution during the simulation. By default, PrimeSim HSPICE reuses saved rational functions in subsequent simulations. See also [Multithreading Acceleration for S-element on Linux](#), [Ensuring Causality in the Rational Function Model](#), and [Rational Function Matrix \(.rfm\) File Format](#).

When `RATIONAL_FUNC` is disabled, the S-parameter evaluation method uses inverse FFT. This method requires equal-spaced sampling with 2^N points. If the S-parameter frequency points do not match the IFFT array sampling, PrimeSim HSPICE uses interpolation and/or extrapolation. This means that the sampling point location for the IFFT depends heavily on the combination of the `FBASE` and `FMAX` values. Accuracy also is strongly dependent on the value of `FMAX`.

Since the recursive convolution method is independent of `FMAX` or `FBASE` values, this method is useful when S-parameters are nonpassive or there are many ports. The S-element `RATIONAL_FUNC` model captures long and multiple delay components automatically without requiring the explicit `DELAYHANDLE` keyword. The recommended configuration for using the S-element is to set `RATIONAL_FUNC=1` and remove all other control keywords, unless there is a specific requirement.

The existence of decoupling capacitors in the S-parameters are automatically detected. The decoupling capacitors are de-embedded and separate sets of rational functions are generated before a recursive convolution is performed.

Both low impedance networks such as power distribution networks (PDN) and networks where the insertion loss is close to an open circuit can effectively be evaluated using rational functions.

The recommended configuration when using recursive convolution is to remove all other control keywords, unless you have a special need.

The convolution integral is commonly used to handle frequency-dependent transfer characteristics. To get a system response at time t , the convolution integral can be done as shown in [Equation 36](#):

Equation 36

$$y(t) = \int_{-\infty}^t x(\tau) \cdot h(t-\tau) d\tau$$

where $x(t)$, $h(t)$, $y(t)$ are input at the t , system response function in the time domain and output at t , respectively. As observed in [Equation 36](#), the convolution integral is computationally expensive, especially if t becomes a long transient simulation due to an increasing time window for each time point evaluation. The conventional S-element obtains $h(t)$ by applying IFFT to the original system function in the frequency domain and performs a discrete linear convolution integral, while [Equation 36](#) is continuous.

On the other hand, the frequency domain transfer function, $h(\omega)$ can be described as:

Equation 37

$$Hs = \frac{A}{s + \omega_c}$$

Converting [Equation 37](#) to the time domain gives the exponential decay function shown in [Equation 38](#):

Equation 38

$$h(t) = Ae^{-\omega_c t}$$

The computational cost of the convolution integral at time point t can be reduced using the convolution result at a previous time point (recursive convolution). Since recursive convolution only requires numerical integration from a previous time point to the current time point, it saves computational time and storage for input signal history. Recursive convolution can be formulated only when the system response can be represented in certain forms of rational functions, as shown in [Equation 39](#):

Equation 39

$$s_{row,col} \text{ or } y'_{row,col} \cong B + sC + \sum_k \frac{Ar_k}{s + \omega r_k} + \sum_l \left(\frac{Ac_l}{s + \omega c_l} + \frac{Ac_l^*}{s + \omega c_l^*} \right)$$

The PrimeSim HSPICE S-element generates a rational function matrix based on a given function and performs recursive convolution. Once the rational function is generated, the S-element stores the intermediate data for reuse in the following form: `MODEL_NAME.yrf`.

When `RATIONAL_FUNC_REUSE=1`, the S-element seeks an available data file and reuses it without running a redundant rational function generation process.

PrimeSim HSPICE also accepts rational function data input as external input. The input file syntax is described in the following section, [Rational Function Matrix \(.rfm\) File Format](#).

PrimeSim HSPICE accepts S- or preconditioned Y- parameter matrices as expressions with pairs of poles and residues. In cases of frequency-dependent scattering parameters, $S(\omega)$, or preconditioned admittance parameter, $Y'(\omega)$ can be represented as rational function matrix components as:

Equation 40

$$\mathbf{S}' = [\alpha \mathbf{I} + (2 - \alpha) \mathbf{S}] [(2 + \alpha) \mathbf{I} - \alpha \mathbf{S}]^{-1}$$

Equation 41

$$\mathbf{Y}' = \mathbf{Y}_c^{\frac{1}{2}} [\mathbf{I} - \mathbf{S}'] [\mathbf{I} + \mathbf{S}']^{-1} \mathbf{Y}_c^{\frac{1}{2}}$$

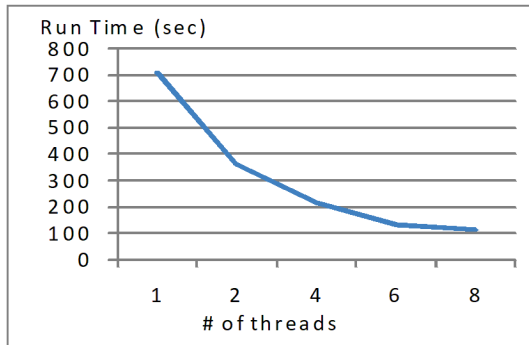
The following sections discuss these topics:

- [Multithreading Acceleration for S-element on Linux](#)
- [Ensuring Causality in the Rational Function Model](#)
- [Rational Function Matrix \(.rfm\) File Format](#)

Multithreading Acceleration for S-element on Linux

One of the benefits of using the rational function model is that since the function can be modeled independent of the transient simulation configuration, generated rational function data can be reused in subsequent simulation runs. To reduce the computational time for this process, the S-element takes the number of threads specified on the command-line invocation of `-mt` as the maximum number of threads to be used. See Running Multithreading (MT) and Distributed Processing (DP) Concurrently in the *PrimeSim HSPICE User Guide: Basic Simulation and Analysis*. The actual number of threads to be used can be smaller depending on the size of the target S-parameters.

Figure 51 Wall clock time for the rational function generation of a 131 port S-parameter block (Linux system with 8 x 2666 MHz Intel® Xeon®)



Ensuring Causality in the Rational Function Model

When using the rational function model, PrimeSim HSPICE enforces the causal behavior. If `RATIONAL_FUNC` is set to 1, the original transfer function is approximated as a summation of the partial rational functions which can be proved to be causal:

$$f_k = \frac{a_k}{\omega_k + s}$$

Therefore, the resulting function must be causal.

In the rational function generation process, potential unstable poles located in the righthand side of the complex plane are filtered out automatically.

Rational Function Matrix (.rfm) File Format

In addition to the rational function matrix (*.yrf) file discussed in the previous section, PrimeSim HSPICE provides syntax for users or third parties to create an ASCII representation of the rational function matrix. The resulting *.rfm file can then be read by the S-element via the S-model `RFMFILE=file_name.rfm` keyword.

The *.rfm file (text file) interface allows you to input transfer function (matrix) in the form of rational function directly instead of using s-parameters. If you have an extraction tool that creates a rational function model directly, using it may reduce numerical errors rather than let the tool convert the function into s-parameters. PrimeSim HSPICE then extracts the rational function from there.

The *.rfm file is divided into two parts:

- The header consists of keywords and setup information for the entire system. This section (first five lines in the following example) contains information about the data that follows, such as number of ports, matrix type, preconditioning factor, and reference impedance.
- The data field consists of rational function coefficients of each matrix component. Each matrix component begins with a BEGIN keyword and ends with the END keyword.

```
Version 200600
NPORT 2
MATRIX_TYPE Y
PRECFACT 0.75
Z0 50

BEGIN 1 1
CONST 0.0
C 0.0
DELAY 0.0
BEGIN_REAL 2
3.50774e+07 -4.54754e-05
2.37196e+08 -0.00327245
BEGIN_COMPLEX 2
3.81668e+08 3.74508e+08 0.00583496 -2.54387
6.88144e+08 2.08242e+08 6.66955e-06 -2.78498
END
```

A single line can only contain single pairs of pole and residue. Therefore, two numbers must appear in a line for a real pole and four numbers must appear in a line for a complex pole. A single complex pole represents a complex conjugate pair of poles. An *.rfm file does not need to include all the matrix components. If certain terms are not found, the S-element regards these terms as ones with no propagation. The comment special character is an exclamation point. Lines that begin with '!' are ignored.

An RFM keyword (with no whitespace) is always the first word on the new line. The following table lists available keywords.

Keyword	Description
VERSION <i>n</i>	Version number.
NPORT <i>n</i>	Number of ports.
MATRIX_TYPE [<i>S</i> <i>Y</i> <i>Z</i>]	Currently, S and Y are supported.
SYMMETRIC	Indicates symmetric matrix. Only a single declaration must appear in the data field for transposing of pair of nondiagonal matrix components.

Keyword	Description
<code>Z0 val(s)</code> or <code>Zo val(s)</code>	Reference impedance of ports. Real number impedance only. When a single value is specified, the value is applied to all the ports. A vector of values with the size of the number of port also can be specified. A single line can only contain single number.
<code>PRECFAC val</code>	Preconditioning factor; must be between 0.5 and 1.0 ($0.5 < < 1.0$).
<code>BEGIN row col</code>	Beginning of a matrix component specified by row and column. The row and column must be 1-based index of the matrix component.
<code>CONST val</code>	Constant term of the rational function “B” term of Equation 39; if not specified, <code>CONST=0</code> .
<code>C val</code>	Reactive term of the rational function “C” term of Equation 39; if not specified, <code>C=0</code> .
<code>DELAY val</code>	Propagation delay from port[col] to port[row]. Must be zero or a positive number. If not specified <code>DELAY=0</code> .
<code>BEGIN_REAL n</code>	Pairs of real poles and residues follow. Following each line must contain a real pole and a real residue in this order. If <code>BEGIN_REAL</code> is not specified, no real pole is constructed. Other keywords must appear before <code>BEGIN_REAL</code> .
<code>BEGIN_COMPLEX n</code>	Pairs of complex pole and residue follows. Following each line must contain a real part and an imaginary part of pole, real and imaginary part of residue in this order. A single complex pole and residue pair represents a conjugate pair of poles. If <code>BEGIN_COMPLEX</code> is not specified, no complex pole is constructed. Other keywords must appear before <code>BEGIN_COMPLEX</code> .
<code>END</code>	End of the matrix component.

Multiport Noise Model for Passive Systems

Multiport passive and lossy circuits, such as transmission lines and package parasitics, can exhibit considerable thermal noise. The passive noise model is used to present such thermal noise for the S-element representing such circuits. The S-element passive noise model supports normal, two-port and multiport noise analysis (`.NOISE=1`) and `.LIN noisecalc=1` for two-port and `.LIN noisecalc=2` for N-port.

The following sections discuss these topics:

- [Input Interface](#)
- [Output Interface](#)

Input Interface

To trigger a passive multiport noise model, the `NOISE` and `DTEMP` keywords in an S-element statement are used:

```
Sxxx n1...nN
+ ...
+ [NOISE=[1|0]] [DTEMP=value]
```

See [S Model Syntax](#) for the description of the `NOISE` and `DTEMP` keywords.

When `NOISE=1`, PrimeSim HSPICE generates a $N \times N$ noise-current correlation matrix from the $N \times N$ S-parameters according to Twiss' Theorem. The result can be stamped into a PrimeSim HSPICE noise analysis as N-correlated noise current sources: j_i ($i=1 \sim N$) as shown [Equation 42](#):

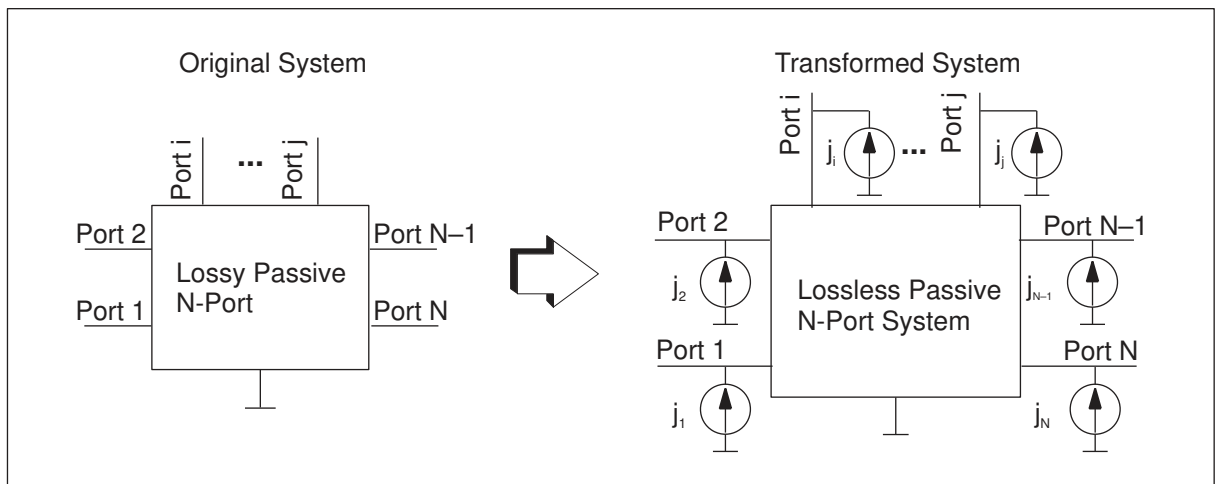
Equation 42

$$C = 2kT(Y + Y^*T) = \begin{bmatrix} \overline{|j_1|^2} & \overline{j_1 j_2^*} & \dots & \overline{j_1 j_N^*} \\ \overline{j_2 j_1^*} & \overline{|j_2|^2} & \dots & \overline{j_2 j_N^*} \\ \dots & \dots & \dots & \dots \\ \overline{j_N j_1^*} & \overline{j_N j_2^*} & \dots & \overline{|j_N|^2} \end{bmatrix}$$

where: $Y = Y_c(I - S)(I + S)^{-1}$

The noise-current correlation matrix represents the frequency-dependent statistical relationship between N noise current sources, j_i ($i=1 \sim N$), shown in [Figure 52](#).

Figure 52 Frequency-dependent statistical relationship between N noise current sources



Output Interface

PrimeSim HSPICE creates a .lis output list file that shows the results of a noise analysis just as any other noisy elements. The format is as follows:

```
**** s element squared noise voltages (sq v/hz)

      element          0:s1
      N(i,j)           data
      r(N(i,j))        data
      ... i,j = 1~N ...
      total            data
```

where:

- $N(i,j)$ = contribution of $j j_i^*$ to the output port.
- $r(N(i,j))$ = transimpedance of j_i to the output port.
- $total$ = contribution of total noise voltage of the S-element to the output port.

S-element Noise Model

This section describes how the S-element supports two-port noise parameters and multiport passive noise models.

The following sections discuss these topics:

- [Two-port Noise Parameter Support in Touchstone Files](#)
- [Input Interface](#)
- [Output Interface](#)
- [Notifications and Limitations](#)

Two-port Noise Parameter Support in Touchstone Files

The S-element can read in two-port noise parameter data from Touchstone data files, and then transform the raw data into a form used for .NOISE and .LIN noisecalc=1[or 2] analysis.

For example, you can represent a two-port with an S-element and then perform a noise analysis (or any other analysis). The S-element noise model supports normal and two-port (.NOISE and .LIN noisecalc=1). See [Noise Parameters in 2-Port and N-Port Networks](#).

Note:

Because Touchstone files currently provide only two-port noise parameters, this type of noise model only supports two-port S-parameter noise analysis for both passive and active systems.

Input Interface

The frequency-dependent two-port noise parameters are provided in a network description block of a Touchstone data file following the S-parameter data block.

The noise parameter data typically is organized by using the following syntax:

```
frequency[Hz] Nfmin[dB] GammaOpt (M) GammaOpt (P) RN/Z0
{ ...data... }
```

where:

- frequency = frequency in units.
- Nfmin[dB] = minimum noise figure (in dB).
- GammaOpt (M) = magnitude of reflection coefficient needed to realize Fmin.
- GammaOpt (P) = phase (in degrees) of reflection coefficient needed to realize Fmin.
- RN/Z0 = normalized noise resistance.
- ! = indicates a comment line.

For example:

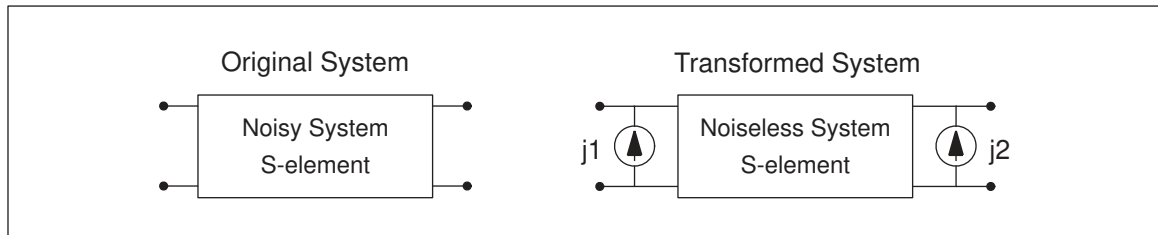
```
! 2-port noise parameter
! frequency[Hz] Nfmin[dB] GammaOpt (M) GammaOpt (P) RN/Z0
0.0000 0.29166 0.98916 180.00 0.11055E-03
0.52632E+08 6.2395 0.59071 -163.50 0.32868
0.10526E+09 7.7898 0.44537 175.26 0.56586
```

Both GammaOpt and RN/Z0 values are normalized for characteristic impedance, Z_0 , specified in the header of the Touchstone data file. PrimeSim HSPICE reads this raw data and converts it to a coefficient of the noise-current correlation matrix. This matrix can be stamped into a PrimeSim HSPICE noise analysis as two correlated noise current sources: j_1 and j_2 , as shown here:

$$C = \begin{bmatrix} \overline{|j_1|^2} & \overline{j_1 j_2^*} \\ \overline{j_2 j_1^*} & \overline{|j_2|^2} \end{bmatrix}$$

The noise-current correlation matrix represents the frequency-dependent statistical relationship between two noise current sources, j_1 and j_2 , as illustrated in [Figure 53](#).

Figure 53 Frequency-dependent statistical relationship between two noise current sources



Output Interface

PrimeSim HSPICE creates a .lis output list file that shows the results of a noise analysis just as any other noisy elements. The format is:

```
**** s element squared noise voltages (sq v/hz)
element      0:s1
  N11         data
r(N11)        data
  N12         data
r(N12)        data
  N21         data
r(N21)        data
  N22         data
r(N22)        data
  total       data
```

where:

- $N11$ = contribution of j_1 to the output port
- $r(N11)$ = transimpedance of j_1 to the output port
- $N12$ = contribution of $j_1 j_2^*$ to the output port
- $r(N12)$ = transimpedance of j_1 to the output port
- $N21$ = contribution of $j_2 j_1^*$ to the output port
- $r(N21)$ = transimpedance of j_2 to the output port
- $N22$ = contribution of j_2 to the output port
- $r(N22)$ = transimpedance of j_2 to the output port
- $total$ = contribution of total noise voltage of the S-element to the output port

Notifications and Limitations

- Because Touchstone files currently provide only two-port noise parameters, this type of noise model only supports two-port S-parameter noise analysis for both passive and active systems.
- If your Touchstone file includes square brackets in a z_0 definition, PrimeSim HSPICE does not support the square brackets. Acceptable syntax is to list the z_0 values without any brackets.

For example:

```
.model s_par s tstonefile='tsn.s4p'  
+ z0=50 50 50 50
```

For readability, parentheses can be used.

```
.model s_par s tstonefile='tsn.s4p'  
+ z0=(50 50 50 50)
```

S-parameter Standalone Manipulation Utility

PrimeSim HSPICE provides several capabilities to manipulate S-parameters including accepting multiple file formats, inter-/extrapolation schemes, data smoothing, rational function approximation, and so on. These capabilities become available when an S-element is specified in a PrimeSim HSPICE circuit netlist and passed to PrimeSim HSPICE. The following sections describe a standalone executable (`sputil`), which is available on Linux and Windows. This standalone utility program is called *SPutil*. This utility allows you to access these S-element features without invoking a PrimeSim HSPICE simulation and additional utility functions. Also, this utility may be used as a tool to predict and verify the behavior of given S-parameters in advance of PrimeSim HSPICE simulation runs.

Note:

You can use `sputil` to manipulate the S-parameters that you plan to use in your W-element. (Remember that the length of the transmission lines that you have extracted S-parameters for must be specified when you use them in a W-element.)

- [SPutil Program Features](#)
- [Invoking the Utility](#)
- [SPutil Runset Format](#)
- [Commands](#)

- [SPUTIL Functions](#)
- [Status Messages](#)
- [Causality Checking](#)
- [Using Interpolation for Causality Checking](#)

SPutil Program Features

SPutil performs many of the functions which are built into the PrimeSim HSPICE S-element. In addition, you can add utility functions on demand. The following list categorizes the current set of *SPutil* functions:

- Read in and combine s/y parameter files
- Interpolate and/or extrapolate necessary output data points
- Passivity check and enforcement
- Rational function approximation
- Output manipulated S-parameter data
- Format conversion (S/Y/Z, Touchstone-v1,v2, CITI file)
- Re-sampling with interpolation, extrapolation, or windowing with multiple sweep blocks
- Causality checking
- Change reference impedance
- Partial view and zero padding
- Port swapping, reordering, and picking
- Connection checking

Invoking the Utility

Note:

The *SPutil* program requires that you check out a PrimeSim HSPICE license.

Run a set of S-parameter manipulation commands in the input file by typing:

```
sputil runset.in
```

SPutil Runset Format

The *SPutil* runset file must contain a list of commands and arguments. The following example shows the basic structure of a *SPutil* runset. Command names must be compatible with PrimeSim HSPICE S-element/model keywords and other PrimeSim HSPICE netlist keywords. *SPutil* directly calls the PrimeSim HSPICE S-element object library to obtain identical behavior to the corresponding control keyword available in the PrimeSim HSPICE S-element.

This example combines three S-parameter files, `ts1.s4p`, `ts2.s4p` and `citi1.citi`, into one Touchstone (version 1) file, and writes results to the `output.s4p` file with a combined list of frequency points specified by the `FREQ_SWEEP` keyword:

```
NPORT 4
TSTONEFILE ts1.s4p
TSTONEFILE ts2.s4p
CITIFILE citi1.citi
INTERPOLATION LINEAR
HIGHPASS 3
LOWPASS 1
TSTONE_OUT output.s4p
FREQ_SWEEP DEC 10 1.0 10e9
```

Note:

1. Accepts both "!" and "*" as comment characters in the runset file.
2. Allows the first line to be a blank line in the runset file.
3. Ignores spaces or extra line feeds at the end of the runset file.

In cases where you want to connect capacitors and subsequently reduce the S-parameter ports, you can use the `MAKEDOTLIN` command. An example command set looks like this:

```
NPORT 4
TSTONEFILE dc.s4p
DATAFORMAT MA
MAKE_DOT_LIN example.sp
```

This command set creates an `example.sp` file, which is modified to remove ports 3 and 4 and add capacitors for nets connected to ports 3 and 4. Running PrimeSim HSPICE on the modified file gives the reduced 2 port S-parameter file.

Commands

A command keyword must always be the first word on the new line. The command keyword is always one continuous word without embedded spaces.

Table 3 Current SPutil Command Set

Keyword	Description
<i>S-element keywords.</i> See also S-element Syntax in <i>PrimeSim Continuum User Guide: Elements</i> for full keyword descriptions.	
NPORT <i>n</i>	Number of ports (optional when the input file is in the Touchstone version 1 (* . sNp) format). Note: Multiple declarations of <i>different</i> NPORT keywords are invalid.
MNAME <i>modelname</i>	Model name used for output.
TYPE [S Y Z]	Parameter type. Currently, only Y and S are supported. Default is S.
TSTONEFILE <i>filename</i>	Specifies a Touchstone file (version 1.0/2.0).
TSTONE_HEADER <i>filename</i>	Specifies the name of the file which contains the necessary comments for the Touchstone file header. SPUTIL will copy all the lines before the index line (that is, the line beginning with "#") from the specified file to the output Touchstone file.
CITIFILE <i>filename</i>	Specifies a CITI file.
BNPFILE <i>filename</i>	Specifies Broadband Network Parameter (BNP) file.
FBASE <i>val</i>	FBASE value for S-element.
FMAX <i>val</i>	FMAX value for S-element.
INTERPOLATION [STEP LINEAR SPLINE HYBRID]	INTERPOLATION setting for S-element.
INTDAT TYP [RI DBA MA]	INTDAT TYP setting for S-element.
HIGHPASS <i>n</i>	HIGHPASS setting for S-element.
LOWPASS <i>n</i>	LOWPASS setting for S-element.
DELAYHANDLE <i>val</i>	DELAYHANDLE setting for S-element.
DELAYFREQ <i>val</i>	DELAYFREQ value for S-element.
RATIONAL_FUNC <i>n</i>	Rational function setting for S-element.
RATIONAL_FUNC_FOR_AC <i>n</i>	Accesses to the original s-parameter data.

Table 3 Current SPutil Command Set (Continued)

Keyword	Description
PASSIVE [0 1]	Activates the passive checker to help debug passive models. The default is 0 for the S-element where 0=deactivate and 1=activate. Using the tolerance value specified by <code>PASSIVE_TOL</code> keyword, PrimeSim HSPICE checks the eigenvalues of matrix $(I-S^*S')$, $ev[i]$. - If any frequency point violates $RE(ev[i]) > -(TOL*0.1)$, PrimeSim HSPICE issues a warning containing a list of violating frequencies with an "E" flag. Also, the checker verifies potential passivity violations by checking the summation of each S-parameter matrix column. - If $Sum > COLSUM_LIMIT$, <i>SPutil</i> issues a warning containing a list of those violating frequencies with a "C" flag.
PASSIVE_TOL val	Tolerance for eigenvalue checking activated by <code>PASSIVE</code> keyword. Default value is 0.01.
COLSUM_LIMIT val	Maximum value allowed for S-parameter matrix column summation. The limiting value will be used for passivity checker and passivity enforcement. Default value is 1.0
ENFORCE_PASSIVE [0 1]	When <code>enforce_passive 1</code> is set, <i>SPutil</i> enforces <code>PASSIVE</code> setting for the S-parameter. Default is 0 (off).
PRECFACT	Preconditioning Factor value for S-element. Nonzero <code>precfac</code> is used only when y-parameter output is requested with S-parameter input.
Control Keywords	
TSTONE_OUT tstone_name	Output Touchstone v. 1.0 file name. If this file name is not provided in causality enforcement, an error message is displayed.
TSTONE2_OUT	Specifies Touchstone v. 2.0 output file.
CAUSAL CHECK	This command invokes causality checks (see Causality Checking). If the command <code>CAUSAL</code> is not followed by an argument, <i>SPUTIL</i> signals an error. Every time this command is invoked, a *.lis file includes a list of the worst causal violations for any port combination that is affected. If a port combination is not affected, it does not appear in that file. Additionally, a Touchstone RI file brings in its real part the value of its Global Difference Measure (GDM), while in the imaginary part it gives a Boolean value according to the threshold entered in <code>CAUSAL_TOL</code>.
CAUSAL_TOL value	This command places as target for causality a value for the threshold. The value starts at 0.0 (causal). If no value is given, the threshold is set to its default value: 0.1.

Table 3 Current SPutil Command Set (Continued)

Keyword	Description
CITIFILE_OUT <i>citifile_name</i>	Output CITI file name.
SELM_OUT <i>selm_file_name</i>	Output sc# file name.
OUT_PORT_BEGIN <i>n</i>	When partial S-parameter matrix output is required, specifies the beginning port index number in the output data. Default value is 1.
OUT_PORT_END <i>n</i>	When partial S-parameter matrix output is required, specifies the ending port index number in the output data. Default is NPORT.
ZREF_OUT <i>value values</i>	Specifies the reference impedance value(s) of the output. A single value of the argument sets the given impedance to all the ports. If a vector (of size the same as the number of ports) is given, each impedance value is assigned to the corresponding port.
SPARDIGIT <i>n</i>	Sets the numerical precision (the number of digits) for the S-parameter output.
FREQDIGIT <i>n</i>	Sets the numerical precision (the number of digits) for the frequency output.
DATAFORMAT	Specifies the format of the output data file: • RI: real-imaginary • MA: magnitude-phase (default format for Touchstone files) • DB: DB(magnitude)-phase
FREQ_SWEEP <i>hspice_sweep</i>	Specifies frequency points in output data file. You can specify any of ORG, LIN, DEC, OCT, or POI. ORG takes frequency points from the input data file, while others specify various frequency sweeps with <i>nsteps</i> , <i>start</i> , and <i>stop</i> values using the following syntax for each type: • LIN <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • POI <i>nsteps freq_values</i> • ORG [<i>fmin fmax</i>]

Table 3 Current SPutil Command Set (Continued)

Keyword	Description
	FREQ_SWEEP with the ORG keyword picks up frequency points given in the input data. The optional <i>f_{min}</i> and <i>f_{max}</i> values, to the ORG keyword, limit the frequency range to be picked up. If the FREQ_SWEEP keyword is not specified, then PrimeSim HSPICE chooses output frequency points as the combination of the input frequency point. For example: If the first input file has f=1 2 3 4 and second input file has f=5 6 7, then the output frequency points (by default, with no FREQ_SWEEP) are f=1 2 3 4 5 6 7.
SYMMETRY_CHECK [0 1]	Default: 0. When SYMMETRY_CHECK 1 is specified, the keyword checks symmetry of the S-parameter matrix at each frequency point (especially useful in evaluating differential signaling systems where preserving symmetric S-parameter matrices is key).
COLUMN_SUM [0 1]	Default: 0. When column_sum 1 is specified, the keyword computes the summation of each column of the S-parameter matrix at each frequency point. A summation of each column indicates whether or not corresponding incidents are amplified according to: $\sum_k S_{kj} a_j$ See Note on the COLUMN_SUM Keyword. Results are printed in an <i>input_filename.colsum</i> text file which can be uploaded to the waveform viewer as a Text/PWL data file. The output file contains <i>real</i>, <i>imaginary</i>, and <i>magnitude</i> as functions of frequency for each column summation.
TO_MIXEDMODE [0 1]	When set to 1, S-parameters will be converted to mixed mode assuming each pair of adjacent terminals forms one differential port. For example, the first and the second ports of the original S-parameter block will be the positive and negative terminals of the first differential port in the resulting mixed mode S-parameter block, respectively.
SWAP_PORT p1 p2	This command swaps the ordering of the two specified ports of the S-parameters.
REORDER_PORT io2bus bus2io port_indices	Swaps the port ordering method used for the S-parameters between <i>I/O pair ordering</i> and <i>bus ordering</i> conventions. io2bus changes from <i>I/O pair ordering</i> (i1 o1 i2 o2...) to <i>bus ordering</i> (i1 i2... o1 o2...). bus2io changes from <i>bus ordering</i> (i1 i2... o1 o2...) to <i>I/O pair ordering</i> (i1 o1 i2 o2...). port_indices, which is an array of integers, specifies port indices for custom port reorder.

Table 3 Current SPutil Command Set (Continued)

Keyword	Description
<code>PICK_PORT indices</code>	Takes an array of S-parameter port indices and outputs only the S-parameters of the specified ports in the specified order. The argument can contain the <code>TO</code> keyword in between two indices. The <code>TO</code> keyword represents all the ports from the first index to the next index in incremental order. For example, <code>2 TO 5</code> is interpreted as <code>2 3 4 5</code>. This command allows you to create a subset of the target system.
<code>REDUCE_W_OPEN array_of_indices</code>	Specifies an array of port indices that are set to open termination, when performing port reduction using the <code>PICK_PORT</code> command.
<code>REDUCE_W_SHORT array_of_indices</code>	Specifies an array of port indices that are set short to the ground/supply termination, when performing port reduction using the <code>PICK_PORT</code> command.
Note: • <code>REDUCE_W_OPEN</code> and <code>REDUCE_W_SHORT</code> arguments accept the <code>TO</code> keyword to represent an array of incremental indices from the left index to the right index. For example, <code>REDUCE_W_OPEN 4 7 to 10</code> is equivalent to <code>REDUCE_W_OPEN 4 7 8 9 10</code>. • <code>REDUCE_W_OPEN</code> and <code>REDUCE_W_SHORT</code> can be specified multiple times. For example, <code>REDUCE_W_SHORT 1 2</code> and <code>REDUCE_W_SHORT 3 4</code> are equivalent to <code>REDUCE_W_SHORT 1 2 3 4</code>. • When performing port reduction using the <code>PICK_PORT</code> command, for ports not specified for either open or short termination, SPUTIL terminates them with the reference impedance.	
<code>SCALE_FREQ scale_factor</code>	Every frequency point in the output file is scaled by this factor.
<code>CHECK_CONNECTION port_index all</code>	Prints all the connections from the specified <code>port_index</code>, in the order of strength. If <code>all</code> is specified, the connection list will be generated for all ports. SPUTIL recognizes a connection between a port <i>n</i> and a port <i>m</i>, when the magnitude of S_{mn} (port <i>n</i> to port <i>m</i> direction) exceeds <code>CONNECTION_TOL</code> at any frequency point.
<code>CONNECTION_TOL value</code>	Specifies the tolerance for the <code>CHECK_CONNECTION</code> command. The default value of this tolerance is 0.1.
<code>XLINELength value</code>	Specifies the line length of the transmission line system from which the S parameter model is extracted. This must be exactly the same as the line length of the extracted system. For information on usage, see RLGC Model Output from S-parameters.
<code>WRITE_XLINE_PARAMS = prefix</code>	Specifies the prefix of the output RLGC file name. For information on usage, see RLGC Model Output from S-parameters.

Table 3 Current SPutil Command Set (Continued)

Keyword	Description
<pre>WINDOWING target Fstart Fend [tolerance [<i>tolerance_fstart</i> <i>tolerance_fend</i>]]</pre>	This command applies a function to roll off the magnitude terms from the initial magnitude value at the <i>Fstart</i> frequency to a value of zero at the <i>Fend</i> frequency. - The value of <i>target</i> can be one of the following: <i>all</i>: All terms. <i>minor</i>: Only minor terms where the maximum magnitude is below the <i>tolerance</i>. <i>major</i>: Only major terms where the maximum magnitude is above the <i>tolerance</i>. <i>diag</i>: Diagonal terms only. <i>offdiag</i>: Off diagonal terms only. <i>tolerance_fstart</i> and <i>tolerance_fend</i> specify the range of frequency for tolerance checking when <i>target</i> is specified as <i>major</i> or <i>minor</i>.
<pre>SCALE_PARAM target scale_value [tolerance [<i>tolerance_fstart</i> <i>tolerance_fend</i>]]</pre>	This command applies a scaling factor, <i>scale_value</i>, to target parameters. For a description of <i>target</i>, <i>tolerance</i>, <i>tolerance_fstart</i>, and <i>tolerance_fend</i> arguments, see the WINDOWING command description.
<pre>ADD_DELAY target delay_value [tolerance [<i>tolerance_fstart</i> <i>tolerance_fend</i>]]</pre>	This command adds a propagation delay, <i>delay_value</i>, to target parameters. For a description of <i>target</i>, <i>tolerance</i>, <i>tolerance_fstart</i>, and <i>tolerance_fend</i> arguments, see the WINDOWING command description.
<pre>MAKE_DOT_LIN filename.sp [start_index] [<i>prefix_string</i>]</pre>	Creates a PrimeSim HSPICE .LIN netlist, <i>filename.sp</i>. The netlist uses an S-element with the given S-parameter file. All the S-element terminals will be terminated with port elements. If the ZREF_OUT keyword is specified, port elements use the specified reference impedance value(s); otherwise, the value(s) specified in the S-parameter file will be used. Optionally, the starting port index can be specified by the <i>start_index</i> keyword. If <i>prefix_string</i> is given, the string will be added to the node name as well as the S-element name. When the given S-parameter file contains sampling frequency information, the resulting netlist will contain an .AC command with all the given frequency points, so that the AC analysis will not need to use interpolation/extrapolation.
<pre>! (comment)</pre>	Lines beginning with "!" or "*" are ignored.

Note on the COLUMN_SUM Keyword

Given incident vector **a**, reflected vector **b**, the S-parameter matrix **S** relates vector **a** and **b** as:

$$\mathbf{b} = \mathbf{S}\mathbf{a}.$$

Therefore, each term of the **b** vector expands as:

$$\begin{aligned} b_1 &= s_{11}a_1 + s_{12}a_2 + s_{13}a_3 \dots \\ b_2 &= s_{21}a_1 + s_{22}a_2 + s_{23}a_3 \dots \\ b_3 &= s_{31}a_1 + s_{32}a_2 + s_{33}a_3 \dots \\ &\dots \end{aligned}$$

As a result, in the output, **b** vector, the energy originated from the a_j term, becomes:

$$\begin{aligned} &(s_{1j} + s_{2j} + s_{3j} + \dots)a_j \\ &= \sum_k s_{kj}a_j \\ &= \text{column sum}[j]a_j \end{aligned}$$

Therefore, $\text{column sum}[j] > 1$ indicates that a_j is amplified.

RLGC Model Output from S-parameters

The `sPutil` command set includes two keywords `XLINELLENGTH` and `WRITEXLINELPARAMS` to output RLGC model information from an S-parameter model.

A `sPutil` script example to output RLGC model information using these two keywords:

```
*****sputil.sp*****
NPORT 4
TSTONEFILE test.s4p
XLINELLENGTH 0.0127
WRITEXLINELPARAMS = sputil_test
*****
```

The following five files are output:

- `$prefix_RLGC.inc`: The RLGC model information specification.
- `$prefix_R.inc`: The frequency-dependent R-parameter file.
- `$prefix_L.inc`: The frequency-dependent L-parameter file.
- `$prefix_G.inc`: The frequency dependent G-parameter file.
- `$prefix_C.inc`: The frequency dependent C-parameter file.

where `$prefix` is `sputil_test` for the preceding example.

Note:

Currently, *SPutil* only supports S-parameters with *nodemap=i1 i2...iN o1 o2...oN*. That is, the input ports are all specified before the output ports.

SPUTIL Functions

The following are the SPUTIL functions:

- `write_fitted_file`

Use the `write_fitted_file` function to generate an S-parameter file from the rational function file `file.srf`. The generated S-parameter file is then used to compare to the original S-parameter file from which the rational function is generated.

```
TSTONEFILE <input_sparam_file>
rational_func=1
write_fitted_file 1
```

- `write_bbs_file`

Use the `write_bbs_file` function to generate a broadband SPICE model from an S-parameter file. This function is used with `rational_func=1` for a broadband SPICE model.

```
TSTONEFILE <input_sparam_file>
rational_func 1
write_bbs_file 1
```

- `rfmfile`

Use the `rfmfile` function to manipulate a rational function matrix file (`.rfm`) in SPUTIL. The runset generates an S-parameters file from the RFM file. Frequency sweep is optional and used to specify a given frequency range in the output file.

Note:

Currently broadband SPICE model output is supported in S-domain rational function model only.

```
RFMFILE <input.rfm>
FREQ_SWEEP <sweep_type>
TSTONE_OUT <output_sparam_file>
```

Status Messages

Status messages are written in the command terminal as an *SPutil* runset proceeds. For example, when the following runset is executed:

```
NPORT 4
TSTONEFILE good_tx.s4p
PASSIVE 1
TSTONE_OUT output.s4p
FREQ_SWEEP DEC 10 1e3 1e9
```

The following types of messages might be returned:

```
% sputil test.in
Reading Touchstone File [good_tx.s4p]...

Read Touchstone File [good_tx.s4p]

s-params: 801 pts 300000Hz - 8.5e+09Hz
no noise parameter read

Performing passivity check...

Warning: s-model [good_tx.s4p] passivity violation detected...
Writing Touchstone File [output.s4p]... done
      See "test.lis" for results and statistics.
```

Causality Checking

- The SPUTIL's causality checker checks the causality of the given frequency points in the raw S-parameter.
- The IFFT causality enforcement during simulation in the PrimeSim HSPICE and PrimeSim HSPICE tool is to ensure causality when preparing data for the IFFT algorithm throughout the extrapolated frequency points. The SPUTIL causality checker is used to check the quality of the raw S-parameters prior to causality enforcement during simulation.

To check the sampled data included in a scattering-parameters Touchstone-file, the input file should look like:

```
TSTONEFILE rlc128p_7169_0_1e11.s128p
CAUSAL CHECK
CAUSAL_TOL 0.953496955
```

In this example, the input file name represents a 128 port-network (whose data has been sampled 7169 times for a frequency range that goes from 0 Hz to 1e11 Hz). Causality checking is asked using a GDM better than 0.953496955.

There are two output files. One of them is a *.lis file that displays a short summary as in the following example:

```
Parsing rlc128p_7169_0_1e11.in ...
Reading TSTONEFILE rlc128p_7169_0_1e11.s128p.....
Touchstone File [rlc128p_7169_0_1e11.s128p] s-params: 7169 pts 0Hz -
1e+11Hz
No noise parameter read.
s-parameters: 7169 pts 0 - 1e+11
no noise parameter read
The three worst e-average Global Difference Measures (1.0 >= eGDM >= 0.0)
for 8192 points are listed below.
Port combination [58,122], its average eGDM is: 0.987236
Port combination [122,58], its average eGDM is: 0.987236
Port combination [3,68], its average eGDM is: 0.988165
Average eGDM for the input Scattering Matrix: 0.999882
Note: 1.0 >= eGDM > 0.65 is good. 0.65 >= eGDM > 0.2 is fair.
0.20 >= eGDM is poor.
More details can be seen by plotting rlc128p_7169_0_1e11REP.s128p
SPUTIL concluded
```

The other output file is an RI Touchstone file where the suffix .s N_p is replaced by REP.s N_p (with N being the number of ports). The file holds, for each frequency point, the GDM value in the corresponding real part and a causality boolean in the imaginary part. That is, one for each GDM is equal to or less than the threshold given in CAUSAL_TOL, or zero when the GDM is greater than that threshold.

For the given example, the name of the second output file is
rlc128p_7169_0_1e11REP.s4p.

Using Interpolation for Causality Checking

When the SPUTIL CAUSAL CHECK command is used, interpolation might be required. The causality checking uses FFT. So, the number of frequency points must be an even power of 2.

The interpolation is defined as follows:

- Number of frequency points after the interpolation:

$$n = 2^{\text{round}(\log_2(k))}$$

- Frequency spacing:

$$\Delta f = f_{\max} / (n-1)$$

In the preceding example, the number of frequency points, k , is 7169 and the maximum frequency, $f_{\max}$, is 100GHz. The number of frequency points is interpolated to 8192

evenly-spaced points. The number of frequency points used in the causality check global difference measurements are printed to the *.lis file.

Preconditioning S-parameters

Certain S-parameters, such as series inductor (2-port), show a singularity when converting S- to Y-parameters. To avoid this singularity, the S-element adds kR_{ref} series resistance to precondition S matrices:

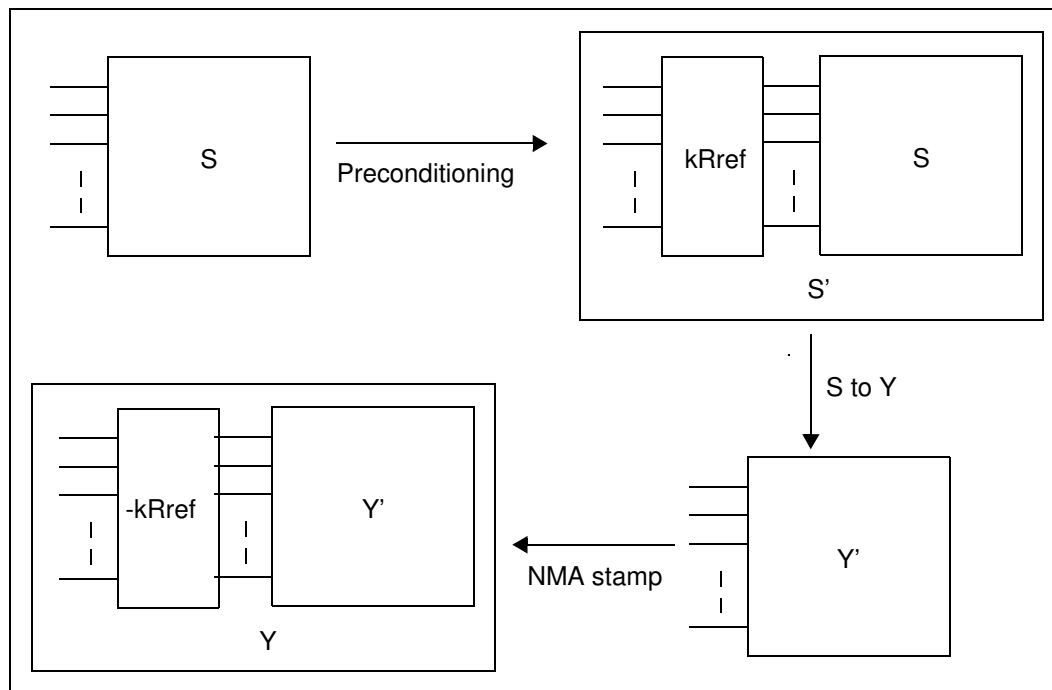
Equation 43

$$\mathbf{S}' = [k\mathbf{I} + (2-k)\mathbf{S}][(2+k)\mathbf{I} - k\mathbf{S}]^{-1}$$

- kR_{ref} is the reference impedance vector.
- k is the preconditioning factor.

To compensate for this modification, the S-element adds a negative resistor ($-kR_{ref}$) to the modified nodal analysis (NMA) matrix in actual circuit compensation. To specify this preconditioning factor, use the `PREFAC` keyword in the S model statement. The default preconditioning factor is 0.75.

Figure 54 Preconditioning S-parameters



Group Delay Handler in Time Domain Analysis

The S-element accepts a constant group delay matrix in time-domain analysis. You also can express a weak dependence of the delay matrix on the frequency as a combination of the constant delay matrix and the phase shift value at each frequency point. To activate or deactivate this delay handler, specify the `DELAYHANDLE` keyword in the S model statement.

The delay matrix is a constant matrix, which PrimeSim HSPICE extracts using finite difference calculation at selected target frequency points. PrimeSim HSPICE obtains the $T_{\omega(i,j)}$ delay matrix component as:

Equation 44

$$T_{\omega(i,j)} = \frac{d\theta_{Sij}}{d\omega} = \frac{1}{2\pi} \cdot \frac{d\theta_{Sij}}{df}$$

- f is the target frequency, which you can set using `DELAYFREQ`. The default target frequency is the maximum frequency point.
- θ_{Sij} is the phase of S_{ij} .

After time domain analysis obtains the group delay matrix, Equation 45 eliminates the delay amount from the frequency domain system-transfer function:

Equation 45

$$y'_{mn(\omega)} = y_{mn(\omega)} \times e^{j\omega T_{mn}}$$

The convolution process uses Equation 46 to calculate the delay:

Equation 46

$$i_{k(t)} = (y'_{k1(t)}, y'_{k2(t)}, \dots, y'_{kN(t)}) \times (v_{1(t-T_{K1})}, v_{2(t-T_{K2})}, \dots, v_{N(t-T_{KN})})^T$$

S-Model Data Smoothing

Four smoothing functions are provided for the S-model. Each of these is available for the S-element and W-element. Scattering parameters are frequently given from measurement instruments such as vector network analyzers (VNA). In measurements, there are many causes of noise injection such as calibration failure, electromagnetic interference (EMI) and so on, especially in high frequency range. For such cases, several data smoothing functions are available to the S-parameter data reader to restore the original noiseless data.

Data smoothing alters the original data to suppress unwanted noise. Therefore, if you are confident of the accuracy of the original data, data smoothing is not recommended.

Data Smoothing Methods

Each smoothed data at i th point $S'i$ is given as a function of original data S_i and its neighbors as:

$$S'i = S_{i-width}, \dots, S_i, \dots, S_{i+width}$$

Four functions for data smoothing are provided:

- Mean: take the average value of $S'i = S_{i-width}, \dots, S_i, \dots, S_{i+width}$
- Median: take the value situated in the middle of: $S'i = S_{i-width}, \dots, S_i, \dots, S_{i+width}$
- 2nd order polynomial fit: perform least square fitting of: $S'i = S_{i-width}, \dots, S_i, \dots, S_{i+width}$ with 2nd order polynomial, and then compute the value at i th frequency.
- 4th order polynomial fit: perform least square fitting of: $S'i = S_{i-width}, \dots, S_i, \dots, S_{i+width}$ with 4th order polynomial, and then compute the value at i th frequency.

The following sections discuss these topics:

- [S-Model Syntax](#)
- [Description](#)
- [Example](#)

S-Model Syntax

```
.model model_name S ....
+ [SMOOTH=val] [SMOOTHPTS=val]
```

Default 0

See [S Model Syntax](#) for the description of the SMOOTH and SMOOTHPTS keywords.

Description

Each smoothing function has different characteristics. You should observe the original data on the waveform viewer when determining the smoothing filter configuration. Typically, the average function has a strong ability of smoothing, but it may lose the necessary bumps in data if they are narrow. Since both the average and median (SMOOTH=1 or 2) takes an intermediate value of the points within the specified window, if these reference points have a wide range of phase difference due to sparse frequency sampling, the smoothing result may lose accuracy.

The Median filter is effective if there are sharp and high noise spikes. These spikes are eliminated by the median filter without changing the offset level. Polynomial fittings are relatively weak in data smoothing, but they preserve narrow bumps. Typically, for

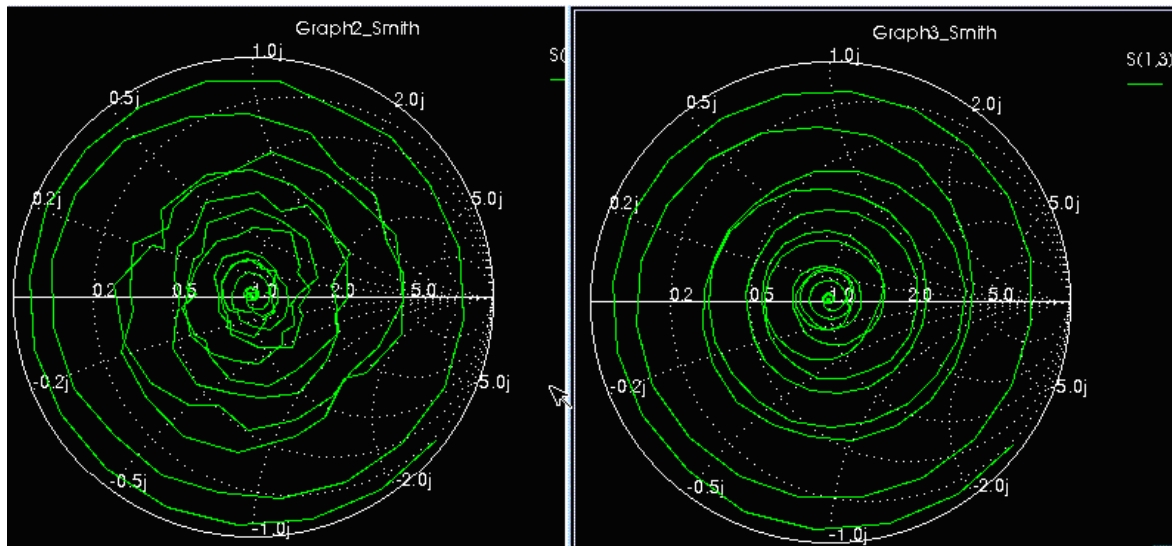
transmission line type S-parameters, polynomial fittings are effective since sinusoidal curves (many narrow bumps) are expected in real and imaginary versus frequency plots due to constant propagation delay.

The 2nd order and 4th order polynomial smoothing methods preserve the overall waveform trend better than the former two methods. However, they are relatively weak in strong, narrow range noises.

Example

The plot on the left side of [Figure 55](#) shows the original measurement data for the propagation term of the differential pair of transmission lines. With `SMOOTH=3` `SMOOTHPTS=5`, 2nd polynomial fitting with 11 points (5 points from each side in addition to the target point) is applied. The plot on the right side shows smoothed data.

Figure 55 Using the smoothing keywords on S Model data



Predicting an Initial Value for FMAX in S-Element Models

When selecting a starting point for the `FMAX` parameter in your S-parameter, it is important to set `FMAX` high enough to account for the fastest edges and higher order harmonics in the input waveforms. There are two methods to determine a starting point for setting `FMAX`. Both these methods require performing a transient analysis and are only meant to provide an initial value. Always check your results to ensure you are getting the accuracy you need. Also, setting `FMAX` without having enough data present in your S-parameter data file may result in extrapolation errors.

Method 1: Based on Risetime using the Knee Frequency

This method is handy for TDR-type simulations where the incident wave has only one rising or falling edge. Most energy in digital pulses concentrates below the knee frequency. The behavior of a circuit at the knee frequency determines its processing of a step edge. The knee frequency for any digital signal is related to the rise and fall time of its digital edges, but not its clock rate. If you want to pass a certain rise time with little degradation, you need the medium it propagates through to be about two times the knee frequency. The knee frequency can be calculated based on a 10–90% or 20–80% risetime measurement. For 10–90%, $FKNEE = (.35/Trise)$. For 20–80%, $FKNEE = (.5/Trise)$. For example, the F_{MAX} needed for a 25-ps risetime measured at 10–90% of the rising edge is $2 \cdot \left(\frac{.35}{25ps} \right) = 28GHz$.

Method 2: Using FFT

You can use the waveform calculator in WaveView to check the frequency and eleventh harmonic.

In WaveView:

1. Select the data signal.
2. Open the Waveform Calculator.
3. Paste the waveform into the calculator with the middle mouse button.
4. Click the **WAVE** button, and select **FFT**.
5. Modify the number of points and start/stop times if desired. Click **OK**.
6. Click the **Graph X** button to plot the FFT.

Note:

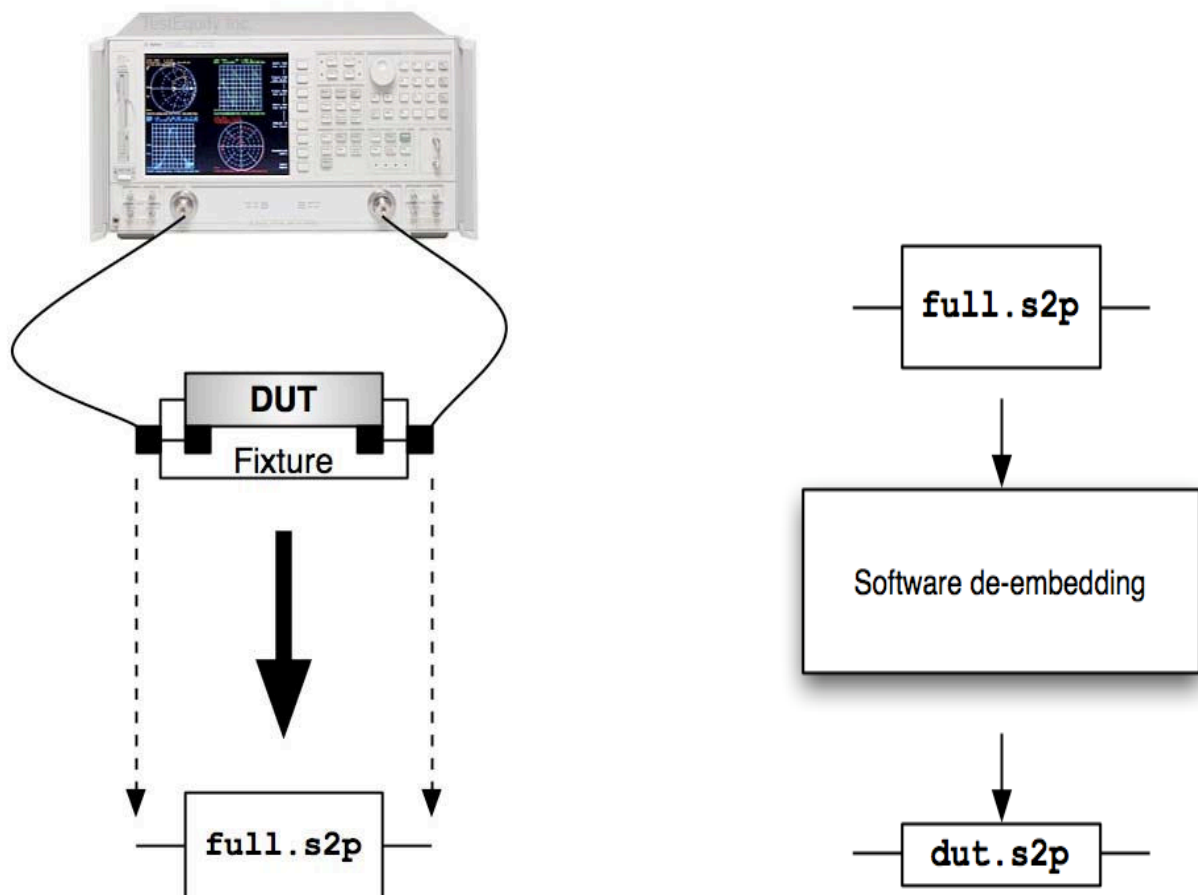
1. PrimeSim HSPICE usually selects a suitable value of F_{BASE} that provides a good tradeoff between the number of sampling points and performance, so allow F_{BASE} to default unless you are not seeing the resolution and accuracy you require.
2. Because FFT requires an array with size of 2^N , PrimeSim HSPICE may select the highest frequency to be sampled and report a different (higher) F_{MAX} value than given as the F_{MAX} in the netlist.

De-Embedding S-parameters

The S-element in PrimeSim HSPICE provides an interface to use measured or extracted S-parameters. When obtaining S-parameters by measurement, the measured S-parameters may contain characteristics of unwanted peripherals such as probes,

connectors, and so on, in addition to the target device under test (DUT) characteristics. If the characteristics of these unwanted peripherals are known as S-parameters, you can use PrimeSim HSPICE to de-embed them through common text-based data file formats such as Touchstone, CITIfile, and the netlist model format, *.sc# by using the existing LIN analysis and S-elements with the following stamping scheme:

Figure 56 (Left) The measured S-parameter model may include the characteristics of fixtures and the DUT itself. (Right) When the characteristics of fixtures are given, software de-embedding extracts pure DUT characteristics.



S-Model Syntax

```
S1 1 2 3 4 5 6 7 8 mname=model_name ...
+ STAMP=DEEMBED
```

or:

```
.model model_name S ...
+ STAMP=DEEMBED
```

The `STAMP=DEEMBED` setting produces a negated stamp to de-embed a given S-parameter block from the adjacent DUT connected in series. For example, the following netlist removes 50-ohm resistance represented by the `S1` element from the DUT of 100-ohm resistor, `R1`. As a result, the LIN analysis produces S-parameters of 50-ohm series resistor.

```
P1 1 0 port=1 ac=1
P2 2 0 port=2
* DUT (w/ unwanted 50 ohm)
R1 m 2 100
* de-embed 50 ohm s-parameters
S1 1 m mname=smodel stamp=DEEMBED
.opt post
.ac POI 4 0.0 1.0 1e8 1e9
.lin format=touchstone
.MODEL Smodel S
+ N=2 FQMODEL=SFQMODEL TYPE=S Z0=50
.MODEL SFQMODEL SP N=2 SPACING=POI MATRIX=NONSYMMETRIC
+ DATA=1
+
+ 0
+ 0.3333333333 0 0.6666666667 0
+ 0.6666666667 0 0.3333333333 0
+
.end
```

Note:

Since negated stamp causes noncausal behavior in time domain analysis, `STAMP=DEEMBED` is only supported in frequency domain analyses. For time domain analyses, generate de-embedded S-parameters first using LIN analysis, and then reuse them.

Small-Signal Parameter Data Frequency Table Model

The small-signal parameter data frequency table model (SP model) is a generic model that describes the frequency-varying behavior.

The following sections discuss these topics:

- [SP Model Syntax](#)
- [Four Valid Forms of the SP Model](#)
- [Examples](#)

SP Model Syntax

```
.MODEL name sp [N=val FSTART=val FSTOP=val NI=val  
+ SPACING=val MATRIX=val VALTYPE=val INFINITY=matrixval  
+ INTERPOLATION=val EXTRAPOLATION=val DC=val]  
+ DATA=(npts ...) | DATAFILE=filename
```

Parameter	Description
name	Model name.
N	Matrix dimension (number of signal terminals). Default is 1. If you use a value other than the default, you must specify that value before you set <code>INFINITY</code> and <code>DATA</code> .
FSTART	Starting frequency point for data. Default=0.
FSTOP	Final frequency point for data. Use this parameter only for the LINEAR and LOG spacing formats.
NI	Number of frequency points per interval. Use this parameter only for the DEC and OCT spacing formats. Default=10.
SPACING	Data sample spacing format: • LIN (LINEAR): uniform spacing with frequency step of $(FSTOP-FSTART)/(npts-1)$. The default. • OCT: octave variation with FSTART as the starting frequency, and NI points per octave. npts sets the final frequency. • DEC: decade variation with FSTART as the starting frequency, and NI points per decade. npts sets the final frequency. • LOG: logarithmic spacing. FSTART and FSTOP are the starting and final frequencies. • POI: nonuniform spacing. Pairs data • (NONUNIFORM) points with frequency points.
MATRIX	Matrix (data point) format: • SYMMETRIC: symmetric matrix. Specifies only lower-half triangle of a matrix (default). • HERMITIAN: similar to SYMMETRIC; off-diagonal terms are complex-conjugates of each other. • NONSYMMETRIC: non-symmetric (full) matrix.
VALTYPE	Data type of matrix elements: • REAL: real entry. • CARTESIAN: complex number in real/imaginary format (default). • POLAR: complex number in polar format. Specify angles in radians.
INFINITY	Data point at infinity. Typically real-valued. This data format must be consistent with <code>MATRIX</code> and <code>VALTYPE</code> specifications. npts does not count this point.
INTERPOLATION	Interpolation scheme: • STEP: piecewise step. This is the default. • LINEAR: piecewise linear. • SPLINE: b-spline curve fit.

Parameter	Description
EXTRAPOLATION	Extrapolation scheme during simulation: • NONE: no extrapolation is allowed. Simulation terminates if a required data point is outside of the specified range. • STEP: uses the last boundary point. The default. • LINEAR: linear extrapolation by using the last two boundary points. If you specify the data point at infinity, then simulation does not extrapolate and uses the infinity value.
npts	Number of data points.
DC	Data port at DC. Normally real-valued. This data format must be consistent with MATRIX and VALTYPE specifications. npts does not count this point. You must specify either the DC point or the data point at frequency=0.
DATA	Data points. • Syntax for LIN spacing: .MODEL name sp SPACING=LIN [N=dim] FSTART=f0 + FSTOP=f1 DATA=npts d1 d2 ... • Syntax for OCT or DEC spacing: .MODEL name sp SPACING=DEC or OCT [N=dim] + FSTART=f0 NI=n_per_intval DATA=npts d1 d2 ... • Syntax for POI spacing: .MODEL name sp SPACING=NONUNIFORM [N=dim] + DATA=npts f1 d1 f2 d2 ...
DATAFILE	Data points in an external file. This file must contain only raw numbers without any suffixes, comments, or continuation letters. The first number in the file must be an integer value to indicate the number of sampling points in the file. Then, sampling data must follow. The order of sampling data must be the same as in the DATA statement. This data file has no limitation on line length so you can enter a large number of data points.

Note:

Interpolation and extrapolation occur after the simulator internally converts the Z- and S-parameter data to Y-parameter data.

Four Valid Forms of the SP Model

The following four sample files are valid forms of the SP model:

SP Model 1: Symmetric complex matrices in linear frequency spacing:

```
.MODEL fmod SP N=2 FSTOP=30MegHz
+ DATA = 2
* matrix at f=0
+ 0.02      0.0
* Re(Y11) Im(Y11)
+ -0.02      0.0      0.02      0.0
* Im(Y21) Im(Y21) (= Y21) Re(Y22) Im(Y22)
* matrix at f=30MHz
+ 0.02      0.0
* Re(Y11) Im(Y11)
```

Chapter 5: S-parameter Modeling Using the S-element Small-Signal Parameter Data Frequency Table Model

```
+ -0.02      0.0              0.02      0.0
* Im(Y21) Im(Y21) (= Y21) Re(Y22) Im(Y22)
```

SP Model 2: Nonsymmetric complex matrices in linear frequency spacing:

```
.MODEL fmod SP N=2 FSTOP=30MegHz MATRIX=NONSYMMETRIC
+ DATA = 2
* matrix at f=0
+ 0.02      0.0      -0.02      0.0
* Re(Y11) Im(Y11) Re(Y12) Im(Y12)
+ -0.02      0.0      0.02      0.0
* Im(Y21) Im(Y21) Re(Y22) Im(Y22)
* matrix at f=30MHz
+ 0.02      0.0      -0.02      0.0
* Re(Y11) Im(Y11) Re(Y12) Im(Y12)
+ -0.02      0.0      0.02      0.0
* Im(Y21) Im(Y21) Re(Y22) Im(Y22)
```

SP Model 3: Symmetric complex matrices in nonuniform frequency spacing:

```
.MODEL fmod SP N=2 SPACING=POI
+ DATA = 1
+ 0.0 * first frequency point
* matrix at f=0
+ 0.02      0.0
* Re(Y11) Im(Y11)
+ -0.02      0.0              0.02      0.0
* Im(Y21) Im(Y21) (= Y21) Re(Y22) Im(Y22)
+ 30e+6 * second frequency point
* matrix at f=30MHz
+ 0.02      0.0
* Re(Y11) Im(Y11)
+ -0.02      0.0              0.02      0.0
* Im(Y21) Im(Y21) (= Y21) Re(Y22) Im(Y22)
```

SP Model 4: Nonsymmetric real matrices in linear frequency spacing:

```
.MODEL fmod SP N=2 FSTOP=30MegHz VALTYPE=REAL
+ MATRIX=NONSYMMETRIC
+ DATA = 2
* matrix at f=0
+ 0.02 -0.02
* Y11 Y12
+ -0.02 0.02
* Y21 Y22
* matrix at f=30MHz
+ 0.02 -0.02
* Y11 Y12
+ -0.02 0.02
* Y21 Y22
```

Examples

Example 1

```
**S-parameter example
.OPTION post=2
.probe v(n2)
V1 n1 0 ac=1v PULSE 0v 5v 5n 0.5n 0.5n 25n
.op
.ac lin 500 1Hz 30MegHz
.tran 0.1ns 10ns
*S1 n1 n2 0 mname=s_model
S1 n1 n2 0 mname=s_model
.model s_model S fmodel=fmod Z0=50 50
*.model s_model S fmodel=fmod2 Z0=50 100
* S parameter for Z0=(50 50)
.MODEL fmod SP N=2 FSTOP=30MegHz DATA = 1
+ 0.333333333 0.0 0.666666667 0.0 0.333333333 0.0
* S parameter for Z0=(50 100)
.MODEL fmod2 SP N=2 FSTOP=30MegHz MATRIX=NONSYMMETRIC
+ DATA = 1
+ 0.5 0.0      0.5 0.0
+ 1.0 0.0      0.0 0.0
Rt1 n2 0 50
.end
```

Example 2

[Figure 57](#) illustrates a transmission line that uses a resistive termination, and [Table 4](#) shows a corresponding input file listing. In this example, the two outputs from the resistor and S-parameter modeling must match exactly.

Figure 57 Transmission Line with Resistive Termination

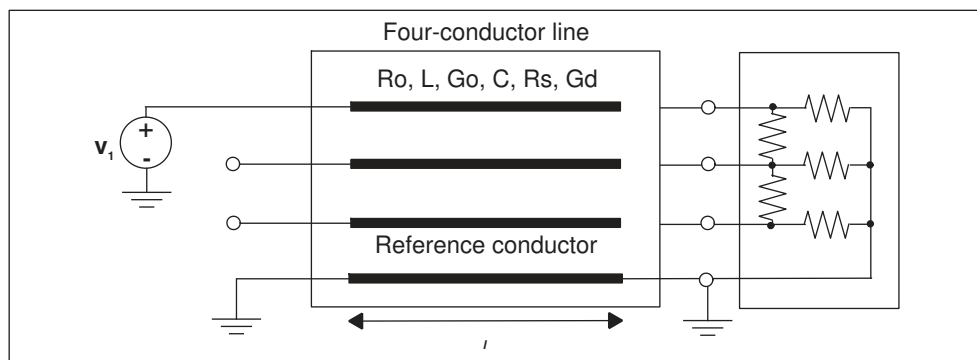


Table 4 *Input File Listing*

Header, options, and sources	*S-parameter x-line with a resistive positive termination.OPTION POST V1 i1 0 ac=1v
Termination	x1 o1 o2 o3 0 terminator
Transmission line (W-element)	W1 i1 i2 i3 0 o1 o2 o3 0 RLGCMODEL= N=3 + L=0.97 .MODEL wrlgc W MODELTYPE=RLGC N=3 + Lo = 2.78310e-07 + 8.75304e-08 3.29391e-07 + 3.65709e-08 1.15459e-07 3.38629e-07 + Co = 1.41113e-10 + -2.13558e-11 9.26469e-11 + -8.92852e-13 -1.77245e-11 8.72553e-11
Frequency model definition	.MODEL fmod sp N=3 FSTOP=30MegHz DATA= 1 + -0.270166 0.0 + 0.322825 0.0 -0.41488 0.0 + 0.17811 0.0 0.322825 0.0 -0.270166 0.0
Resistor elements	.SUBCKT terminator n1 n2 n3 ref R1 n1 ref 75 R2 n2 ref 75 R3 n3 ref 75 R12 n1 n2 25 R23 n2 n3 25 .ends terminator
Analysis	.AC lin 500 0Hz 30MegHz .DC v1 0v 5v 1v
Equivalent S-parameter element	.ALTER S parameter case .SUBCKT terminator n1 n2 n3 ref S1 n1 n2 n3 ref + FQMODEL=fmod .ENDS terminator .END

Example 3

The transmission-line example shown here uses capacitive network termination. The two outputs from the resistor and S-parameter modeling in Example 2 differ slightly due to the linear frequency dependency relative to the capacitor. To remove this difference, use the linear interpolation scheme in .MODEL.

Frequency model definition	.MODEL fmod sp N=3 FSTOP=30MegHz + DATA= 2 + 1.0 0.0 + 0.0 0.0 1.0 0.0 + 0.0 0.0 0.0 0.0 1.0 0.0 + 0.97409 -0.223096 + 0.00895303 0.0360171 0.964485 -0.25887 + -0.000651487 0.000242442 0.00895303 + 0.0360171 0.97409 -0.223096
-------------------------------	---

Using capacitive
elements

```
.SUBCKT terminator n1 n2 n3 ref
  C1 n1 ref 10pF
  C2 n2 ref 10pF
  C3 n3 ref 10pF
  C12 n1 n2 2pF
  C23 n2 n3 2pF
.ENDS terminator
```

Example 4

Figure 58 and Table 5 show an example of a transmission line that uses the S-parameter.

Figure 58 3-Conductor transmission line

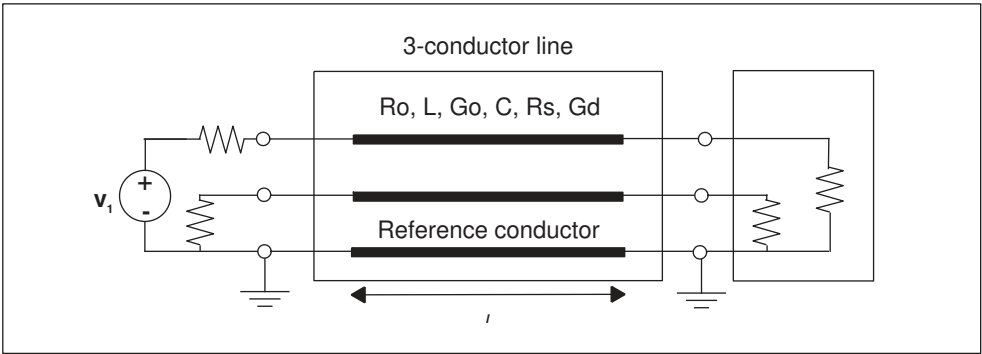


Table 5 Input File Listing

Header, options, and sources	<pre>*S parameter ex3: modeling x-line by using + S parameter .OPTION POST vin in0 0 ac=1</pre>
Analysis	<pre>.AC lin 100 0 1000meg .DC vin 0 1v 0.2v</pre>
Transmission line	<pre>W1 in1 in2 0 out1 out2 0 N=2 RLGCMODEL=m2</pre>
Termination	<pre>R1 in0 in1 28 R2 in2 0 28 R3 out1 0 28 R4 out2 0 28</pre>
W-element RLGC model definition	<pre>.MODEL m2 W ModelType=RLGC, N=2 + Lo= 0.178e-6 0.0946e-7 0.178e-6 + Co= 0.23e-9 -0.277e-11 0.23e-9 + Ro= 0.97 0 0.97 + Go= 0 0 0 + Rs= 0.138e-3 0 0.138e-3 + Gd= 0.29e-10 0 0.29e-10</pre>

Table 5 *Input File Listing (Continued)*

Frequency model definition	<pre>.MODEL SM2 sp N=4 FSTART=0 FSTOP=1e+09 + SPACING=LINEAR DATA= 60 0.00386491 0 + 0 0 0.00386491 0 0.996135 0 0 0 0.00386491 0 + 0 0 0.996135 0 0 0 0.00386491 0 + -0.0492864 -0.15301 + 0.00188102 0.0063569 -0.0492864 + -0.15301 0.926223 -0.307306 0.000630484 + -0.00154619 0.0492864 -0.15301 + 0.000630484 -0.00154619 0.926223 + -0.307306 0.00188102 0.0063569 + -0.0492864 -0.15301 -0.175236 -0.241602 + 0.00597 0.0103297 -0.175236 -0.241602 + 0.761485 -0.546979 0.00093508 + -0.00508414 -0.175236 -0.241602 + 0.00093508 -0.00508414 0.761485 + -0.546979 0.00597 0.0103297 -0.175236 + -0.241602 + ...</pre>
Equivalent S-parameter element	<pre>.SUBCKT terminator n1 n2 n3 ref S1 n1 n2 n3 ref FQMODEL=SM2 .ENDS terminator .END</pre>

References

[1] Dmitri Borisovich Kuznetsov and Jose E. Schutt-Aine, “Optimal Transient Simulation of Transmission Lines”, IEE Transaction on Circuits and Systems-I: Fundamental Theory and Applications. Vol. 43, No. 2, February 1996

[2] Bjorn Gustavsen and Adam Semlyen, “Rational Approximation of Frequency Domain Responses by Vector Fitting,” IEEE Transaction on Power Delivery, Vol.14, No.3, pp. 1052-1061, July 1999

6

W- and T-elements Transmission-Line Modeling

Describes how to use basic and advance transmission-line models and the various methods available to input transmission-line descriptions.

PrimeSim HSPICE ships many examples for your use. For W-element-related demo files, see Transmission (W-element) Line Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

The following sections discuss these topics:

- [Overview of W-element Usage](#)
- [Using the W-element](#)
- [Controlling W-element Accuracy](#)
- [Understanding Frequency-dependent Matrices and Transmission Line Theory](#)
- [Overview of T-element Usage](#)
- [Using T-element for Predicting Simple Tx Line Effects](#)
- [References](#)

Overview of W-element Usage

The W-element is a versatile transmission line model that you can apply to efficiently and accurately simulate transmission lines, ranging from a simple lossless line to complex frequency-dependent lossy-coupled lines. Unlike the U-element, the W-element can output accurate simulation results without fine-tuning optional parameters. For more information on U-elements, see [Appendix B, Simulating Circuits with Xilinx FPGAs](#).

A transmission line is a passive element that connects any two conductors at any distance apart. One conductor sends the input signal through the transmission line and the other conductor receives the output signal from the transmission line. The signal that transmits from one end of the pair to the other end is voltage between the conductors.

Examples of transmission lines include:

- Traces on printed circuit boards and multichip modules (MCMs)
- Bonding wires in semiconductor IC packages
- On-chip interconnections

This chapter describes the basic transmission-line simulation equations. It explains how to use these equations as an input to the transmission line model, the W-element. (For more information about the W-element, see [\[1\]](#).)

For an optional method to compute the parameters of the transmission line equations by using the *field solver model*, see [Chapter 7, Using the Field Solver to Extract Tx Line Parameters](#). This simple (but efficient and accurate) 2D electromagnetic field solver calculates the electrical parameters of a transmission line system based on its cross-section.

For less complex transmission line analysis, see [Using T-element for Predicting Simple Tx Line Effects](#) at the end of this chapter.

The following section discusses these topics:

- [W-element Capabilities](#)

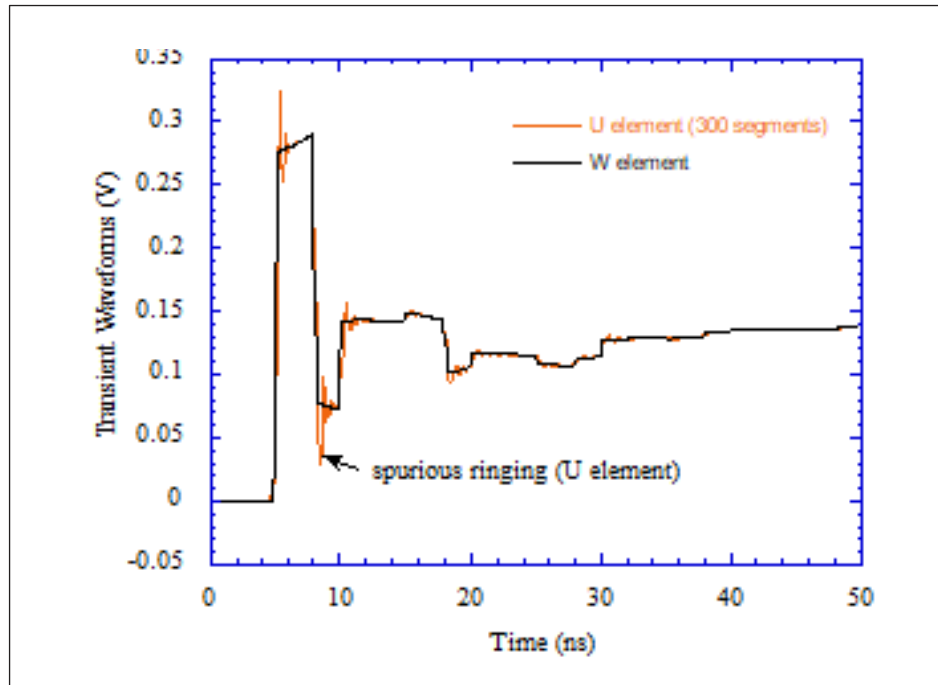
W-element Capabilities

The W-element is a multiconductor lossy frequency-dependent transmission line. It provides advanced modeling capabilities for transmission lines. The W-element provides:

- Ability to extract analytical solutions for AC and DC.
- No limit on the number of coupled conductors.
- No restriction on the structure of RLGC matrices; all matrices can be full.
- No spurious ringing, such as is produced by the lumped model. (See [Figure 59](#).)
- Accurate modeling of frequency-dependent loss in the transient analysis.

- Built-in 2D field solver, which you can use to specify a physical line shape.

Figure 59 Spurious ringing in U-element



The W-element supports the following types of analysis:

- DC
- AC
- Transient
- RF analyses (HB, HBAC, HBNOISE, PHASENOISE, LIN)
- Parameter sweeps
- Optimization
- Monte Carlo
- StatEye

Using the W-element

For detailed information on the W-element syntax and examples, see W-element (Distributed Transmission Lines) in *PrimeSim Continuum User Guide: Elements*.

The following sections discuss these topics:

- [Input Model 1: W-element, RLGC Model](#)
- [Input Model 2: U-element, RLGC Model](#)
- [Input Model 3: Built-in Field-Solver Model](#)
- [Input Model 4: Frequency-Dependent Tabular Model](#)
- [Input Model 5: S Model](#)
- [Input for W-element Passive Noise Model](#)

Input Model 1: W-element, RLGC Model

[Equations and Parameters](#) describes the inputs of the W-element per unit length matrices: R_o (DC resistance), L , G , C , R_s (skin effect), and G_d (dielectric loss).

The W-element does not limit any of the following parameters:

- Number of coupled conductors.
- Shape of the matrices.
- Line loss.
- Length or amount of frequency dependence.

The RLGC text file contains frequency-dependent RLGC matrices per unit length. The W-element also handles frequency-independent RLGC, and lossless (LC) lines. It does not support RC lines.

Because RLGC matrices are symmetrical, the RLGC model specifies only the lower triangular parts of the matrices.

The following sections discuss these topics:

- [Syntax](#)
- [Sample Netlist](#)
- [Simulation Results](#)
- [Specifying the RLGC Model in an External File](#)

Syntax

The syntax of the RLGC model for the W-element is:

```
.MODEL name W MODELTYPE=RLGC N=val  
+ Lo=matrix_entries
```

```
+ Co=matrix_entries [Ro=matrix_entries Go=matrix_entries]
+ Rs=matrix_entries wp=val Gd=matrix_entries Rognd=val
+ Rsgnd=val Lgnd=val
```

Parameter	Description
N	Number of conductors (same as in the element card).
Lo	DC inductance matrix, per unit length $\left[\frac{H}{m} \right]$.
Co	DC capacitance matrix, per unit length $\left[\frac{F}{m} \right]$.
Ro	DC resistance matrix, per unit length $\left[\frac{\Omega}{m} \right]$.
Go	DC shunt conductance matrix, per unit length $\left[\frac{S}{m} \right]$.
Rs	Skin effect resistance matrix, per unit length $\left[\frac{\Omega}{m\sqrt{Hz}} \right]$.
Gd	Dielectric loss conductance matrix, per unit length $\left[\frac{S}{m \cdot Hz} \right]$.
wp	Angular frequency of the polarization constant [radian/sec] (see Introducing the Complex Dielectric Loss Model). When the wp value is specified, the unit of Gd becomes [S/m]. The wp parameter is an array property; you can specify as many polarization frequencies as you want to achieve Equation 61 .
Lgnd	DC inductance value, per unit length for grounds $\left[\frac{H}{m} \right]$ (reference line).
Rognd	DC resistance value, per unit length for ground $\left[\frac{\Omega}{m} \right]$.
Rsgnd	Skin effect resistance value, per unit length for ground $\left[\frac{\Omega}{m\sqrt{Hz}} \right]$.

Sample Netlist

The following input netlist file shows RLGC input for the W-element:

```
* W-Element example, four-conductor line
W1 N=3 1 3 5 0 2 4 6 0 RLGCMODEL=example_rlc l=0.97
V1 1 0 AC=1v DC=0v pulse(4.82v 0v 5ns 0.1ns 0.1ns 25ns)
.AC lin 1000 0Hz 1GHz
.DC v1 0v 5v 0.1v
.tran 0.1ns 200ns

* RLGC matrices for a four-conductor lossy
.MODEL example_rlc W MODELTYPE=RLGC N=3
+ Lo=
+ 2.311e-6
+ 4.14e-7 2.988e-6
```

```
+ 8.42e-8 5.27e-7 2.813e-6
+ Co=
+ 2.392e-11
+ -5.41e-12 2.123e-11
+ -1.08e-12 -5.72e-12 2.447e-11
+ Ro=
+ 42.5
+ 0 41.0
+ 0 0 33.5
+ Go= + 0.000609
+ -0.0001419 0.000599
+ -0.00002323 -0.00009 0.000502
+ Rs=
+ 0.00135
+ 0 0.001303
+ 0 0 0.001064
+ Gd=
+ 5.242e-13
+ -1.221e-13 5.164e-13
+ -1.999e-14 -7.747e-14 4.321e-13
.end
```

Simulation Results

The following three figures show plots of the simulation results:

- [Figure 60](#) shows DC sweep.
- [Figure 61](#) shows AC response.
- [Figure 62](#) shows transient waveforms.

These figures also demonstrate that the transmission line behavior of interconnects has a significant and complicated effect on the integrity of a signal. This is why it is very important to accurately model transmission lines when you verify high-speed designs.

Figure 60 Simulation results: DC sweep

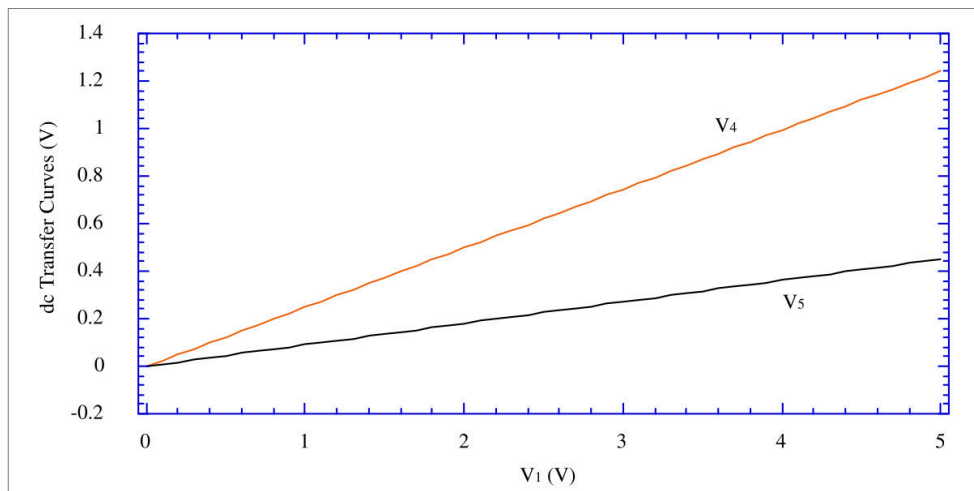


Figure 61 Simulation results: AC response

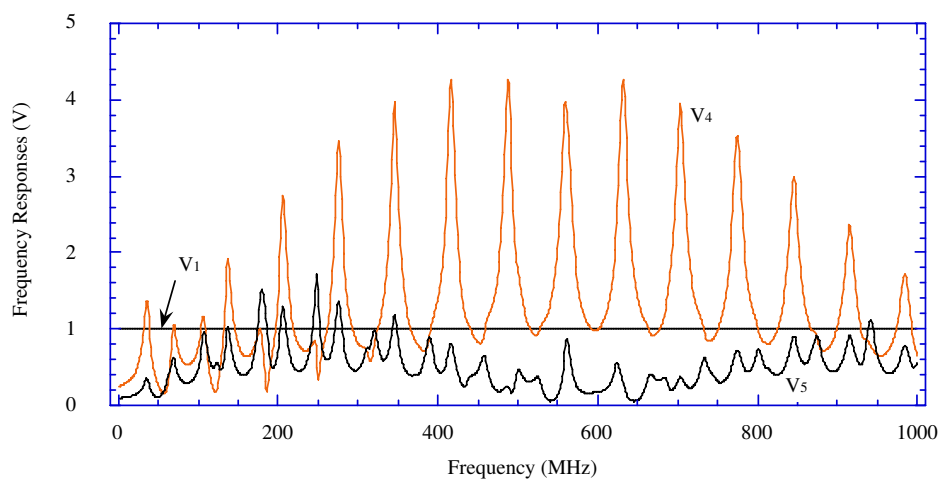
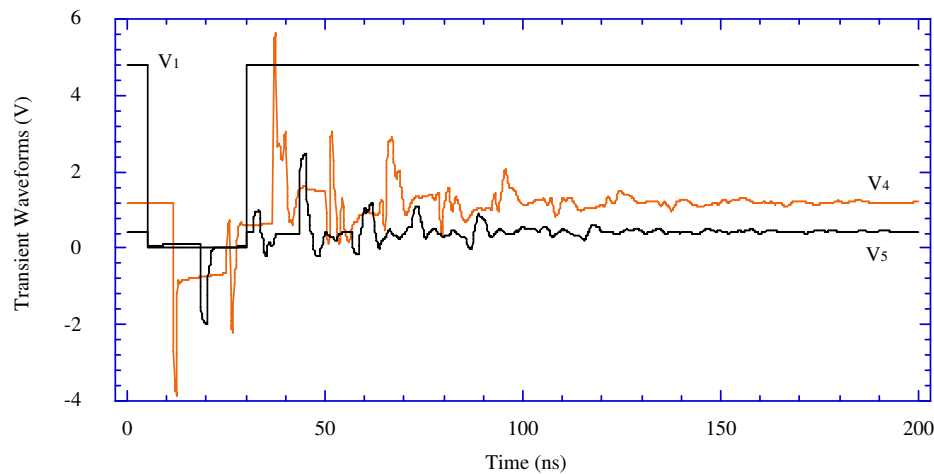


Figure 62 Simulation results: transient waveforms



Specifying the RLGC Model in an External File

You can also specify RLGC matrices in an RLGC file. Its file format is more restricted than the RLGC model; for example:

- You cannot include any parameters.
- The file does not support ground inductance and resistance.

Note:

This format does not provide any advantage over the RLGC model so do not use it unless you already have an RLGC file. It is supported for backward-compatibility.

The RLGC file only specifies the lower-triangular parts of the matrices and is order-dependent. Its parameters are in the following order:

Table 6 Parameters in RLGC File for W-element

Parameter	Description
N	Number of conductors (same as in the element card).
L	DC inductance matrix, per unit length $\left[\frac{H}{m} \right]$.
C	DC capacitance matrix, per unit length $\left[\frac{F}{m} \right]$.
R _o	(Optional) DC resistance matrix, per unit length $\left[\frac{\Omega}{m} \right]$.
G _o	(Optional) DC shunt conductance matrix, per unit length $\left[\frac{S}{m} \right]$.

Table 6 *Parameters in RLGC File for W-element (Continued)*

Parameter	Description
R_s	(Optional) Skin effect resistance matrix, per unit length $\left[\frac{\Omega}{m\sqrt{Hz}} \right]$.
G_d	(Optional) Dielectric loss conductance matrix, per unit length $\left[\frac{S}{m \cdot Hz} \right]$.

Note:

You can skip the optional parameters, because they default to zero. But if you specify an optional parameter, then you must specify all preceding parameters, even if they are zero.

An asterisk (*) in an RLGC file comments out everything until the end of that line. You can use any of the following characters to separate numbers:

space tab newline , ; () [] { }

This RLGC file is for the same netlist example used for the RLGC model in the [Input Model 1: W-element, RLGC Model](#):

```
* W- Element example, four-conductor line

W1 N=3 1 3 5 0 2 4 6 0 RLGCfile=example.rlc l=0.97
V1 1 0 AC=1v DC=0v pulse(4.82v 0v 5ns 0.1ns 0.1ns 25ns)

.AC lin 1000 0Hz 1GHz
.DC v1 0v 5v 0.1v
.tran 0.1ns 200ns

.end
```

Calls this example.rlc file:

```
* RLGC parameters for a four-conductor lossy
* frequency-dependent line
* N (number of signal conductors)
3
* Lo
2.311e-6
4.14e-7 2.988e-6
8.42e-8 5.27e-7 2.813e-6
* Co
2.392e-11
-5.41e-12 2.123e-11
-1.08e-12 -5.72e-12 2.447e-11
* Ro
42.5
```

```
0 41.0
0 0 33.5
* Go
0.000609
-0.0001419 0.000599
-0.00002323 -0.00009 0.000502
* Rs
0.00135
0 0.001303
0 0 0.001064
* Gd
5.242e-13
-1.221e-13 5.164e-13
-1.999e-14 -7.747e-14 4.321e-13
```

The RLGC file format does not support scale suffixes, such as:

n (10⁻⁹) or p (10⁻¹²)

Input Model 2: U-element, RLGC Model

The W-element accepts the U Model as an input to provide backward compatibility with the U-element. It also uses the geometric and measured-parameter interfaces of the U model.

To use the W-element with the U Model on the W-element card, specify:

`Umodel=U-model_name`

The W-element supports all U model modes, including:

- geometric, Elev=1
 - planar geometry, Plev=1
 - coax, Plev=2
 - twin-lead, Plev=3
- RLGC, Elev=2
- measured parameters, Elev=3
- skin-effect, Nlay=2

The only exception is Llev=1, which adds the second ground plane to the U model. The W-element does not support this. To model the extra ground plane, add an extra conductor to the W-element in Elev=2, or use an external lumped capacitor in Elev=1 or Elev=3. For information about the U model, see [Appendix B, Simulating Circuits with Xilinx FPGAs](#)

The following sections discuss these topics:

- [Using RLGC Matrices](#)
- [Sample Netlist](#)

Using RLGC Matrices

RLGC matrices in the RLGC model of the W-element are in the Maxwellian format. In the U model, they are in self/mutual format. For conversion information, see [Determining Matrix Properties](#). When you use the U model, the W-element performs the conversion internally. [Table 7](#) shows how the RLGC matrices in the U Model are related to the RLGC matrices in the W-element, and how the W-element uses these matrices.

Handling the Dielectric-loss Matrix

Because the U model does not input the G_d dielectric loss matrix, the W-element defaults G_d to zero when it uses the U model input.

Handling the Skin-effect Matrix

The U- and W-elements use the R_s skin-effect resistance in different ways.

- In a W-element, the R_s matrix specifies the square-root dependence of the frequency-dependent resistance (see the equations in [Frequency-Dependent Matrices](#)).
- In a U-element, R is the value of skin resistance at the frequency:

Equation 47

$$R \cong R_c + R_s$$

In the preceding equation, the core resistance (R_c) is equivalent to the DC resistance (R_o) in the W-element. The frequency at which the U-element computes the R matrix is:

Equation 48

$$f_{skin} = \frac{1}{15 \cdot RISETIME}$$

Table 7 RLGC Matrices for U and W elements

For U models with	W-element
RLGC input; Elev=2	Uses the R_s values that you specify in the U model.

For U models with	W-element
Geometric input; Elev=1	Divides the R_s (which the U model computes internally), by $\sqrt{f_{skin}}$ to obtain the R_s value. For Elev=1, the R_s value in the U model printout is not the same as the R_s value in the W-element.
Measured-parameter input; Elev=3	Does not support the skin effect.

If you do not specify the RISETIME option, the U-element uses Tstep from the .TRAN card.

Table 8 RLGC Matrices in the W-element and the U Model

W-element parameters	U Model parameters
L, C	$\begin{bmatrix} L_{11} & & \\ L_{12} & L_{22} & \\ L_{13} & L_{23} & L_{33} \end{bmatrix}$
Go, Gd	$\begin{bmatrix} G_{r1}+G_{12}+G_{13} & & \\ -G_{12} & G_{r2}+G_{12}+G_{23} & \\ -G_{13} & -G_{23} & G_{r3}+G_{13}+G_{23} \end{bmatrix}$
Ro	Nlay=1 (no skin effect) $\begin{bmatrix} R_{11}+R_{rr} & & \\ R_{rr} & R_{22}+R_{rr} & \\ R_{rr} & R_{rr} & R_{33}+R_{rr} \end{bmatrix}$ Nlay=2 (skin effect present) $\begin{bmatrix} R_{1c}+R_{rc} & & \\ R_{rc} & R_{2c}+R_{rc} & \\ R_{rc} & R_{rc} & R_{3c}+R_{rc} \end{bmatrix}$
Rs	Nlay=1 (no skin effect) $\begin{bmatrix} 0 & & \\ 0 & 0 & \\ 0 & 0 & 0 \end{bmatrix}$ Nlay=2 (skin effect present) $\frac{1}{\sqrt{f_{skin}}} \begin{bmatrix} R_{1s}+R_{rs} & & \\ R_{rs} & R_{2s}+R_{rs} & \\ R_{rs} & R_{rs} & R_{3s}+R_{rs} \end{bmatrix}$

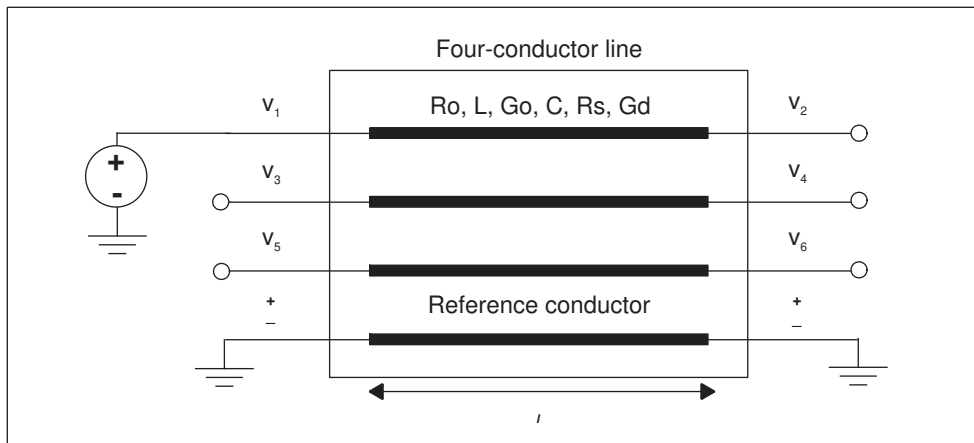
Sample Netlist

The following netlist is for a 4-conductor line as shown in [Figure 63](#).

```
* W Element example, four-conductor line, U model
W1 1 3 5 0 2 4 6 0 Umodel=example N=3 l=0.97
.MODEL example U LEVEL=3 NL=3 Elev=2 Llev=0 Plev=1 Nlay=2
+ L11=2.311uH
```

```
+ L12=0.414uH L22=2.988uH
+ L13=84.2nH L23=0.527uH L33=2.813uH
+ Cr1=17.43pF
+ C12=5.41pF Cr2=10.1pF
+ C13=1.08pF C23=5.72pF Cr3=17.67pF
+ R1c=42.5 R2c=41.0 R3c=33.5
+ Gr1=0.44387mS
+ G12=0.1419mS Gr2=0.3671mS
+ G13=23.23uS G23=90uS Gr3=0.38877mS
+ R1s=0.00135 R2s=0.001303 R3s=0.001064
V1 1 0 AC=1v DC=0v pulse(4.82v 0v 5ns 0.1ns 0.1ns 25ns)
.AC lin 1000 0Hz 1GHz
.DC v1 0v 5v 0.1v
.TRAN 0.1ns 200ns
.END
```

Figure 63 4-Conductor line



Input Model 3: Built-in Field-Solver Model

Instead of RLGC matrices, you can directly use geometric data with the W-element by using a built-in field solver. To use the W-element with a field solver, specify `F$model=model_name` on the W-element card. For a description of the built-in field solver, see [Field Solver Model Syntax](#).

Input Model 4: Frequency-Dependent Tabular Model

You can use the tabular RLGC model as an extension of the analytical RLGC model to model any arbitrary frequency-dependent behavior of transmission lines. (This model does not support RC lines.)

You can use this extension of the W-element syntax to specify a table model (use a `.MODEL` statement of type `w`). To accomplish this, the `.MODEL` statement refers to `.MODEL` statements where the “type” is `SP` (described in [Small-Signal Parameter Data Frequency Table Model](#)), which contain the actual table data for the RLGC matrices.

Note:

To ensure accuracy, the W-element tabular model requires the following:

- R and G tables require zero frequency points.
- L and C tables require infinity frequency points as well as zero frequency points.

To specify a zero frequency point, you may use `DC` keyword or `f=0` data entry in the `DATA` field of the `SP` model. To specify an infinity frequency point, use the `INFINITY` keyword of the `SP` model. See also, [Small-Signal Parameter Data Frequency Table Model](#).

The following sections discuss these topics:

- [Notation Used](#)
- [Table Model Card Syntax](#)
- [Examples: 4-Conductor Tx Line and RLGC Model List](#)
- [Introducing Causality Check for W-element RLGC Table Model](#)

Notation Used

- Lower-case variable: Scalar quantity
- Upper-case variable: Matrix quantity
- All upper-case words: Keyword
- Parentheses and commas: Optional

Table Model Card Syntax

```
.MODEL name W MODELTYPE=TABLE [FITGC=0|1] N=val  
+ LMODEL=l_freq_model CMODEL=c_freq_model  
+ [RMODEL=r_freq_model GMODEL=g_freq_model]
```

Parameter	Description
FITGC	Keyword for W Model (w/ <code>MODELTYPE=TABLE</code>) 1=causality check on (eliminates ringings for segmented lines), 0= causality check off (default)
N	Number of signal conductors (excluding the reference conductor).
LMODEL	SP model name for the inductance matrix array.

Parameter	Description
CMODEL	SP model name for the capacitance matrix array.
RLMODEL	SP model name for the resistance matrix array. By default, it is zero.
GMODEL	SP model name for the conductance matrix array. By default, it is zero.

The following is an example netlist of a two-line system.

```
.MODEL ex1 W MODELTYPE=TABLE N=2 LMODEL=lmod1
+ CMODEL=cmod1 RMODEL=rmod1 GMODEL=gmod1
.MODEL cmod1 sp N=2 SPACING=NONUNIFORM VALTYPE=REAL
+ INFINITY=(5.602360e-11 -7.047240e-12 5.602360e-11)
+ DATA=( 1,
+ (0.000000e+00 5.602360e-11 -7.047240e-12 5.602360e-11)
+ )
.MODEL cmod1 sp N=2 SPACING=NONUNIFORM VALTYPE=REAL
+ INFINITY=(3.93346e-7 4.93701e-8 3.93346e-7)
+ DATA=( 34,
+ (0.000000e+00 3.933460e-07 4.937010e-08 3.933460e-07)
+ (3.746488e+06 4.152139e-07 4.937010e-08 4.151959e-07)
+ .....
+ (4.000000e+09 3.940153e-07 4.937010e-08 3.940147e-07)
+ )
.MODEL lmod1 sp N=2 SPACING=NONUNIFORM VALTYPE=REAL
+ DATA=( 34,
+ (0.000000e+00 8.779530e-02 6.299210e-03 8.779530e-02)
+ (3.746488e+06 6.025640e-01 6.299210e-03 6.021382e-01)
+ .....
+ (4.000000e+09 1.690795e+01 6.299210e-03 1.689404e+01)
+ )
.MODEL rmod1 sp N=2 SPACING=NONUNIFORM VALTYPE=REAL+ DATA=( 34,
+ (0.000000e+00 8.779530e-02 6.299210e-03 8.779530e-02)
+ (3.746488e+06 6.025640e-01 6.299210e-03 6.021382e-01)
+ .....
+ (4.000000e+09 1.690795e+01 6.299210e-03 1.689404e+01)
+ )
.MODEL gmod1 sp N=2 SPACING=NONUNIFORM VALTYPE=REAL
+ DATA=( 34,
+ (0.000000e+00 5.967166e-11 0.000000e+00 5.967166e-11)
+ (3.746488e+06 1.451137e-05 -1.821096e-06 1.451043e-05)
+ .....
+ (4.000000e+09 1.549324e-02 -1.944324e-03 1.549224e-02)
+ )
```

Examples: 4-Conductor Tx Line and RLGC Model List

[Table 9](#) is an example of a four-conductor transmission line system, and [Table 10](#) is a list of a tabular RLGC model.

Table 9 *Input File Listing*

Listing Type	W-element Tabular Model Example
Header, options and sources	.OPTION POST V1 7 0 ac=1v dc=0.5v pulse(0.5v 1.5v 0ns 0.1ns) V2 8 0 dc=1v
Analysis	.DC v1 0.5v 5.5v 0.1v SWEEP length POI 2 1.2 2 .AC lin 200 0Hz 1GHz SWEEP Ro POI 3 400 41.6667 400 .TRAN 0.1ns 50ns
Termination	R1 7 1 50 R2 4 0 450 R3 5 0 450 R8 6 0 450 R5 4 5 10800 R6 5 6 10800 R7 4 6 1393.5
Analytical RLGC model (W-element)	.SUBCKT sub 1 2 3 4 5 6 7 8 W1 1 2 3 4 5 6 7 8 l=0.1 fgd=5e6 RLGCMODEL=analogmod n=3 .MODEL analogmod W MODELTYPE=RLGC N=3 + Lo=2.41667e-6 + 0.694444e-6 2.36111e-6 + 0.638889e-6 0.694444e-6 2.41667e-6 + Co=20.9877e-12 + -12.3457e-12 29.3210e-12 + -4.01235e-12 -12.3457e-12 20.9877e-12 + Ro=41.6667 + 0 41.6667 + 0 0 41.6667 + Go=0.585937e-3 + 0 0.585937e-3 + 0 0 0.585937e-3 + Rs=0.785e-5 + 0 0.785e-5 + 0 0 0.785e-5 + Gd=0.285e-6 + 0 0.285e-6 + 0 0 0.285e-6 .ENDS sub
Tabular RLGC model (W-element)	.ALTER Tabular Model .SUBCKT sub 1 2 3 4 5 6 7 8 W1 n=3 1 2 3 4 5 6 7 8 l=0.1 fgd=5e6 tablem odel=trmod .INCLUDE table.txt .ENDS sub

Table 10 *Tabular RLGC Model*

Listing Type	W-element Tabular Model Example
RLGC table model definition	.MODEL trmod W MODELTYPE=TABLE N=3 + LMODEL=lmod CMODEL=cmod RMODEL=rmod GMODEL=gmod
C model	.MODEL cmod sp N=3 VALTYPE=REAL INTERPOLATION=LINEAR + DATA=(1 2.09877e-11 -1.23457e-11 2.9321e-11 + -4.01235e-12 -1.23457e-11 2.09877e-11)
L model	.MODEL lmod sp N=3 VALTYPE=REAL INTERPOLATION=LINEAR + INFINITY= 2.41667e-06 6.94444e-07 2.36111e-06 + 6.38889e-07 6.94444e-07 2.41667e-06 FSTOP=1e+07 + DATA=(25 2.41667e-06 6.94444e-07 2.36111e-06 + 6.38889e-07 6.94444e-07 2.41667e-06 2.41861e-06 + 6.94444e-07... 2.41707e-06 6.94444e-07 2.36151e-06 + 6.38889e-07 6.94444e-07 2.41707e-06)
R model	.MODEL rmod sp N=3 VALTYPE=REAL INTERPOLATION=LINEAR + FSTOP=1e+10 DATA=(200 41.6667 0 41.6667 0 0 41.6667 + 41.7223 0 41.7223 0 0 41.7223 ... + 42.4497 0 42.4497 0 0 42.4497 42.4517 0 42.4517 0 0 + 42.4517)
G model	.MODEL gmod sp N=3 VALTYPE=REAL INTERPOLATION=LINEAR + FSTOP=1e+08 + DATA=(100 0.000585937 0 0.000585937 + 0 0 0.000585937 0.282764 0 0.282764 0 0 0.282764 + ... 1.42377 0 1.42377 0 0 1.42377 1.42381 0 1.42381 + 0 0 1.42381)

Introducing Causality Check for W-element RLGC Table Model

To improve the accuracy of W-element RLGC table model, you can introduce a complex dielectric coefficient to assure causality for the W-element RLGC table model. You can use the keyword `FITGC=[1|0]` when using the W-element RLGC table model to turn on or turn off this method. By default, `FITGC=0`.

Although the dielectric properties have only a slight frequency-dependent character, they have an impact on transmission line simulation accuracy in that not only does that signal appear at the output port before the delay time is reached, but there is also a non-consistency between the segmented lines and an integral line. [Figure 64](#) and [Figure 65](#) show that when a long transmission line is separated into 50 series connected segments, the output signal at the far end of the lines has large discrepancies compared to a single line, while they should be the same. When applying `DELAYOPT=3`, the discrepancy becomes even bigger. This is due to the errors introduced by inter-/extrapolation at a high frequency band which are more accurately fitted, and, therefore, more explicitly exposed in the simulation results.

Figure 64 Non-consistency between segmented lines and one integral line; `DELAYOPT=0`

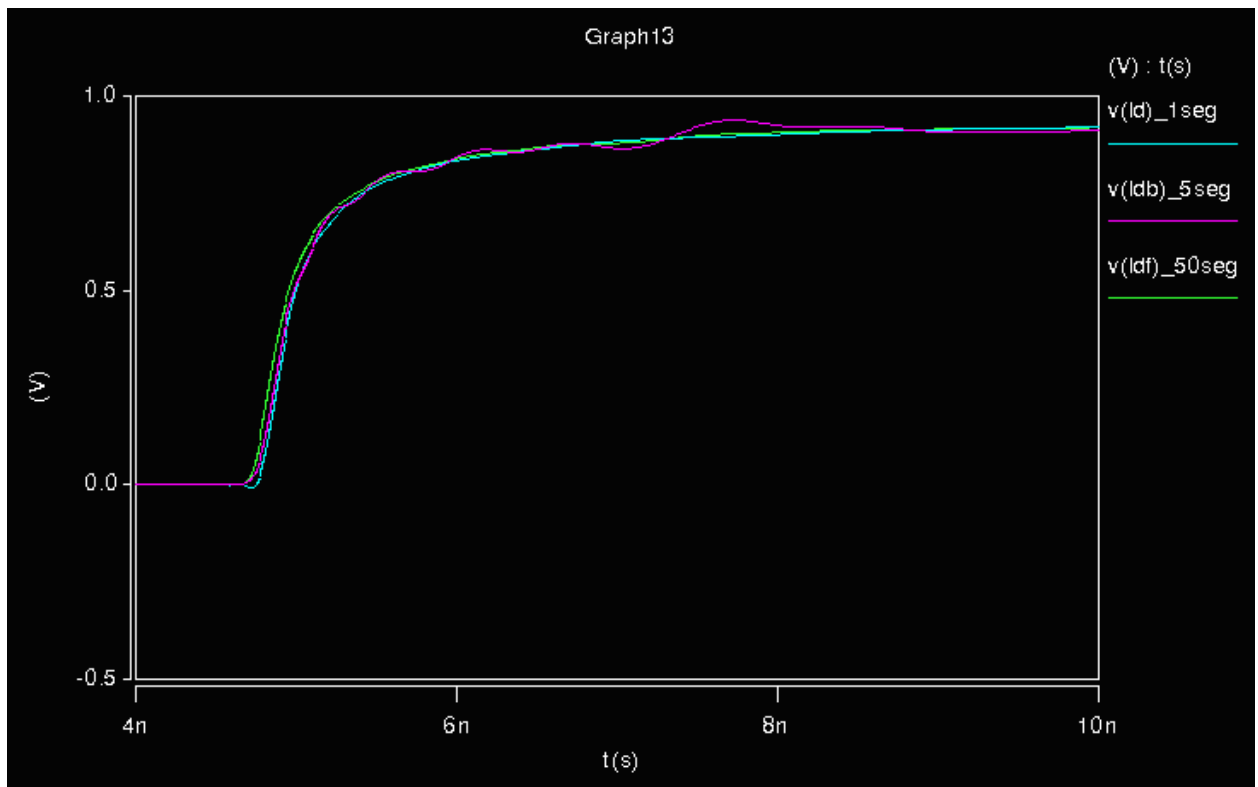
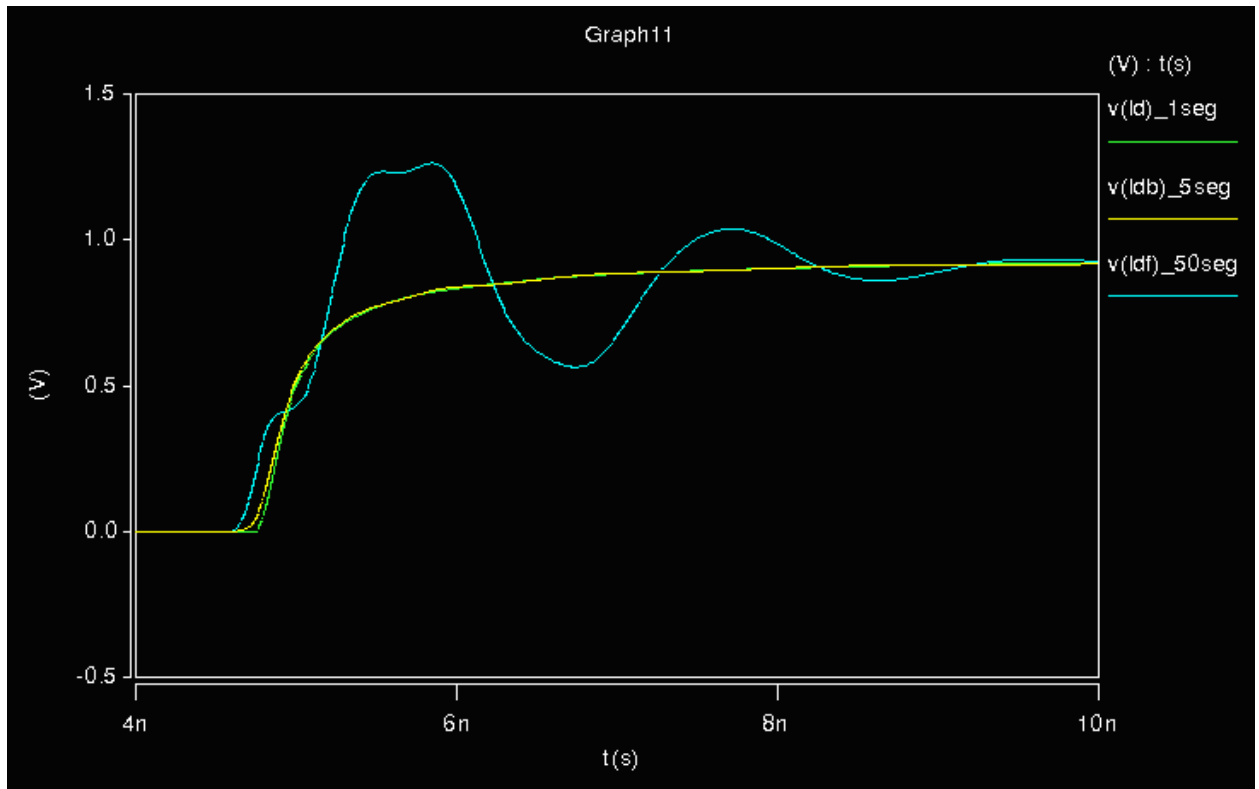


Figure 65 Non-consistency between segmented lines and one integral line; DELAYOPT=3



The solution to this issue lies in the dielectric properties of a transmission line system. In an ideal capacitor, the current that flows through the capacitor is exactly 90 degrees out of phase with the voltage sine wave. If the ideal capacitor were filled with an insulator with a dielectric constant of ϵ_r , the capacitance would increase to $C = \epsilon_r C_0$. However, real dielectric materials have some resistivity associated, which leads to leakage current. This current is completely in phase with the voltage. It can be modeled as an ideal resistor. By conventional transmission line theory, it is modeled by conductor G. Both of C and G are frequency-dependent parameters in a real transmission line system. The frequency-dependent character comes from the dipoles in dielectric material. Actually, both of these two terms relate to the number of dipoles, how large they are and how they are able to move. These characters are described by dielectric constant of the material.

Equation 49

$$\epsilon_r(\omega) = \epsilon_r'(\omega) + j\omega_r''(\omega)$$

$$tg\delta = \frac{\epsilon_r''}{\epsilon_r'}$$

$$C(\omega) = \epsilon_r'(\omega) C_0$$

$$C(\omega) = \omega C(\omega) tg\delta = \omega \epsilon_r''(\omega) C_0$$

The real part of ϵ corresponds to the motion of the dipoles that are out of phase with the applied field and contributes to increasing capacitance, while the imaginary part corresponds to the motion of the dipoles that are in phase with the applied voltage and contribute to the losses.

Since the frequency-dependent character of both of these terms relates to the motion of dipoles, which can be described by $\epsilon_r \omega$, the real and imaginary part of ϵ_r must satisfy the Kramers-Kronig relationship. And therefore, we can fit with rational function.

Equation 50

$$\epsilon_r(\omega) = D + j\omega E + \sum_{m=1}^N \frac{C_m}{j\omega - A_m}$$

Once you get $\epsilon_r \omega$, we can calculate accurate $C(\omega)$ and $G(\omega)$ interpolation and extrapolation, using [Equation 50](#).

Once PrimeSim HSPICE gets a successful fitting, you can get a $G(\omega)$ and $C(\omega)$ matrix from the fitting result. Since only the RMS error of the real part is considered during the fitting process, you get only the $C(\omega)$ matrix from the fitting result. For $G(\omega)$, you still use the conventional linear interpolation and extrapolation because of its strong dependency on (ω) .

Input Model 5: S Model

The W-element can accept the transmission line-based S-parameters as input. To use the W-element with the S Model on the W-element card, specify the following line:

```
SMODEL=Smodel_name NODEMAP=XiYj...
```

where:

- `Smodel_name` is an S-model, which is normally used for an S-element. Use the `XLINLENGTH` keyword in the S-model statement to indicate the line length of the system where the S-parameters are extracted. This keyword is required only when you use an S-model with a W-element. See S-element Syntax in *PrimeSim Continuum User Guide: Elements* for more information.
- `NODEMAP` is a string that assign each index of the S-parameter matrix to one of the W-element terminals. This string must be an array of pairs that consists of a letter and a number, (for example, Xn), where
 - X= I, i, N, or n to indicate near end (input side) terminal of the W-element
 - X= O, o, F, or f to indicate far end (output side) terminal of the W-element.

For example, `NODEMAP = I1I2O1O2` represents that the

- 1st port of the s-matrix corresponds to the 1st near end terminal of the W-element.
- 2nd port of the s-matrix corresponds to the 2nd near end terminal of the W-element.
- 3rd port of the s-matrix corresponds to the 1st far end terminal of the W-element.
- 4th port of the s-matrix corresponds to the 2nd far end terminal of the W-element.

NODEMAP = I1I2I3...InO1O2O3...On is the default setting.

The following sections discuss these topics:

- [S-Model Conventions](#)
- [S-Model Example](#)

S-Model Conventions

When specifying an S model, you must adhere to the following rules and conventions:

- The size of the NODEMAP array must be the same as twice the line number of the W-elements and also must be the same as the port count of the S-parameter matrices.
- If the W-element input model is S-MODEL, an S-model definition must accompany that input model.
- S-parameters must have even number of terminals.
- S-parameters must be symmetric.
- S-parameters must be passive.
- Transmission-line based S-parameters can be used with different lengths of a system when the varying length keyword (L) in a W-element instance statement is present.
- The XLINLENGTH keyword must be set when used in S-models that use W-elements.

S-Model Example

The following input netlist file shows S-model input for the W-element:

```
**** W Element Example: S Model ****
rout out 0 50
vin in gnd LFSR (1 0 0 0.1n 0.1n 1g 1 [5,2] rout=50)
*+ pulse(0 1 0 0.1n 0.1n 0.9n 2n)
W1 in gnd out gnd SMODEL=smodel N=1 l=0.3
+ NODEMAP=I1O1
.MODEL smodel S TSTONEFILE=w.s2p
+ XLINLENGTH=0.3
.opt accurate post
.tran .01n 20n
.end
```

Input for W-element Passive Noise Model

The W-element is a passive transmission line model. When the transmission lines are lossy, they generate thermal noise. The W-element passive noise model is used to describe these noise effects. This model supports normal, 2-port and multiport (`.NOISE` and `.LIN noisecal=1` [or `2` for N-port]). See [Noise Parameters in 2-Port and N-Port Networks](#).

The following sections discuss these topics:

- [Input Interface](#)
- [Output Interface](#)

Input Interface

To trigger a passive noise model, the `NOISE` and `DTEMP` keywords in an W-element statement are used:

```
W i1 i2 ... iN iR o1 o2 ... oN oR N=val L=val
+ ...
+ [NOISE=[1|0]] [DTEMP=val]
```

Parameter	Description
NOISE	Activates thermal noise. • 1 (default): element generates thermal noise • 0: element is considered noiseless
DTEMP	Temperature difference between the element and the circuit, expressed in °C. The default is 0.0. Element temperature is calculated as: $T = \text{Element temperature (°K)}$ $= 273.15 \text{ (°K)} + \text{circuit temperature (°C)}$ $+ \text{DTEMP (°C)}$ Where circuit temperature is specified using either the <code>.TEMP</code> statement, or by sweeping the global <code>TEMP</code> variable in <code>.DC</code> , <code>.AC</code> , or <code>.TRAN</code> statements. When a <code>.TEMP</code> statement or <code>TEMP</code> variable is not used, the circuit temperature is set by <code>.OPTION TNOM</code> , which defaults to 25 °C unless you use <code>.OPTION SPICE</code> , which raises the default to 27 °C.

When `NOISE=1`, PrimeSim HSPICE generates a $2N \times 2N$ noise-current correlation matrix from the N-conductor W-element admittance matrix according to Twiss' Theorem. The

result can be stamped into a PrimeSim HSPICE noise analysis as 2N-correlated noise current sources: j_i ($i=1\sim 2N$), as shown in [Equation 51](#):

Equation 51

$$C = 2kT(Y + Y^*T) = \begin{bmatrix} \overline{j_1|^2} & \overline{j_1 j_2^*} & \dots & \overline{j_1 j_{2N}^*} \\ \overline{j_2 j_1^*} & \overline{j_2|^2} & \dots & \overline{j_2 j_{2N}^*} \\ \dots & \dots & \dots & \dots \\ \overline{j_{2N} j_1^*} & \overline{j_{2N} j_2^*} & \dots & \overline{j_{2N}^2} \end{bmatrix}$$

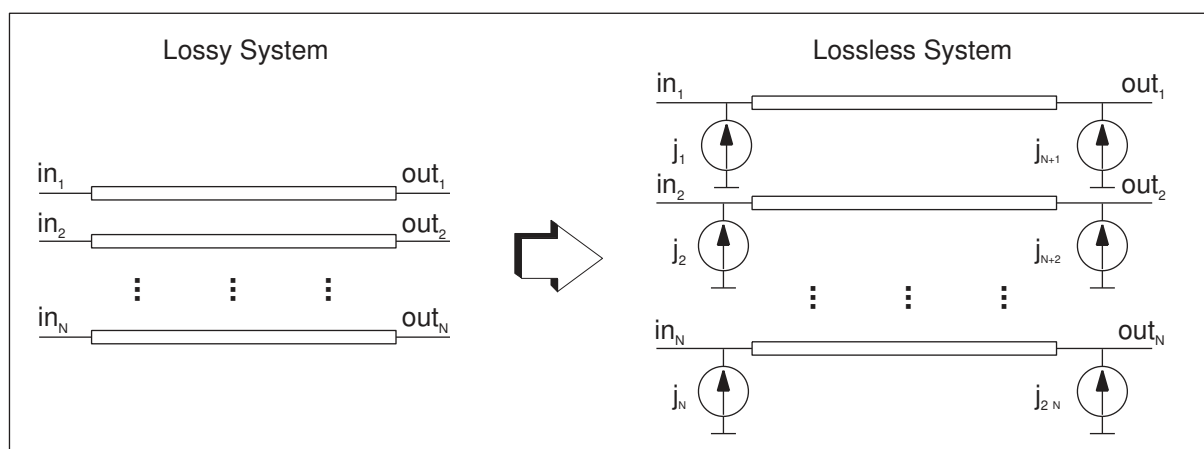
where:

$i=1\sim N$ corresponding to N input terminals

$i=N+1\sim 2N$ corresponding to N output terminals.

The noise-current correlation matrix represents the frequency-dependent statistical relationship between 2N noise current sources, j_i ($i=1\sim 2N$), shown in the following figure.

Figure 66 Frequency-dependent relationship, 2N noise current sources



Output Interface

PrimeSim HSPICE creates a `.lis` output list file that shows the results of a noise analysis just as any other noisy elements. The format is as follows:

```
**** w element squared noise voltages (sq v/hz)
element      0:w1
N(i,j)       data
r(N(i,j))    data
... i,j = 1~N ...
total       data
```


where:

- $N(i, j)$ = contribution of $j_1 j_i^*$ to the output port
- $r(N(i, j))$ = transimpedance of j_i to the output port
- $total$ = contribution of total noise voltage of the W-element to the output port.

Controlling W-element Accuracy

The following sections discuss these topics:

- [Setting Frequency Range of Interest for Greater Accuracy](#)
- [Using Time-Step Control](#)

Setting Frequency Range of Interest for Greater Accuracy

This section describes the keywords you can use for achieving greater accuracy of the W-element by controlling the frequency of interest.

The following sections discuss these topics:

- [Setting .OPTION RISETIME](#)
- [Using DELAYOPT Keyword for Higher Frequency Ranges](#)
- [Using DCACC Keyword for Lower Frequency Ranges](#)

Setting .OPTION RISETIME

The `RISETIME` option is used to improve accuracy of the W-element analysis by setting the focal frequency. It is applicable to all W-element modeling methods. If not explicitly set, PrimeSim HSPICE automatically determines `RISETIME` and the focal frequency from independent sources by examining their edge rates and using the fastest of those as the effective `RISETIME` setting.

If you know the focal frequency, you can explicitly set `RISETIME`. The focal frequency is user-determined, but the knee frequency of the design is typically a valid approximation. Use the reciprocal of the focal frequency to set `RISETIME`; e.g. $1/10\text{GHz} = 100\text{p}$. The minimum value for `RISETIME` is 0.1p and anything smaller defaults to that value. Do not set `RISETIME` to a value of 0 (zero).

Be cautious when setting `RISETIME` as it can have a direct impact on transient simulation results. If it is not properly set (too low or too high compared to the maximum frequency contained in the actual signal), responses to the high frequency components might be omitted or inaccurately modeled.

It is generally a good idea to explicitly set `RISETIME`, especially if you are using behavioral (controlled) sources or Verilog-A sources that do not have edge rates that can be determined by statically parsing the netlist.

You may want to let PrimeSim HSPICE determine `RISETIME` under the following conditions:

- The sources are independent and not subject to unexpected incidents that modify the predicted edge rates.
- You don't want to risk forgetting to reset `RISETIME` when you modify your sources.
- You share the netlist others who may use it without knowing that the `RISETIME` is set such as in the case of intellectual property.
- There is a potential for making a mistake in `RISETIME` estimation.

Using DELAYOPT Keyword for Higher Frequency Ranges

Long transmission lines fabricated in a high polymer insulator, such as PCB traces, show high losses in high frequencies due to dielectric loss. In such cases, the propagation delay of the system becomes a non-constant function of frequency. To take this phenomenon accurately, a novel pre-process function was introduced for constructing W-element transient (recursive convolution) model with a higher level of accuracy. To activate this new function, you can add the `DELAYOPT` keyword to the W-element instance line. You can use `DELAYOPT=0 | 1 | 2` to deactivate, activate, and automatically determine, respectively. The default value is 0 (deactivate). If this function is deactivated, the W-element behaves identically to the previous versions.

You can use `DELAYOPT=3` to achieve a level of accuracy up to a tens of GHz operation and involve harmonics up to THz order. With this option, line length limits are removed, which frees the simulation from segmenting, and allows independence in the behavior of the `RISETIME` option setting. A setting of `DELAYOPT=3` automatically detects whether or not frequency-dependent phenomena need to be recorded, which makes it identical to the `DELAYOPT=0` setting if it produces a high enough accuracy.

Note:

The `DELAYOPT=3` option activates additional evaluation functions in transient analysis, which might take longer CPU time.

To set this parameter as a global option for all W-elements in a netlist, see `.OPTION WDELAYOPT` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Using DCACC Keyword for Lower Frequency Ranges

The W-element can take an additional step in making a time domain model check the accuracy of low frequency and DC coverage. It automatically adds rational function terms

if necessary. This process may cause slight additional computational cost and slight difference in element behavior in DC offset. Should you choose to use this conventional behavior, set `DCACC=0` in the W-element instance or model line to deactivate this process.

Using Time-Step Control

This section describes using static and dynamic time-step controls in the time domain.

The following sections discuss these topics:

- [Static Time-Step Control](#)
- [Dynamic Time-Step Control using WACC](#)

Static Time-Step Control

The W-element provides accurate results with just one or two time steps per excitation transient (0.1 ns in [Figure 59](#)). Like the T-element, the W-element supports the `TLINLIMIT` parameter. The `TLINLIMIT=0` default setting enables special breakpoint building, which limits the maximum time step by the smallest transmission line delay in the circuit. This improves transient accuracy for short lines, but reduces efficiency. Setting `TLINLIMIT=1` disables this special breakpoint building.

Longer transmission lines might experience prolonged time intervals when nothing happens at the terminals, while the wave propagates along the line. If you increase the time step, the accuracy of the simulation decreases when the wave reaches the terminal. To prevent this for longer lines excited with short pulses, set `.OPTION DELMAX` to limit the time step to between 0.5 and 1 of the excitation transient.

Dynamic Time-Step Control using WACC

Static time step control achieves certain accuracy by setting static breakpoints. The `TLINLIMIT=0` parameter limits the maximum time step by the minimum transmission line delay, which results in poor performance for cases with ultra-short delay transmission lines because too many redundant time points are calculated, especially when the transmission line terminal signals do not vary rapidly. The same problem exists with the `DELMAX` option where time steps are evenly set in spite of terminal signal variation. This is inefficient.

The `WACC` option was added to solve this problem by providing dynamic step control for W-element transient analysis. Setting `WACC` to a positive value removes the static breakpoints and the necessary time points are set dynamically according to the variations in terminal currents and voltages.

The `WACC` option has the following syntax:

```
.OPTION WACC=value
```

where WACC is a non-negative real value between 0.0 and 10.0. PrimeSim HSPICE assigns `WACC -1` if you do not set a `WACC` option, or if you set `.OPTION WACC`. When a value of 1 is specified, PrimeSim HSPICE assigns `WACC` with a positive value. If a non-negative value is set in the `.option` line (`.OPTION WACC=XXX`), PrimeSim HSPICE uses the specified `WACC` value for all the W-elements. When a positive `WACC` value is set, the dynamic time step control algorithm is activated. When `WACC` is zero, the conventional static time step control method is used. Larger `WACC` values result in less restriction in time point intervals (therefore faster simulation), while smaller values result in denser time points with higher accuracy.

A positive `WACC` is selected by default to activate the dynamic time step control. PrimeSim HSPICE automatically finds the optimum `WACC` value based on the netlist properties such as transmission line system delay, risetime, and transient command configurations. Since the W-elements in the netlist may have different properties, each has its own `WACC` values. If a user-specified positive `WACC` value is found in the netlist, PrimeSim HSPICE uses the user-defined `WACC` value for all the W-elements in the netlist. If the user-specified `WACC` is larger than the automatic estimation, PrimeSim HSPICE outputs a warning message.

For cases containing IBIS, PKG, EBD, or ICM blocks, PrimeSim HSPICE turns `WACC` off automatically. If you want to use the dynamic time step control algorithm for IBIS related cases, you must set it explicitly in the netlist for example:

```
.option WACC          $ Make PrimeSim HSPICE use automatically generated WACC
                        value for each W element
```

or:

```
.option WACC=value    $ Use this value for all the W elements
```

Understanding Frequency-dependent Matrices and Transmission Line Theory

The following sections discuss these topics:

- [Equations and Parameters](#)
- [Frequency-Dependent Matrices](#)
- [Wave Propagation](#)

Equations and Parameters

Maxwell's equations for the transverse electromagnetic (TEM) waves on multiconductor transmission lines reduce to the telegrapher's equations. The general form of the telegrapher's equation in the frequency domain is:

Equation 52

$$-\frac{\partial}{\partial z} \mathbf{v}(z, \omega) = [\mathbf{R}(\omega) + j\omega \mathbf{L}(\omega)] \mathbf{i}(z, \omega)$$

Equation 53

$$-\frac{\partial}{\partial z} \mathbf{i}(z, \omega) = [\mathbf{G}(\omega) + j\omega \mathbf{C}(\omega)] \mathbf{v}(z, \omega)$$

The preceding equations use the following definitions:

- Lower-case symbols denote vectors.
- Upper-case symbols denote matrices.
- $\mathbf{v}$ is the voltage vector across the lines.
- $\mathbf{i}$ is the current vector along the lines.

For the TEM mode, the transverse distribution of electromagnetic fields at any instant of time is identical to that for the static solution.

From a static analysis, you can derive the four parameter matrices for multiconductor TEM transmission lines:

- resistance matrix, $\mathbf{R}$
- inductance matrix, $\mathbf{L}$
- conductance matrix, $\mathbf{G}$
- capacitance matrix, $\mathbf{C}$

The telegrapher's equations and the four parameter matrices from a static analysis completely and accurately describe TEM lines.

Not all transmission lines support pure TEM waves; some multiconductor systems inherently produce longitudinal field components. In particular, waves propagating in either the presence of conductor losses or the absence of dielectric homogeneity (but not dielectric losses) must have longitudinal components.

However, if the transverse components of the fields are significantly larger than the longitudinal components, the telegrapher's equations (and the four parameter matrices obtained from a static analysis) still provide a good approximation (a quasi-static approximation).

Multiconductor systems in which this approximation is valid take the name *quasi-TEM lines*. For typical micro-strip systems the quasi-static approximation holds up to a few GHz.

Frequency-Dependent Matrices

The static (constant) L and C matrices are accurate for a wide range of frequencies. In contrast, the static (DC) R matrix applies to only a limited frequency range, mainly due to the skin effect. A good approximate expression of the R resistance matrix with the skin effect is:

Equation 54

$$R(f) = R_o + (1 + j)\sqrt{f} R_s$$

or:

Equation 55

$$R(f) = \sqrt{R_o^2 + (1 + j)^2 f R_s^2}$$

where:

- R_o is the DC resistance matrix.
- R_s is the skin effect matrix.

The imaginary term depicts the correct frequency response at high frequency; however, it might cause significant errors for low-frequency applications. In the W-element, [Equation 54](#) can be chosen by `RSTYPE=1` (default) or [Equation 55](#) by `RSTYPE=2`. Also, you can exclude this imaginary term:

```
Wxxx i1 i2 ... iN iR o1 o2 ... oN oR N=val L=val INCLUDERSIMAG=NO
```

In contrast, the G (loss) conductance matrix often approximates as:

Equation 56

$$G(f) \cong G_o + \frac{f}{\sqrt{1 + (f / f_{gd})^2}} G_d$$

where:

- G_o models the shunt current due to free electrons in imperfect dielectrics.
- G_d models the power loss due to the rotation of dipoles under the alternating field (C. A. Balanis, Advanced Engineering Electromagnetics, New York: Wiley, 1989).
- f_{gd} is a cut-off frequency.

If you do not set f_{gd} , or if you set f_{gd} to 0, then $G(f)$ keeps linear dependency on the frequency. In the W-element, the default f_{gd} is zero (that is, $G(f)$ does not use the f_{gd} value).

You can specify an alternate value in the W-element statement:

```
Wxxx i1 i2 ... iN iR o1 o2 ... oN oR N=val L=val fgd=val
```

If you prefer to use the previous linear dependency, set f_{gd} to 0.

Note:

f_{gd} estimates frequency-dependent shunt loss conductance described as [Equation 52](#) for the RLGC model without `INCLUDEGDIMAG=yes` only (see [Fitting Procedure Triggered by INCLUDEGDIMAG Keyword](#)). When you specify `INCLUDEGDIMAG=yes`, the RLGC model estimates frequency-dependent shunt (C and G) parameters as described in [Equation 56](#) and does not use the f_{gd} value. Both of these are ways to fit the RLGC model with actual measurements. If you measure or computationally extract a tabular RLGC model, you are likely to accurately extract parameters.

The following sections discuss these topics:

- [Introducing the Complex Dielectric Loss Model](#)
- [Fitting Procedure Triggered by INCLUDEGDIMAG Keyword](#)
- [Example: Complex Dielectric Loss Model](#)
- [Determining Matrix Properties](#)
- [Using the PRINTZO Option](#)
- [Defining Frequency-dependent Impedance in Mixed Mode Printing](#)
- [File Description for *.wzo](#)

Introducing the Complex Dielectric Loss Model

When the `INCLUDEGDIMAG` keyword = `yes` and there is no `wp` input, the W-element regards the G_d matrix as the conventional model and then automatically extracts constants for the complex dielectric model. In conventional use, with the PrimeSim HSPICE W-element RLGC model, you approximate frequency-dependent conductance as [Equation 56](#) where the increase of shunt conductance is due to dielectric loss.

These pure real nonconstant functions of frequency violate causality [2]. As system operating frequency becomes high, even for PCB systems that use high polymer dielectric materials like FR4, the appearance of the dielectric loss becomes significant and significant noncausality of [Equation 56](#) appears. The frequency-dependent loss of the shunt conductance in the dielectric is mainly due to dielectric polarization.

This polarization loss leads to a complex permittivity, $\varepsilon(\omega)$, for the dielectric material [3].

Equation 57

$$\varepsilon(\omega) = \varepsilon'(\omega) - j\varepsilon''(\omega)$$

You can specify loss tangent of the dielectric material as the ratio of imaginary part of $\varepsilon(\omega)$ to the real part:

Equation 58

$$\tan \delta(\omega) = \frac{\varepsilon''(\omega)}{\varepsilon'(\omega)}$$

You can write complex electric permittivity for a single dielectric dipolar moment:

Equation 59

$$\varepsilon(\omega) = \varepsilon_{\infty} + \omega_p \frac{\varepsilon_{dc} - \varepsilon_{\infty}}{j\omega + \omega_p}$$

where:

- ε_{dc} and ε_{∞} are low and high frequency limits of dielectric permittivity, which are real numbers.
- ω_p is the angular frequency that corresponds to the polarization time constant of the dielectric material.

In Equation 59, frequency-dependent complex shunt loss conductance is expressed as [4],

Equation 60

$$G(\omega) = G_o + G_d \frac{j\omega}{j\omega + \omega_p}$$

Where the imaginary part of the conductance contributes reactively. In cases of multiple dielectric materials surrounding the system, the complex loss conductance extends as linear combinations of multiple dipole moments as:

Equation 61

$$G(\omega) = G_o + \sum_k G_{dk} \frac{j\omega}{j\omega + \omega_{pk}}$$

Since Equation 61 satisfies the Krong-Kramers condition, you can ensure the passivity/causality of the system. Activating this new model changes the definition of G_d from conventional [S/m*Hz] to [S/m].

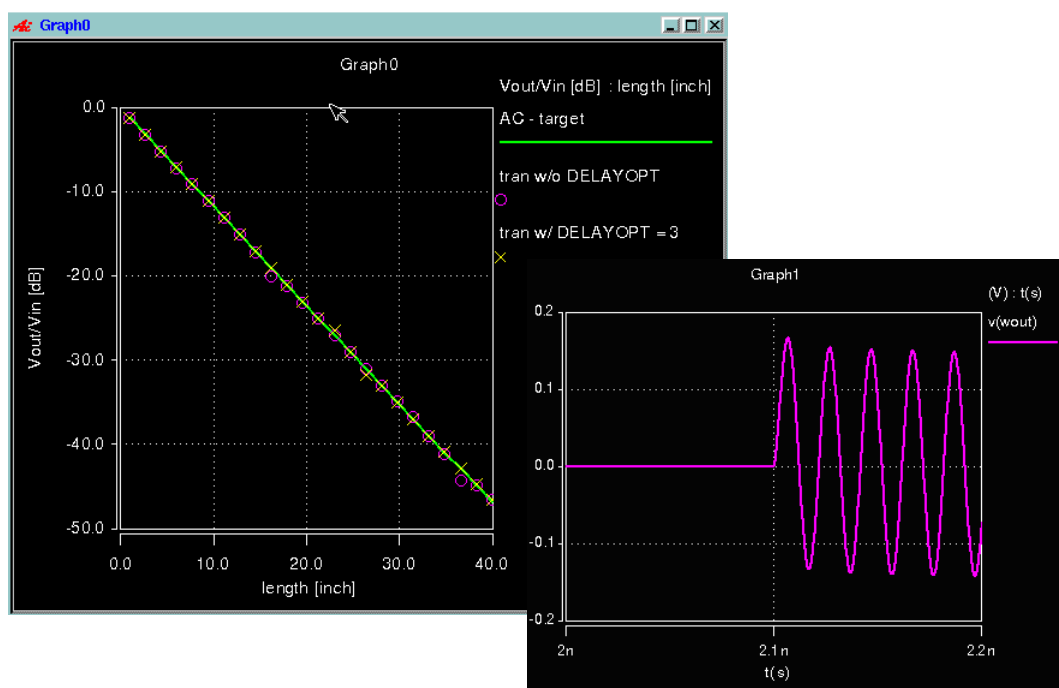
Fitting Procedure Triggered by INCLUDEGDIMAG Keyword

A fitting procedure generates a complex dielectric model with as close behavior as possible to the conventional pure real loss conductance model while preserving passivity. The `INCLUDEGDIMAG` keyword is the trigger to activate the new complex dielectric loss model. You activate the model with conventional `Go/Gd` input with `INCLUDEGDIMAG=yes` without polarization constant (`wp`) input, and the W-element automatically generates the new model by fitting.

In this fitting process, the W-element automatically computes wp and Gd values for Equation 61 where the real part of the function fits with conventional pure real dielectric loss model, $G(f) = G_0 + fG_d$. Then the imaginary part of derived model contributes to the frequency dependency of the capacitance.

Because the model ensures causality, frequency domain and time domain responses maintain better consistency (see Figure 67). Also, you reduce the functional overhead of the `DELAYOPT=3` for passive transfer functions. Thus, you improve performance of the `DELAYOPT` function.

Figure 67 Improved consistency using the `INCLUDEGDIMAG` keyword



To set the `INCLUDEGDIMAG` keyword as a global option for all W-elements in a netlist, see `.OPTION WINCLUDEGDIMAG` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Example: Complex Dielectric Loss Model

This fully executable example shows use of two methods:

1. Method 1- Specifying Gd/wp pairs
2. Method 2 - Automatic parameter generation

```
* complex dielectric loss model example
* -- method 1: specify Gd/wp pairs -----
.model use_wp W modeltype=rlgc n=1
+ Lo= 3.842520e-07
+ Co= 1.212598e-10
+ Ro= 7.559055e+01
+ Go= 0.000000e+000
+ Rs= 1.510184e-03
+ Gd =
+ 0.0008840 $ for wp[1]
+ 0.00365   $ for wp[2]
+ 0.04848   $ for wp[3]
+ 0.39910   $ for wp[4]
+ wp=
+ 0.04396e9 $ wp[1]
+ 0.3768e9  $ wp[2]
+ 3.1e9     $ wp[3]
+ 30e9      $ wp[4]
W1 in1 0 out1 0 N=1 RLGCmodel=use_wp l=1
+ includegdimag=yes
P11 in1 0 port=1 ac=1 sin 0 1 1e8
P12 out1 0 port=2

* -- method 2: automatic parameter generation -----
* use INCLUDEGDIMAG=yes
* without polarization constant input
.model auto_fit W modeltype=rlgc n=1
+ Lo= 3.842520e-07
+ Co= 1.212598e-10
+ Ro= 7.559055e+01
+ Go= 0.000000e+000
+ Rs= 1.510184e-03
+ Gd= 5.983933e-11
W2 in2 0 out2 0 N=1 RLGCmodel=auto_fit l=1
+ INCLUDEGDIMAG=yes
P21 in2 0 port=3 ac=1 sin 0 1 1e8
P22 out2 0 port=4
.opt post
.ac dec 10 1 1e10
.tran 0.1n 20n
.end
```

Determining Matrix Properties

All matrices in [Frequency-Dependent Matrices](#) are symmetric.

- The diagonal terms of L and C are positive, nonzero.
- The diagonal terms of R_o , R_s , G_o , and G_d are nonnegative (can be zero).
- Off-diagonal terms of the L , R_o impedance matrices are nonnegative.

R_o can have negative off-diagonal terms, but a warning appears. Negative off-diagonal terms normally appear when you characterize R_o at a frequency higher than zero. Theoretically, R_o should not contain negative off-diagonal terms because these might cause errors during analysis.

- By convention, off-diagonal terms of admittance matrices C , G_o , and G_d are negative.
- Off-diagonal terms of all matrices can be zero.

The elements of admittance matrices relate to the self/mutual admittances (such as those that the U-element generates):

Equation 62

$$Y_{ii} = Y_{ri}^{(self)} + \sum_{k(k \neq i)} Y_{ki}^{(mutual)}$$

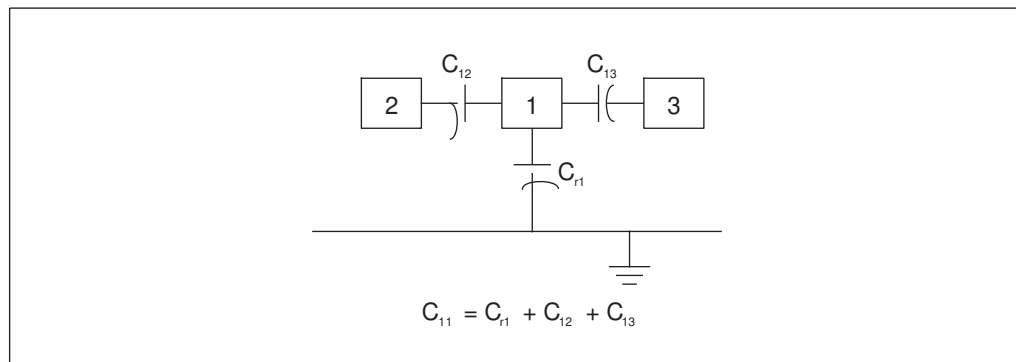
Equation 63

$$Y_{ij} = -Y_{ij}^{(mutual)} \quad (i \neq j)$$

In the preceding equations, Y stands for either C , G_o , or G_d .

A diagonal term of an admittance matrix is the sum of all self and mutual admittance in this row. This term is larger (in absolute value) than the sum of all off-diagonal terms in its row or column. Admittance matrices are strictly diagonally dominant (except for a zero matrix).

For example, you can express diagonal terms for capacitance matrix c as shown in the following figure:



You can obtain loop impedance matrix terms from the partial impedance matrix:

Equation 64

$$Z_{ij}^{(loop)} = Z_{ij}^{(partial)} - Z_{ig}^{(partial)} - Z_{jg}^{(partial)} + Z_{gg}^{(partial)}$$

where partial impedance is a ratio of induced current versus applied voltage without considering the series and coupling effect of the return path. Then in the preceding equation, the g index denotes a reference node which does not appear in Maxwellian format matrices.

Using the PRINTZO Option

The `PRINTZO` option outputs the W-element complex characteristic impedance matrix to a `.wzo` file. For simplicity, since the W-element is a symmetric system, the `zo` matrix is a symmetric matrix. Therefore, PrimeSim HSPICE only outputs the lower half of the matrix. For discussion of the `*.wzo` file, see [File Description for *.wzo](#) and the following sections. For example, the following frequency-sweep example shows the use of the `PRINTZO` option with the W-element to check for characteristic impedance.

Input:

```
W1 N=2 in1 in2 gnd out1 out2 gnd RLGCMODEL=line2 l=0.1
+ PRINTZO=POI 3 1e6 1e9 1e12
```

The file `line2.wzo` stores the output (see the following example external file).

```
* w-element model [2_line] Characteristic Impedance Matrix:
.MODEL ZO SP N=2 SPACING=POI MATRIX=SYMMETRIC
+ DATA=3
+ 1.0e6
+ 175.362 -156.577
+ 3.54758 -2.53246 175.362 -156.577
+ 1.0e9
+ 48.7663 -1.3087
+ 1.69417 -0.0073233 48.7663 -1.3087
+ 1e12
+ 48.9545 0.238574
+ 1.66444 0.0348332 48.9545 0.238574
```

The following example shows a `PRINTZO` statement with the `MIXEDMODEZO` option enabled. The syntax is:

```
W1 N=2 in1 gnd out1 out2 gnd RLGCMODEL=line2_mx l=0.1
+ PRINTZO=POI 3 1e6 1e9 1e12
+ MIXEDMODEZO=1
```

Then the mixed mode characteristic impedance will be stored in: `line2_mx.wzo`. For more information about `*.wzo`, see [File Description for *.wzo](#).

Defining Frequency-dependent Impedance in Mixed Mode Printing

This section discusses the PrimeSim HSPICE ability to print a complex characteristic impedance matrix in differential and common modes at given frequency points. This functionality supports high-speed network designs where differential data transfer systems commonly achieve a higher data transfer rate with low loss. For such designs, impedance information in mixed (differential and common) mode is more useful than single-ended representation. To learn more about the SP model syntax which has a complex number matrix by default, see [Small-Signal Parameter Data Frequency Table Model](#).

[Equation 65](#) provides a choice to output transmission-line characteristic impedance in mixed mode. [Equation 65](#) also gives the characteristic impedance as a frequency-independent constant matrix for the ideal lossless transmission line system:

Equation 65

$$\mathbf{Z_0} = (\mathbf{L} \cdot \mathbf{C}^{-1})^{1/2}$$

where $\mathbf{L}$ and $\mathbf{C}$ are inductance and capacitance matrices of the system, respectively, and in this case, the characteristic impedance is a real matrix. When the system becomes lossy, in other words, the system has nonzero resistance, $\mathbf{R}$, and/or nonzero shunt loss conductance, $\mathbf{G}$. [Equation 66](#) expresses characteristic impedance as a function of frequency, ω :

Equation 66

$$\mathbf{Z_0} = ((\mathbf{R} + j\omega\mathbf{L}) \cdot (\mathbf{G} + j\omega\mathbf{C})^{-1})^{1/2}$$

In this case, $\mathbf{Z_0}$ becomes a complex matrix. Knowledge of the characteristic impedance ($\mathbf{Z_0}$) matrix of the transmission line system at given frequency point enables circuit designers to establish well matched signal transfer conditions. This ability preserves integrity of the system, especially for high frequency operation. This feature allows you to check the complex characteristic impedance matrix of the system.

Note:

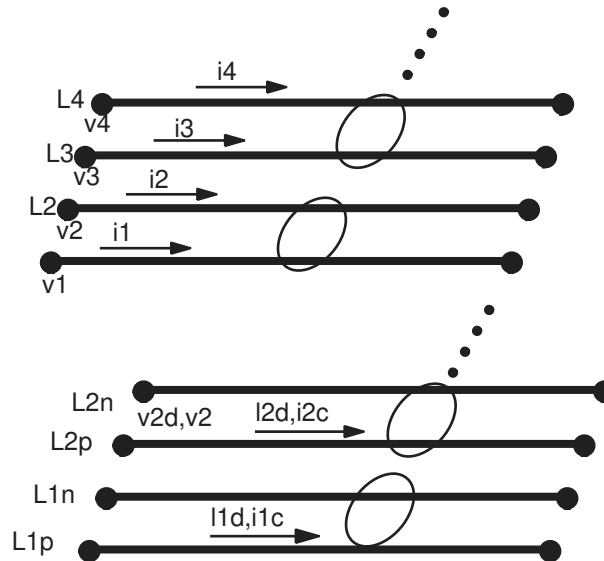
The PRINTZO function does not evaluate [Equation 66](#), but it obtains $\mathbf{Z_0}$ directly from the W-element's AC model. It does this so you can get $\mathbf{Z_0}(f)$ from other types of W-element models as well as S-parameter models.

In cases of lossless transmission line structure, the PRINTZO result may differ from [Equation 66](#) at very low frequencies because the RLGC-based lossless W-element adds a small amount of loss in the very low frequency range when it initializes the AC model. The effect of this on your actual AC or transient simulation is negligible, but is important to achieve stable simulation.

The PrimeSim HSPICE W-element creates its own frequency-dependent characteristics when the construct uses RLGC parameters (RLGC or RLGC table model), structural (field

solver) mode, U-element model, or scattering (S-parameter) model. You can compute the characteristic impedance, not only from the RLGC model, but also from any other of W-element configurations when you use the keyword `PRINTZO` to specify the frequency point.

Figure 68 Definition of mixed-mode impedance and derivation from single-ended impedance



Equation 68 defines differential and common voltage as:

where $\mathbf{M}_V$ and $\mathbf{M}_i$ are voltage and current transformation matrices. Both single-ended and mixed mode representations satisfy relationships of voltage and current vector through characteristic impedance matrices as:

Equation 67

$$V = Z_o I$$

$$V_{mixed} = Z_{o_{mixed}} \cdot I_{mixed}$$

Substitute Equation 67 for Equation 69, and the mixed-mode characteristic impedance can relate to the single-ended one as shown in Equation 69:

Equation 68

$$\mathbf{V}_{mixed} = \begin{pmatrix} v_{d1} \\ v_{c1} \\ v_{d2} \\ v_{c2} \\ \vdots \\ \vdots \end{pmatrix} = \begin{pmatrix} v_1 - v_2 \\ (v_1 + v_2)/2 \\ v_3 - v_4 \\ (v_3 + v_4)/2 \\ \vdots \\ \vdots \end{pmatrix} = \begin{pmatrix} 1 & -1 & 0 & 0 & \dots \\ 1/2 & 1/2 & 0 & 0 & \dots \\ 0 & 0 & 1 & -1 & \dots \\ 0 & 0 & 1/2 & 1/2 & \dots \\ \vdots & \vdots & \vdots & \vdots & \dots \\ \vdots & \vdots & \vdots & \vdots & \dots \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ \vdots \\ \vdots \end{pmatrix} = \mathbf{M}_V \mathbf{V}$$

$$\mathbf{I}_{mixed} = \begin{pmatrix} i_{d1} \\ i_{c1} \\ i_{d2} \\ i_{c2} \\ \vdots \\ \vdots \end{pmatrix} = \begin{pmatrix} (i_1 - i_2)/2 \\ i_1 + i_2 \\ (i_3 - i_4)/2 \\ i_3 + i_4 \\ \vdots \\ \vdots \end{pmatrix} = \begin{pmatrix} 1/2 & -1/2 & 0 & 0 & \dots \\ 1 & 1 & 0 & 0 & \dots \\ 0 & 0 & 1/2 & -1/2 & \dots \\ 0 & 0 & 1 & 1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \dots \\ \vdots & \vdots & \vdots & \vdots & \dots \end{pmatrix} \begin{pmatrix} i_1 \\ i_2 \\ i_3 \\ i_4 \\ \vdots \\ \vdots \end{pmatrix} = \mathbf{M}_I \mathbf{I}$$

Equation 69

$$\mathbf{M}_V \cdot \mathbf{V} = \mathbf{Z}_{o_{mixed}} \cdot \mathbf{M}_I \cdot \mathbf{I}$$

$$\mathbf{V} = \mathbf{M}_V^{(-1)} \cdot \mathbf{Z}_{o_{mixed}} \cdot \mathbf{M}_I \cdot \mathbf{I}$$

thus:

Equation 70

$$\mathbf{Z}_{o_{mixed}} = \mathbf{M}_V \cdot \mathbf{Z}_o \cdot \mathbf{M}_I^{-1}$$

For example, for a system with one differential pair of lines, the transformation matrix would be:

Equation 71

$$\mathbf{M}_V = \begin{pmatrix} 1 & -1 \\ 1/2 & 1/2 \end{pmatrix}, \mathbf{M}_I = \begin{pmatrix} 1/2 & -1/2 \\ 1 & 1 \end{pmatrix}$$

Therefore, Equation 72 expresses mixed-mode characteristic impedance as:

Equation 72

$$\begin{aligned} \mathbf{Z}_{o_{mixed}} &= \mathbf{M}_V \cdot \mathbf{Z}_o \cdot \mathbf{M}_I^{-1} \\ &= \frac{1}{4} \begin{pmatrix} 2 & -2 \\ 1 & 1 \end{pmatrix} \cdot \mathbf{Z}_o \cdot \begin{pmatrix} 2 & 1 \\ -2 & 1 \end{pmatrix} \\ &= \frac{1}{4} \begin{pmatrix} 2 & -2 \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} Z_{11} & Z_{12} \\ Z_{21} & Z_{22} \end{pmatrix} \cdot \begin{pmatrix} 2 & 1 \\ -2 & 1 \end{pmatrix} \\ &= \frac{1}{4} \begin{pmatrix} 4Z_{11}-4Z_{12} & -4Z_{21}+4Z_{22} & 2Z_{11}+2Z_{12}-2Z_{21} & -2Z_{22} \\ 2Z_{11}-2Z_{12} & +2Z_{21}-2Z_{22} & Z_{11}+Z_{12}+Z_{21} & +Z_{22} \end{pmatrix} = \begin{pmatrix} Z_{dd} & Z_{dc} \\ Z_{cd} & Z_{cc} \end{pmatrix} \end{aligned}$$

where:

- Z_{dd} is differential (mode) impedance
- Z_{cc} is common mode impedance.

Differential impedance is useful for designers to check matching characteristics of differential signal transfer systems.

Typically, for a symmetric two-line structure with weak coupling, single-ended characteristic impedance matrix components become: $Z_{11} = Z_{22} = Z_{self} (\sim 50\Omega)$ $Z_{12} = Z_{21} = Z_{mutual} (\sim 0)$. Therefore, mixed-mode characteristic impedance is:

Equation 73

$$\begin{pmatrix} Z_{dd} & Z_{dc} \\ Z_{cd} & Z_{cc} \end{pmatrix} = \begin{pmatrix} 2Z_{self} - 2Z_{mutual} & 0 \\ 0 & \frac{1}{2}(Z_{self} + Z_{mutual}) \end{pmatrix} = \begin{pmatrix} 100\Omega & 0 \\ 0 & 25\Omega \end{pmatrix}$$

Equation 74

$$\begin{pmatrix} Z_{dd} & Z_{dc} \\ Z_{cd} & Z_{cc} \end{pmatrix} = \begin{pmatrix} 2Z_{self} - 2Z_{mutual} & 0 \\ 0 & \frac{1}{2}(Z_{self} + Z_{mutual}) \end{pmatrix} = \begin{pmatrix} 100\Omega & 0 \\ 0 & 25\Omega \end{pmatrix}$$

File Description for *.wzo

When you include the `PRINTZO` option in a W-element statement, you also generate the *.wzo file. The prefix for this file is the W-element model name:

```
W1 N=2 i1 i2 gnd o1 o2 gnd RLGCMODEL=line l=10m PRINTZO=POI 2 10meg 10g
+ mixedmodezo=1 $ 1= mixed mode 0= common mode
```

The netlist fragment creates two impedance matrices, one at 10 MHz and the other at 10 GHz. The netlist fragment then writes them to the file `line.wzo` for this two-conductor W-element.

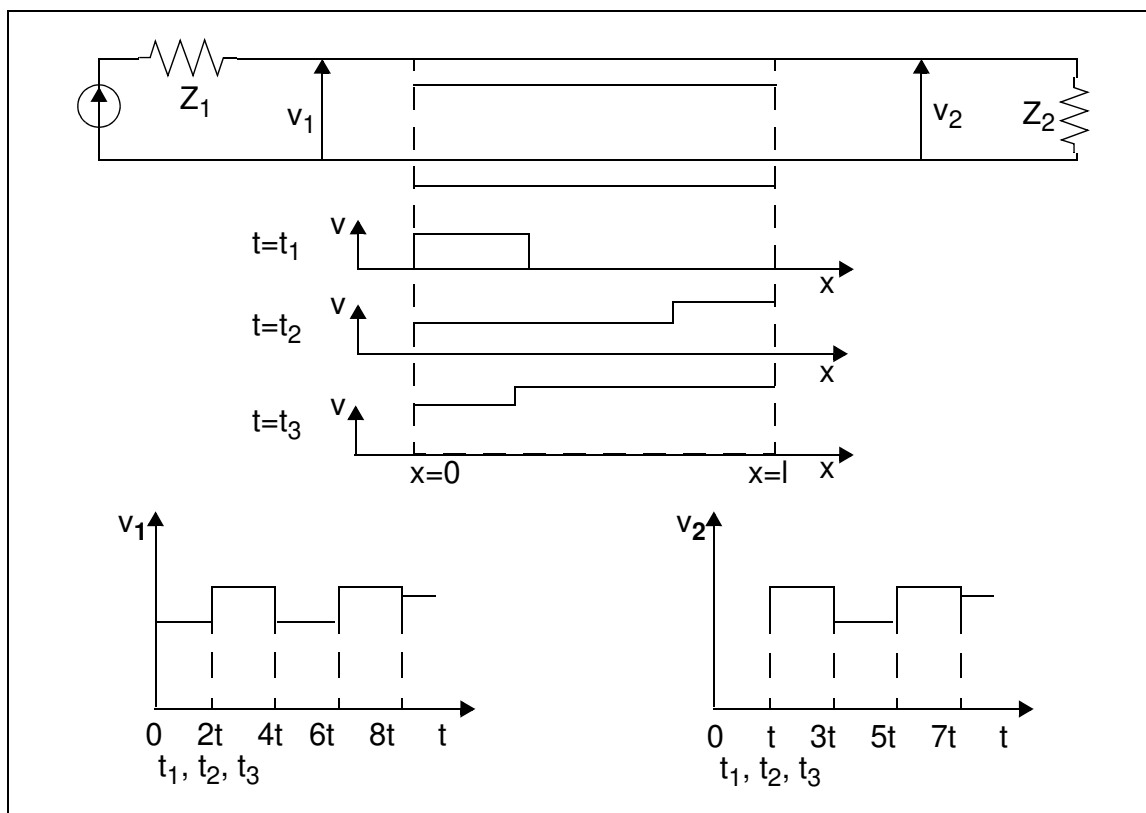
The *.wzo file contains complex impedances in the form $R + jX$, where the first term of each pair is R and the second is X . The off-diagonal (negative) terms relate to the interactions between conductors (common mode). For a single-ended analysis, the diagonal terms are identical.

For the mixed-mode configuration, the $[1,1]$ term is the differential mode impedance, and the $[2,2]$ term is the common mode impedance. It is typical for the matrices to show a 3 or 4 to 1 ratio of the $[1,1]$ to the $[2,2]$ terms. Loosely-coupled conductors means that the ratio is close to 4:1. This is because $Z_{11} = Z_{22} = Z_{self}$ and $Z_{12} = Z_{21} = Z_{mutual} = 0$.

Wave Propagation

To illustrate the physical process of wave propagation and reflection in transmission lines, [Figure 69](#) shows lines where the voltage step excites simple termination.

Figure 69 Propagation of a voltage step in a transmission line

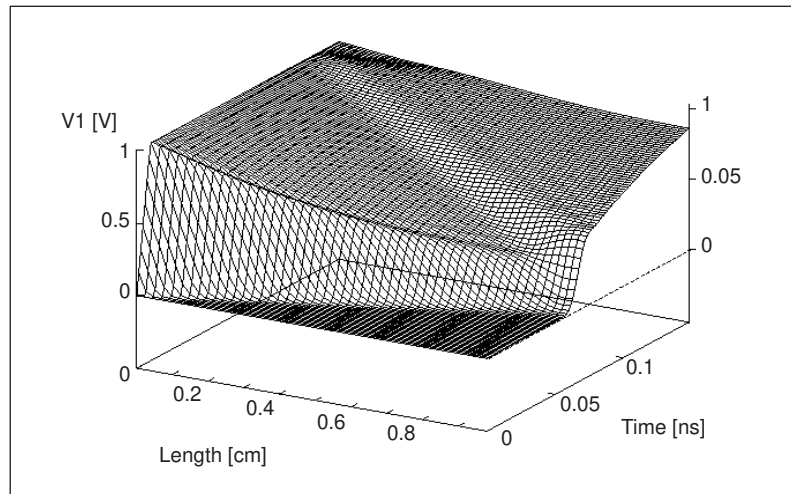


- At time $t=t_1$, a voltage step from the e_1 source, attenuated by the Z_1 impedance, propagates along the transmission line.
- At $t=t_2$, the voltage wave arrives at the far end of the transmission line, is reflected, and propagates in the backward direction. The voltage at the load end is the sum of the incident and reflected waves.
- At $t=t_3$, the reflected wave arrives back at the near end, is reflected again, and again propagates in the forward direction. The voltage at the source end is the sum of attenuated voltage from the e_1 source, the backward wave, and the reflected forward wave.

The surface plot in [Figure 70](#) shows voltage at each point in the transmission line. The input incident propagates from the left (length = 0) to the right. You can observe both

reflection at the end of the line (length = 1) and a reflected wave that goes backward to the near end.

Figure 70 Surface plot for the transmission line shown in Figure 69



You can find more information about transmission lines in this resource: H.B. Bakoglu, *Circuits, Interconnections and Packaging for VLSI*. Reading, MA: Addison-Wesley, 1990.

The following sections discuss these topics:

- [Propagating a Voltage Step](#)
- [Handling Line-to-Line Junctions](#)

Propagating a Voltage Step

This section is a summary of the process in [Figure 69](#) to propagate a voltage step in a transmission line.

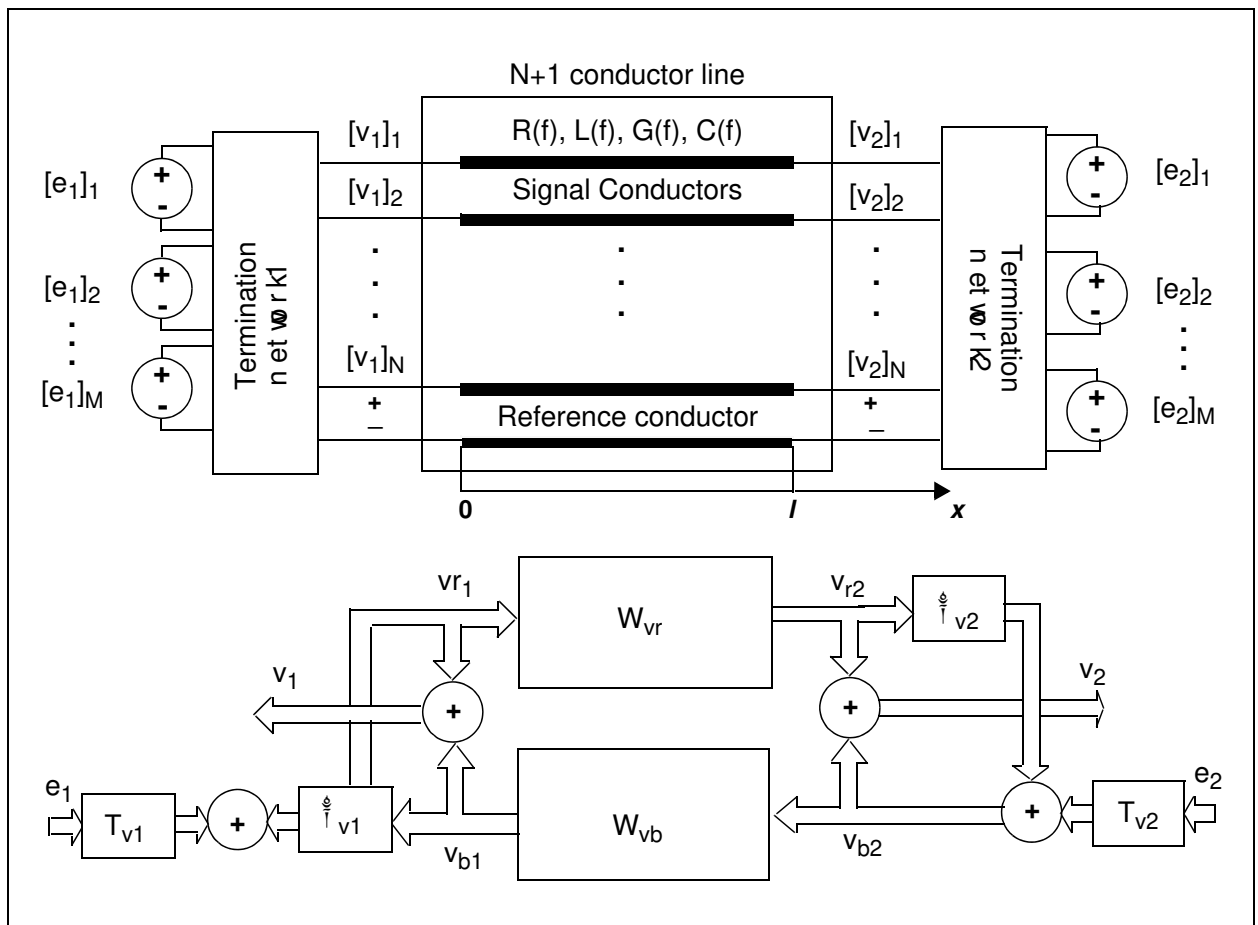
- Signals from the excitation source spread-out in the termination networks, and propagate along the line.
- As the forward wave reaches the far-end termination, it does the following:
 - Reflects.
 - Propagates backward.
 - Reflects from the near-end termination.

- Propagates forward again.
- Continues in a loop.
- The voltage at any point along the line, including the terminals, is a superposition of the forward and backward propagating waves.

Figure 71 shows the system diagram for this process, where:

- W_{vr} and W_{vb} are forward and backward matrix propagation functions for voltage waves.
- T_1 and T_2 stand for the near-end matrix transmission and reflection coefficients.
- $\Gamma_1\Gamma_2$, (Gamma_1,Gamma_2) stand for the far-end matrix transmission and reflection coefficients.

Figure 71 System model for transmission lines



This model reproduces the general relationship between the physical phenomena of wave propagation, transmission, reflection, and coupling in a distributed system. It can represent an arbitrarily-distributed system, such as:

- Transmission line
- Waveguide
- Plane-wave propagation

You can use this model for:

- System analysis of distributed systems, or
- Writing a macro solution for a distributed system without complicated mathematical derivations.

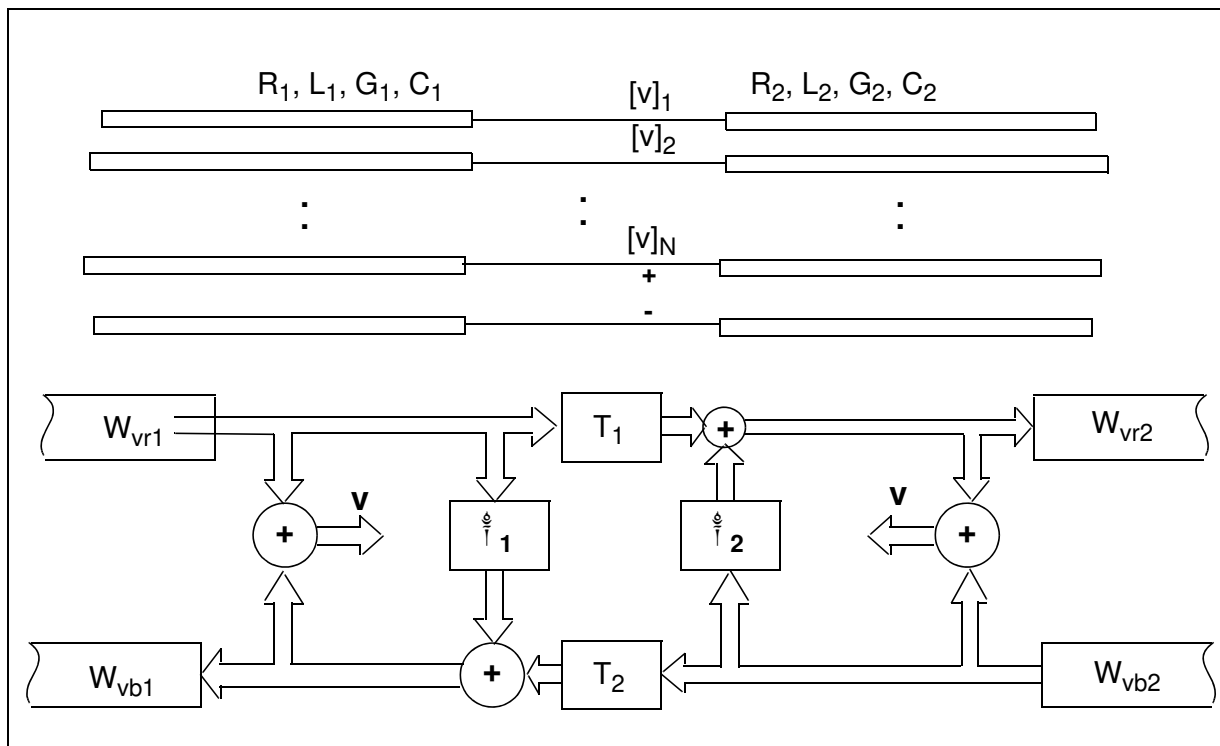
[Figure 60](#) shows how transmission lines and terminations form a feedback system. Because the feedback loop contains a delay, both the phase shift, and the sign of the feedback change periodically with the frequency. This causes oscillations in the frequency-domain response of the transmission lines, such as those in [Figure 60](#).

Handling Line-to-Line Junctions

A special case occurs when the line terminates in another line. [Figure 61](#) shows the system diagram for a line-to-line junction. Use this diagram to:

- Solve multi-layered plane-wave propagation problems.
- Analyze common waveguide structures.
- Derive generalized transmission and reflection coefficient formulas.
- Derive scattering parameter formulas.

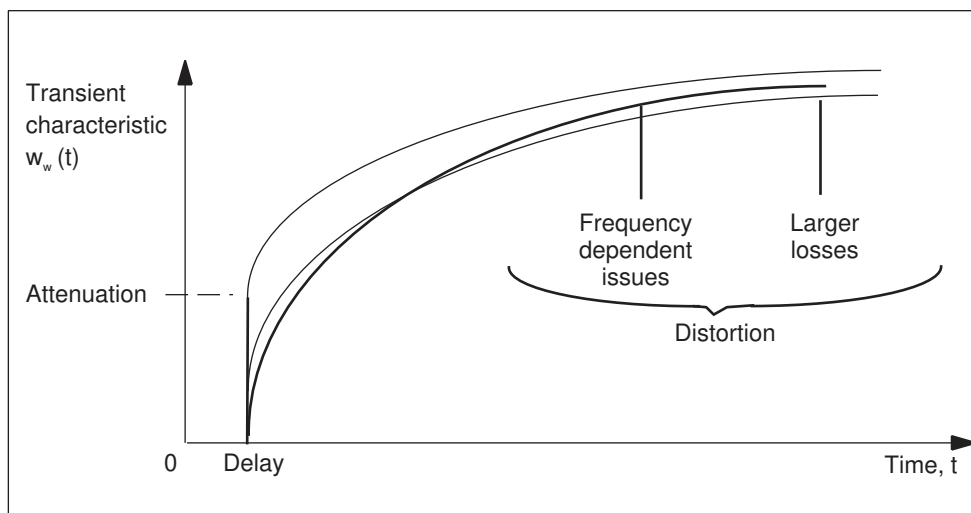
Figure 72 System model for a line-to-line junction



The W_{vr} and W_{vb} propagation functions describe how propagation (from one termination to another) affects a wave. These functions are equal for the forward (W_{vr}) and backward (W_{vb}) directions. The off-diagonal terms of the propagation functions represent the coupling between conductors of a multiconductor line.

As a wave propagates along the line, it experiences delay, attenuation, and distortion (see [Figure 73](#)). Lines with frequency-dependent parameters (that is, all real lines) do not contain the frequency-independent attenuation component.

Figure 73 Propagation function transient characteristics (unit-step response)



Overview of T-element Usage

Some applications do not require the sophisticated models used by the PrimeSim HSPICE W-element or S-element to effectively predict transmission line effects. In such cases, the less complex T-element and U-element transmission line models can provide adequate solutions. For discussion of the rarely used U-element model, see *PrimeSim Continuum User Guide: Elements*. For detailed information on the T-element syntax and examples, see T-element (Ideal Transmission Lines) in *PrimeSim Continuum User Guide: Elements*.

For information regarding the physics of modeling transmission lines with ideal and lumped equivalents, see [Appendix A, Simulating Circuits with Signetics Drivers](#).

The following sections discuss these topics:

- [Invoking Ideal T-element Transmission Lines](#)
- [Selecting Ideal or Lossy Transmission Line Elements](#)

Using T-element for Predicting Simple Tx Line Effects

The following sections discuss these topics:

- [T-element Capabilities](#)
- [Invoking Ideal T-element Transmission Lines](#)

T-element Capabilities

The T-element is useful for modeling ideal transmission lines in terms of their impedance and delay. The U-element is useful for modeling lossy, nonideal transmission lines in terms of lumped-element equivalent circuits. The T- and U-elements can model effects such as:

- Time delay
- Phase shift
- Coupling and Crosstalk
- Distortion

Invoking Ideal T-element Transmission Lines

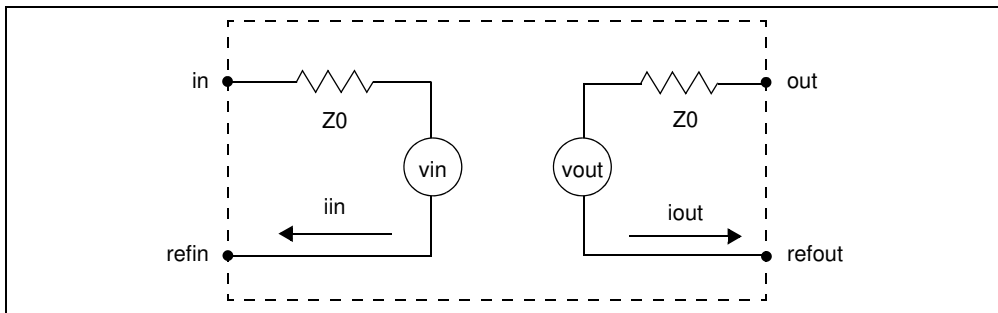
The following sections discuss these topics:

- [Ideal Transmission Line Model](#)
- [Selecting Ideal or Lossy Transmission Line Elements](#)
- [Lossless Voltage and Current Propagation](#)

Ideal Transmission Line Model

The ideal transmission model is based on a characteristic impedance (Z_0), and propagation delay (TD). The Z_0 and TD values may be obtained from a U model.

Figure 74 Ideal element circuit



The input and output of an ideal transmission line have these relationships:

$$Vin|_t = V(out-refout)|_{t-TD} + (iout \times Z_0)|_{t-TD}$$

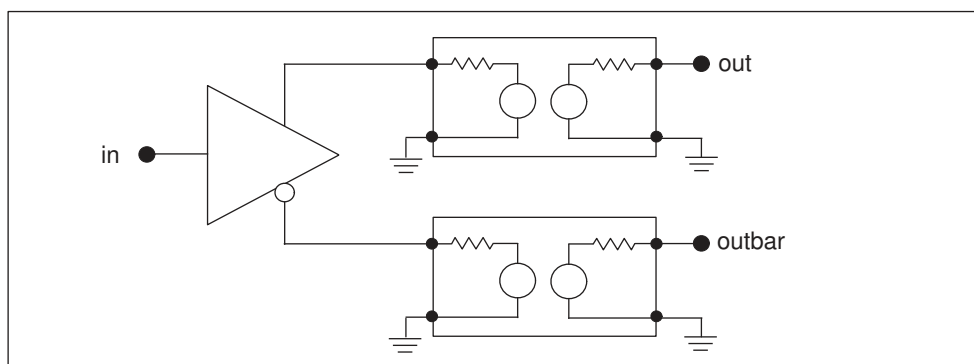
$$Vout|_t = V(in-refin)|_{t-TD} + (iin \times Z_0)|_{t-TD}$$

The signal (propagation) delays for ideal transmission lines are specified as one of the following:

- $TD_{eff}=TD$ (if only TD is given)
- $TD_{eff}=TD*L$ if TD and L are given
- $TD_{eff}=NL/F$ if NL and F are given
- $TD_{eff}=TD$ if you use a U model

The ideal transmission line only delays the difference between the signal and the reference. Note that although you can specify length as an input parameter, it is used only to compute the effective propagation delay. Some applications, such as a differential output driving twisted pair cable, require both differential and common mode propagation. Use a W-element or U-element, if you need to model both propagation modes and coupling. If not, you can use two T-elements (see [Figure 75](#)) to model uncoupled differential signals. In this figure, the two lines are completely uncoupled but the delay and impedance values are correctly modeled.

Figure 75 Use of Two T-elements for Differential Signals



You cannot implement coupled lines with the T-element so use W-elements or U-elements for applications requiring two or three coupled conductors.

Synopsys circuit simulation uses a transient timestep that does not exceed half the minimum line delay. Very short transmission lines (relative to the analysis time step) cause long simulation times. You can replace very short lines with a single R, L, or C-element (see [Figure 76](#)).

Selecting Ideal or Lossy Transmission Line Elements

An electrical model that simulates the behavior of an interconnect or transmission line must consider all of the following:

- Physical nature or electrical properties of the interconnect
- Bandwidth or risetime of signals of interest
- Interconnect's actual time delay or equivalent electrical length
- Complexity and accuracy of the model

When accuracy is the utmost priority, and all high-frequency effects must be taken into account, use the W-element or S-element with models based on measurements or EM analyses. When basic models are adequate, you can choose from the following circuit models for interconnects:

- No model at all. Use a common node to connect two elements.
- Lumped models with R, L, and C-elements as described in the *PrimeSim Continuum User Guide: Elements*. These include a series resistor (R), a shunt capacitor (C), a series inductor and resistor (RL), or a series resistor and a shunt capacitor (RC).
- Transmission line models such as an ideal transmission line (T-element) or a lossy transmission line (U-element)

The following tables and charts explain the factors involved when determining the best choice between RLC-elements, the T-element, or U-element transmission line models.

The following sections discuss these topics:

- [Source Properties](#)
- [Interconnect Properties](#)
- [Comparison of Ideal and Lossy Transmission Line Elements](#)
- [Transmission Lines: Example](#)

Source Properties

t_{rise} = source risetime

R_{source} = source output impedance

Interconnect Properties

Z_0 = characteristic impedance

TD = time delay of the interconnection

or:

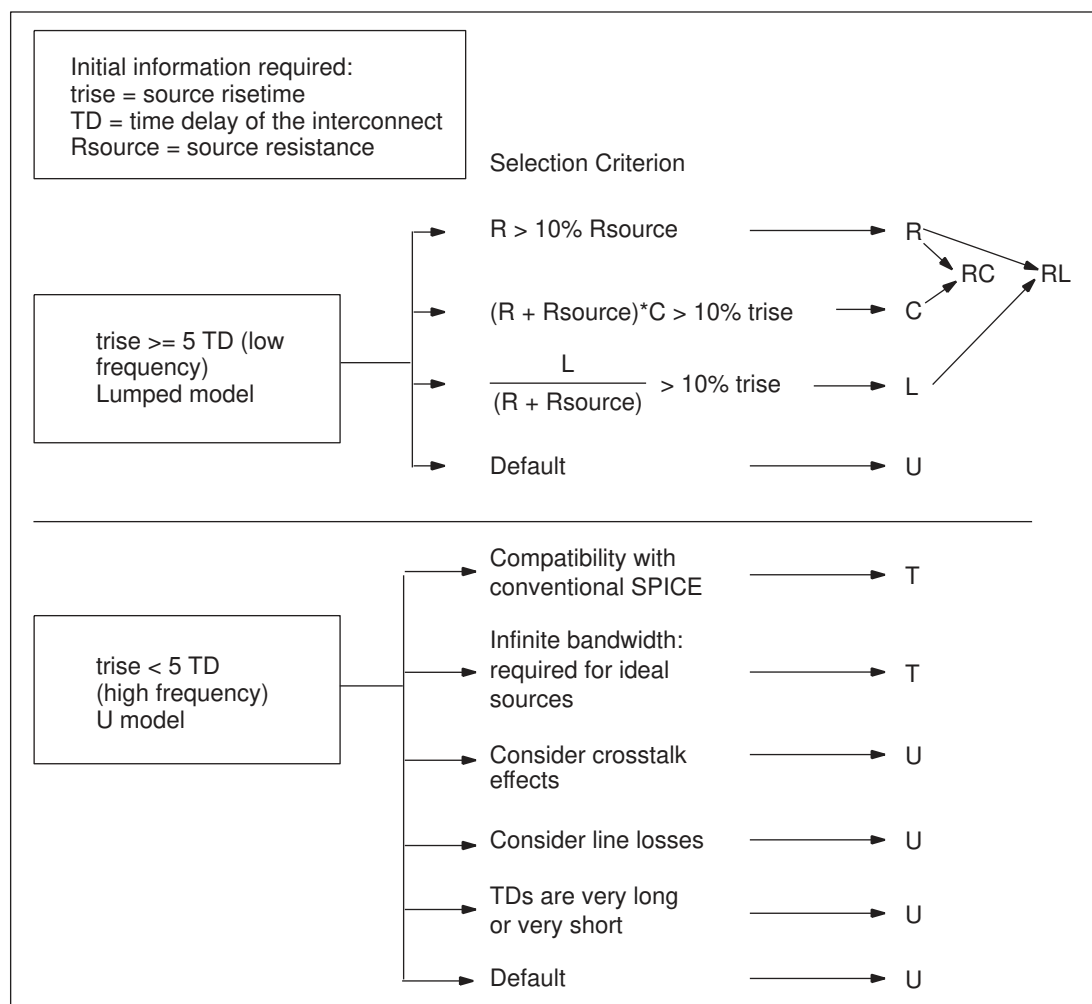
R = equivalent series resistance

C = equivalent shunt capacitor

L = equivalent series inductance

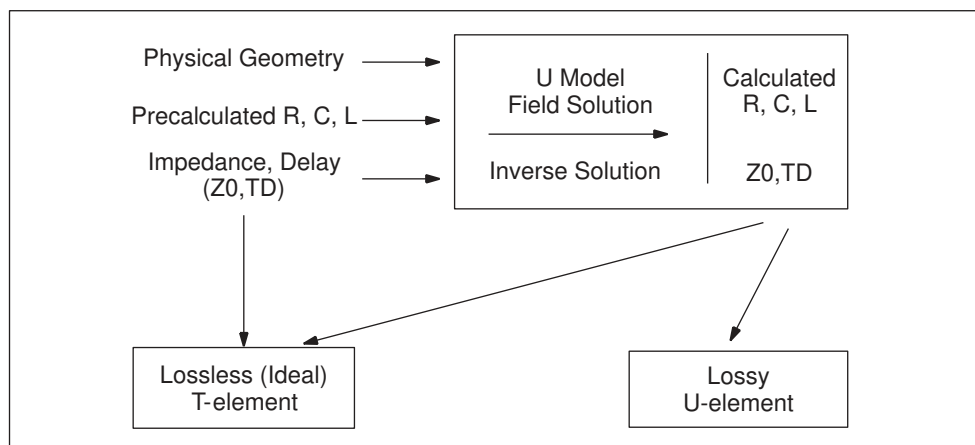
Figure 76 shows how to select a model based on source and interconnect properties.

Figure 76 Wire model selection chart



Use the U model with either the ideal T-element or the lossy U-element. You can also use the T-element alone without the U model. PrimeSim HSPICE offers both a flexible definition of the conventional SPICE T-element and an accurate U-element lossy simulation.

Figure 77 U Model, T-element, and U-element relationship



The T- and U-elements do not support the $[M=va]$ multiplier function. If a U- or T-element is used in a subcircuit and an instance of the subcircuit has a multiplier applied, that multiplier is not applied to the T- or U-element.

A warning message similar to the following is issued in both the status file (.st0) and the output file (.lis) if the smallest transmission line delay is less than $TSTOP/10e6$:

```

**warning**: the smallest T-line delay (TD) = 0.245E-14 is too small
Please check TD, L and SCALE specification
  
```

This feature is an aid to finding errors that cause excessively long simulations.

Note:

All transmission lines have a ground reference for the signal conductors. In this manual, the ground reference is called the reference plane so that it is not confused with SPICE ground. The reference plane is the shield or the ground plane of the transmission line element. The reference plane nodes may or may not be connected to SPICE ground.

Comparison of Ideal and Lossy Transmission Line Elements

The ideal and lossy transmission line models each have particular advantages, and they may be used in a complementary manner. Both model types are fully functional in AC analysis and transient analysis. Some of the comparative advantages and uses of each type of model are listed in [Table 11](#).

Table 11 Ideal versus Lossy Transmission Line

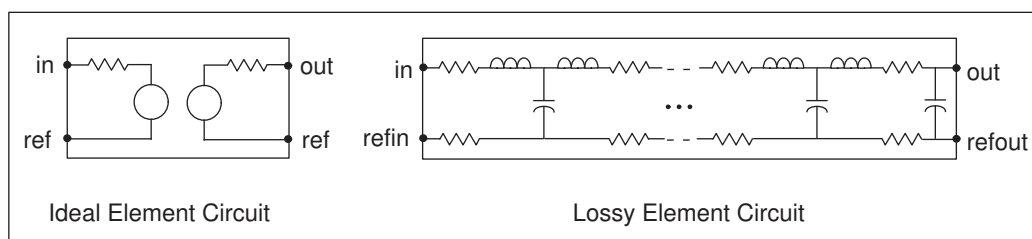
Ideal Transmission Line	Lossy Transmission Line
Lossless	Includes loss effects

Table 11 *Ideal versus Lossy Transmission Line (Continued)*

Ideal Transmission Line	Lossy Transmission Line
Used with voltage sources	Used with buffer drivers
No limit on input risetime	Prefiltering necessary for fast rise
Less CPU time for long delays	Less CPU time for short delays
Differential mode only	Supports common mode simulation
No ground bounce	Includes reference plane reactance
Single conductor	Up to five signal conductors allowed
AC and transient analysis	AC and transient analysis

The ideal line is modeled as a voltage source and a resistor. The lossy line is modeled as a multiple lumped filter section as shown in [Figure 78](#).

Figure 78 *Ideal versus lossy transmission line model*



Because the ideal element represents the complex impedance as a resistor, the transmission line impedance is constant, even at DC values. On the other hand, you must prefilter the lossy element if ideal piecewise linear voltage sources are used to drive the line.

Transmission Lines: Example

The following file fragment is an example of how both T-elements and U-elements can be referred to a single U-model as indicated in [Figure 77](#). The file specifies a 200 mm printed circuit wire implemented as both a U-element and a T-element. The two implementations share a U-model that is a geometric description (`ELEV=1`) of a planar structure (`PLEV=1`).

```
T1 in gnd t_out gnd micro1 L=200m
U1 in gnd u_out gnd micro1 L=200m
.model micro1 U LEVEL=3 PLEV=1 ELEV=1 wd=2m ht=2m th=0.25m
+ KD=5
```

The next section provides details of element and model syntax.

Table 12 Transmission Line Parameters

Parameter	Description
T1, U1	Element names
micro1	The model name
in, gnd, t_out, and u_out	Nodes
L	The length of the signal conductor
wd, ht, th	Dimensions of the signal conductor and dielectric
KD	The relative dielectric constant

Lossless Voltage and Current Propagation

For the ideal transmission line, voltage and current propagate without loss along the length of the line ($\pm x$ direction) with spatial and time-dependence given according to the following equation:

$$v(x, t) = \text{Re} \left[A e^{j(\omega t - \beta x)} + B e^{j(\omega t + \beta x)} \right]$$

$$v(x, t) = \text{Re} \left[\frac{A}{Z_0} e^{j(\omega t - \beta x)} - \frac{B}{Z_0} e^{j(\omega t + \beta x)} \right]$$

The A represents the incident voltage, B represents the reflected voltage, Z_0 is the characteristic impedance, and β is the propagation constant. The latter is related to the transmission line inductance (L) and capacitance (C) by the following equation:

$$Z_0 = \sqrt{\frac{L}{C}}$$

$$\beta = \omega \sqrt{LC}$$

The L and C terms are in per-unit-length units (Henries/meter, Farads/meter). The following equation gives the phase velocity:

$$v_p = \frac{\omega}{\beta} = \frac{1}{\sqrt{LC}}$$

At the end of the transmission line ($x = l$), the propagation term βl becomes the following equation:

$$\beta l = \omega \sqrt{LC} \cdot l = \omega \frac{l}{v_p}$$

This is equivalent to an ideal delay with the following value:

$$T = \frac{l}{V_P} = \sqrt{LC} \cdot l$$

where:

T : absolute time delay (sec)

l : physical length (L) (meters)

V_P : phase velocity (meters/sec)

Using standard distance=velocity*time relationships, the PrimeSim HSPICE T-element parameter values are related to these terms according to:

$$V_P = f \cdot \lambda = \frac{1}{t_d}$$

where:

f : frequency

λ : wavelength

t_d : relative time delay (TD) (sec/meter)

$$T = \frac{l}{V_P} = t_d \cdot l = \frac{l}{f \cdot \lambda} = \frac{l / \lambda}{f} = \sqrt{LC} \cdot l$$

where:

l : physical length (L) (meters)

l / λ : normalized length (NL)

f : frequency at NL (F) (Hz)

$$T = TD \cdot L = \frac{NL}{L} = \sqrt{LC} \cdot L$$

The T-element is therefore an ideal behavioral model given in terms of Z_0 and delay, yet you can relate it back to inductance, capacitance, velocity, and/or length quantities as needed. PrimeSim HSPICE allows you to specify the T-element transmission line in four different ways:

- Z_0 , TD
- Z_0 , TD, L

- Z_0 , NL, F
- L, with $\sqrt{\frac{L}{C}}$ and $\sqrt{LC}$ values taken from a U model.

References

- [1] Dmitri Kuznetsov, "Optimal Transient Simulation of Transmission Lines," IEEE Trans., Circuits Syst., vol.43, pp. 110-121, Feb., 1996.
- [2] Luca Daniel and Joel Philips, "Model Order Reduction for Strictly Passive and Causal Distributed Systems," proceeding of DAC, June, 2002
- [3] [2] Colin Gordon, Thomas Blazeck, and Raj Mittra, "Time-Domain Simulation of Multiconductor Transmission Lines with Frequency-Dependent Losses," IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN, VOL. 11, NO. 11, pp. 1372-1387, NOVEMBER 1992
- [4] [3] Qinglian Yu and Omar Wing, "Computational Models of Transmission Lines with Skin Effects and Dielectric Loss," IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS-I: FUNDAMENTAL THEORY AND APPLICATIONS. VOL. 41, NO. 2, pp. 107-119, FEBRUARY 1994
- [5] Eric Bogatin, "Signal Integrity Simplified", Prentice HallBjorn Gustavsen and Adam Semlyen, "Rational Approximation of Frequency Domain Responses by Vector Fitting," IEEE Transaction on Power Delivery, Vol.14, No.3, pp. 1052-1061, July 1999
- [6] Ravi Kollipara, et al "Practical Design Considerations for 10 to 25 Gbps Copper Backplane Serial Links," DesignCon 2006, Santa Clara, CA, U.S.A. E. Hammerstad and O. Jensen, "Accurate models for microstrip computer aided design," IEEE MTT-S Int. Microwave Symp. Dig., 1980,. pp. 407-409.

7

Using the Field Solver to Extract Tx Line Parameters

Describes how to use the built-in PrimeSim HSPICE Field Solver for transmission line parameter extraction.

Transmission line simulation is challenging and time-consuming, because extracting transmission line parameters from physical geometry requires a significant effort. To minimize this effort, you can use a simple (but efficient and accurate) 2D electromagnetic field solver, which calculates the electrical parameters of a transmission line system, based on its cross-section.

The built-in 2-D electromagnetic field solver is highly-optimized for interconnects in stratified media. This field solver uses the W-element, and it supports optimization and statistical analysis within transient simulation.

The solver is based on:

- An improved version of the boundary-element method, and
- The filament method that is also implemented in the Synopsys product, Raphael.

See K. S. Oh, D. B. Kuznetsov, and J. E. Schutt-Aine, “Capacitance computations in a multi-layered dielectric medium using closed-form spatial Green s functions,” IEEE Trans. Microwave Theory and Tech., vol. 42, pp. 1443-1453, August 1994 for more information on the boundary-element method.

To learn more about BEM and Green’s Function, see the *Raphael Reference Manual*.

The following sections discuss these topics:

- [Using the Field Solver Model](#)
- [Field-Solver-Related Netlist Statements](#)
- [Accelerating the W-element Field Solver Using Multicore Technology](#)
- [Visualizing Cross-Sectional Geometric Information](#)
- [Field Solver Examples](#)

Using the Field Solver Model

Use the field-solver model to specify a geometry model for the W-element transmission line. In the field-solver model:

- The list of conductors must appear last.
- Conductors cannot overlap each other.
- The Field Solver assumes that floating conductors are electrically disconnected, and does not support non-zero fixed charges. Because the field solver is designed as 2-D, it ignores displacement current in floating conductors.
- The Field Solver treats metal layers in the layer stack as the reference node.
- Conductors defined as `REFERENCE` are all electrically-connected, and correspond to the reference node in the W-element.
- You must place signal conductors in the same order as the terminal list in the W-element statement. For example, the *i*th signal conductor (not counting reference and floating conductors) is associated with the *i*th input and output terminals specified in the corresponding W-element.
- Floating and reference conductors can appear in any order.

Note:

It is strongly recommended that a separate model be created for complex structures when an internal ground plane is introduced.

The following sections discuss these topics:

- [Filament Method](#)
- [Modeling Geometries](#)
- [Solver Limitation](#)

Filament Method

This section describes the filament method for the skin-effect resistance and inductance solver. The 2-D filament method uses data about magnetic coupling when it extracts frequency-dependent resistance and inductance. To use this solver, set `COMPUTE_RS=yes` in a `.FSOPTIONS` statement.

The following process explains the filament method:

1. The filament method divides the original conductor system into thin filaments.
2. From the coupling of these filaments, this method then derives the distributed magnetic coupling of the inside and outside of the conductor.
3. After dividing the conductors into thin filaments, this method creates the impedance matrix of the filament system:

$$\mathbf{Z}_f = \mathbf{R}_f + j\omega\mathbf{L}_f$$

4. This method uses the following equation to solve the current matrix ($\mathbf{i}_f$):

$$\mathbf{v}_f = \mathbf{Z}_f \mathbf{i}_f$$

In the preceding equation, the $\mathbf{v}_f$ vector excites the filament system.

5. The filament method uses the result of this equation to calculate the partial current matrix of the conductor system $\mathbf{i}_p$ as a sum of all filament currents:

Equation 75

$$i_{p(j,k)} = \sum_{\text{filaments in conductor } j} i_f (@ \text{ k-th excitation vector})$$

6. The filament method uses the following equation to solve the partial impedance matrix ($\mathbf{Z}_p$):

$$\mathbf{v}_p = \mathbf{Z}_p \mathbf{i}_p$$

7. From the components of the partial impedance matrix, the filament method uses the following relationship to calculate the components of the loop $Z_{p(j,k)}[j, k:0 \sim n]$ impedance matrix:

Equation 76

$$Z_{l(j,k)} = Z_{p(j,k)} - Z_{p(j,0)} - Z_{p(k,0)} + Z_{p(0,0)}$$

In the preceding equation, n is the number of signal (non-reference) conductors in the system.

Note:

W-element analysis uses these loop impedance components.

For full discussion of the `.FSOPTIONS` command, see [Accelerating the W-element Field Solver Using Multicore Technology](#) or `.FSOPTIONS` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Modeling Geometries

In geometry modeling:

- The number of dielectric layers is arbitrary.
- You can arbitrarily shape the conductor cross-section, including an infinitely-thin strip.
- The number of conductors is unlimited.
- The current dielectric region must be planar.
- Conductors must not overlap each other.
- Magnetic materials are not supported.

Geometric modeling outputs the Maxwellian (short-circuit) transmission line matrices: C, L, Ro, Rs, Go, and Gd. (See [Equations and Parameters](#).)

Solver Limitation

When the field solver computes the conductance matrices (Go and Gd), if the media are not homogeneous, then the solver uses the arithmetic average values of conductivities and loss tangents.

Field-Solver-Related Netlist Statements

[Table 13](#) describes the netlist statements that specifically relate to the field solver. For the syntax and examples of these statements, see the links to the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Table 13 *Field-Solver Statement Syntax*

Statement	Usage
.MATERIAL	Use this statement to define the properties of a material.
.LAYERSTACK	Use this statement to define a stack of dielectric or metal layers.
.SHAPE	Use this statement to define a shape. The Field Solver uses these shapes: • Rectangle • Circle • Strip • Polygon • Trapezoid to describe a cross-section of the conductor.

Table 13 *Field-Solver Statement Syntax (Continued)*

Statement	Usage
.FSOPTIONS	Use this statement to set various options for the field solver.
.MODEL W MODELTYPE=FieldSolver	Type of transmission-line model.

The following sections discuss these topics:

- [Field Solver Model Syntax](#)
- [Using the Field Solver to Extract an RLGC Tabular Model](#)
- [Accounting for Surface Roughness Effect in W-element](#)

Field Solver Model Syntax

Use a `.MODEL` statement to define a field solver.

```
.MODEL mname W MODELTYPE=FieldSolver
+ LAYERSTACK=name [FSOPTIONS=name]
+ [RLGCFILE=name] [COORD=0|DESCART|1|POLAR]
+ [OUTPUTFORMAT=RLGC|RLGCFILE]
+ CONDUCTOR=SHAPE=name [MATERIAL=name]
+ [ORIGIN=(val1,val2)] [TYPE=SIGNAL|REFERENCE|FLOATING] ...
```

Parameter	Description
mname	Model name.
LAYERSTACK	Name of the associated layer stack.
FSOPTIONS	Associated option name. If you do not specify this entry, the Field Solver uses the default options.
RLGCFILE	Use the output file for RLGC matrices, instead of the standard error output device. An RLGC file name must start with an alphabetic letter (not a number). If the specified file already exists, then the Field Solver appends the output. To generate output, you must set <code>PRINTDATA</code> in <code>.FSOPTIONS</code> to <code>YES</code> ; setting it to <code>APPEND</code> , appends the extracted RLGC model to the specified file.
COORD	The polar field solver is invoked only when <code>COORD=1</code> or <code>COORD=POLAR</code> .
OUTPUTFORMAT	Model syntax format for RLGC matrices in the W-element. Specified in the RLGC file. Default format is an RLGC model.
SHAPE	Shape name.
MATERIAL	Conductor material name. If you do not specify this entry, the Field Solver defaults to the predefined metal name <code>PEC</code> (perfect electrical conductor).

Parameter	Description
ORIGIN=(val1, val2)	val1, val2 can be (x,y) for regular coordinates and (radius, degree) for polar coordinates.
TYPE	One of the following conductor types: • SIGNAL: a signal node in the W-element (the default). • REFERENCE: the reference node in the W-element. • FLOATING: floating conductor, no reference to W-element.

Using the Field Solver to Extract an RLGC Tabular Model

You can use the Field Solver to extract an RLGC tabular model which allows higher flexibility of dependence on frequency. (When PRINTDATA=APPEND, RLGC model output is appended to the specified output file.)

Note:

The default for ACCURACY is HIGH.

The following sections discuss these topics:

- [Syntax for Extracting RLGC Tabular Model](#)
- [COMPUTE_TABLE/COMPUTETABLE Keyword](#)
- [Default](#)
- [RLGC Tabular Model Sample Output](#)

Syntax for Extracting RLGC Tabular Model

```
.FSOPTIONS name [ACCURACY=HIGH|MEDIUM|LOW] + [GRIDFACTOR=val] +  
[COMPUTE_GO=YES|NO] [COMPUTE_GD=NO|YES] + [COMPUTE_RO=YES|NO]  
[COMPUTE_RS=NO|YES|DIRECT|ITER]  
+ [COMPUTE_TABLE=frequency_sweep]  
+ [PRINTDATA=NO|YES|APPEND]
```

Notes

- The forms of the following arguments are interchangeable:

COMPUTE_GO : COMPUTEGO
COMPUTE_GD : COMPUTEGD
COMPUTE_RO : COMPUTERO
COMPUTE_RS : COMPUTERS
COMPUTE_TABLE : COMPUTETABLE
- If you only set COMPUTE_GD=yes in the .FSOPTIONS statement, you do not generate data in the Gd matrix of the RLGC file. Since Gd is the dielectric loss conductance

matrix, you also need to define the loss tangent values for each dielectric material you have in your layer stack. For example:

```
.MATERIAL die1 DIELECTRIC ER=4.1 LOSSTANGENT=.012
```

The Gd matrix is derived from Co matrix and loss tangent value as $Gd = 2 \cdot \pi \cdot \tan \delta \cdot Co$

when multiple loss tangent values are given to the `.layerstack` structure, the average value (including background material property) is used.

COMPUTE_TABLE/COMPUTETABLE Keyword

Frequency sweep	Definition
COMPUTE_TABLE or COMPUTETABLE	Specifies a type of frequency sweep. You can specify either LIN, DEC, OCT, POI. Specify the nsteps, start, and stop values using the following syntax for each type of sweep: • LIN <i>nsteps start stop</i> • DEC <i>nsteps start stop</i> • OCT <i>nsteps start stop</i> • POI <i>nsteps freq_values</i> Note: To reduce the risk of instability in subsequent simulations, the field solver adds points to ensure that there is at least one frequency point per decade up to 1 THz.

Once a frequency sweep is specified, the W-element computes transmission line parameters at specified frequency points. In addition, since resistance and conductance at DC (zero frequency) and capacitance and inductance at infinite frequency are essential to ensure the accuracy, these four matrices are automatically computed regardless of the sweep configuration.

In the table model extraction, series impedance, $Z(\omega) = R(\omega) + j\omega L(\omega)$ is computed directly from the filament method solver. For shunt admittance, $Y(\omega) = G(\omega) + j\omega C(\omega)$, the static capacitance solver is still used. From the static capacitance, C, and corresponding dielectric loss term, $Gd = 2 \cdot \pi \cdot \tan \delta \cdot C$, a complex dielectric loss model is derived. For further detail about the complex dielectric loss model generation, see [Fitting Procedure Triggered by INCLUDEGDIMAG Keyword on page 233](#).

Default

If the `COMPUTE_TABLE` keyword is not specified, a conventional RLGC model is generated.

Note:

The direct solver is chosen (`COMPUTERS=DIRECT`) by default for table model extraction as well as for the RLGC (non-table) model.

RLGC Tabular Model Sample Output

```
.MODEL rmod1 sp N=3 MATRIX=SYMMETRIC
+ SPACING=POI VALTYPE=REAL INTERPOLATION=LINEAR
+ DC = 3.850667e+04
+ 7.700000e+03 3.850667e+04
+ 7.700000e+03 7.700000e+03 3.850667e+04
+ INFINITY = 1.444000e+05
+ 3.082000e+04 1.402667e+05
+ 2.389333e+04 3.082000e+04 1.444000e+05
+ DATA = ( 20
+ 0.000000e+00
+ 3.850667e+04
+ 7.700000e+03 3.850667e+04
+ 7.700000e+03 7.700000e+03 3.850667e+04
+ .....
+
.MODEL lmod1 sp N=3 MATRIX=SYMMETRIC
+ SPACING=POI VALTYPE=REAL INTERPOLATION=LINEAR
+ INFINITY = 6.624667e-07
+ 2.603333e-07 7.253333e-07
+ 1.406667e-07 2.603333e-07 6.624667e-07
+ DATA = ( 20
+ 0.000000e+00
+ 9.733333e-07
+ 4.876000e-07 9.806667e-07
+ 3.454000e-07 4.876000e-07 9.733333e-07
+ .....
+
.MODEL gmod1 sp N=3 MATRIX=SYMMETRIC
+ SPACING=POI VALTYPE=REAL
+ DC = 0.000000e+00
+ 0.000000e+00 0.000000e+00
+ 0.000000e+00 0.000000e+00 0.000000e+00
+ DATA=( 1
+ 0.000000e+00
+ 0.000000e+00
+ 0.000000e+00 0.000000e+00
+ 0.000000e+00 0.000000e+00 0.000000e+00
+ .....
+
.MODEL cmod1 sp N=3 MATRIX=SYMMETRIC
+ SPACING=POI VALTYPE=REAL+ INFINITY = 3.930057e-17
+ -1.300346e-17 3.969922e-17 + -3.026179e-18 -1.300788e-17
3.929558e-17
+ DATA=( 20+ 0.000000e+00
+ 3.930057e-17 + -1.300346e-17 3.969922e-17
+ -3.026179e-18 -1.300788e-17 3.929558e-17 + .....
```


Accounting for Surface Roughness Effect in W-element

In real devices operating at high frequency range, skin effect causes a secondary effect since the surfaces of conductors are not flat but have some roughness. When a majority of current propagates across the rough surfaces, there is a non-negligible increase in series impedance. Since the influence of this phenomenon depends on the dominance of current around the conductor surface, this effect is also frequency-dependent.

PrimeSim HSPICE provides two ways to take this frequency-dependent increase of series impedance into account:

- Scaling RS matrix
- Calculating root mean square (RMS) surface roughness height

To scale the RS matrix use the `SCALE_RS` keyword with scaling factor and apply it to the skin-effect (R_s) matrix. Thus, the series impedance is expressed as:

Equation 77

$$R(f) = R_0 + (1 + j) \cdot \sqrt{f} \cdot SCALE_RS \cdot R_s$$

To calculate the RMS surface roughness height, the ratio of resistivity increment, SR , may be empirically estimated as:

Equation 78

$$SR = \frac{2}{\pi} \tan^{-1} \left(1.4 \left(\frac{\Delta}{\delta_s(f)} \right)^2 \right)$$

Where Δ and δ_s are the RMS surface roughness height and skin depth of the conductor material.

When `ROUGHNESS_METHOD=1` (default) is chosen, the W-element field solver will re-scale the metal resistivity of each conductor filament at each frequency point using the surface roughness factor as:

Equation 79

$$\rho'(f) = (1 + SR)\rho$$

When `ROUGHNESS_METHOD=2` is chosen, the W-element field solver first solves frequency dependent impedance without surface roughness effect. Then roughness factor (SR in Equation 62) will be applied to scale the impedance as:

Equation 80

$$Z_{rough}(f) = Z_o + (Z(f) - Z_o) \times (1 + SR)$$

Where $Z_{\text{rough}}(f)$, Z_o and $Z(f)$ are impedance with surface roughness effect, DC impedance and impedance without surface roughness effect, respectively.

The following sections discuss these topics:

- [Syntax for Scaling RS Matrix](#)
- [Syntax for Taking RMS Surface Roughness of Conductor Materials](#)
- [Syntax for Choosing Surface Roughness Method](#)
- [Example: Transmission Line Surface Roughness](#)

Syntax for Scaling RS Matrix

```
Wxxx ni1 ni2... ref_in no1 no2... ref_out  
+ [SCALE_RS=value]
```

Keyword

SCALE_RS: Scaling factor to the RS matrix

Syntax for Taking RMS Surface Roughness of Conductor Materials

```
.material copper metal conductivity=value [roughness=value]
```

Keyword

ROUGHNESS: RMS surface roughness height.

Syntax for Choosing Surface Roughness Method

```
.fsoptions [ROUGHNESS_METHOD=1|2]
```

Keyword

ROUGHNESS_METHOD: Surface roughness computation method.

Example: Transmission Line Surface Roughness

```
*** no surface roughness ***  
P1 in1 0 port=1 ac=1  
P2 out1 0 port=2  
W1 in1 gnd out1 gnd FSmodel=line1 N=1 l=0.1  
  
*** use copper_roughs w/ roughness=2um ***  
P3 in2 0 port=3 ac=1  
P4 out2 0 port=4  
W2 in2 gnd out2 gnd FSmodel=line1_rough N=1 l=0.1
```

Chapter 7: Using the Field Solver to Extract Tx Line Parameters Accelerating the W-element Field Solver Using Multicore Technology

```

*** use SCALE_RS=1.1 ***
P5 in3 0 port=5 ac=1
P6 out3 0 port=6
W3 in3 gnd out3 gnd FSmodel=line1 N=1 l=0.1 SCALE_RS=1.1

.material diel dielectric er=4.3
.material copper metal conductivity=57.6meg
.material copper_rough metal conductivity=57.6meg
+ROUGHNESS=2e-6

.shape rect rectangle width=400e-6 height=40e-6
.layerstack stack1 background=air
+layer=(copper,10e-6)
+layer=(diel,200e-6)
.fsoptions opt1 printdata=yes computegd=no computers=yes
.model line1 W Modeltype=fieldsolver,
+layerstack=stack1,
+fsoptions=opt1,
+Rlgcfile=line1.rlgc,
+conductor=(shape=rect,origin=(0,110e-6),material=copper)
.model line1_rough W Modeltype=fieldsolver,
+layerstack=stack1,
+fsoptions=opt1,
+Rlgcfile=line1_rough.rlgc,
+conductor=(shape=rect,origin=(0,110e-6),material=copper_rough)
.opt post
.ac dec 100 1e6 1e10
.end

```

Accelerating the W-element Field Solver Using Multicore Technology

You can increase the speed of the W-element magnetic coupling field solver with the multithread option when a multicore system is available. Since the skin effect solver has to solve multiple frequency points to capture the frequency-dependent effect, it consumes a majority of field solver run-time. When the multicore option (`-mt #`) is used in the command-line invocation, PrimeSim HSPICE parallelizes frequency sampling in the tabular RLGC model extraction.

Syntax

```

FSOPTIONS name [ACCURACY=HIGH|MEDIUM|LOW+ [GRIDFACTOR=val]
[PRINTDATA=NO|YES|APPEND]+ [COMPUTE_GO=NO|YES] [COMPUTE_GD=YES|NO]+
[COMPUTE_RO=YES|NO] [COMPUTE_RS=NO|YES|DIRECT|ITER]
+ [COMPUTE_TABLE=frequency_sweep]

```

Keyword

```

COMPUTE_RS | COMPUTERS

```

COMPUTE_RS Options	Definition
YES	Activate filament solver with direct matrix solver
NO	Do not to perform filament solver
DIRECT	Activate filament solver with direct matrix solver (same as YES)
ITER	Activate filament solver with iterative matrix solver

For full discussion of the `.FSOPTIONS` command, see `.FSOPTIONS` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Visualizing Cross-Sectional Geometric Information

When PrimeSim HSPICE runs with the W-element field solver model, it generates the `model_name.str` file, which contains Tcl/Tk scripts to display cross-sectional geometrical information of the target structure. Once PrimeSim HSPICE is executed, locate the `model_name.str` file. To invoke Tcl/Tk graphics, at the command prompt, enter either:

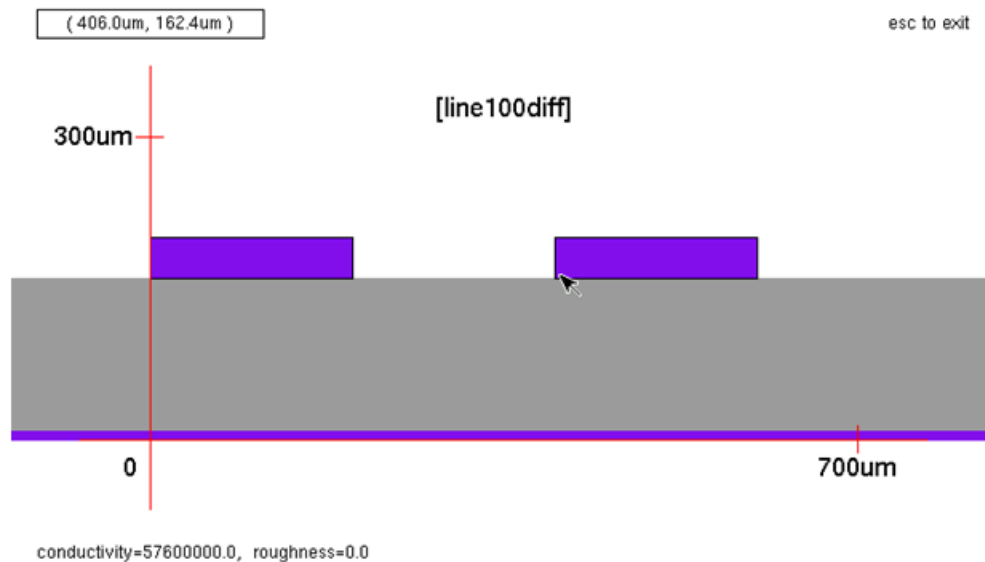
```
% wish model_name.str

or

% chmod +x model_name.str
% model_name.str
```

Example output is seen in [Figure 79](#).

Figure 79 Structural output example



The following dynamic functions are available on the structural display window:

- Move the mouse cursor over an object to display corresponding material properties.
- Drag the mouse cursor to display geometrical locations.

Note:

To use this function, the Tcl/Tk environment must be installed on users' systems. On a Linux system, Tcl/Tk is included in the default installation package. Manual installations are needed for the other platforms. The Tcl/Tk environment is available for all the platforms supported by PrimeSim HSPICE.

Limitation: the `POLAR` coordinate is not currently supported.

For further information go to: <http://www.tcl.tk/software/tcltk/8.0.html>

Field Solver Examples

The following examples show you how to use the Field Solver. All of the examples shown in this section run with the `HIGH` accuracy mode and with `GRIDFACTOR = 1`.

- [Example 1: Cylindrical Conductor Above a Ground Plane](#)
- [Example 2: Stratified Dielectric Media](#)
- [Example 3: Two Traces Between Two Ground Planes](#)

- [Example 4: Using Field Solver with Monte Carlo Analysis](#)
- [Example 5: Modeling Coaxial and Shielded Twin-Lead Lines Using the Polar Field Solver](#)

Example 1: Cylindrical Conductor Above a Ground Plane

This is an example of a copper cylindrical conductor above an ideal (lossless) ground plane.

With these formulas, you can derive the exact analytical formulas for all transmission line parameters:

Equation 81

$$L = \frac{1}{\mu\epsilon} C^{-1}$$

Equation 82

$$G = \frac{\sigma_d}{\epsilon} C = \omega \cdot \tan(\delta) \cdot C$$

Equation 83

$$R = \frac{1}{\sigma_c \delta \pi d} \left[\frac{2H/d}{\sqrt{(2H/d)^2 - 1}} \right] = \sqrt{f} \sqrt{\frac{\pi \mu}{\sigma_c}} \frac{1}{\pi d} \left[\frac{2H/d}{\sqrt{(2H/d)^2 - 1}} \right]$$

[Figure 80](#) shows the geometry of a copper cylindrical conductor above an ideal ground plane.

Equation 84

$$C = \frac{2\pi\epsilon}{\operatorname{acosh}\left(\frac{2H}{d}\right)}$$

Figure 80 Cylindrical conductor above a perfect electrical conductor ground plane

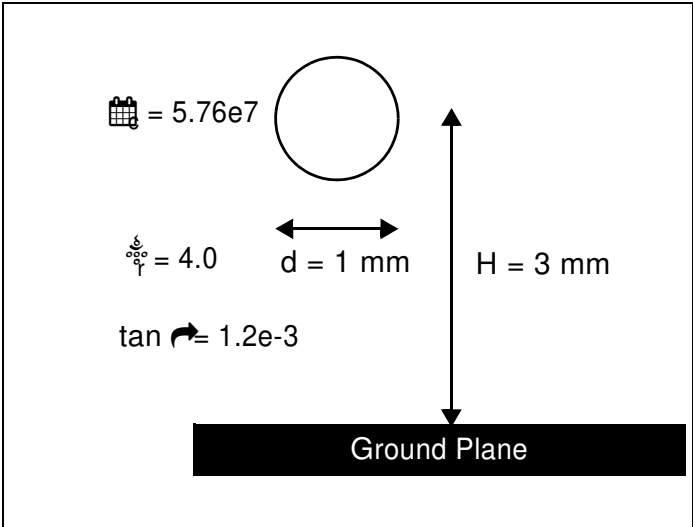


Table 14 lists the corresponding netlist.

Table 14 Input File Listing

Listing Type	Field Solver Cylindrical Example
Header, options and sources	* Example: cylindrical conductor .OPTION PROBE POST VIMPULSE in1 gnd PULSE 4.82v 0v 5n 0.5n 0.5n 25n
W-element	W1 in1 gnd out1 gnd FSmodel=cir_trans N=1 l=0.5
Materials	.MATERIAL diel_1 DIELECTRIC ER=4, LOSSTANGENT=1.2e-3 .MATERIAL copper METAL CONDUCTIVITY=57.6meg
Shapes	.SHAPE circle_1 CIRCLE RADIUS=0.5mm
Defines a half-space	.LAYERSTACK halfSpace + BACKGROUND=diel_1,LAYER=(copper,1mm)
Option settings	.FSOPTIONS opt1 PRINTDATA=YES, + COMPUTE_RS=yes, COMPUTE_GD=yes
Model definition	.MODEL cir_trans W MODELTYPE=FieldSolver + LAYERSTACK=halfSpace, FSOPTIONS=opt1, RLGCFILE=ex1.rlgc + CONDUCTOR=(SHAPE=circle_1, ORIGIN=(0,4mm), + MATERIAL=copper)
Analysis, outputs and end	.TRAN 0.5n 100n .PROBE v(out1) .END

Compare the computed results with the analytical solutions in Table 15. The Field Solver computes the resistance and conductance at the frequency of 200 MHz, but does not include the DC resistance (Ro) and conductance (Go) in the computed values.

Table 15 Comparison Result

Value	Analytical	Computed
C (pF/m)	89.81	89.66
L (nH/m)	494.9	496.4
G (mS/m)	0.1354	0.1352
R (Ω /m)	1.194	1.086

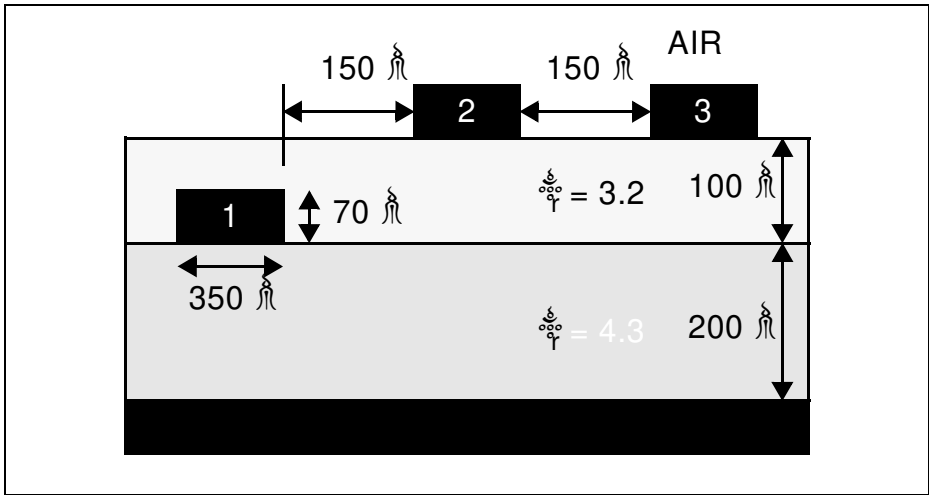
Note:

The analytical resistance value does not consider ground plane resistivity.

Example 2: Stratified Dielectric Media

This is an example of three traces immersed in a stratified dielectric media (see [Figure 81](#)).

Figure 81 Three traces immersed in stratified dielectric media



[Table 16](#) shows the input file.

Table 16 Input File for Three Traces Immersed in Stratified Dielectric Media

Listing Type	Field Solver Stratified Dielectric Example
Header, options and sources	* Example: three traces in dielectric .OPTION PROBE POST VIMPULSE in1 gnd PULSE 4.82v 0v 5n 0.5n 0.5n 25n

Table 16 *Input File for Three Traces Immersed in Stratified Dielectric Media (Continued)*

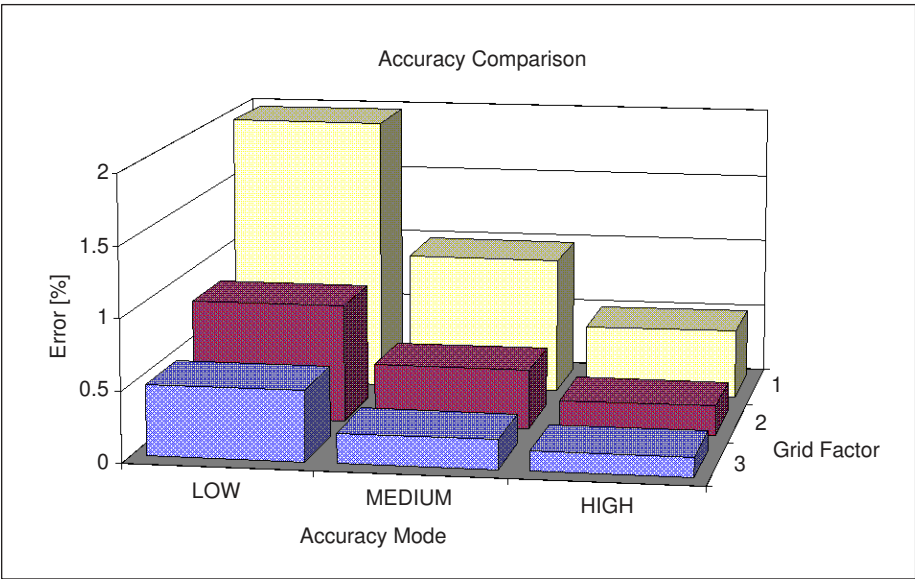
Listing Type	Field Solver Stratified Dielectric Example
W-element	W1 in1 in2 in3 gnd out1 out2 out3 gnd + FSmodel=cond3_sys N=3 l=0.5
Materials	.MATERIAL diel_1 DIELECTRIC ER=4.3 .MATERIAL diel_2 DIELECTRIC ER=3.2 .MATERIAL copper METAL CONDUCTIVITY=57.6meg
Shapes	.SHAPE rect_1 RECTANGLE WIDTH=0.35mm, HEIGHT=0.07mm
Uses the default AIR background	.LAYERSTACK stack_1 + LAYER=(copper,1um),LAYER=(diel_1,0.2mm), + LAYER=(diel_2,0.1mm)
Option settings	.FSOPTIONS opt1 PRINTDATA=YES
Three conductors share the same shape	.MODEL cond3_sys W MODELTYPE=FieldSolver, + LAYERSTACK=stack_1, FSOPTIONS=opt1, RLGCFILE=ex2.rlgc + CONDUCTOR=(SHAPE=rect_1,MATERIAL=copper, ORIGIN=(0,0.201mm)), + CONDUCTOR=(SHAPE=rect_1,MATERIAL=copper,ORIGIN=(0.5mm,0.301mm)), + CONDUCTOR=(SHAPE=rect_1,MATERIAL=copper, ORIGIN=(1mm,0.301mm))
Analysis, outputs and end	.TRAN 0.5n 100n .PROBE v(out1) .END

Note:

W. Delbare and D. D. Zutter, “Space-domain Green’s function approach to the capacitance calculation of multi-conductor lines in multi-layered dielectrics with improved surface charge modeling,” IEEE Trans. Microwave Theory and Tech., vol. 37, pp. 1562-1568, October 1989.

Figure 82 shows the results of convergence analysis, based on the total capacitance of the first conductor with respect to the GRIDFACTOR parameter.

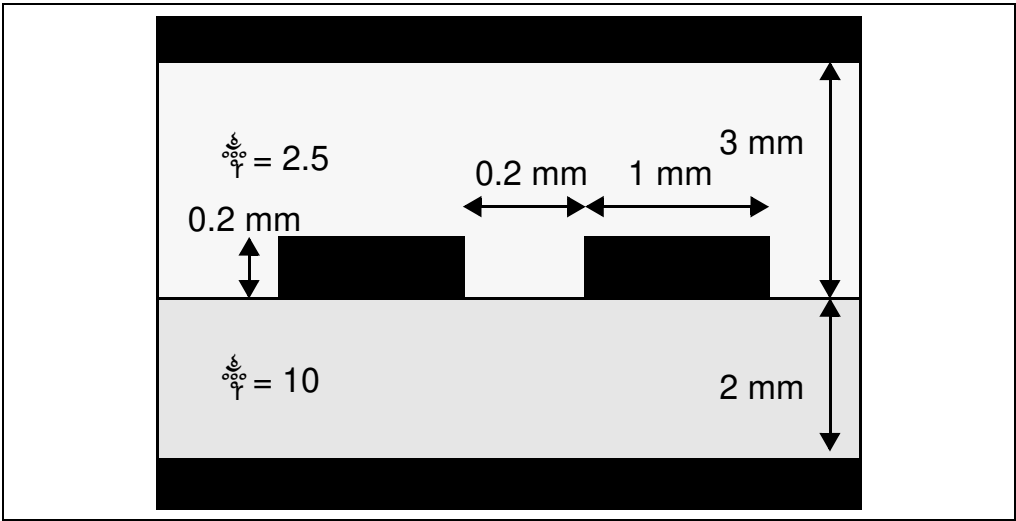
Figure 82 Convergence of accuracy modes



Example 3: Two Traces Between Two Ground Planes

This is an example of two traces between two ground planes (in other words, a coupled strip line) (see [Figure 83](#)).

Figure 83 Example of a coupled strip line



[Table 17](#) lists the complete input netlist.

Table 17 *Input Netlist for Two Traces Between Two Ground Planes*

Listing Type	Field Solver Ground Planes Example
Header, options and sources	* Example: two traces between gnd planes .OPTION PROBE POST VIMPULSE in1 gnd PULSE 4.82v 0v 5n 0.5n 0.5n 25n
W-element	W1 in1 in2 gnd out1 out2 gnd FSmodel=cond2_sys +N=2 l=0.5
Materials	.MATERIAL diel_1 DIELECTRIC ER=10.0 .MATERIAL diel_2 DIELECTRIC ER=2.5 .MATERIAL copper METAL CONDUCTIVITY=57.6meg
Shapes	.SHAPE rect RECTANGLE WIDTH=1mm, + HEIGHT=0.2mm,
Top and bottom ground planes	.LAYERSTACK stack_1, + LAYER=(copper,1mm), LAYER=(diel_1,2mm), + LAYER=(diel_2,3mm), LAYER=(copper,1mm)
Option settings	.FSOPTIONS opt1 PRINTDATA=YES
Two conductors share the same shape	.MODEL cond2_sys W MODELTYPE=FieldSolver, + LAYERSTACK=stack_1, FSOPTIONS=opt1 RLGCFILE=ex3.rlgc + CONDUCTOR=(SHAPE=rect,MATERIAL=copper, ORIGIN=(0,3mm)), + CONDUCTOR=(SHAPE=rect,MATERIAL=copper, ORIGIN=(1.2mm,3mm))
Analysis, outputs and end	.TRAN 0.5n 100n .PROBE v(out1) .END

Table 18 compares the computed result with the Finite Element (FEM) solver result.

Table 18 *Comparison Between Computed and FEM Solver Results*

Computed	$\begin{bmatrix} 214.1 & -105.2 \\ -105.2 & 214.1 \end{bmatrix}$ (pF/m)
FEM Solver	$\begin{bmatrix} 217.7 & -108.2 \\ -108.2 & 217.7 \end{bmatrix}$ (pF/m)

Example 4: Using Field Solver with Monte Carlo Analysis

The following example shows how to use Monte Carlo transient analysis to model variations in the manufacturing of a microstrip.

The transient output waveforms are shown in [Figure 84](#).

Figure 84 Monte carlo analysis with field solver and W-element

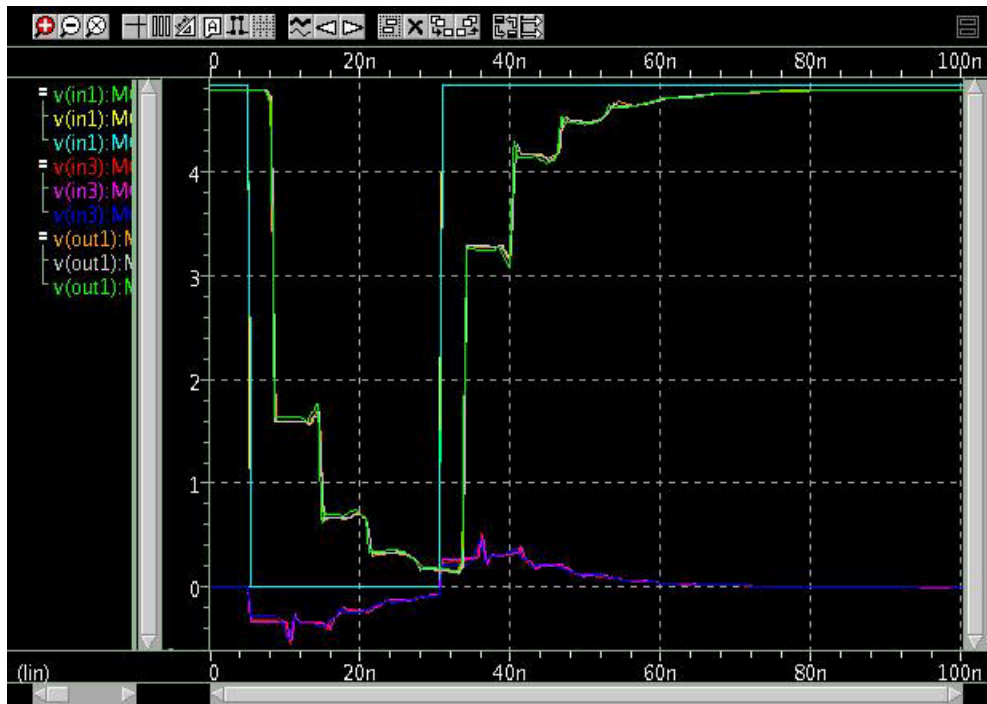


Table 18 shows the input listing with the W-element.

Table 19 Input File Listing with the W-element

Listing Type	Field Solver Monte Carlo Example
Header, options and sources	PETL *Example: with Monte Carlo .OPTION PROBE POST + VIMPULSE in1 gnd AC=1v PULSE 4.82v 0v 5ns + 0.5ns 0.5ns 25ns
Parameter definitions	.PARAM x1=Gauss(0,0.02,1) x2=Gauss(0.5mm,0.02,1) + x3=Gauss(1mm,0.02,1) .PARAM dRef=1u dY1=Gauss(2mm,0.02,1) + dY2=Gauss(1mm,0.02,1)
W-element	W1 in1 in2 in3 0 out1 out2 out3 0 + FSMODEL=cond3_sys N=3 l=0.5
Materials	.MATERIAL diel_1 DIELECTRIC ER=4.3 .MATERIAL diel_2 DIELECTRIC ER=3.2 .MATERIAL copper METAL CONDUCTIVITY=57.6meg
Shapes	.SHAPE r1 RECTANGLE WIDTH=0.35mm, HEIGHT=0.070mm
Uses the default AIR background	.LAYERSTACK stack_1 LAYER= (copper,dRef), + LAYER=(diel_1,dY1), LAYER= (diel_2,dY2)

Table 19 Input File Listing with the W-element (Continued)

Listing Type	Field Solver Monte Carlo Example
Three conductors share the same shape	<pre>.MODEL cond3_sys W MODELTYPE=FieldSolver, + LAYERSTACK=stack1, + CONDUCTOR=(SHAPE=r1,MATERIAL=copper,ORIGIN=(x1,'dRef+dY1')), + CONDUCTOR=(SHAPE=r1,MATERIAL=copper,ORIGIN=(x2,'dRef+dY1+dY2')), + CONDUCTOR=(SHAPE=r1,MATERIAL=copper,ORIGIN=(x3,'dRef+dY1+dY2'))</pre>
Analysis, outputs and end	<pre>.PROBE TRAN v(in1) v(out1) v(in3) .PROBE AC v(out1) v(out3) .PROBE DC v(in1) v(out1) v(out3) .AC LIN 200 0Hz 0.3GHz .DC VIMPULSE 0v 5v 0.01v .TRAN 0.5ns 100ns SWEEP MONTE=3 .END</pre>

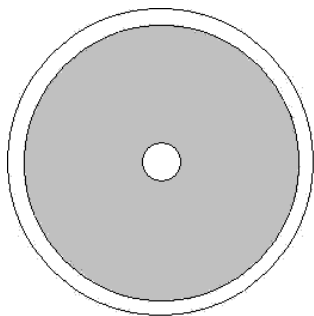
Example 5: Modeling Coaxial and Shielded Twin-Lead Lines Using the Polar Field Solver

The following examples show how to model a coaxial line and a twin-lead line. The keyword `COORD=polar` (or `COORD=1`) invokes the polar field solver. When the polar field solver is used, the conductor position is defined in polar coordinates (radius, angle in degrees). Only one dielectric is permitted and the dielectric layer is surrounded by ground.

- [Coax Line](#)
- [Shielded Twin-Lead Line](#)

Coax Line

Figure 85 Polar field solver for modeling coaxial lines



```
*PETL Example: Coaxial Line  
.OPTION PROBE POST  
VIMPULSE in1 gnd AC=1v PULSE 4.82v 0v 5ns  
+0.5ns 0.5ns 25ns  
  
*W element
```

Chapter 7: Using the Field Solver to Extract Tx Line Parameters

Field Solver Examples

```

W1 in1 gnd out1 gnd FSMODEL=coax N=1, L=1
R1 out1 gnd 50

* [[ Material List ]]
.MATERIAL diel_1 DIELECTRIC ER=4
.MATERIAL copper METAL CONDUCTIVITY=57.6meg

* [[ Shape List ]]
.SHAPE circle_1 CIRCLE RADIUS=0.5m

* [[ Layer Stack ]]
.LAYERSTACK coaxial LAYER=(diel_1 11m) $ only one

* [[ Field solver option ]]
.FSOPTIONS myOpt printdata=yes compute_rs=yes compute_gd=yes
+ compute_go=yes

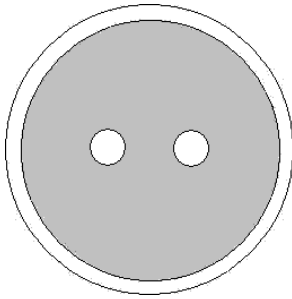
* [[ Field solver model ]]
.MODEL coax W MODELTYPE=FIELDSOLVER FSOPTIONS=myOpt COORD=polar
+ LAYERSTACK=coaxial, RLGCFILE=coax.rlgc
+ CONDUCTOR = ( SHAPE=circle_1, MATERIAL=copper, ORIGIN=(0, 0) )

.TRAN 0.5n 100n
.PROBE v(in1) v(out1)
.END

```

Shielded Twin-Lead Line

Figure 86 Polar field solver for modeling twin-lead line



```

*PETL Example: Shield twin-lead lines
.OPTION PROBE POST
VIMPULSE in1 gnd AC=1v PULSE 4.82V 0v 5ns
+0.5ns 0.5ns 25ns
*W element
W1 in1 in2 0 out1 out2 0 FSMODEL=twin, N=2, L=1
R1 out1 gnd 50
R2 out2 gnd 50
R3 in2 gnd 50

* [[ Material List ]]
.MATERIAL diel_1 DIELECTRIC ER=4

```

```
.MATERIAL copper METAL CONDUCTIVITY=57.6meg

* [[ Shape List ]]
.SHAPE circle_1 CIRCLE RADIUS=0.5m

* [[ Layer Stack ]]
.LAYERSTACK coaxial LAYER=(diel_1 11m) $ only one

* [[ Field solver option ]]
.FSOPTIONS myOpt printdata=yes compute_rs=yes compute_gd=yes
compute_go=yes
* [[ Field solver model ]]
.MODEL twin W MODELTYPE=FIELDSOLVER FSOPTIONS=myOpt COORD=polar
+ LAYERSTACK=coaxial, RLGCFILE=twin.rlgc
+ CONDUCTOR = ( SHAPE=circle_1, MATERIAL=copper,
ORIGIN=(4.5m, 0) )
+ CONDUCTOR = ( SHAPE=circle_1, MATERIAL=copper,
ORIGIN=(4.5m, 180) )

.TRAN 0.5n 100n
.PROBE v(in1) v(out1) v(out2)
.END
```

8

Using IBIS Files

Describes how IBIS model components are included in a PrimeSim HSPICE simulation, with usage models for IBIS buffer, component package, board-level components (EBD), and Interconnect Specification (ICM).

PrimeSim HSPICE ships numerous of examples for your use. Find paths to IBIS-related demo files under IBIS Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

For information about IBIS-Algorithmic Modeling Interface (AMI) usage with PrimeSim HSPICE, see [Chapter 9, Using IBIS-AMI Equalizer Models with StatEye](#).

The Input/Output Buffer Information Specification (IBIS) specifies a standard ASCII format for presenting information describing the behavior of various I/O buffers that send electrical signals outside the silicon chip or receive such signals.

IBIS promotes tool independent I/O models for system level Signal Integrity work and has become:

- The ANSI/SAE-ITC and IEC 62014-1 standard for describing the analog behavior of the buffers of digital devices using plain ASCII text-formatted data.
- An alternative to models - IBIS files are not models, they contain the data to be used by the behavioral models and algorithms inside the simulator.

Except for SPICE or Verilog-A formatted buffers, IBIS specifies formats for the following types of information:

- Output I-V curves for output buffers in LOW and HIGH states.
- V(t) curves describing the exact form of transitions from LOW to HIGH states and from HIGH to LOW states for a given load.
- Values for die capacitance.

The IBIS standard was developed by the IBIS Open Forum, affiliated with the Society of Automotive Engineers - Industry Technologies Consortia (SAE-ITC). IBIS specifies only the “form” for the information; it does not specify how the information is processed or used by the simulator.

For introductory information on using IBIS with PrimeSim HSPICE, see [Simulating Circuits with IBIS Models in PrimeSim HSPICE](#) in this user guide.

The following sections discuss these topics:

- [Terminology](#)
- [Verifying IBIS Files with the Golden Parser](#)
- [Specifying Models for IBIS Buffers](#)
- [Troubleshooting Signal Propagation Issues/Overclocking](#)
- [Multiple Rising and Falling Waveforms Support](#)
- [Differential Pins](#)
- [Buffers in Subcircuits](#)
- [Netlist Example with Output Buffer, Transmission Line, and Input Buffer](#)
- [Using the IBIS Component Command](#)
- [Using IBIS Package Modeling](#)
- [Using IBIS Board-Level Components](#)
- [Using IBIS Interconnect Modeling \(ICM\)](#)
-

Terminology

The following terms are frequently used in this chapter:

Table 20 *Terminologies Used in This Chapter*

Term	Definition
B-element	The B-element is a single instance of an IBIS buffer, whereas the .IBIS component defines a component that has multiple buffers and can also include package and pin information. Model selection is not available when using the B-element.
.IBIS component	The .IBIS component enables model selection using <code>mod_sel</code> . The <code>mod_sel</code> keyword in the <code>.IBIS</code> statement supports multiple model assignments. If you want to model package effects, the .IBIS component allows the use of package models that are defined in the IBIS file or in a separate package file.
card, buffer card	Denote lines from the netlist that specifies the buffer name (should begin with the letter b), a list of external nodes, required keyword, and optional keywords.

Table 20 Terminologies Used in This Chapter (Continued)

Term	Definition
buffer, I/O buffer, input/output buffer	One of 14 IBIS models as specified in the standard, version 5.0, and implemented in the Synopsys IBIS device models.
RWF, FWF	Rising waveform, falling waveform
I/O	Input/Output
I/V curve	Current-voltage curve
PU, PD	Pullup, pulldown
PC, GC	Power clamp, ground clamp

Verifying IBIS Files with the Golden Parser

The IBIS standard contains a section devoted to recommendations on how to derive information from either the simulation or silicon measurement. The IBIS Open Forum has sponsored development of a parser for IBIS files—called the golden parser. The golden parser is freely available as an executable and should be used for verification of IBIS files. The golden parser is incorporated into Synopsys circuit simulators. When processing an IBIS file, the golden parser produces warnings or error messages that appear in the PrimeSim HSPICE output by default.

Specifying Models for IBIS Buffers

This section describes IBIS buffer usage in PrimeSim HSPICE. The I/O buffer element type is called *buffer* and in PrimeSim HSPICE is commonly referred to as the B-element or b-element. For information on IBIS syntax conventions for I/O buffers and buffer types, see Modeling Input/Output Buffers Using IBIS Files in *PrimeSim Continuum User Guide: Elements*.

Using buffers is similar to using other simulation elements, such as transistors: specify a name for the buffer, specify a list of nodes that connect the buffer to the rest of the circuit, and specify parameters. Only parameters that specify a model for the buffer (file name and model name) are required. The IBIS model name is limited to 60 characters. Messages similar to the following are issued if the name exceeds 60 characters.

```
**error** Model
'EDD2516AETA_NORMAL_iobuf_XXXXXX_XXXXX_XXXXX_XXXXX_XXXXX_XXXXX_XXXXX' is
not found
**error** .\md8.sp:174) difficulty in reading input
```

Except for SPICE or Verilog-A formatted buffers, there are two significant differences from the use of other elements:

- You can specify a total of 3 to 8 external nodes depending on the buffer type.
- If nodes are supposed to connect to power or ground rails, do not connect them in the netlist because the simulation connects the nodes by default.

This chapter is not intended to introduce the IBIS standard, because it is a large document; familiarity with the standard is assumed. A significant amount of information is available on the Internet.

The official IBIS Open Forum web site is located at:

<http://www.ibis.org>

This site contains articles introducing IBIS, text of the IBIS standard, examples of IBIS files, and tools such as the golden parser. The site also links to other web sites devoted to IBIS.

Three types of analysis are supported for input/output buffers:

- DC analysis
- Transient analysis
- AC analysis

Troubleshooting Signal Propagation Issues/Overclocking

If you connect a source to your buffer using the correct nodes and do not see any output or have a distorted waveform, your input voltage swings may be out of range, or your input stimulus (digital signal) is switching too fast, such that *overclocking* occurs.

In the case of overclocking, you may see a “jump” when a transition is triggered before the previous one is completed. This shows up as an artificial skew in simulation.

PrimeSim HSPICE can handle overclocking issues without making modifications to an IBIS model by using IBIS keywords that remove initial delays and flat tails from the rising and falling waveforms.

The keywords are as follows:

- `rm_dly_rwf= default | rdly_time_value`
- `rm_dly_fwf= default | fdly_time_value`
- `rm_tail_rwf= default | rtail_time_value`
- `rm_tail_fwf= default | ftail_time_value`

`rm_dly_rwf` and `rm_dly_fwf` remove initial delays from [Rising Waveform] and [Falling Waveform], respectively.

`rm_tail_rwf` and `rm_tail_fwf` remove flat tails from [Rising Waveform] and [Falling Waveform], respectively.

For each keyword, if the value is set as a positive number, then that amount of time is removed from the corresponding V-T waveforms.

For each keyword, if the value is set as `default`, then for each V-T waveform, PrimeSim HSPICE computes the amount of no-switching time based on the criteria that the voltage change should be less than 0.2% of full voltage swing, then selects the minimum one as the final value used.

The removal of initial no-switching time by 0.2% criteria is the default for both B-element and `.IBIS` cards. So there is no need to set `rm_dly_rwf=default` or `rm_dly_fwf=default`. These two keywords are just for backward compatibility.

For detailed information on the keywords, see Specifying Required and Optional Common Keywords in *PrimeSim Continuum User Guide: Elements*.

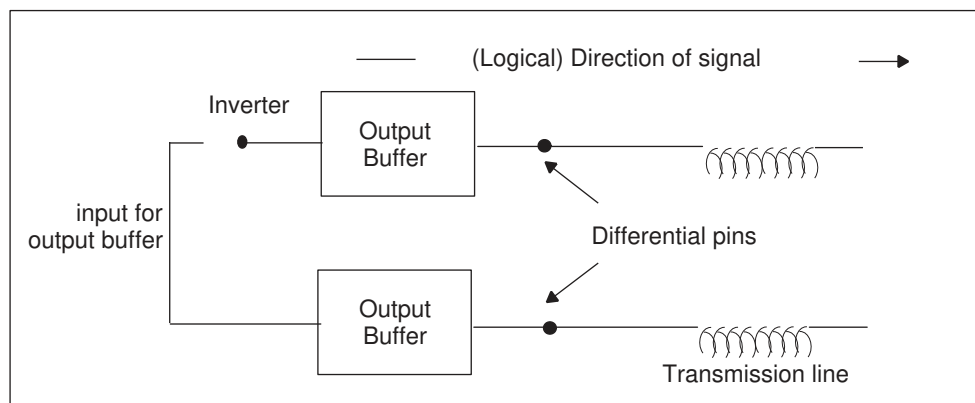
Multiple Rising and Falling Waveforms Support

An IBIS model specification can contain more than one rising or falling VT waveform table. If more than two rising or falling VT waveforms exist in the model, the PrimeSim HSPICE simulator selects two suitable waveforms for accurate modeling. The selection is based on the initial voltage of the VT waveforms at time zero. Before the buffer transition, PrimeSim HSPICE sorts all rising or falling waveforms by initial voltage and then selects two waveforms so that the current die pad voltage lies between the initial voltages of these two waveforms. PrimeSim HSPICE uses the PU and PD scaling coefficients derived from the two VT waveforms for the transient simulation.

Differential Pins

Differential pins refer to the relationship between buffers. This facilitates creating pseudo-differential buffer by two B-elements. Specify these pins in the “Component Description” section of the IBIS standard. [Figure 87](#) and [Figure 88](#), and the examples that follow these figures, show how you can simulate differential pins using the Synopsys implementation of IBIS.

Figure 87 Output buffers

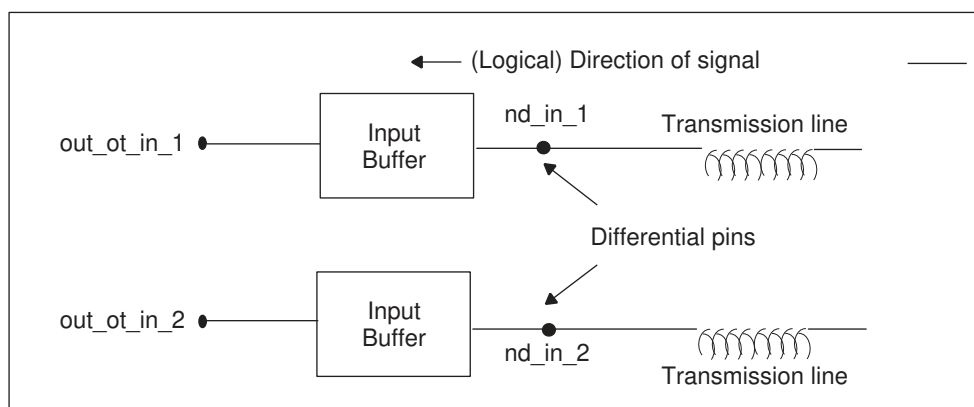


You must use two separate buffers, each specified in a separate card in the netlist. They are related only through their input, which is differential. To implement the inverter in this situation, you must specify two independent voltage sources that have opposite polarity.

You must specify the `out_of_in_1`, and `out_of_in_2` nodes, even if they are not used. Differential input buffers probe the voltage between `nd_in_1` and `nd_in_2`. A voltage-dependent voltage source processes the voltage.

`V_diff` is a differential voltage parameter from the IBIS file (default is 200 mV). Add a definition of the `V_diff` parameter, the `E_diff_out_of_in` voltage controlled voltage source, and a `R_diff_out_of_in` resistor.

Figure 88 Input buffers



For example,

```
.PARAM V_diff = 0.2
E_diff_out_of_in diff_out_of_in 0 PWL(1) nd_in_1 nd_in_2
```

```
+ '- V_diff' 0 '+' V_diff' 1
R_diff_out_of_in diff_out_of_in 0 1x
```

Use the voltage across `R_diff_out_of_in` as the output of the differential input buffer as shown below:

```
If V(nd_in_1) - V(nd_in_2) < V_diff, V(diff_out_of_in) = 0
if V(nd_in_1) - V(nd_in_2) > V_diff, V(diff_out_of_in) = 1
```

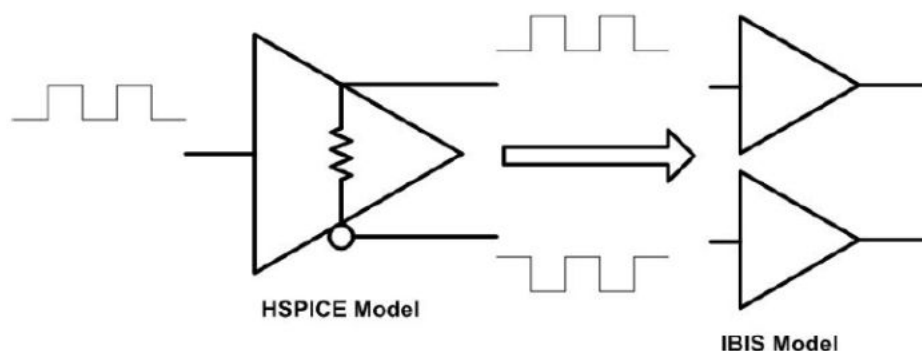
In IBIS 4.1, a true differential buffer with SPICE-formatted external model is introduced. You can use it with the rules in Multilingual Model Support and [SPICE or Verilog-A Formatted B-element Naming Rules](#) sections.

Differential and Pseudo-Differential Buffers

While you use the traditional IBIS model for a single-ended IO buffer, you can use two B-elements for low voltage differential signal (LVDS). For this case:

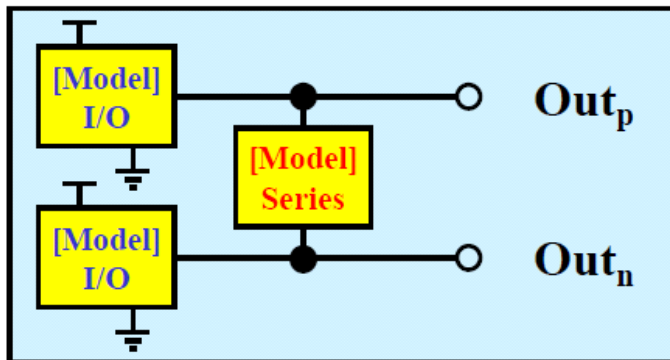
- Use inverse stimulus, if the same IBIS model is used.
- Use the same stimulus, if two IBIS models are used (for the same model data except for a polarity flag)
- For the receiver, add an E-element for A2D translation (from diffV to 0/1 signal).

Figure 89 Pseudo-differential buffer



- For greater accuracy, use a series buffer for differential mode characteristics.

Figure 90 Series buffer for best accuracy



- [External Model] is a good choice for differential buffer modeling.

Buffers in Subcircuits

The following examples demonstrate usage of buffers in subcircuit definition. Also, modification in the circuit description, by changing the power keyword value, is explained.

```
*****
* example 1 * buffers in subcircuit, power=on
*****
v_in1 nd_in1 0 pulse
+ ( 0V 1.0V CLK_Q_PRD DLT_TIME DLT_TIME CLK_H_PRD CLK_PRD )
v_en1 nd_en1 0 1V
v_in2 nd_in2 0 pulse
+ ( 1.1V 0V CLK_Q_PRD DLT_TIME DLT_TIME CLK_H_PRD CLK_PRD )
v_en2 nd_en2 0 1V
x1 nd_out1 nd_in1 nd_en1 nd_outofin1 buffer11
x2 nd_out2 nd_in2 nd_en2 nd_outofin2 buffer11

R_load nd_out1 nd_out2 50

.subckt buffer11 nd_out0 nd_in0 nd_en0 nd_outofin0
b_io_0 nd_pu0 nd_pd0 nd_out nd_in0 nd_en0 nd_outofin0 nd_pc0
+ nd_gc0
+ file = '921v090b.ibs'
+ model = 'DS92LV090A_DOUT'
+ typ=typ power=on
+ buffer=3
+ interpol=1
xpin nd_out nd_out0 pin22
.ends

.subckt pin22 nd_out nd_out0
R_pin nd_out_c nd_out0 50m
C_pin nd_out_c 0 0.3p
```

```
L_pin nd_out nd_out_c 2n
.ends
```

In this example, buffers are connected to power sources implicitly, inside the subcircuit. Subcircuit external terminals does not need to include *nd_pu*, *nd_pd*, *nd_pc*, and *nd_gc*.

```
*****
* example 2* buffers in subcircuit, power=off
*****
v_in1 nd_in1 0 pulse
+ ( 0V 1.0V CLK_Q_PRD DLT_TIME DLT_TIME CLK_H_PRD CLK_PRD )
v_en1 nd_en1 0 1V
v_in2 nd_in2 0 pulse
+ ( 1.1V 0V CLK_Q_PRD DLT_TIME DLT_TIME CLK_H_PRD CLK_PRD )
v_en2 nd_en2 0 1V

x1 nd_power 0 nd_out1 nd_in1 nd_en1 nd_outofin1 nd_power 0
+ buffer11
x2 nd_power 0 nd_out2 nd_in2 nd_en2 nd_outofin2 nd_power 0
+ buffer11

R_load nd_out1 nd_out2 50

.subckt buffer11 nd_pu0 nd_pd0 nd_out0 nd_in0 nd_en0
+ nd_outofin0 nd_pc0 nd_gc0
r_0 nd_pu0 nd_pd0 1.23456789x
b_io_0 nd_pu0 nd_pd0 nd_out nd_in0 nd_en0 nd_outofin0 nd_pc0
+ nd_gc0
+ file = '92lv090b.ibs'
+ model = 'DS92LV090A_DOUT'
+ typ=typ power=off
+ buffer=3
+ interpol=1
xpin nd_out nd_out0 pin22
.ends

.subckt pin22 nd_out nd_out0
R_pin nd_out_c nd_out0 50m
C_pin nd_out_c 0 0.3p
L_pin nd_out nd_out_c 2n
.ends

V_power nd_power 0 3.3V
```

In the example above, only one voltage source (*V_power*) is used to power all buffers. Specify the power nodes, *nd_pu*, *nd_pd*, *nd_pc*, and *nd_gc*. In other words, if subcircuit external terminals include *nd_pu*, *nd_pd*, *nd_pc*, or *nd_gc*, take care that the two X elements calling the same subcircuit are not be connected to the same node of *pu*, *pd*, *pc*, or *gc*. Otherwise, this results in a voltage loop error.

Netlist Example with Output Buffer, Transmission Line, and Input Buffer

An example of a netlist that contains an output buffer, transmission line, and input buffer is shown below. A digital signal is supplied to the `nd_in` node. The output buffer transmits to a network, goes through a transmission line, is received by an input buffer. It is transformed into digital form and available on the `out_of_in` node.

```
* IBIS Buffer Test
.option post
.tran 0.05n 70n
*
* input source
v1 in 0 pulse ( 0V 1V 1n 1n 1n 9n 20n )
r1 in in1 50
* tristate enable
v5 enable 0 1V

* transmission line
wline1 n1 0 n2 0 RLGCmodel=pcb N=1 L=0.3

* IBIS buffers
b1 nd_pc nd_gc n2 out_of_in
+ file = 'at16245.ibs'
+ model = 'AT16245_IN'

b4 nd_pu nd_pd n1 in1 enable
+ file = 'at16245.ibs'
+ model = 'AT16245_OUT'
+ ramp_fwf=0 ramp_rwf=0

* load
rload n2 0 50

* RLCG parameters for W-element
.model pcb w modeltype=rlgc n=1
+L0=3.94266e-7
+C0=1.12727e-10
+R0=5.7739
+G0=0
+Rs=0.00141445
+Gd=0

.end
```

Using the IBIS Component Command

The `.IBIS` command creates all buffers, package and interconnection for an entire component in an IBIS file.

For the syntax of the `.IBIS` command, see `.IBIS` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [How .IBIS Creates Buffers](#)
- [Required Keywords](#)
- [Optional Keywords](#)
- [Component Calls for SPICE or Verilog-A Formatted Pins](#)
- [Component Calls for SPICE or Verilog-A Formatted \[External Circuit\]](#)
- [Buffer Power](#)

How .IBIS Creates Buffers

The `.IBIS` command adds a buffer to the netlist for every pin, according to the `signal_name` and `model_name` defined in the [Pin] keyword in the `*.ibs` file.

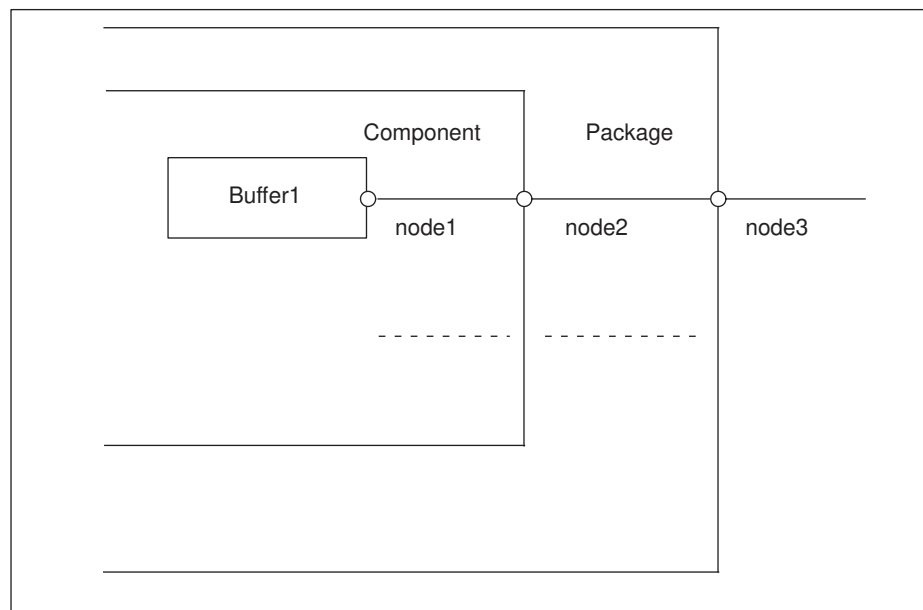
Note:

The `.IBIS` command does not create a buffer if the pin name is a reserved model name, such as `POWER`, `GND`, or `NC`.

```
buffer_name = 'cname'_'pin_name'
```

- `cname` is defined in the `.IBIS` card in the `*.sp` netlist.
- `pin_name` is defined in the [Pin] keyword in the `*.ibs` file

Figure 91 Package and component combined



PrimeSim HSPICE connects the new buffers that the `.IBIS` command creates to the following nodes:

- name of buffer 1: 'cname'_'pin_name'
- name of node 1: 'cname'_'pin_name'_i
- name of node 2: 'cname'_'pin_name'_o
- name of node 3: 'cname'_'pin_name'

Note that:

- 'cname'_'pin_name'_en is the enable node if the buffer has enable node.
- 'cname'_'pin_name'_i is the outofin node if the buffer is an input buffer; for an input/output buffer, the outofin node is 'cname'_'pin_name'_outofin.

Required Keywords

file='file_name'

This keyword identifies the IBIS file. The `file_name` parameter must be lower case and must specify either the absolute path for the file or the path relative to the directory from which you run the simulation.

Example

```
file = '.ibis/at16245.ibs'  
file = '/home/oneuser/ibis/models/abc.ibs'
```

component='component_name'

This keyword identifies the component for an .IBIS command from the IBIS file, specified using the `file='...'` keyword. The `component_name` keyword is case-sensitive, and it must match one of the components from the IBIS file.

Example

```
component = 'procfast'  
component = 'Virtex_SSTL_3-I_BG432'
```

Optional Keywords

The following sections discuss these optional keywords:

- [package](#)
- `pkgfile='pkg_file_name'`
- [\[Model Selector\] Support](#)
- [Other Optional Keywords](#)

package

This keyword specifies the type of package to add.

```
package = [0|1|2|3|4]
```

When `package` is set to

- 0, then the RLC package is not added into the component.
- 1, then [Package] (in the `ibs` file) is added.
- 2, then [Pin] (in the `ibs` file) is added. If the package information is not set in [Pin], set the package with [Package] as a default.
- 3 (default), and if [Package Model] is defined, set package with a package model. If the [Package Model] is not defined, set the package with [Pin]. If the package information is not set in [Pin], set the package with [Package] as a default. You can define the [Package Model] in IBIS or PKG files.

- 4, then the package model of [Package Model] is added.

You can define the [Package Model] in an IBIS file specified by the `file` keyword or a PKG file specified by the `pkgfile` keyword. The `pkgfile` keyword is useful only when `package = 3` or 4.

Example

```
.ibis p_test
+ file = 'comp.ibs'
+ component = 'cpu_133mhz_ff'
+ hsp_ver = 2002.4 nowarn
+ package = 3
+ pkgfile = 'test.pkg'
```

This card combines the `.pkg` and `.ibs` files. If you use [Package Model] in the `cpu_133mhz_ff` component, HSPICE searches for the `pkg` model in the `comp.ibs` and `test.pkg` files.

`pkgfile='pkg_file_name'`

You can define the package model file using the `pkgfile` keyword.

```
pkgfile = 'pkg_file_name'
```

If you cannot find a package model defined within a component in both the PKG and IBIS files, HSPICE issues an error message.

[Model Selector] Support

You can select a model using the `mod_sel` keyword.

```
mod_sel = 'selName_1=modName_1, selName_2=modName_2,...,selName_n=modName_n'
```

If `selName` and `modName` cannot be found in a related IBIS file or they cannot be matched, then an error occurs. In addition, the input string should be less than 1024 characters. If a model selector is used for a pin of a component, but `mod_sel` is not set, then the first model under the corresponding [Model Selector] is selected as default.

Other Optional Keywords

The following keywords are all optional keywords of the `.IBIS` command. They have the same function as for the B-element (I/O buffer). For detailed function descriptions, see Specifying Required and Optional Common Keywords in *PrimeSim Continuum User Guide: Elements*.

- `typ`
- `interpo`

- `ramp_rwf`
- `ramp_fwf`
- `ss_state`
- `time_control`
- `dt_scale`
- `pd_scal`
- `pu_scal`
- `pc_scal`
- `gc_scal`
- `rwf_scal`
- `fwf_scal`
- `nowarn`
- `hsp_ver`
- `c_com_pd`
- `c_com_pu`
- `c_com_pc`
- `c_com_gc`
- `detect_oti_mid`

If any keyword above is used, then it will be applied to all buffers created by the `.IBIS` command.

Component Calls for SPICE or Verilog-A Formatted Pins

The syntax to call a [Component] having SPICE or Verilog-A formatted [Pin] is:

```
.IBIS ibis_command_name  
+ file='ibis_filename' component='component_name'  
+ [package=3|0|1|2] [pkgfile='pkg_file_name']  
+ [nowarn]  
+ [mod_sel = 'selName_1=modName_1,  
+ selName_2=modName_2,...,selName_n=modName_n']
```

In the preceding syntax,

- The *component_name* specification must involve one or more SPICE formatted [Pin].
- Usage of the other parameters follow .ibis conventions.

Other parameters not shown above have no affect on a SPICE formatted buffer. If the buffer is a series buffer or a SPICE or Verilog-A formatted true differential buffer in which two pins are involved, then:

```
buffer_name='cname'_'pin1_name1'_'pin2_name'
```

For related node names, refer to [SPICE or Verilog-A Formatted B-element Naming Rules](#).

Component Calls for SPICE or Verilog-A Formatted [External Circuit]

[External Circuit] in a component can be divided into two types. One is used to describe the buffer and is similar to [External Model]; the other is used to describe interconnection inside a component. For the former, [Pin Mapping] can not be used in order to avoid ambiguous connections. For the latter, a die node declared in Port_map of [Circuit Call] can be probed in a netlist by the naming rule: 'cname'_'die_node_name'. Here, cname is defined in the .ibis card in the .sp netlist. In addition, die pad must not occur in Port_map since related connection syntax is still not available in the latest IBIS spec. If a buffer is described by [External Circuit], then it is set as power off, so voltage sources are not be created automatically to connect to external terminals of Pullup, Pulldown, Powerclamp and Groundclamp.

Buffer Power

The buffer component creates buffers that are always connected to the power sources that are specified in the IBIS file. You can specify power sources in two different ways, as described in the following sections:

- [Buffer Power ON](#)
- [Buffer Power OFF](#)

Buffer Power ON

If [Pin Mapping] is not defined in the *ibs* file or *hsp_ver* <= 2002.2, PrimeSim HSPICE automatically sets the power of buffers to ON. Use the [Voltage Range], [Pullup Reference], [Pulldown Reference], [POWER Clamp Reference], and [GND Clamp Reference] keywords in the *ibs* file to set the voltage source inside the buffer.

The buffer nodes expect input and output node names in this format:

```
'buffer_name' _node_name
buffer_name = 'cname' _'pin_name'
```

- `cname` is defined in the .ibis card in the .sp netlist.
- `pin_name` is defined in the [Pin] keyword in the ibs file
- `node_name` is different for different types of buffers as shown in the following list:

INPUT	pc , gc
OUTPUT	pu , pd , pc , gc
INPUT_OUTPUT	pu , pd , en , outofin , pc , gc
THREE_STATE	pu , pd , en , pc , gc
OPEN_DRAIN	pu , pd , pc , gc
IO_OPEN_DRAIN	pu , pd , en , outofin , pc , gc
OPEN_SINK	pu , pd , pc , gc
IO_OPEN_SINK	pu , pd , en , outofin , pc , gc
OPEN_SOURCE	pu , pd , pc , gc
IO_OPEN_SOURCE	pu , pd , en , outofin , pc , gc
INPUT_ECL	pc , gc
OUTPUT_ECL	pu , pc , gc
IO_ECL	pu , en , outofin , pc , gc
THREE_STATE_ECL	pu , en , pc , gc

Note:

For more information about nodes for different buffers, see Buffer Types in *PrimeSim Continuum User Guide: Elements*.

Table 21 shows the names of the input and output nodes for the buffers:

Table 21 *Input and Output Node Names for Buffers*

Buffer Type	Node Names
INPUT and INPUT_ECL buffers	'cname'_'pin_name' (for in node)'cname'_'pin_name'_i (for outofin node)

Table 21 *Input and Output Node Names for Buffers (Continued)*

Buffer Type	Node Names
Other types of buffers	'cname'_'pin_name'_'i' (for in node)'cname'_'pin_name' (for out node)

If the buffer has an enable terminal, you must create a node named `buffer_name_en` to enable the buffer.

You can test the following sample cases using the same method explained.

In this example, you can test the buffers with the B-element.

```
*****
.tran 50p 30n
.option post probe
.subckt twobus_b out1 out2
boutput1 nd_pu1 nd_pd1 out1 nd_in1 nd_pc1 nd_gc1
+ file = 'pinmap.ibs'
+ model = 'out50v'
+ buffer=2 power_on
+ nowarn
boutput2 nd_pu2 nd_pd2 out2 nd_in2 nd_pc2 nd_gc2
+ file = 'pinmap.ibs'
+ model = 'out50v'
+ buffer=2 power=on
+ nowarn
Vin1_b nd_in1 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Vin2_b nd_in2 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
.ends

Xtwobus_b out1_b out2_b twobus_b

Rout1_b out1_b 0 50
Rout2_b out2_b 0 50

.probe tran
+ out1_of_output_b = v(out1_b)
+ out2_of_output_b = v(out2_b)
+ in_of_output_b = v(Xtwobus_b.nd_in1)
.end
```

In this example, you can test the buffers that the `.IBIS` command creates.

```
*****
.tran 50p 30n
.option post probe
* add component
.ibis pcomp
+ file = 'pinmap.ibs'
+ component = 'NO_PINMAPPING'
```

```
+ nowarn
+ package = 0
Vin1 pcomp_1_i 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Vin2 pcomp_2_i 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Rout1_1 pcomp_1 0 50
Rout2_1 pcomp_2 0 50

.probe tran
+ out1_of_nopm = v(pcomp_1)
+ out2_of_nopm = v(pcomp_2)
+ in_of_output = v(pcomp_1_i)
.end
```

The following ibs file is:

```
| *****
| IBIS file pinmap.ibs
| *****
[IBIS ver] 3.1
[File name] pinmap.ibs
[File Rev] 1.0
[Date] 5/24/2002
[Source] spice models
[Notes] This ibis file tests the pinmap
results of HSPICE .ibis command. Pinmapping
is considered only if hsp_ver > 2002.2.

| *****
| Component NO_PINMAPPING
| *****
|
[Component] NO_PINMAPPING
|
[Manufacturer] TEST
|
```

	[Package]	typ	min	max
R_pkg	0	NA	NA	
L_pkg	0	NA	NA	
C_pkg	0	NA	NA	

[Pin]	signal_name	model_name	R_pin	L_pin	C_pin
OUT1	out50v	50.0m	2.01554n	0.33960p	
OUT2	out50v	50.0m	1.83759n	0.31097p	
GND1	GND	50.0m	1.89274n	0.32012p	

VCC1	POWER	50.0m	1.74394n	0.31941p
GND2	GND	50.0m	1.89274n	0.32012p
VCC2	POWER	50.0m	1.74394n	0.31941p

```
.....
|model information
|.....
[END]
```

Buffer Power OFF

If [Pin Mapping] is defined both in the `ibs` file and `hsp_ver > 2002.2`, PrimeSim HSPICE automatically uses the [Pin Mapping] keyword in the `ibs` file to load voltage sources to the buffers. PrimeSim HSPICE turns OFF the power of buffers that the component created.

In this case, the `nd_pc`, `nd_pu`, `nd_gc`, and `nd_pd` nodes connect to the power and ground bus according to the information in [Pin Mapping]. All other nodes are the same as in [Buffer Power ON](#).

The following two `.sp` files are equivalent. The first file uses the B-element to describe buffers:

```
* test for IO,OUTPUT,3-STATE,INPUT BUFFER in component w/pin
mapping
.option post probe
.tran 50p 30n
.param vhi = 2.5

.subckt io_o_3st_i_b in st gnd1 vcc1 io output
bin vcc1 gnd1 in nd_out1
+ file = 'pinmap.ibs'
+ model = 'DS92LV090A_IN1'
+ buffer=1 power=off
+ nowarn
b3st vcc1 gnd1 st nd_in1 nd_en1 vcc1 gnd1
+ file = 'pinmap.ibs'
+ model = 'DS92LV090A_RO'
+ buffer=4 power=off
+ nowarn
bio vcc1 gnd1 io nd_in2 nd_en2 nd_ofi vcc1 gnd1
+ file = 'pinmap.ibs'
+ model = 'DS92LV090A_DOUT'
+ buffer=3 power=off
+ nowarn
boutput vcc1 gnd1 output nd_in3 vcc1 gnd1
+ file = 'pinmap.ibs'
+ model = 'out50v'
+ buffer=2 power=off
+ nowarn
```

Chapter 8: Using IBIS Files

Using the IBIS Component Command

```
Vin1_b nd_in1 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Ven1_b nd_en1 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Vin2_b nd_in2 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Ven2_b nd_en2 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Vin3_b nd_in3 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Rout_b nd_out1 0 50
.ends

Xio_o_3st_i_b in_b st_b gnd1_b vcc1_b io_b output_b io_o_3st_i_b
Vvcc1_b vcc1_b 0 3.3v
Vgnd1_b gnd1_b 0 0v
Vin_b in_b 0 0V PULSE (0 vhi 1ns 0.5ns 0.5ns 5ns 10ns)
Rout1_b st_b 0 50
Rout2_b io_b 0 50
Rout3_b output_b 0 50
.probe tran
+ out_of_in_b = v(Xio_o_3st_i_b.nd_out1)
+ out_of_3st_b = v(st_b)
+ out_of_io_b = v(io_b)
+ out_of_output_b = v(output_b)
+ in_of_output_b = v(Xio_o_3st_i_b.nd_in3)
.end
```

The second file uses a the `.IBIS` command to create buffers:

```
* test with component
.option post post_version=9601 probe
.tran 50p 30n
.param vhi = 2.5
.ibis pcomp
+ file = 'pinmap.ibs'
+ component = 'IO_OUTPUT_3ST_INPUT'
+ nowarn
+ package = 0

Vin1 pcomp_3_i 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Ven1 pcomp_3_en 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Vin2 pcomp_6_i 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Ven2 pcomp_6_en 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Vin3 pcomp_7_i 0 0V pulse ( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
Rout pcomp_2_i 0 50

Vvcc1 pcomp_5 0 3.3v
Vgnd1 pcomp_4 0 0v
Vin pcomp_2 0 0V PULSE (0 vhi 1ns 0.5ns 0.5ns 5ns 10ns)
Rout1 pcomp_3 0 50
Rout2 pcomp_6 0 50
Rout3 pcomp_7 0 50

.probe tran
+ out_of_in = v(pcomp_in_2)
+ out_of_3st = v(pcomp_3)
+ out_of_io = v(pcomp_6)
```

```
+ out_of_output = v(pcomp_7)
+ in_of_output = v(pcomp_7_i)
.end
```

The .ibs file is:

```
| *****
| IBIS file pinmap.ibs
| *****
[IBIS ver] 3.1
[File name] pinmap.ibs
[File Rev] 1.0
[Date] 5/24/2002
[Source] spice models
[Notes] This ibis file tests the pinmap
results of HSPICE .ibis command. Pinmapping
is considered only if hsp_ver > 2002.2.
| *****
| Component IO,OUTPUT,3-STATE,INPUT BUFFER
| *****
```

[Component]	IO_OUTPUT_3ST_INPUT[Manufacturer]
TEST	

	[Package]	typ	min	max
R_pkg	0	NA	NA	
L_pkg	0	NA	NA	
C_pkg	0	NA	NA	

```
|
| *****
|
```

[Pin]	signal_name	model_name	R_pin	L_pin	C_pin
NC	NC	50.0m	2.11433n	0.27505p	
IN	DS92LV090A_IN1	50.0m	2.01554n	0.33960p	
ST	DS92LV090A_RO	50.0m	1.94357n	0.32802p	
GND1	GND	50.0m	1.89274n	0.32012p	
VCC1	POWER	50.0m	1.74394n	0.31941p	
IO	DS92LV090A_DOUT	50.0m	1.83759n	0.31097p	

```

OUTPUT      out50v      50.0m      1.83759n      0.31097p
-----

| ***** |
|
| [Pin Mapping] pullup_ref  pullup_ref  gnd_clamp_ref  power_clamp_ref
|
| NC      NC      NC      NC
|
| NC      NC      GND1     VCC1
|
| GND1     VCC1     GND1     NC
|
| NC      VCC1     NC      NC
|
| GND1     VCC1     GND1     VCC1
|
| GND1     VCC1     GND1     VCC1
|
|
| .....
|model information
| .....
| [END]

```

Using IBIS Package Modeling

In addition to `.IBIS`, PrimeSim HSPICE provides the `.PKG` command to parse package model definition in either `.ibs` or `.pkg` files. The `.PKG` command automatically creates a series of W-elements or discrete R, L and C components. For the complete description of this command, see the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [Syntax](#)
- [Examples](#)
- [Accessing Nets inside a Package Model](#)

Syntax

```
.PKG pkgname
+ pkgfile='pkgfilename'
+ model='pkgmodelname'
```

Examples

Example 1

```
.pkg p_test
+ pkgfile='processor_clk_ff.ibs'
+ model='FCPGA_FF_PKG'
```

Example 2—This example shows how in the second line, pin1 is referenced as the element name changes:

```
p_test_pin1_dia and p_test_pin1
w_p_test_pin1_? ? or r_p_test_pin1_? ? ...
```

Accessing Nets inside a Package Model

You can access the nets (internal pins) of the package model defined in the IBIS file. The node name is `compName_pinName`. Where `compName` is the component name defined in the `.IBIS` command and `pinName` is the Pin name given by the [Pin] definition in the package model. For example

```
.ibis buff
+ file = 'buff1.ibs'
+ component = 'Comp12'
```

The IBIS file contains the following package information for the pins that you want to access.

[Package Model]	[Pin]	signal_name	model_name	R_pin	L_pin
	C_pin				
vdd	vdd		POWER		
gnd	gnd		GND		
out	out		driver		
out2	out2		driver		

Using `.PROBE` statements, you can probe the nets as follows:

```
.probe v(buff_vdd) $ pin vdd defined in the package model
+ v(buff_gnd)      $ pin gnd defined in the package model
+ v(buff_out)      $ pin out defined in the package model
+ v(buff_out2)     $ pin out2 defined in the package model
```

Note:

If [pin mapping] is not defined in the IBIS file, then there is no connection to the pins of *vdd* and *gnd* and the nodes of *buff_vdd* and *buff_gnd* are not created. PrimeSim HSPICE creates voltage sources automatically for pins of *out* and *out2*. If [pin mapping] is defined in the IBIS file, then PrimeSim HSPICE connects PU, PD, PC, and GC of *out* and *out2* to pins of *vdd* and *gnd* (in fact,

to nodes of *buff_vdd_o* and *buff_gnd_o*, due to package) according to [pin mapping].

Using IBIS Board-Level Components

A board-level component is used to describe a printed circuit board (PCB) or substrate that can contain components or even other boards and can connect to another board through a set of user visible pins. The electrical connectivity of such a board level component is referred to as an “Electrical Board Description” (EBD). The `.EBD` command provides connectivity descriptions of chip to chip, or multiple-chips on a single package.

The `.EBD` command is associated with `.IBIS` command, because the EBD file contains the reference designator of the component. PrimeSim HSPICE search paths between the component pins and the EBD pins are displayed in the [Path Description] of the EBD file. The component pins are designated with `keyword:component` in the `.EBD` command, which automatically create Lumped RLCs and distributed RLCs (W-elements) to express those paths. PrimeSim HSPICE then simulates the newly-created circuit.

In the following netlist, you can see the important elements and internal nodes that are created in the naming rules of the `.EBD` command.

```
*****
*      Analysis And Options
*****

.op
.tran 10p 30n

.option probe
*****
*      Stimulus
*****

Vin cmp1_5_i 0 0V pulse ( 0V 3V 2n 0.1n 0.1n 7.5n 15n )

Vpu cmp1_1 0 2.5v
vpc cmp1_2 0 2.5v
vpd cmp1_3 0 0v
vgc cmp1_4 0 0

*****
*      Rload
*****

Rd1 ebd1_a3 0 50
Rd2 ebd1_a3 0 50

*****
*      Define Component
```



```

*****
.ibis cmp1
+ file = 'pinmap.ibs'
+ component = 'Component1'
+ package = 0

*****
*   Define EBD
*****

.ebd ebd1
+ file = 'pinmap.ebd'
+ model = 'Board1'
+ component = 'cmp1:u21'

*****
*   Output
*****

.probe tran
+ cmp1_5_out = v(cmp1_5)      $ buf_5(Output): cmp1_5
+ cmp1_5_in  = v(cmp1_5_i)    $ no package , so the out node is cmp1_5,
                                $ not cmp1_5_o

+ ebd1_a3    = v(ebd1_a3)
+ ebd1_a4    = v(ebd1_a4)

.end

```

The following sections discuss these topics:

- [.EDB and .IBIS Command Syntax](#)
- [Circuit Topology Created by the .EBD and .IBIS Commands](#)
- [B-element Naming Rules](#)
- [Circuit Topology Created with SPICE or Verilog-A Formatted Pins](#)
- [IBIS Board-Level Component Examples](#)

.EDB and .IBIS Command Syntax

Use the following syntax when using IBIS board-level components:

```

.EDB 'ebd_name'
+ file = 'ebd_file_name' $ case-sensitive
+ model = 'ebd_model_name' $ case-sensitive
+ component = 'ibis_or_ebd_name:ref_des' $ case-insensitive
+ [component = 'ibis_or_ebd_name:ref_des' [...]]

.IBIS 'ibis_name'

```

```
+ file = 'ibis_file_name' $ case-sensitive
+ component = 'component_name' $ case-sensitive
+ [package = 0|1|2|3]
+ [other_keyword = value [...]]
```

Note:

Parameters surrounded by arrow brackets (< >) are not required.

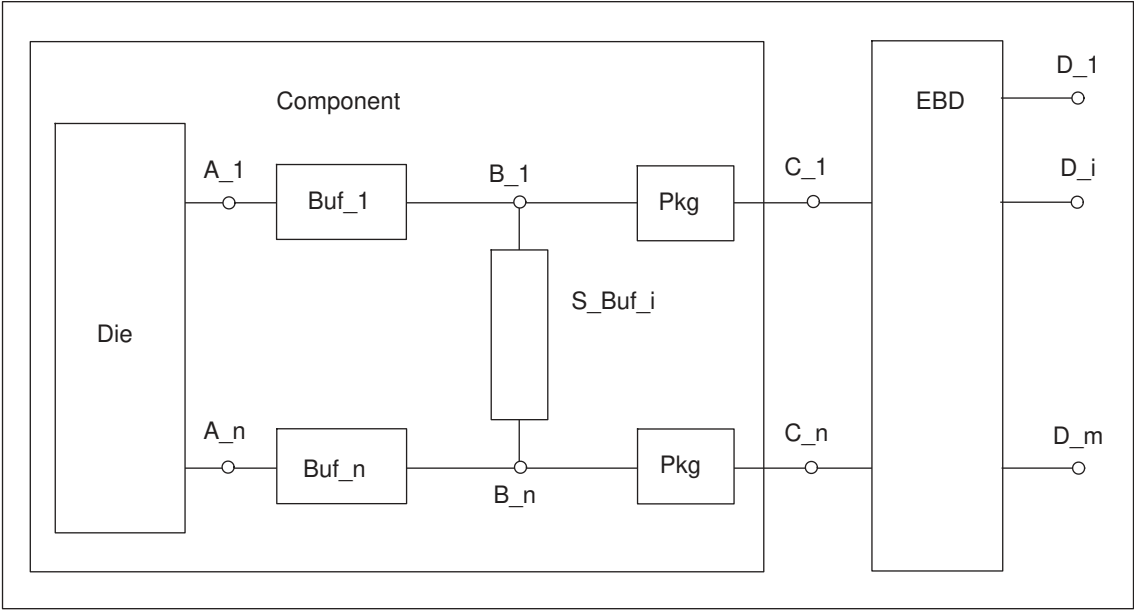
Table 22 .EBD/.IBIS Required Argument Descriptions

Parameter	Description
ebd_name	Specifies the name of the .EBD command. This argument is case-insensitive.
ibis_or_ebd_name	Specifies the name of associated .IBIS or .EBD command. This argument is case-insensitive.
ebd_file_name	Identifies the EBD file. You must specify either the absolute path for the file or the path relative to the directory from where you run the simulation. This argument is case-sensitive. For example: file = './ebd/test.ebd' file = '/home/usr/ebd/test.ebd'
ebd_model_name	Identifies the Board-Level Component. Visible in the [Begin Board Description] keyword in the EBD file. This argument is case-sensitive.
ibis_or_ebd_name: ref_des	Maps an IBIS buffer component or EBD board to a reference designator. - ibis_or_ebd_name: Name of the associated .IBIS command or associated .EBD command to identify IBIS buffer component or EBD board. - ref_des: Reference designator. Visible in the Node sub-parameter of the [Path Description] keyword in the EBD file. For example: component = 'cmp2:u22'

Circuit Topology Created by the .EBD and .IBIS Commands

The circuit topology in [Figure 92](#) is created by the `.EBD` and `.IBIS` commands:

Figure 92 Circuit topology created by the `.EBD` and `.IBIS` commands



B-element Naming Rules

The following rules in [Table 23](#) apply when naming B-elements in IBIS board-level components:

Table 23 B -element Naming Rules

Buffer	Name	Note
Buf_#	'ibis_name'_'ibis_pin_name'	If model_name shows NC/POWER/GND in the column of the [Pin] keyword in the IBIS file, no Buf_# buffer exists.
S_Buf_#	'ibis_name'_'pin1_name'_'pin2_name'	If the [Series Pin Mapping] keyword is not in the IBIS file, no S_Buf_# series/series switch buffer exists.

Table 24 *Node Naming Rules*

Node	Name	Corresponding Buffer Node
A_#	'ibis_name'_'ibis_pin_name'_'i	Terminator buffer: No A_# nodes Input/Input_ecl buffer: nd_out_of_in Other buffers: nd_inlf no Buf_#, no A_# nodes
B_#	'ibis_name'_'ibis_pin_name'_'o	Input/input_ecl buffer: nd_in Other buffers: nd_out Note: If no package (i.e. the package keyword in the .IBIS command is set to 0 or no relative sub- parameter is given in [Pin]/[Package] in IBIS file), then no B_# nodes. The corresponding nodes are named as C_# nodes.
C_#	'ibis_name'_'ibis_pin_name'	
D_#	'ebd_name'_'ebd_pin_name'	
Enable	'ibis_name'_'ibis_pin_name'_'en	Buffer that has the nd_en Enable node. Other buffers: No
Out_of_in	'ibis_name'_'ibis_pin_name'_'outofin	Buffer that has out_of_in node except the Input/Input_ECL buffer
Pulldown of Buf_#	'ibis_name'_'ibis_pin_name'_'pd or 'ibis_name'_'ibis_pin_pd_name_o'(if pin mapping is defined in the IBIS file)	Buffer that has the Pulldown nd_pd node. Other buffers: No
Pullup of Buf_#	'ibis_name'_'ibis_pin_name'_'pu or 'ibis_name'_'ibis_pin_pu_name_o' (if pin mapping is defined in the IBIS file)	Buffer that has the Pullup nd_pu node. Other buffers: No
GND Clamp of Buf_#	'ibis_name'_'ibis_pin_name'_'pd or 'ibis_name'_'ibis_pin_gc_name_o' (if pin mapping is defined in the IBIS file)	Buffer that has the GND Clamp nd_gc node. Other buffers: No
POWER Clamp of Buf_#	'ibis_name'_'ibis_pin_name'_'pc or 'ibis_name'_'ibis_pin_pc_name_o' (if pin mapping is defined in the IBIS file)	Buffer that has POWER Clamp nd_pc node. Other buffers: No

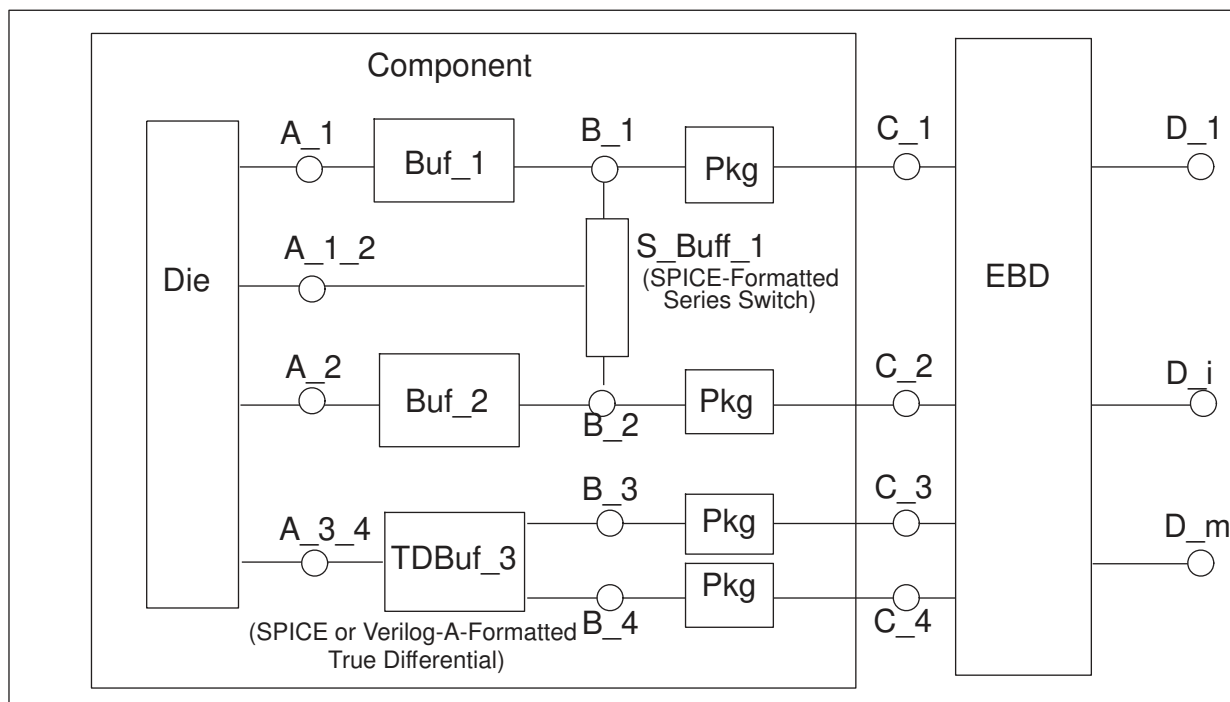
In the following node names, the first matched pin name in [Pin] consists of ground/power connections given in [Pin Mapping] in the *.ibs file.

- 'ibis_name': Name of .IBIS command.
- 'ebd_name': Name of .EBD command.
- 'ibis_pin_name': Pin Name, the first column in Keyword: [Pin] in the IBIS file.
- 'ebd_pin_name': Pin Name, the pin name given in [Path Description] in the EBD file.
- 'pin1_name': One of two pins joined by a series model, the first column in Keyword: [Series Pin Mapping] in the IBIS file.
- 'Pin2_name': The other of two pins joined by a series model, the second column in Keyword: [Series Pin Mapping] in the IBIS file.
- 'ibis_pin_pd_name'
- 'ibis_pin_pu_name'
- 'ibis_pin_gc_name'
- 'ibis_pin_pc_name': The first matched pin name in [Pin] which is the ground/power connections given in [Pin Mapping] in the IBIS file.

Circuit Topology Created with SPICE or Verilog-A Formatted Pins

The circuit topology in [Figure 93](#) is created by the `.EBD` and `.IBIS` commands where the IBIS component model contains SPICE or Verilog-A formatted pins:

Figure 93 Circuit topology with SPICE or Verilog-A formatted pins



SPICE or Verilog-A Formatted B-element Naming Rules

The following rules ([Table 25](#)) apply when naming SPICE or Verilog-A formatted B-elements in IBIS board level components:

Table 25 SPICE or Verilog-A Formatted B-element Naming Rules

Buffer	Name	Comment
Buf_#	'ibis_name'_'ibis_pin_name'	If model_name shows NC/POWER/GND in the column of the [Pin] keyword in the IBIS file, no Buf_# buffer exists.
TDBuf_#	'ibis_name'_'pin1_name'_'pin2_name'	If [Diff Pin] keyword is not in the IBIS file, or it is not a true differential buffer, no TDBuf_# buffer exists.
S_Buf_#	'ibis_name'_'pin1_name'_'pin2_name'	If the [Series Pin Mapping] keyword is not in the IBIS file, no S_Buf_# series/series switch buffer exists.

The following rules (Table 26) apply when naming SPICE or Verilog-A formatted nodes in IBIS board level components:

Table 26 Node naming rules—SPICE/Verilog-A formatted buffers in component

Node	Name	Corresponding buffer node/port
A_#	'ibis_name'_'ibis_pin_name'_i	Terminator buffer: No A_# nodes Input/Input_ecl buffer: nd_out_of_in Other buffers: nd_in
B_#	ibis_name_'ibis_pin_name'_o	Input/input_ecl buffer: nd_in Other buffers: nd_out Note: If no package (i.e. the package keyword in the .IBIS command is set to 0 or no relative sub-parameter is given in [Pin] / [Package] in IBIS file), then no B_# nodes. The corresponding nodes are named as C_# nodes.
C_#	'ibis_name'_'ibis_pin_name'	
D_#	'ebd_name'_'ebd_pin_name'	
A_#_#	'ibis_name'_'pin1_name'_'pin2_name'_i	Input_diff buffer: nd_out_of_in Other buffers: nd_in
enable	'ibis_name'_'ibis_pin_name'_en or 'ibis_name'_'pin1_name'_'pin2_name'_en (if buffer is true differential buffer)	Buffer that has an enable node (nd_en)
Out_of_in	'ibis_name'_'ibis_pin_name'_outofin or'ibis_name'_'pin1_name'_'pin2_name'_outofin(if buffer is true differential buffer)	Buffer that has out_of_in node except Input, Input_ECL and, Input_diff buffers
Pulldown of Buf_#	'ibis_name'_'ibis_pin_name'_pd or'ibis_name'_'ibis_pin_pd_name'_o' (if pin mapping is defined in the IBIS file)	A_pdref
Pullup of Buf_#	'ibis_name'_'ibis_pin_name'_pu or 'ibis_name'_'ibis_pin_pu_name'_o (if pin mapping is defined in the IBIS file)	A_puref
GND Clamp of Buf_#	'ibis_name'_'ibis_pin_name'_gc or 'ibis_name'_'ibis_pin_gc_name'_o (if pin mapping is defined in the IBIS file)	A_gcref

Table 26 Node naming rules—SPICE/Verilog-A formatted buffers in component (Continued)

Node	Name	Corresponding buffer node/port
POWER Clamp of Buf_#	clamp'ibis_name'_ 'ibis_pin_name'_pc or 'ibis_name'_ 'ibis_pin_pc_name'_o (if pin mapping is defined in the IBIS file)	A_pcref
External Reference of Buf_#	reference 'ibis_name'_ 'ibis_pin_name'_ext or 'ibis_name'_ 'ibis_pin_ext_name'_o (if pin mapping is defined in the IBIS file)	A_extref
Global reference	0	A_gnd
Non-inverting node of S_Buf_#	'ibis_name'_ 'pin1_name'_o	A_pos
Inverting node of S_Buf_#	'ibis_name'_ 'pin2_name'_o	A_neg
Non-inverting node of TDBuf_#	'ibis_name'_ 'pin1_name'_o	A_signal_pos
Inverting node of TDBuf_#	'ibis_name'_ 'pin2_name'_o	A_signal_neg
other nodes of Buf_#	'ibis_name'_ 'ibis_pin_name'_ 'port_name'	other ports defined by user
other nodes of TDBuf_#	'ibis_name'_ 'pin1_name'_ 'pin2_name'_ 'port_name'	other ports defined by user

The following names are the same as in [Table 24](#) except that for TDBuf_#, 'pin1_name', and 'pin2_name' are two pins joined by a true differential buffer model. They are the first and second columns in the keyword: [Diff Pin] in the IBIS file:

- 'ibis_name'
- 'ibis_pin_name'
- 'pin1_name'
- 'pin2_name'
- 'ibis_pin_pd_name'
- 'ibis_pin_pu_name'
- 'ibis_pin_gc_name'
- 'ibis_pin_pc_name'

IBIS Board-Level Component Examples

The following examples show how you can use the .IBIS and .EBD commands for board-level components.

Example 1: .IBIS Command File

The IBIS model for this example is the file `4mx32.ibs`. To use the `.IBIS` command, it is necessary to find the `[Component]` keyword in the `.ibs` file as in the following part of the `.ibs` file:

```
|
[Component]      K4D263238A |
[Manufacturer]   TEST
[Package]
| variable      typ          min          max
R_pkg          0.17         77.00m        0.77
L_pkg          2.84nH       0.56nH       6.04nH
C_pkg          3.20pF       0.55pF       5.20pF
|
[Pin]  signal_name      model_name      R_pin      L_pin      C_pin
B3     DM0              CTRL_PIN      0.71       5.75nH     5.20pF
B5     DQ3              DQ_PIN      0.12       0.86nH     0.57pF
B6     DQ2              DQ_PIN      83.00m     0.64nH     0.92pF
B7     DQ0              DQ_PIN      88.00m     0.56nH     0.53pF
C6     DQ1              DQ_PIN      0.11       0.78nH     0.56pF
M11    CK               CLK_PIN      0.36       1.64nH     2.52pF
```

In the example above, the component name is `K4D263238A`. In addition to the `[Component]` keyword, this portion of the `.ibs` file also shows the pin list. This information becomes important later.

Note:

There are three models relating to this component: `CTRL_PIN`, `DQ_PIN`, and `CLK_PIN`. If `ramp_rwf` and `ramp_fwf` keywords are set, then all models take the keyword values. Of course, such keywords may be nonsense for some model types.

This component only uses `[Ramp]` to define the V-t characteristics of the buffers so; the `ramp_rwf` and `ramp_fwf` keywords can be set to 0 for this example. There is package information for this component that is contained in the package file `4mx32.pkg`. The package information can be specified in the `.IBIS` command. The `.IBIS` command requires an instance name of the component to be specified. For this case, the name `bc1` is used. The `ibis` component definition can then be written as:

```
* ibis component
.ibis bc1 component='K4D263238A'
+ file='4mx32.ibs'
+ ramp_rwf=0 ramp_fwf=0
+ package=3
+ pkgfile='4mx32.pkg'
```

Note:

The component keyword is case sensitive and the specified component name must match the case of [Component] in the .ibs file or PrimeSim HSPICE prints an error message that the component is not defined.

Example 2: .IBIS Command File

```
** HSPICE Netlist **
.ibis cmp1
+ file = 'test1.ibs'
+ component = 'Component1'
+ package = 1

.ebd ebd1
+ file = 'test.ebd'
+ model = 'Board1'
+ component = 'cmp1:u21'

** IBIS file: test1.ibs **
[Component] Component1
| ...
[Pin] signal_name model_name R_pin L_pin C_pin
|
1 RAS0 Buffer1 200.0m 5.0nH 2.0pF
2 gnd GND 209.0m NA 2.5pF
3 pwr POWER NA 6.3nH NA
| ...
[Pin Mapping] pulldown_ref pullup_ref gnd_clamp_ref power_clamp_ref
ext_ref
|
1 GNDBUS1 PWRBUS1 |
2 GNDBUS1 NC |
3 NC PWRBUS1
| ...
[Series Pin Mapping] pin_2 model_name function_table_group
2 3 CBTSeries 1 | Four independent groups
```

The nodes listed in [Table 27](#) are used with B-elements in the example above.

Table 27 *B -element Nodes*

Buffer/Node	Name
A_1	cmp1_1_i
B_1	cmp1_1_o
B_2	cmp1_2_o
B_3	cmp1_3_o
C_1	cmp1_1

Table 27 *B -element Nodes (Continued)*

Buffer/Node	Name
C_2	cmp1_2
C_3	cmp1_3
D_1	ebd1_a1
D_2	ebd1_a2
D_3	ebd1_a3
Pulldown of Buf_1	cmp1_2_o
Pullup of Buf_1	cmp1_3_o
GND Clamp of Buf_1	cmp1_1_gc
POWER Clamp of Buf_1	cmp1_1_pc
Buf_1	cmp1_1
S_Buf_1	cmp1_2_3

Example 3: .IBIS Command File

The following is an example where the component keyword of use the map of another ebd to reference designator:

```

** HSPICE Netlist **
.ibis cmp1
+ file = 'testibis.ibs'
+ component = 'testibis'
+ nowarn
.ebd ebd1
+ file = 'testebd1.ebd'
+ model = 'testebd1'
+ component = 'cmp1:u2'
+ component = 'ebd2:u20'
.ebd ebd2
+ file = 'testebd2.ebd'
+ model = 'testebd2'
+ component = 'cmp1:u2'
** IBIS file: testibis.ibs **
[Component]      testibis
| ...
[Pin]      signal_name      model_name      R_pin      L_pin      C_pin
|C8      DQ1      IO      0.0980      1.1970nH      0.4340pF

```

Example 1: .EBD Command File

To use the `.EBD` command, it is necessary to find the name of the board as given by the [Begin Board Description] keyword in the `.ebd` file.

The complete netlist, IBIS model, package and `.ebd` files for this example can be found in `$installdir/demo/hspice/ibis/ebd`. The name of the board is used for the model name in the `.EBD` command. In this example, the name of the board description is Board1. The reference designators of the components on the board are given by the [Reference Designator Map] keyword and are used by the component keyword of the `.EBD` command. In this example, there is only one reference designator, U1 which is part of the `.ebd` file:

```
[Begin Board Description] Board1
[Manufacturer]           Test
|
[Number Of Pins] 2
|
[Pin List]   signal_name
B7          DQ0
C6          DQ1
[Path Description] Trace1
Pin B7
Len = 0.5 L=8.35n C=3.34p R=0.01 /
Len = 0.1 L=2.1n  C=1.15p R=0.01 /
Node u1.B7
[Path Description] Trace2
Pin C6
Len = 0.5 L=8.35n C=3.34p R=0.01 /
Len = 0.1 L=2.1n  C=1.15p R=0.01 /
Node u1.C6
[Reference Designator Map]
| Ref Des   File name      Component name
u1         4mx32.ibs       K4D263238A
[End Board Description]
```

The `.EBD` command requires an instance name of the board to be specified. For this case, the name `sb1` is used. The board definition can be written as:

```
* board description
.ebd sb1
+ file='4mx32.ebd'
+ model='board1'
+ component=bcl:u1
```

Now that both the ibis component and board description are defined, the input, enables, and terminations for the buffers can be defined. The node naming convention follows the node naming rules defined in [Table 24](#). The complete netlist can be written as:

```
* EBD Usage Example
* ibis component
.ibis bcl component='K4D263238A'
+ file='4mx32.ibs'
```

Chapter 8: Using IBIS Files

Using IBIS Board-Level Components

```
+ ramp_rwf=0 ramp_fwf=0
+ pkgfile='4mx32.pkg'
+ package=3

* board description
.ebd sb1
+ file='4mx32.ebd'
+ model='board1'
+ component=bcl:u1

* enables
v3 bcl_b7_en 0 1
v4 bcl_c6_en 0 1
* data
vdq0 bcl_b7_i 0 pulse(0 1 1n 0.1n 0.1n 5n 20n)
vdq1 bcl_c6_i 0 pulse(0 1 1n 0.1n 0.1n 5n 15n)

* terminations
r1 sb1_b7 0 50
r2 sb1_c6 0 50
```

Example 2: .EBD Command File

Here is an example of an .ebd file:

```
[Begin Board Description] Board1
|... ...
[Pin List] signal_name
A1 GND
A2 data1
A3 data2
|... ...
[Path Description] CAS_2
Pin A1
Len = 0.5 L=8.35n C=3.34p R=0.01 /
Node u21.1
Len = 0.5 L=8.35n C=3.34p R=0.01 /
Node u21.2
Len = 0.5 L=8.35n C=3.34p R=0.01 / | this section is
| discarded because
| the Node u23.3
| reference
| designator:u23 is not
| designated to IBIS
| component in the .ebd
| ebd1 command.
```

Example 3: .EBD Command Files

The following are examples of two .ebd files:

```
** EBD file: testebd1.ebd **
[Begin Board Description] testebd1
```

```
[Manufacturer]          Test
|...
[Pin List]  signal_name
12          RDQ8
[Path Description] 12
Pin 12
Len=0.10604 L=8.39999e-009 C=2.74272e-012 R=0.25139 /
Len=0 R=15.00000 /
Fork
  Len=0.07935 L=8.39999e-009 C=2.74272e-012 R=0.25139 /
  Node U2.C8
Endfork
Len=0.07063 L=8.39999e-009 C=2.74272e-012 R=0.25139 /
Node U20.A13

** EBD file: testebd2.ebd **
[Begin Board Description] testebd2
[Manufacturer]          Test
|...
[Pin List]  signal_name
A13          DQ1
[Path Description] A13
Pin A13
Len=0.18690 L=8.38871e-009 C=2.32868e-012 R=0.12121 /
Node U2.C8
```

Using IBIS Interconnect Modeling (ICM)

ICM is a standard for behavioral descriptions of interconnect electrical characteristics. The text describes a consistent format that can be parsed by software, allowing the simulation tool and interconnect vendors to create models compatible with their own products.

The PrimeSim HSPICE `.ICM` command automatically creates port names that reference the pin name of an ICM model and generates a series of element nodes on the pin.

Syntax

```
.ICM icmname
+ file='icmfilename'
+ model='icmmodelname'
```

Example

```
.ICM icm1
+ file='test1.icm'
+ model='FourLineModel1'
```

For the complete description of the command, see the *PrimeSim HSPICE Reference Manual: Commands and Control Options*, `.ICM`.

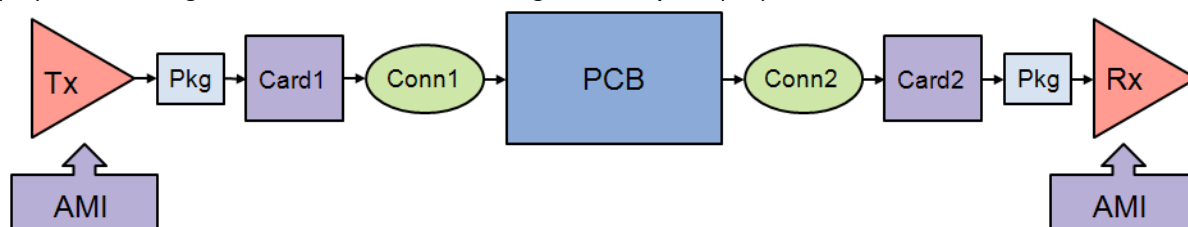
9

Using IBIS-AMI Equalizer Models with StatEye

Describes use of the Algorithmic Modeling Interface (AMI) to enable StatEye analysis of systems with Serializer-Deserializer (SERDES) devices for equalizers in both transmitter and receiver sides.

PrimeSim HSPICE supports an interface that uses the IBIS Version 5.0 interface to the C/C++ language-based compiled shared object called Algorithmic Modeling Interface (AMI). The AMI models complex linear and non-linear equalizers and enables the StatEye analysis of systems with Serializer-Deserializer (SERDES) devices for equalizers in both transmitter (TX) and receiver (RX) sides.

Figure 94 High-speed serial links can be composed of three parts: Transmitter algorithmic part (Tx), the analog channel, and Receiver algorithmic part (Rx)



See [Chapter 3, Statistical Eye Analysis](#), for a full discussion of the `.STATEYE` command and application examples.

These topics are discussed in the following sections:

- [IBIS-AMI Model Structure](#)
- [Using the Standalone AMI Object Checking Program 'amicheck'](#)
- [How AMI Works in PrimeSim HSPICE](#)
- [StatEye Analysis with AMI Using Demo Example](#)
- [Parametric Control over AMI File for Quoted Expressions](#)
- [Using an IBIS-AMI Mid-Channel Repeater Model](#)
- [References](#)

IBIS-AMI Model Structure

An IBIS-AMI model is an executable “black box” that is:

- Written in C/C++
- Object files are platform dependent
- Limited to one IBIS-AMI model per device
- Accompanied by parameter definition file
- Supports statistical simulation

In the IBIS file (*.ibs), the model is defined by the keywords:

```
[Algorithmic Model]
[End Algorithmic Model]
```

Example

```
[Algorithmic Model]
Executable Linux_gcc_32 libtx_ami.so
tx_ami_params.ami
[End Algorithmic Model]
```

IBIS-AMI Data Flow

Each IBIS-AMI object file contains the following sections:

- AMI_Init
 - Initializes impulse response filter
 - Sets up data structures
- AMI_GetWave
 - Enables waveform processing
 - Sets up clock and data recovery
- AMI_Close
 - Frees memory
 - Closes files, etc.
- Parameter file (*.ami) for model and simulation control

Requirements for Using IBIS-AMI Models with PrimeSim HSPICE

The following are the requirements for PrimeSim HSPICE-AMI usage:

- IBIS-AMI models are to be used only with PrimeSim HSPICE .STATEYE analysis.
- The IBIS-AMI parameter file (*.ami) can be encrypted using 3DES encryption.
- Required files:
 - AMI parameter file – *.ami
 - PrimeSim HSPICE netlist
 - Either C/C++ files that define the model, or Makefile – required to compile the model
 - Or platform-specific binary AMI object files, *.so or *.dll

Using the Standalone AMI Object Checking Program 'amicheck'

PrimeSim HSPICE includes a standalone utility program which provides a simple way to examine PrimeSim HSPICE's AMI filtering flow without composing a full simulation netlist. By combining this utility program with a memory profiling tool such as `valgrind` on the Linux platform, AMI model creators may detect possible memory access violations and problems in filtering operations in advance of the model release to users.

For more information on using `valgrind`, see <http://valgrind.org/>

Caution:

Consider the following before using the `amicheck` utility:

- Ensure that the `amicheck` executable bitness matches with that of the object file.
- Ensure to check all AMI files received from a model supplier using the `amicheck` utility.

To run the checker program, on the command line type:

```
% amicheck object_filename parameter_filename
```

For the 64 bit version of `amicheck`, the syntax is:

```
% amicheck -64 object_filename parameter_filename
```

Note:

PrimeSim HSPICE and `amicheck` uses standard shared object loader function which attempts to find the target library in the path specified in the `LD_LIBRARY_PATH` environment variable.

PrimeSim HSPICE and `amicheck` fails to locate the target library if you do not have your current directory (".") included in the `LD_LIBRARY_PATH` environment variable.

The following sections discuss these topics:

- [amicheck Flow](#)
- [Windows OS 'amicheck' Setup](#)
- [amicheck Output](#)

amicheck Flow

On launch, the `amicheck` flow follows this process:

1. Applies linear time invariant (LTI) filters to the entire system using `AMI_init`
2. When supplied in AMI object, applies non-LTI filters to the bit stream response waveforms using the `AMI_GetWave` Closes the AMI interface with `AMI_Close`
3. Calls `AMI_Init`, `AMI_GetWave` (if available) and `AMI_Close` with falling step input
4. Calls `AMI_Init`, `AMI_GetWave` (if available) and `AMI_Close` with pulse input
5. Unloads the AMI object and exits

Windows OS 'amicheck' Setup

On the Windows OS, use the following steps to set up **amicheck**.

1. Go to **My Computer** or the **Computer** folder on your desktop and click the right mouse button.
2. Select **Properties** and then **Advanced system settings**.
3. On the **Advanced** tab click the **Environment Variables** button.
4. In the **System Variables** field, select the **Path** variable and edit the value to add your PrimeSim HSPICE installation path. For example, if the path to the PrimeSim HSPICE

executable is `C:\synopsys\Hspice_G-2012.09\BIN` then the `amicheck.exe` utility is also in this path.

5. Execute `amicheck` by using the DOS command line. For example:

```
C:\users\test> amicheck test_ami.dll test_param.ami
```

amicheck Output

The `amicheck` program generates these output files:

- `parameter_filename.lis`: program status output.
- `parameter_filename.tree`: parsed parameter tree
- `object_filename.rise`: rise step response
- `object_filename.fall`: fall step response
- `object_filename.pulse`: pulse response

WaveView can be used to display the waveform response files, `object_filename.{rise, fall, pulse}`. These files contain the following columns:

- Input signal to AMI Object
- Output of `AMI_Init` command
- Output of `AMI_GetWave` command

How AMI Works in PrimeSim HSPICE

The compact standalone utility program (`amicheck`) detects potential problems in AMI objects and parameter files before running PrimeSim HSPICE simulations. The `amicheck` program takes the AMI object file name and parameter file name as command line arguments to load and filter processes as used in actual PrimeSim HSPICE simulations. AMI functionality is called at appropriate step(s) from incident and/or probe port(s) in StatEye analysis when these port elements are configured with an AMI object (shared library) file name and an AMI parameter file (`*.ami`) which contains specific keywords to control the behavior of the AMI objects.

When a port (P-) element includes the keywords `AMI_OBJ=filename` and `AMI_PARAM=filename`, StatEye opens interface(s) to the AMI object(s) and applies waveform filtering processes using `AMI_Init`, `AMI_GetWave`, and `AMI_Close` functions at appropriate analysis steps.

These topics are presented in the following sections:

- [Input Syntax](#)
- [Saving AMI Pulse/Step Responses and Filters](#)
- [Visualizing Eye Diagram Output of an IBIS-AMI RX Model](#)
- [Usage Example](#)

Input Syntax

```
Pi n1 n2 port=1 ...  
+ [AMI_OBJ=filename]  
+ [AMI_PARAM=filename]  
+ [AMI_CDR=0|1]  
+ [AMI_CORNER=idx]
```

Keyword	Description
AMI_OBJ	Specifies an AMI shared object (typically, lib *.so for UNIX, *.dll for Windows)
AMI_PARAM	Specifies an AMI parameter file (*.ami)
AMI_CDR	When AMI_CDR=1, StatEye analysis with specific pattern (LFSR/PAT) input takes into account the clock data recovery (CDR) output information from the receiver side AMI_GetWave function. Default = 0.
AMI_CORNER	Specifies an index number. The IBIS keyword "...(Corner val1 val2 ...)" defines the corner values in the IBIS-AMI file specified by AMI_PARAM. When you use AMI_CORNER, <i>idx</i> is used to chose the <i>idx</i> th value defined by the "...(Corner ...)" keyword. For example, if AMI_CORNER =2 is specified in a port element and the associated AMI parameter file has the following: (My_AMI_param1 ... (Corner 1 2 3)) (My_AMI_param2 ... (Corner 4 5 6)) (My_AMI_param3 ... (Corner 7 8 9)) Then, 2, 5 and 8 are chosen for My_AMI_param{1,2 and 3} respectively.

See also [Statistical Eye Analysis Setup](#).

Note:

PrimeSim HSPICE's AMI parameter parsing rule—When GUI form contents are given in the AMI parameter file, PrimeSim HSPICE uses your specified

Default value. If you do not specify a `Default` value in the GUI content, then PrimeSim HSPICE:

- Tries to set a “typical” value based on the `Value/Range/List/Step/Increment` keyword. Then, when a value is determined, PrimeSim HSPICE describes it in the `*.lis` file.
- Issues a warning.

For example, for this syntax:

```
(my_param
  (Range 30 0 100) (Usage Info) (Type Integer) (Default 10)
)
```

PrimeSim HSPICE sets `(my_param 10)`, while for this syntax:

```
(my_param
  (Range 30 0 100) (Usage Info) (Type Integer)
)
```

PrimeSim HSPICE issues a warning and sets `(my_param 30)`.

Saving AMI Pulse/Step Responses and Filters

When `SAVE_TR=ascii` is specified with AMI configurations, any system step/pulse responses after AMI filtering are saved in `netlist.saveAMI#/#` directory where `#` represents the `.alter` index number. If AMI filters are active, when `LOAD_TR=ascii` is specified, StatEye loads pre-AMI filtered pulse/step responses from the `netlist.save#/#` directory, then applies the AMI filters again. This way, users can efficiently examine multiple AMI filters with the same netlist.

Note:

When an explicit directory path is not given for the AMI object file name (for example, only the file name is given), PrimeSim HSPICE and the `amicheck` utility search the object file under directories given in the `LD_LIBRARY_PATH` environment variable. Typically, when the current working directory (“.”) is not a part of the `LD_LIBRARY_PATH` environment variable, specifying only the file name with the target object located under the current directory causes a load failure. For such cases, add the current directory to your `LD_LIBRARY_PATH` variable or use the explicit path.

Visualizing Eye Diagram Output of an IBIS-AMI RX Model

To see the statistical eye at output of the IBIS-AMI RX model when you connect an IBIS-AMI Rx model in your circuit, you need to properly assign port elements to the inputs and outputs of the system and identify them in the `.STATEYE` command using the

INCIDENT_PORT and PROBE_PORT lists. You also need to add .PROBE statements to output the desired eye diagram.

For example:

```
.STATEYE T=200p trf=25p incident_port=1 probe_port=2
.PROBE stateye eye(2)      $ eye diagram at port 2
```

Usage Example

This example demonstrates PrimeSim HSPICE's StatEye analysis setup with a receiver side equalizer described by AMI.

```
*
* StatEye with an AMI model
*
.param _BW=0.05n
.param trf = '_BW*0.1'
P1 in 0 port=1
P2 out 0 port=2
+ AMI_OBJ='./my_AMI_lib.so' AMI_PARAM='./
my_AMI_params.ami'
Xdut in out DUT
stateye T='_BW' trf='trf' incident_port=1
probe_port=2_port=2
.subckt DUT _in _out
*** your DUT
.ends
.end
```

StatEye Analysis with AMI Using Demo Example

The PrimeSim HSPICE installation includes an example to perform .StatEye analysis with an AMI object to model a simple receiver side Finite Impulse Response (FIR) filter.

Locate the demo example in the *\$installdir/demo/hspice/si/stateyeAMI* directory. The example contains the PrimeSim HSPICE netlist (*stateye_ami.sp*), AMI parameter file (*SNPS_Rx.ami*), C/C++ source code, and make systems (Makefiles) for each platform with a pre-compiled AMI object. In the *stateye_ami.sp* file, the second port element (declared as the probe port for .StatEye analysis) contains AMI object assignment statements which are originally commented out as:

```
P2 out 0 port=2
+ AMI_OBJ='libSNPS_Rx.so' AMI_PARAM=SNPS_Rx.ami
*** AMI_OBJ='./Linux64/libSNPS_Rx.so' AMI_PARAM=SNPS_Rx.ami
```

By activating one AMI assignment statement for your platform, you can examine the effect of the AMI DFE module in the .StatEye analysis. Also with AMI parameter file,

SNPS_Rx.ami, you can modify the DFE behavior by changing the number of taps, tap locations, and tap weights described in following part of the file.

```
(Model_Specific
  (TAP
    (Description "TAP (location weight) ")
    (-1 -0.200)
    (0 0.900)
    (1 -0.030)
    (2 0.035)
    (3 -0.010)
  )
)
```

Taking these tap coefficients, the example DFE model outputs:

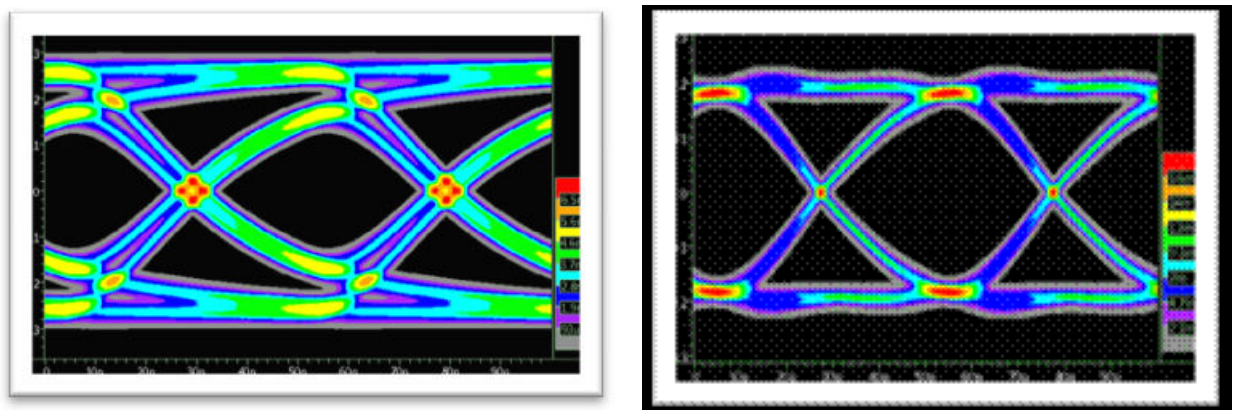
Equation 85

$$V_{DFEout}(t) = \sum_i w_i \times V_{DFEin}(t + T \cdot x_i)$$

where:

- $V_{DFEout}(t)$: DFE output (equalized waveform)
- $V_{DFEin}(t)$: DFE input (pre-equalized waveform)
- x_i : i-th tap location
- w_i : i-th tap weight
- T : Unit interval

Figure 95 Output statistical eye diagram from the demo example: (left) without equalization (right) with equalization



Note:

Descriptions about definitions of controllable model specific parameters must be provided by each AMI model provider. Follow vendor guidelines before changing the contents of vendor supplied *.ami parameter files.

Parametric Control over AMI File for Quoted Expressions

StatEye analysis can evaluate quoted expressions in the AMI parameter file (*.ami file) by using parameter values defined in a PrimeSim HSPICE netlist. This capability increases usability for optimizing transmitter and receiver equalizations in high speed serial links.

In a PrimeSim HSPICE netlist, you can define an arbitrary number of parameters by use of the .PARAM command. For example,

```
.PARAM param_name=value
```

Then, in the AMI parameter file (.ami file), you can insert expressions as (single) quoted strings. When loading the AMI parameter file, PrimeSim HSPICE evaluates these quoted strings as expressions, then builds a string to be passed to the AMI_Init function. Once PrimeSim HSPICE parses the AMI parameter tree evaluates expressions the tool prints the resulting parameter tree and the parameter string for the AMI_Init function to the listing file.

Example

The following PrimeSim HSPICE netlist performs two .StatEye runs with different parameter sets for an AMI parameter file.

```
*
* StatEye with SNPS Rx Equalizer Model in AMI
*
.param _BW=0.05n
.param trf = '_BW*0.01'
.param opfreq = '1.0/_BW'
P1 in 0 port=1
*+ LFSR (1 -1 0 'trf' 'trf''opfreq' 1 [14,5,2] )
P2 out 0 port=2
+ AMI_OBJ='./libSNPS_Rx.so'
  AMI_PARAM=SNPS_Rx_param.ami
X1 in out CONN
.stateye T='_BW' trf='0.1*_BW' incident_port=1 probe_port=2
.opt post accurate delmax=1e-12
.subckt CONN _in _out
Rtest _in _out 50 RS=1e-3 $ skin effect resistor
Cin _in 0 0.3p
Cout _out 0 0.3p
.ends

.probe stateye eye(2)
```



```
.param ami_filt_mag = 1.0
.param ami_tap_n1 = 0.0 ami_tap_0 = 1.0
.param ami_tap_1 = 0.0 ami_tap_2 = 0.0
.param ami_tap_3 = 0.0
.alter 2nd param set
.param ami_filt_mag = 0.9
.param ami_tap_n1 = -0.18 ami_tap_0 = 1.0
.param ami_tap_1 = -0.03 ami_tap_2 = 0.04
.param ami_tap_3 = -0.01
.end
```

The resulting AMI parameter file, `SNPS_Rx_param.ami`, contains expressions using parameters which are defined in the PrimeSim HSPICE netlist.

```
(SNPS_Rx
  (Description "SNPS Rx Equalizer Model")
  (Reserved_Parameters
    (Ignore_Bits 10)
    (Max_Init_Aggressors 0)
    (Init_Returns_Impulse False)
    (GetWave_Exists True)
    (Use_Init_Output False)
  )
  (Model_Specific
    (TAP
      (Description "TAP (loc weight) ")
      (-1 ' ami_filt_mag * ami_tap_n1 ' )
      (0 ' ami_filt_mag * ami_tap_0 ' )
      (1 ' ami_filt_mag * ami_tap_1 ' )
      (2 ' ami_filt_mag * ami_tap_2 ' )
      (3 ' ami_filt_mag * ami_tap_3 ' )
    )
  )
)
```

This is the resulting parameter tree with expression evaluation at first alter (to be printed out in the `.lis` file):

```
-----
, SNPS_Rx
, , Description "SNPS Rx Equalizer Model"
, , Reserved_Parameters
, , , Ignore_Bits 10
, , , Max_Init_Aggressors 0
, , , Init_Returns_Impulse False
, , , GetWave_Exists True
, , , Use_Init_Output False
, , Model_Specific
, , , TAP
, , , , Description "TAP (loc weight) "
, , , , -1 0
, , , , 0 1
, , , , 1 0
```

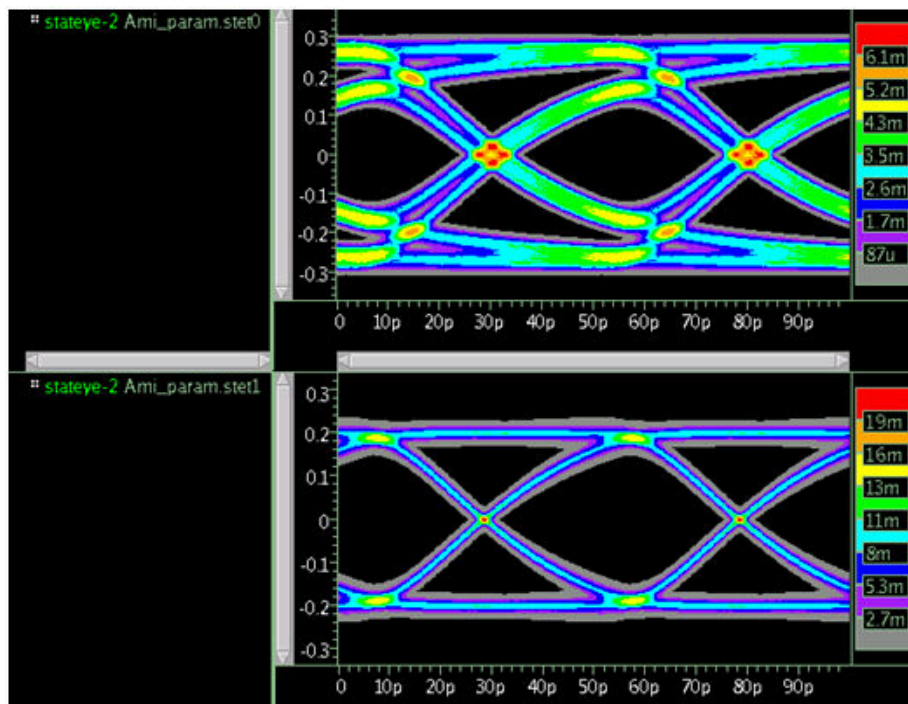
```
, , , , 2 0  
, , , , 3 0  
-----
```

The resulting parameter tree with expression evaluation at the second alter is also printed to the `.lis` file):

```
-----  
, SNPS_Rx  
, , Description "SNPS Rx Equalizer Model"  
, , Reserved_Parameters  
, , , Ignore_Bits 10  
, , , Max_Init_Aggressors 0  
, , , Init_Returns_Impulse False  
, , , GetWave_Exists True  
, , , Use_Init_Output False  
, , Model_Specific  
, , , TAP  
, , , , Description "TAP (loc weight) "  
, , , , -1 -0.162  
, , , , 0 0.9  
, , , , 1 -0.027  
, , , , 2 0.036  
, , , , 3 -0.009  
-----
```

Here, the resulting eye diagrams show the effect of equalizer tap weight.

Figure 96 Effect of equalizer tap weight



Backchannel Communication Interface for IBIS-AMI's Link Training Protocol

The target feature adds keywords for the Backchannel Communication Interface (BCI) to the IBIS-AMI simulation flow to allow information exchange between IBIS-AMI equalizer models at Tx and Rx.

PrimeSim HSPICE interprets the new AMI reserved parameters such as `BCI_Protocol`, `BCI_State`, `BCI_ID`, `BCI_Message_Interval_UI`, and `BCI_Training_UI` from *.ami files and implements channel training flow.

The BCI equalizer training flow is implemented in the edge mode only. The repeater support implementation is planned after upcoming release depending on the customer requirements.

Input Syntax

No additional netlist keyword is necessary. PrimeSim HSPICE reads the reserved keywords in the *.ami file and automatically activates channel training flow.

Output

There is no change in the output. The output from `AM_Init` and `AMI_GetWave` may include `BCI_State` for either one of the following *Off*, *Training*, *Converged*, *Failed*, and *Error* outputs.

BIC_ID Naming Convention

PrimeSim HSPICE decides and substitutes the string value for the `BCI_ID` keyword. The ID is used as a prefix of the file names and used by Tx and Rx equalizers for communication. StatEye uses the following naming convention.

Syntax

```
(BCI_ID (netlist_name.bci#/BCI_protocol_name_))
```

where

netlist_name: main (PrimeSim HSPICE) netlist body name (without suffix, for example, .sp)

#:alter index

For example, `my_stateye.bic0/channel_` for the netlist, `my_stateye.sp`.

Using an IBIS-AMI Mid-Channel Repeater Model

The IBIS version 6.0 and later specification allows placing an IBIS-AMI equalizer model to realize a mid-channel repeater. PrimeSim HSPICE's StatEye allows this analysis flow by specifying the `redriver=(rx_index, tx_index)` or the `retimer=(rx_index, tx_index)` keywords to connect two channels in series.

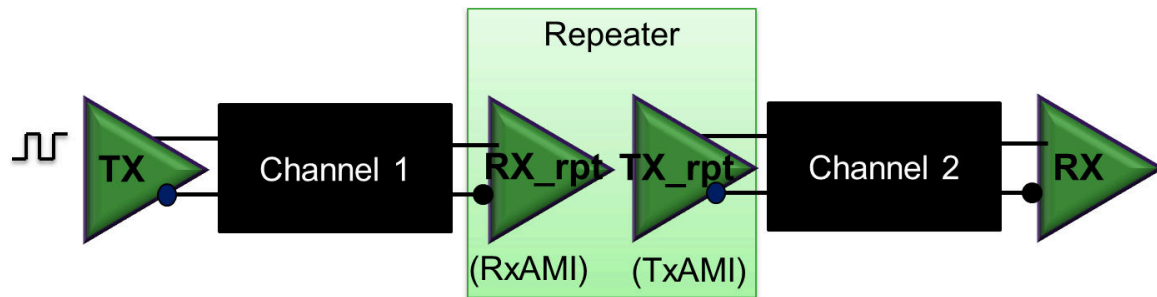
The `redriver` keyword specifies a probe port index and an incident port index pair. The mid-channel repeater configured as a re-driver will then be placed in between the specified probe port and the incident port. The output of the upstream channel as the waveform at the probe port will be fed into the incident port as an input to the downstream channel.

The `retimer` keyword specifies a probe port index and incident port index pair. The mid-channel repeater configured as a re-timer will be placed between the specified probe port and incident port. The output of the upstream channel as the waveform at the probe port will be used to generate a digital stream where the CDR clock information will be used when an AMI model is attached to the probe port. The generated digital stream will be fed into the incident port as an input to the downstream channel.

Note:

The mid-channel repeater analysis, using either of the two preceding keywords, requires StatEye's convolution mode (`mode=conv`) to be set.

Figure 97 Mid-Channel repeater



Mid-Channel Repeater Example

```
***
.param _UI = xxx

.param trf = '_UI*0.05'
.param opfreq = '1.0/_UI'

P1 in1 0 port=1
+ LFSR (-1 1 0 'trf' 'trf' 'opfreq' 2 [14 7 2] )
X1 in1 out1 CONN
P2 out1 0 port=2
+ AMI_OBJ='my_rx1.so' AMI_PARAM=my_rx1.ami

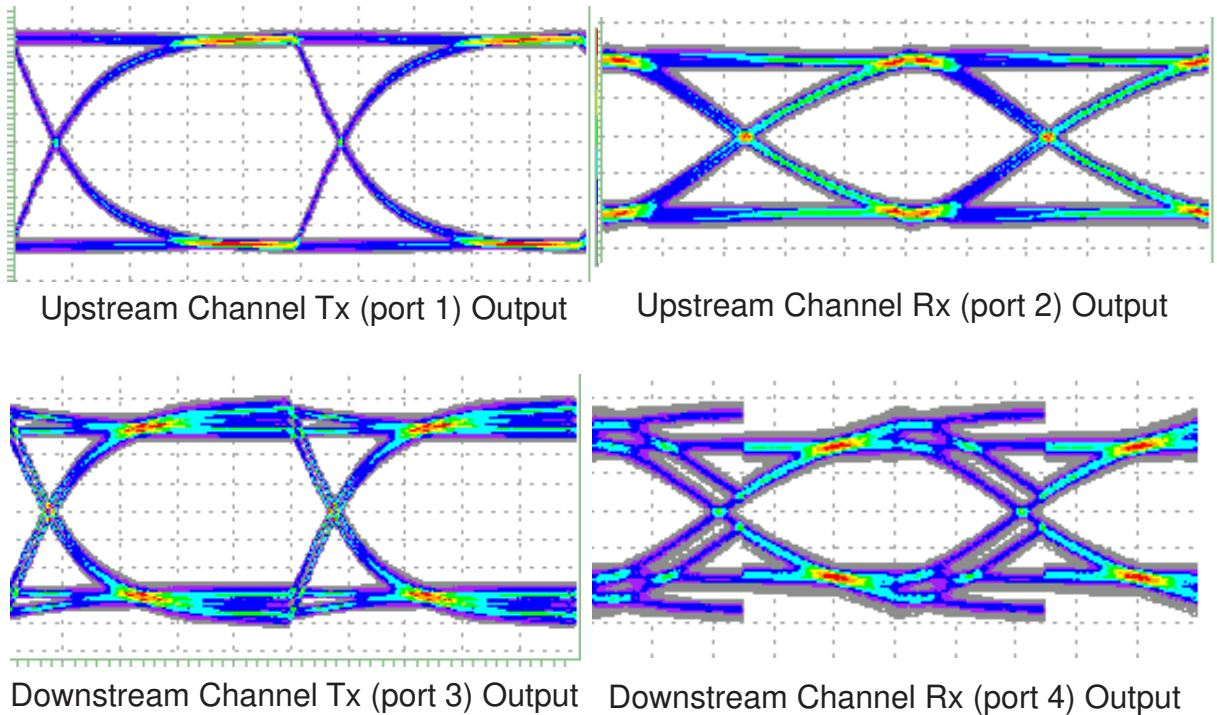
P3 in2 0 port=3
+ AMI_OBJ='my_tx2.so' AMI_PARAM=my_tx2.ami
X2 in2 out2 CONN
P4 out2 0 port=4
+ AMI_OBJ='my_rx2.so' AMI_PARAM=my_rx2.ami

.stateye T='_UI' trf='trf'
+ incident_port=1 3 probe_port=1 2 3 4
+ retimer = (2,3)
+ mode=conv
+ VD_RANGE=2.0 0.8 2.0 0.8

.probe stateye eye(1) eye(2) eye(3) eye(4)
.option post runlvl=6
```

```
.subckt CONN _in _out
:
.ends
.end
```

Results



References

- [1] IBIS Version 5.0 Specification: https://ibis.org/ver5.0/ver5_0.txt
- [2] IBIS (I/O Buffer Information Specification) home page: <http://www.eigroup.org/ibis/default.htm>
- [3] IBIS Version 7.0 Specification: (Training/Analysis flow for channels with no repeater): https://ibis.org/ver7.0/ver7_0.pdf
- [4] IBIS Specification v7.0 : <http://ibis.org/specs/>

Part 3: Feed-Forward Equalization with the PrimeSim HSPICE Tool

Describes how the Feed-Forward Equalization can be modeled in a variety of methods in PrimeSim HSPICE .

The following sections discuss this topic:

- [Introduction to Feed-Forward Equalization](#)

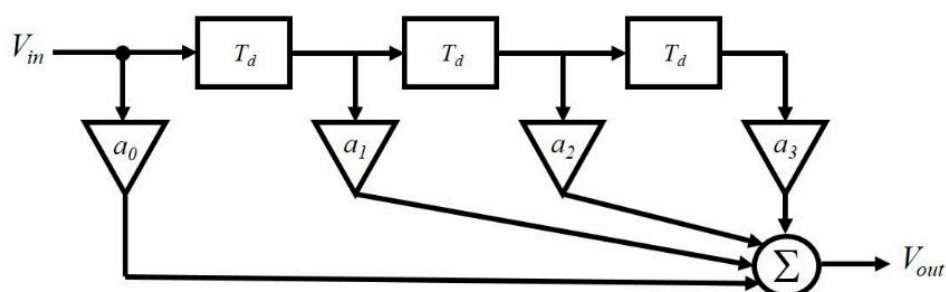
10

Introduction to Feed-Forward Equalization

Equalization is important in high-speed data channels where frequency dependent losses and dispersion result in inter symbol interference (ISI). Equalizers are used at both transmit and receive positions to compensate for channel distortion.

Feed-forward equalization (FFE) is useful at the transmitter (TX) end, where it can be used to increase the transmitted ratio of high-frequency to low-frequency symbol power to help offset channel losses at higher frequencies. The basic FFE equalizer concept is shown in [Figure 98](#). Delayed and scaled versions of the transmitted signal are combined to reshape its power spectrum.

Figure 98 A 4-tap FFE



This demo example shows three different approaches

- [FFE Modeling With Verilog-A](#)
- [Built-in FFE Model Using Port Emphasis Parameters](#)
- [E-element Delay Based Modeling](#)

FFE Modeling With Verilog-A

The time-sampled (Z-domain) transfer function of a 4-tap FFE can be written as

$$H(z) = a_0 + a_1z^{-1} + a_2z^{-2} + a_3z^{-3}$$

Using Verilog-A, the transfer function is easily implemented in PrimeSim HSPICE using a linear discrete-time Z-transform filter operator. The core of an example Verilog-A model implementation is below:

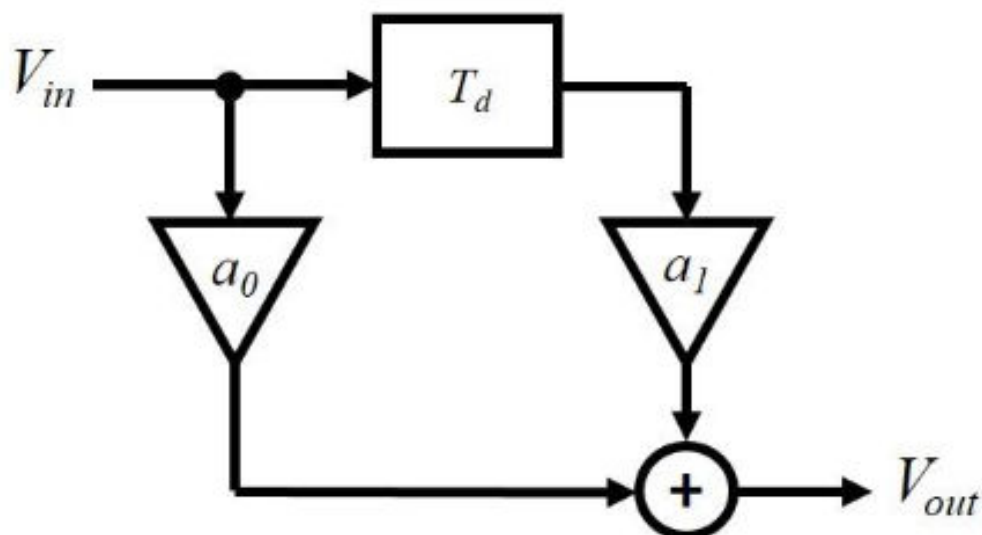
```
// Z-transformFFE model in Verilog-A
...
analog begin
    V(out) <+ zi_nd( V(in), {a0,a1,a2,a3}, {1}, sampleT );
end
...
```

The `zi_nd()` function implements the numerator-denominator form of a Z-transform filter. The real parameter values `a0` to `a3` represent the FFE tap coefficients, and `sampleT` is the sampling interval. An advantage of this linear Verilog-A implementation is that it can be analyzed for both linear and nonlinear simulations. In fact, the Z-domain model has a continuous linear time response in PrimeSim HSPICE. This means its evaluation for both TranEye and StatEye simulations are the same, and it can be used in StatEye with no topology restrictions.

Built-in FFE Model Using Port Emphasis Parameters

PrimeSim HSPICE also includes a simple built-in 2-tap analog continuous-time feed-forward equalizer for sources specified with the Port element. This built-in FFE is useful for adding emphasis or de-emphasis to clock and data streams. The 2-tap configuration is shown in

Figure 99 2-tap FFE



This FFE has a continuous input/output relationship given by

$$V_{out}(t) = a_0 \cdot V_{in}(t) + a_1 \cdot V_{in}(t - T_e)$$

where the `Emphasis_Time` parameter sets the T_e delay value (typically one-half the bit-width or $UI/2$) and the `Emphasis_Level` or e_{level} parameter controls both tap coefficients according to

$$a_0 = 1 + e_{level}$$

$$a_1 = -|e_{level}|$$

Note that negative values of e_{level} results in de-emphasis, while positive values give emphasis. This 2-tap combination restricts the emphasis/de-emphasis waveform effect to the `Emphasis_Time` interval on rising and falling edges.

E-element Delay Based Modeling

Another option for constructing FFEs is to use the PrimeSim HSPICE E-element as a delay and scaling element:

```
Exxx n+ n- [VCVS] DELAY in+ in- TD=val SCALE=val
```

where the propagation delay is specified with the input parameter `TD` and the scaling is given by the `SCALE` parameter. By using a combination of E-elements as delays and VCVS components (for scaling and summing), a two-tap FFE can be realized as shown in [Figure 100](#).

Figure 100 2-tap FFE using stacked E-elements

```
** 1st tap (0.5*UI of propagation delay added)
Edo d0 0 DELAY in 0 TD='0.5*UI' scale='1.0+e_level'
** 2nd tap:
Ed1 d1 d0 DELAY in 0 TD='UI' scale=' -abs(e_level)'
** Output impedance:
Rout out_E d1 R=50
```

In this example, the main tap has an initial propagation delay of 0.5Unit Intervals(UI). The subsequent tap has an additional delay of $UI/2$. Note that this example corresponds to the sampled Z-domain transfer function.

$$H(z) = a_0 + a_1 z^{-1}$$

But with an additional propagation delay and with a sampling interval of $UI/2$, E-element FFE implementations are also linear, and can be used with all PrimeSim HSPICE analyses including StatEye.

Appendices

Describes how to simulate circuits with Signetics drivers and Xilinx FPGAs.

The following topics are covered in this appendix:

- [Simulating Circuits with Signetics Drivers](#)
- [Simulating Circuits with Xilinx FPGAs](#)

A

Simulating Circuits with Signetics Drivers

This appendix describes how to simulate circuits with Signetics drivers.

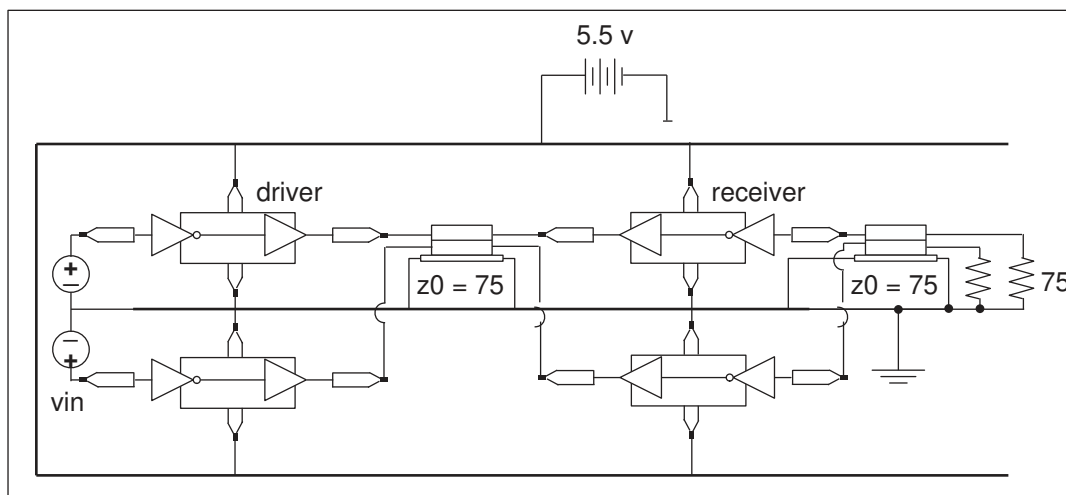
The following topics are covered in this appendix:

- [Simulating Circuits with Signetics Drivers—Examples](#)

Simulating Circuits with Signetics Drivers—Examples

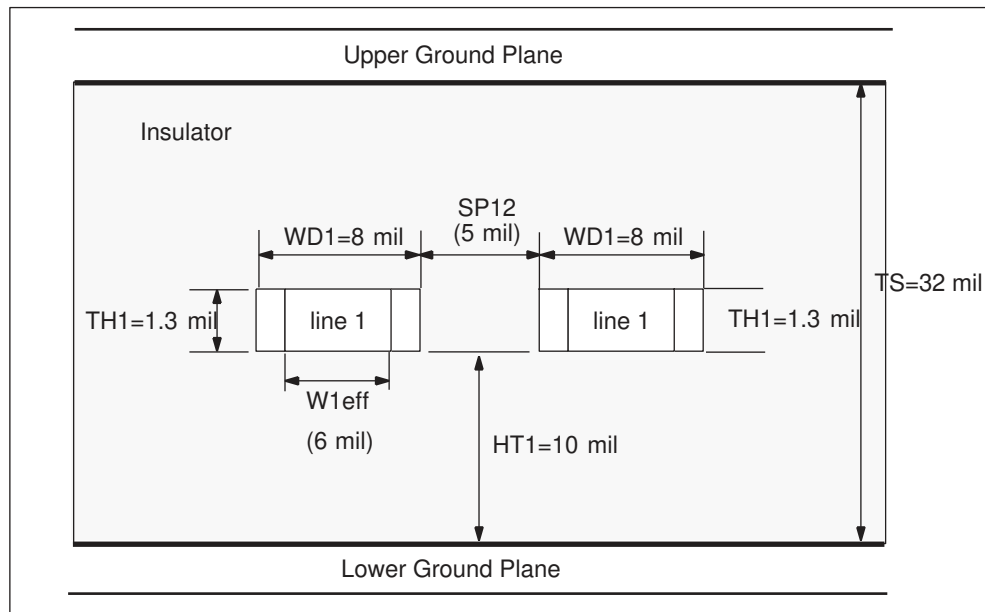
PrimeSim HSPICE includes a Signetics I/O buffer library in the `$installdir/parts/signet` directory. You can use these high-performance parts in the backplane design. An example describes of how to combine these parts with transmission line models.

Figure 101 I/O drivers/receivers with package lead inductance, parallel 4" lossy microstrip connectors



The schematic in [Figure 101](#) shows a pair of drivers driving 4 inches of transmission line to a pair of receivers that drive 4 inches of transmission line. A cross-section of the transmission line is shown in [Figure 102](#).

Figure 102 Planar transmission line DLEV=2: microstrip sea of dielectric

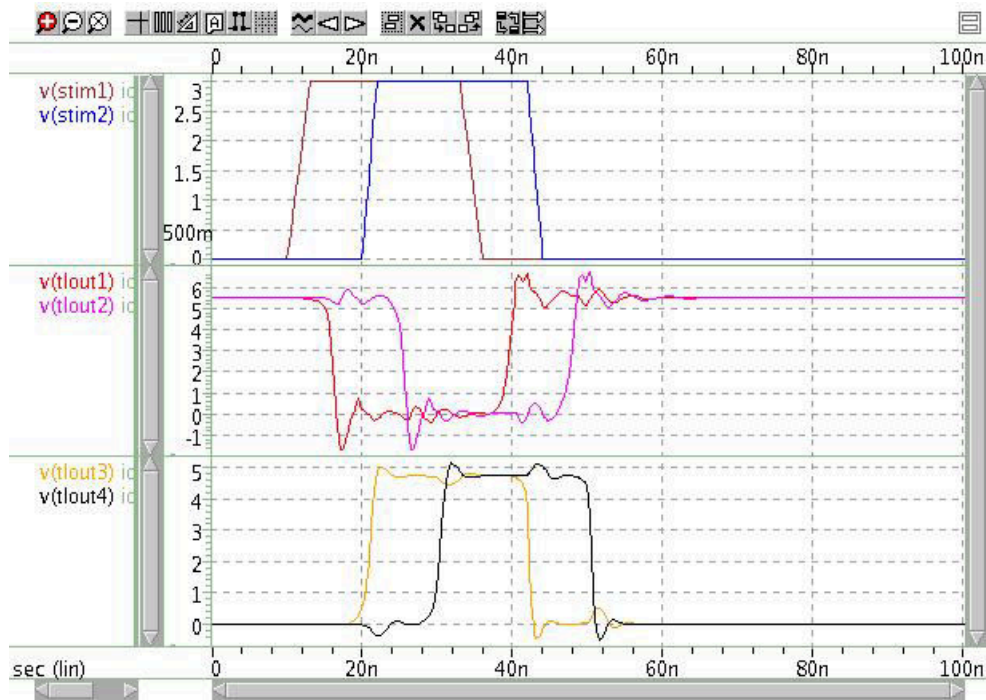


The following is an example of a chip model with package inductance:

```
.SUBCKT IO_CHIP IN OUT VCC XGND PIN_VCC=7n PIN_GND=1.8n
X1 IN1 INVOUT VCC1 XGND1 ACTINPUT
X2 INVOUT OUT1 VCC1 XGND1 AC109EQ
* Package inductance
LIN_PIN IN IN1 PIN_IN
LOUT_PIN OUT1 OUT PIN_OUT
LVCC VCC VCC1 PIN_VCC
LGND XGND1 XGND PIN_GND
.ENDS
$ TLINE MODEL - 2 SIGNAL CONDUCTORS WITH GND
$ PLANE
.MODEL USTRIP U LEVEL=3 ELEV=1 PLEV=1
+ TH1=1.3mil HT1=10mil TS=32mil KD1=4.5 DLEV=0 WD1=8mil
+ XW=-2mil KD2=4.5 NL=2 SP12=5mil
$ ANALYSIS / PRINTS
.TRAN .1NS 100NS
.PROBE V(STIM1) V(STIM2) $ Inputs
.PROBE V(TLOUT1) V(TLOUT2) V(TLOUT3) V(TLOUT4) $ Outputs
.END
```

Simulation results for this model are shown in [Figure 103](#).

Figure 103 Simulation results of I/O Chips connected with Tlines



Here's a full netlist example of how I/O chips connect with transmission lines:

```
.OPTION SEARCH='$installdir/parts/signet'
.OPTION POST=2 TNOM=27 NOMOD LIST METHOD=GEAR
.TEMP 27
$ DEFINE PARAMETER VALUES
.PARAM LV=0 HV=3 TD1=10n TR1=3n TF1=3n TPW=20n
+ TPER=100n TD2=20n TR2=2n TF2=2n LNGTH=101.6m
$ POWER SUPPLY
VCC VCC 0 DC 5.5
$ INPUT SOURCES
VIN1 STIM1 0 PULSE LV HV TD1 TR1 TF1 TPW TPER
VIN2 STIM2 0 PULSE LV HV TD2 TR2 TF2 TPW TPER
$ FIRST STAGE: DRIVER WITH TLINE
X1ST_TOP STIM1 OUTPIN1 VCC GND IO_CHIP PIN_IN=2.6n
+ PIN_OUT=4.6n
X1ST_DN STIM2 OUTPIN2 VCC GND IO_CHIP PIN_IN=2.9n
+ PIN_OUT=5.6n
U_1ST OUTPIN1 OUTPIN2 GND TLOUT1 TLOUT2 GND USTRIP L=LNGTH
$ SECOND STAGE: RECEIVER WITH TLINE
X2ST_TOP TLOUT1 OUTPIN3 VCC GND IO_CHIP PIN_IN=4.0n
+ PIN_OUT=2.5n
X2ST_DN TLOUT2 OUTPIN4 VCC GND IO_CHIP PIN_IN=3.6n
+ PIN_OUT=5.1n
U_2ST OUTPIN3 OUTPIN4 GND TLOUT3 TLOUT4 GND USTRIP L=LNGTH
$ TERMINATING RESISTORS
```

Appendix A: Simulating Circuits with Signetics Drivers

Simulating Circuits with Signetics Drivers—Examples

```

R1 TLOUT3 GND 75
R2 TLOUT4 GND 75
$ IO CHIP MODEL - SIGNETICS
.SUBCKT IO_CHIP IN OUT VCC XGND PIN_VCC=7n PIN_GND=1.8n
X1 IN1 INVOUT VCC1 XGND1 ACTINPUT
X2 INVOUT OUT1 VCC1 XGND1 AC109EQ
*Package Inductance
LIN PIN IN IN1 PIN_IN
LOUT_PIN OUT1 OUT PIN_OUT
LVCC VCC VCC1 PIN_VCC
LGND XGND1 XGND PIN_GND
.ENDS
$ TLINE MODEL - 2 SIGNAL CONDUCTORS WITH GND
$ PLANE
.MODEL USTRIP U LEVEL=3 ELEV=1 PLEV=1
+ TH1=1.3mil HT1=10mil TS=32mil KD1=4.5 DLEV=0 WD1=8mil
+ XW=-2mil KD2=4.5 NL=2 SP12=5mil
$ ANALYSIS / PRINTS
.TRAN .1NS 100NS
.PROBE V(STIM1) V(STIM2)
.PROBE V(TLOUT1) V(TLOUT2) V(TLOUT3) V(TLOUT4)
.END

```

B

Simulating Circuits with Xilinx FPGAs

This appendix describes how to simulate circuits with Xilinx FPGAs.

The following topics are covered in this appendix:

- [Introduction to Simulating Circuits with Xilinx FPGAs](#)
- [Syntax for IOB \(xil_job\) and IOB4 \(xil_job4\)](#)
- [Ground-Bounce Simulation](#)
- [Coupled Line Noise](#)

Introduction to Simulating Circuits with Xilinx FPGAs

Synopsys and Xilinx maintain a library of PrimeSim HSPICE device models and transistor-level subcircuits for the Xilinx 3000 and 4000 series Field Programmable Gate Arrays (FPGAs). These subcircuits model the input and output buffer. See Signal Integrity Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

The following simulations described in this appendix use the Xilinx input/output buffer (xil_job.inc) to simulate:

- Ground bounce, as a function of package, temperature, part speed, and technology
- Coupled noise, both on-chip and chip-to-chip
- Full transmission line effects at the package level and the printed circuit board level
- Peak current and instantaneous power consumption for power supply bus considerations and chip capacitor placement

Syntax for IOB (xil_job) and IOB4 (xil_job4)

Example of call for 1.2u PART:

```
X1 I O PAD TS FAST PPUB TTL VDD GND XIL_IOB
+ XIL_SIG=0 XIL_DTEMP=0 XIL_SHRINK=0
```


Example of call for 1.08u part:

```
X1 I O PAD TS FAST PPUB TTL VDD GND XIL_IOB
+ XIL_SIG=0 XIL_DTEMP=0 XIL_SHRINK=1
```

Nodes	Description
I (IOB only)	output of the TTL/CMOS receiver
O (IOB only)	input pad driver stage
I1 (IOB4 only)	input data 1
I2 (IOB4 only)	input data 2
DRIV_IN (IOB4 only)	
PAD	bonding pad connection
TS	three-state control input (5 V disables)
FAST	slew rate control (5 V fast)
PPUB (IOB only)	pad pull-up enable (0 V enables)
PUP (IOB4 only)	pad pull-up enable (0 V enables)
PDOWN (IOB4 only)	pad pull-up enable (5 V enables)
TTL (IOB only)	CMOS/TTL input threshold (5 V selects TTL)
VDD	5-volt supply
GND	ground
XIL_SIG	model distribution: (default 0) -3==> slow 0==> typical +3==> fast
XIL_DTEMP	Buffer temperature difference from ambient. The default = 0 degrees if ambient is 25 degrees, and if the buffer is 10 degrees hotter than XIL_DTEMP=10.
XIL_SHRINK	Old or new part; (default is new): 0==>old 1==>new

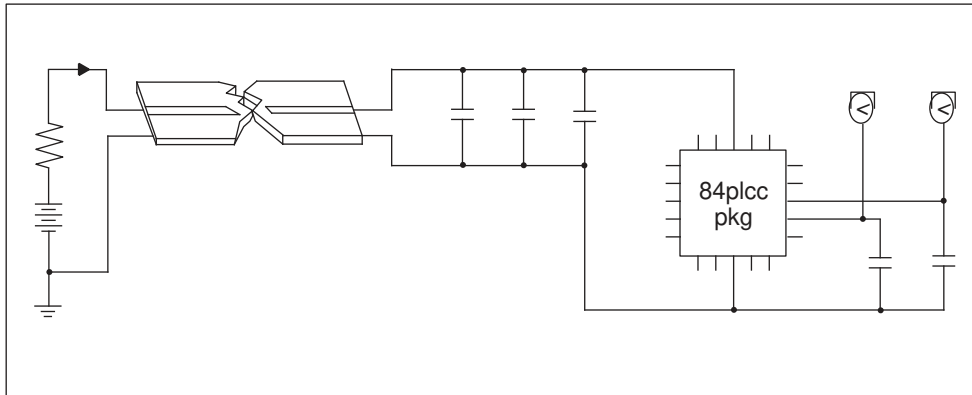
All grounds and supplies are common to the external nodes for the ground and VDD. You can redefine grounds to add package models.

Ground-Bounce Simulation

Ground-bounce simulation duplicates the Xilinx internal measurements methods. It simultaneously toggles 8 to 32 outputs. The simulation loads each output with a 56 pF

capacitance. Simulation also uses an 84-pin package mode and an output buffer held at chip ground to measure the internal ground bounce.

Figure 104 Ground bounce simulation

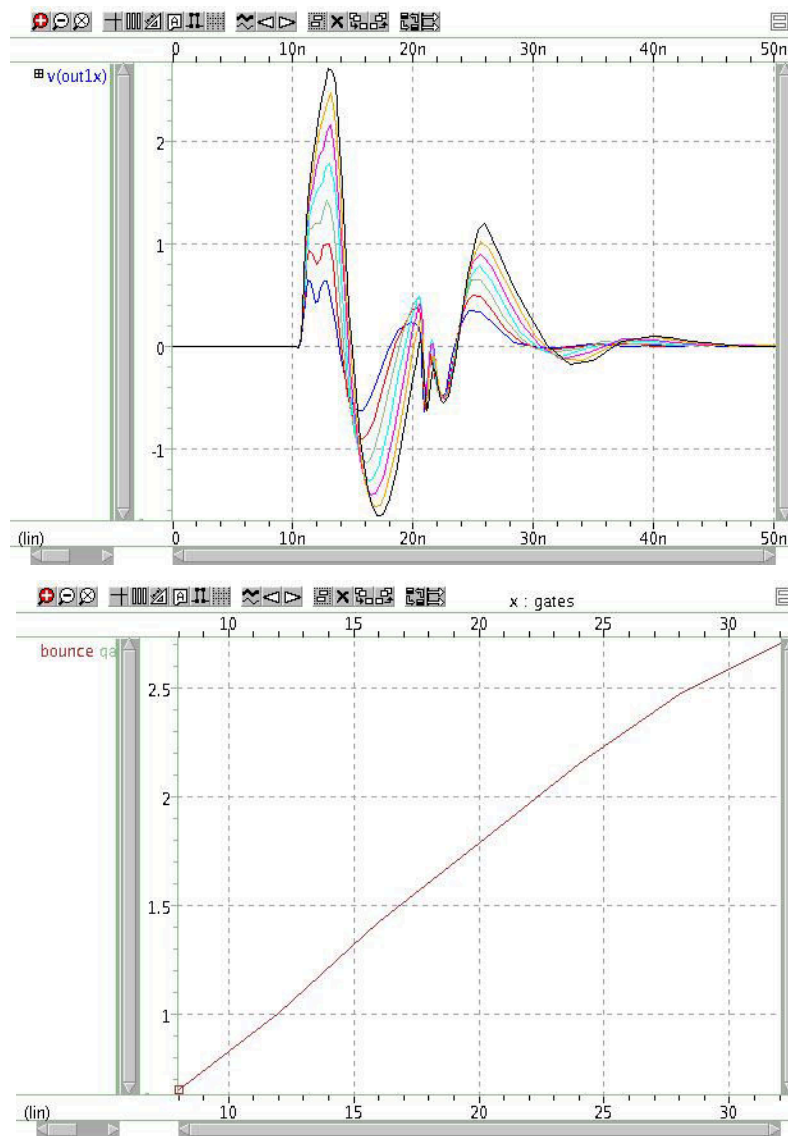


PrimeSim HSPICE adjusts the simulation model for the oscilloscope recordings so you can use it for the two-bond wire ground. For example, the following netlist simulates ground bounce:

```
qabounce.sp test of xilinx i/o buffers
.OPTION SEARCH='$installdir/parts/xilinx'
.op
.option post list
.tran 1ns 50ns sweep gates 8 32 4
.measure bounce max v(out1x)
*.tran .1ns 7ns
.param gates=8
.print v(out1x) v(out8x) i(vdd) power
$.param xil_dtemp=-65 $ -40 degrees c
$ (65 degrees from +25 degrees)
vdd vdd gnd 5.25
vgnd return gnd 0
upower1 vdd return ioblvdv ioblvgnd pcb_power
+ L=600mil
* local power supply capacitors
xcla ioblvdv ioblvgnd cap_mod cval=.1u
xclb ioblvdv ioblvgnd cap_mod cval=.1u
xclc ioblvdv ioblvgnd cap_mod cval=.1u
xgnd_b ioblvdv ioblvgnd out8x out1x xil_gnd_test
xcout8x out8x ioblvgnd cap_mod m=gates
xcout1x out1x ioblvgnd cap_mod m=1
.model pcb_power u LEVEL=3 elev=1 plev=1 nl=1 llev=1
+ th=1.3mil ht=10mil kd=4.5 dlev=1 wd=500mil xw=-2mil
.macro cap_mod node1 node2 cval=56p
Lr1 node1 node1x L=2nh R=0.05
cap node1x node2x c=cval
Lr2 node2x node2 L=2nh R=0.05
```

```
.eom
.macro xil_gnd_test vdd gnd outx outref
+ gates=8
* example of 8 iobuffers simultaneously switching
* through approx. 4nh lead inductance
* 1 iob is active low for ground bounce measurements
vout drive chipgnd pwl 0ns 5v, 10ns 5v, 10.5ns 0v,
$+ 20ns 0v, 20.5ns 5v, 40ns 5v R
x8 I8 drive PAD8x TS FAST PPUB TTL chipvdd chipgnd
+ xil_iob xil_sig=0 xil_dtemp=0 xil_shrink=1 M=gates
x1 I1 gnd PAD1x TS FAST PPUB TTL chipvdd chipgnd
+ xil_iob xil_sig=0 xil_dtemp=0 xil_shrink=1 m=1
*Control Settings
rts ts chipgnd 1
rfast fast chipvdd 1
rppub ppub chipgnd 1
rttl ttl chipvdd 1
* pad model plcc84 rough estimates
lvdd vdd chipvdd L=3.0nh r=.02
lgnd gnd chipgnd L=3.0nh r=.02
lout8x outx pad8x L='5n/gates' r='0.05/gates'
lout1x outref pad1x L=5nh r=0.05
c_vdd_gnd chipvdd chipgnd 100n
.eom
.end
```

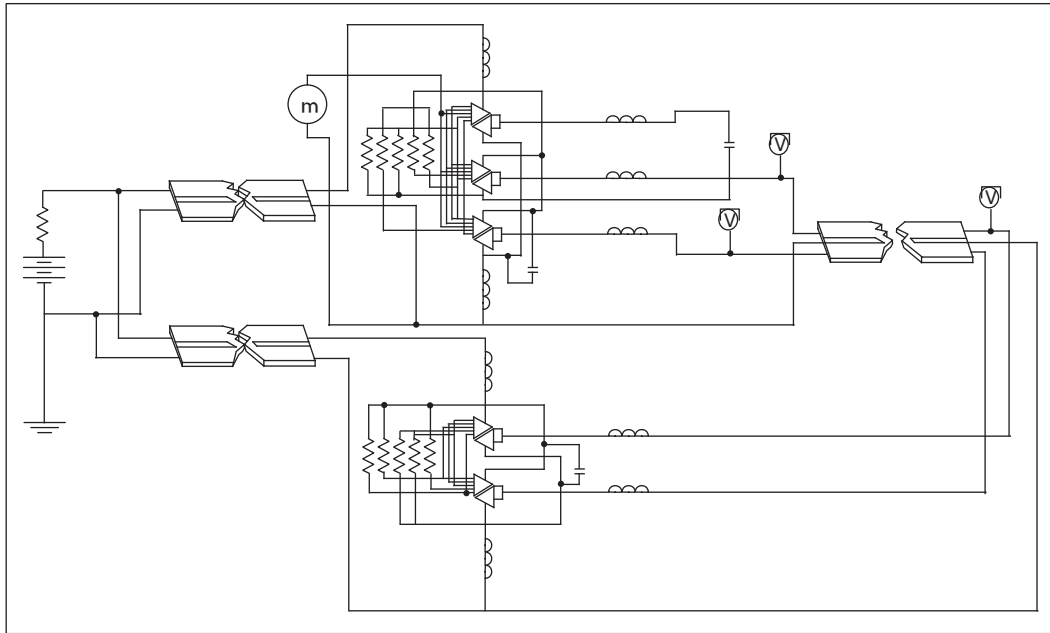
Figure 105 Results of ground bounce simulation



Coupled Line Noise

This example uses coupled noise to separate IOB parts. The output of one part drives the input of the other part through 0.6 inches of PCB. This example also monitors an adjacent quiet line.

Figure 106 Coupled noise simulation



Here is an example netlist for coupled noise simulation:

```
Input File, for qa8.sp test of xilinx 0.8u i/o buffers
.OPTION SEARCH='$installdir/parts/xilinx'
.op
.option nomod post=2
*.tran .1ns 5ns sweep xil_sig -3 3 3
.tran .1ns 15ns
.print v(out1x) v(out3x) i(vdd) v(irec)
vdd vdd gnd 5
vgnd return gnd 0
upower1 vdd return iob1vdd iob1gnd pcb_power L=600mil
upower2 vdd return iob2vdd iob2gnd pcb_power L=600mil
x4io iob1vdd iob1gnd out3x out1x outrec irec xil_iob4
cout3x out3x iob1gnd 9pf
ulx out1x outrec iob1gnd i_o_in i_o_out iob2gnd pcb_top
+ L=2000mil
xrec iob2vdd iob2gnd i_o_in i_o_out xil_rec
.ic i_o_out 0v
.model pcb_top u LEVEL=3 elev=1 plev=1 nl=2 llev=1
+ th=1.3mil ht=10mil sp=5mil kd=4.5 dlev=1 wd=8mil xw=-2mil
.model pcb_power u LEVEL=3 elev=1 plev=1 nl=1 llev=1
+ th=1.3mil ht=10mil kd=4.5 dlev=1 wd=500mil xw=-2mil
.macro xil_rec vdd gnd tri1 tri2
* example of 2 iobuffers in tristate
xtri1 Irec 0 pad_tri1 TSrec FAST PPUB TTL
+ chipvdd chipgnd xil_iob xil_sig=0 xil_dtemp=0 xil_shrink=1
+ m=1
```

Appendix B: Simulating Circuits with Xilinx FPGAs

Coupled Line Noise

```

xtri2 Irec 0 pad_tri2 TSrec FAST PPUB TTL
+ chipvdd chipgnd xil_iob xil_sig=0 xil_dtemp=0
+ xil_shrink=1 m=1
*Control Setting
rin_output 0 chipgnd 1
rtsrec tsrec chipvdd 1
rfast fast chipvdd 1
rppub ppub chipgnd 1
rttl ttl chipvdd 1
* pad model plcc84 rough estimates
lvdd vdd chipvdd L=1nh r=.01
lgnd gnd chipgnd L=1nh r=.01
ltril tri1 pad_tri1 L=3nh r=.01
ltril2 tri2 pad_tri2 L=3nh r=.01
c_vdd_gnd chipvdd chipgnd 100n
.eom
.macro xil_iob4 vdd gnd out3x out1x outrec Irec
* example of 4 iobuffers simultaneously switching
* through approx. 3nh lead inductance
* 1 iob is a receiver (tristate)
vout 0 chipgnd pwl 0ns 0v, 1ns 0v, 1.25ns 4v, 7ns 4v,
+ 7.25ns 0v, 12ns 0v R
x3 I3 0 PAD3x TS FAST PPUB TTL chipvdd chipgnd xil_iob
+ xil_sig=0 xil_dtemp=0 xil_shrink=1 m=3
x1 I1 0 PAD1x TS FAST PPUB TTL chipvdd chipgnd xil_iob
+ xil_sig=0 xil_dtemp=0 xil_shrink=1 m=1
xrec Irec 0 PADrec TSrec FAST PPUB TTL chipvdd chipgnd xil_iob
+ xil_sig=0 xil_dtemp=0 xil_shrink=1 m=1
* control settings
rts ts chipgnd 1
rtsrec tsrec chipvdd 1
rfast fast chipvdd 1
rppub ppub chipgnd 1
rttl ttl chipvdd 1
* pad model plcc84 rough estimates
lvdd vdd chipvdd L=1nh r=.01
lgnd gnd chipgnd L=1nh r=.01
lout3x out3x pad3x L=1nh r=.0033
lout1x out1x pad1x L=4nh r=.01
loutrec outrec padrec L=4nh r=.01
c_vdd_gnd chipvdd chipgnd 100n
.eom
.end

```

Figure 107 Results of coupled noise simulation

